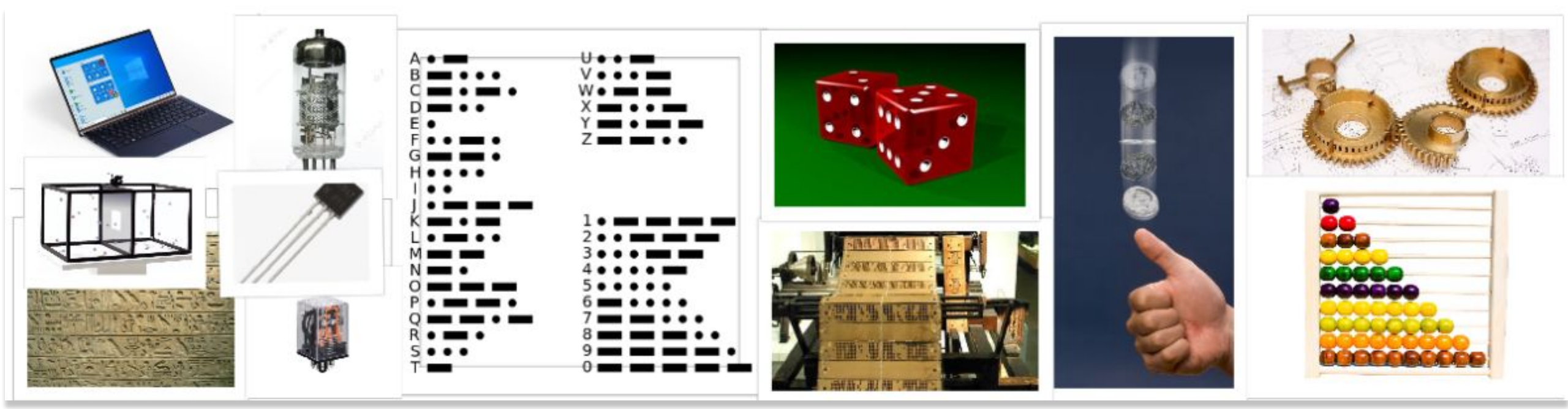


Computer Architecture (CS F342)

Motivation and Introduction

Automatic, Single & General Purpose Computing

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Class Schedule

- Google Classroom (Class materials)
- Moodle or Quanta (Labs)

Day	Venue	Time
Tuesday	LT-1	10 AM to 10:50 AM
Wednesday	LT-1	5 PM to 5:50 AM
Thursday	LT-1	10 AM to 10:50 AM
Thursday [Lab.]	CC-219	2 PM to 3:50 PM

Instructors

- Dr Kanchan Manna (kanchanm@goa.bits-pilani.ac.in)

TAs

TAs

Tentative Evaluation Guideline

Components	Duration (Mints)	Weightage (%)	Date	Nature of Component
Regular Labs.	-	20	Lab 1: Aug 6 Lab 2: Aug 13 Lab 3: Aug 20 Lab 4: Aug 27 Lab 5: Sep 03 Lab 5: Sep 10 Lab 6: Sep 17 Lab 7: Sep 24 Lab 9: Oct 1 Lab 10: Oct 15 Lab 11: Oct 22 Lab 12: Oct 29 Lab 13: Nov 5 Lab 14: Nov 12 Lab 15: Nov 19 Lab 16: Nov 26	Open Book Best n-2 of n
Lab. Test	120	15	Lab : Lab :	TBA
Midsem	90	25	As per the timetable	TBA
Comprehensive	180	40	As per the timetable	TBA

Text and Reference Books

Textbooks:

(T1) Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy.
(T2) Computer Architecture: A quantitative Approach by David A. Patterson and John L. Hennessy.

Reference Books:

(R1) Digital Design: With a Introduction to the Verilog HDL by M. Morris Mano & Michael D. Ciletti
(R2) Verilog HDL: A Guide to Digital Design and Synthesis by Samir Palnitkar.
(R3) Computer Organisation & Architecture: Designing for performance by William Stallings.

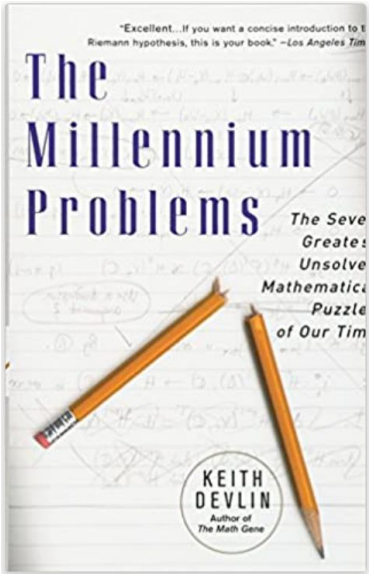
We shall answer the following questions

1. What's computation?
2. What's uncomputable?
3. What's automatic computation/automation?
4. What's a single-purpose microprocessor?
5. What's a general-purpose microprocessor?
6. How does a laptop solve the problems?
7. Past & Future of Microprocessors

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

What is the meaning of Computable?

- What is meant to be computable?
  - Is a number prime?
  - An algorithm is present
- Is there anything to uncomputable?
  - Is a number random?
  - Is P =? NP
  - An algorithm is not present till now



Unsolved problems: https://en.wikipedia.org/wiki/Millennium_Prize_Problems?fbclid=IwAR1I9mERgOdeaKJASjcXE9AaMht3U6x_4zc0orU_aEC49Vt5U35ZOZB3-Lo
https://en.wikipedia.org/wiki/Smale%27s_problems?fbclid=IwAR3K5n01NUIpeQy_AsXen6dKtMSDzl5RuQ6xQrXuIcNRmdIvR_kPjqVnyEQ

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

A problem: Addition of two numbers

- Two numbers: -2 and 8
- Result: 6
- How do humans calculate it?
- How does a calculator/computer/integrated circuit calculate it?
- Is there an algorithm available for this?

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

A problem: Addition of two numbers

- How does a calculator/computer/IC calculate it?
- Representation of numbers (sign & magnitude)
  - Binary (Digital) system and width
- Is there an algorithm available for this?

Half-Adder

Input		Output	
A	B	S	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

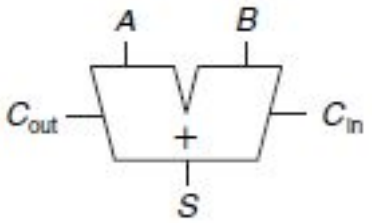
S = A (XOR) B
Cout = A (AND) B

Full-Adder (1-bit)

Input			Output	
Cin	A	B	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Find the relation between input variables and output variables for compressing the table.

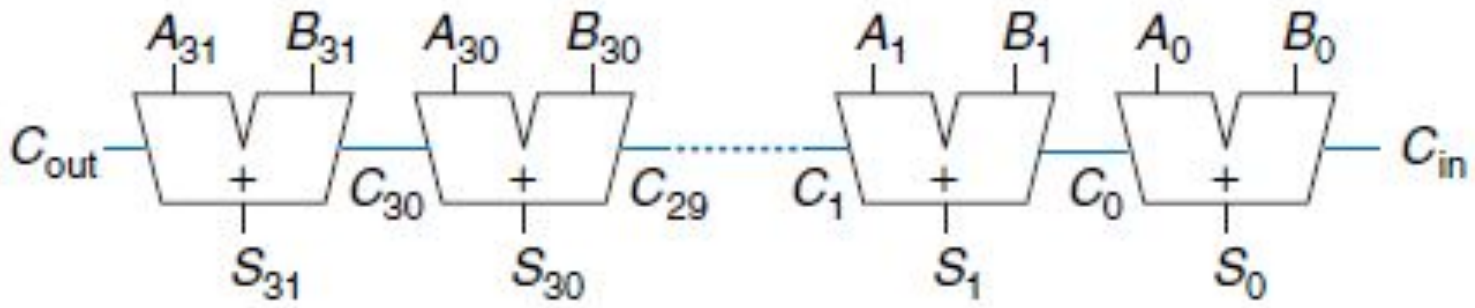
S = A (XOR) B (XOR) Cin
Cout = A (AND) B + A (AND) Cin + B (AND) Cin



© Kanchan Manna; BITS-Pilani, Goa Campus, India.

A problem: Addition of two numbers

- How does a calculator/computer calculate it?
- Is there an algorithm available for this?
  - Ripple-Carry Adder/Algorithm
- Are there efficient algorithms available for this?



- Datapath
- Controller

Can we write a C-code for the above circuit?

Book- COD by P&H –ch-appendix-B

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Basic Elements:
Mapping of High-level Construct onto Digital Construct

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Basic Elements:
Mapping of High-level Construct onto Digital Construct

How does one represent the data?

For example,
C language provides datatype to represent and store the data.

How is this data stored in digital components such as mobile phones, laptops, smartwatches, etc.?

In the storage elements: Registers and Memory

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Basic Elements:
Mapping of High-level Construct onto Digital Construct

Register:

32-bit register



© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Basic Elements:
Mapping of High-level Construct onto Digital Construct

Register:

Each location of the register is made of a latch or flip-flop

A latch or flip-flop

- RS
- **D**
- T

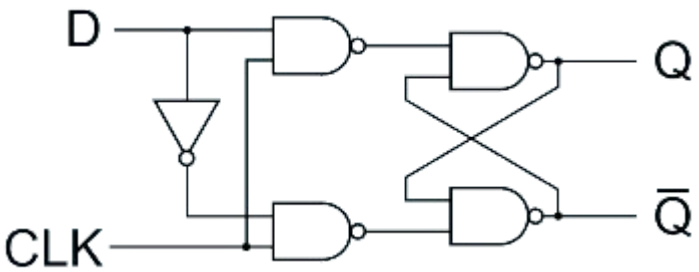


Fig: D-Latch

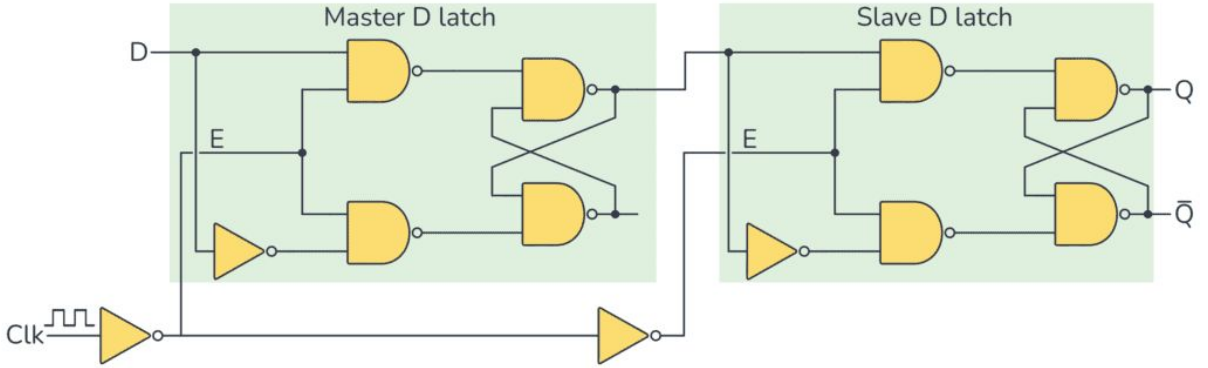


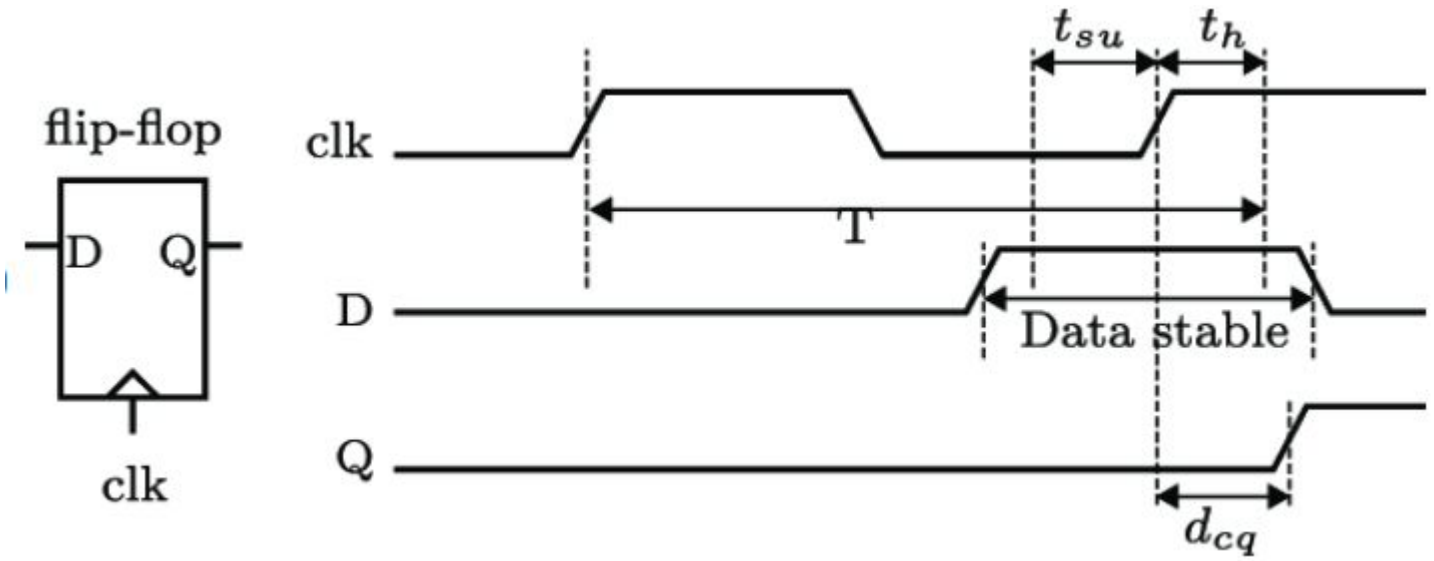
Fig: D-ff

Why do we need a clock in the CKT?

https://www.electronicsforu.com/technology-trends/latch-not-bad-latch-vs-flip-flop
https://www.build-electronic-circuits.com/d-flip-flop/

Book- COD by P&H –ch-appendix-B

Basic Elements: Clock

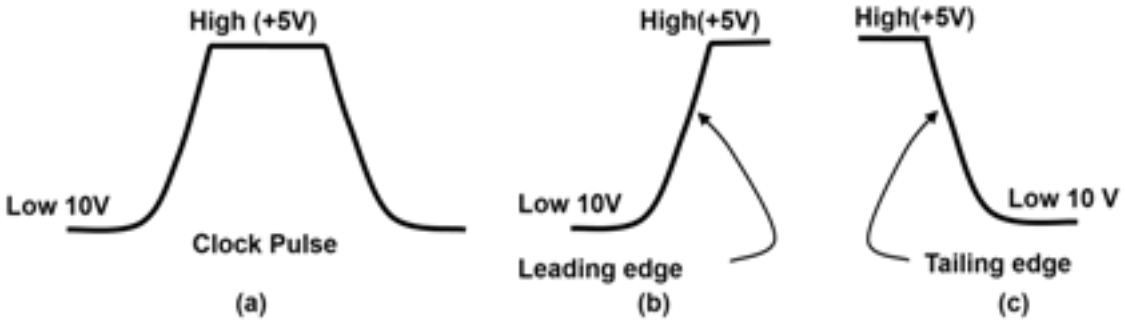


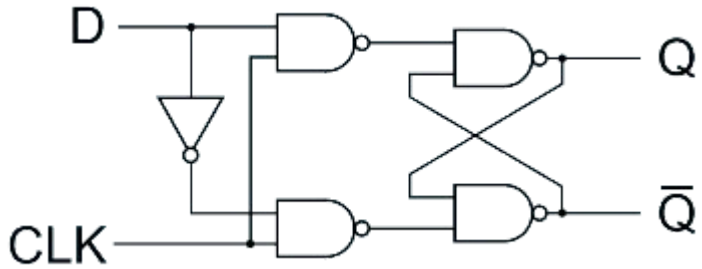
t_{su}: setup time

t_h: Hold time

t_{cq}: Clock-to-Q (q)

T: Clock (clk) Period





CLK edge sensitive

+5 volts, logic 1
+10 volts, logic 0

Book- COD by P&H –ch-appendix-B

Basic Elements:
Verilog D-Latch and D-F/F

```
module d_latch (  input d,          // 1-bit input pin for data
                  input en,         // 1-bit input pin for enabling the latch
                  input rstn,       // 1-bit input pin for active-low reset
                  output reg q);    // 1-bit output pin for data output

   // This always block is "always" triggered whenever en/rstn/d changes
   // If reset is asserted then output will be zero
   // Else as long as enable is high, output q follows input d
   always @ (en or rstn or d)
      if (!rstn)
         q <= 0;
      else
         if (en)
            q <= d;
endmodule
```

```
module dff(clk,d,q);
input clk,d;
output reg q;

always @ (posedge clk)begin
    q <= d;
end

endmodule
```

https://circuitfever.com/d-flip-flop-in-verilog

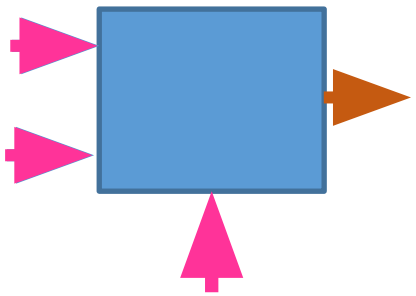
How to map Algorithm's elements onto Architectural elements

High-level Construct	Digital Construct	Type of Component
Scalars/variable	Registers or wires	Sequential if Reg.
Arrays	Memories	Sequential
Operators (+,-,*,/, etc)	Functional unit	Combinational
Control flows [if-else, switch & loop]	Control unit	Combinational/Sequential

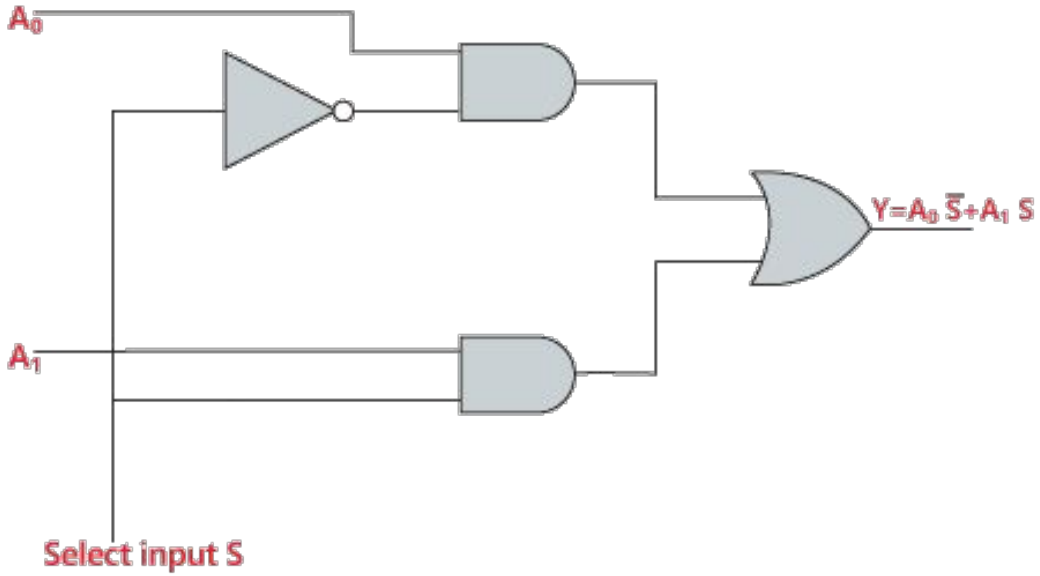
© Kanchan Manna; BITS-Pilani, Goa Campus, India.

How to map Algorithm's elements onto Architectural elements


```
if (sel)
   a = 10;
else
   a = 5;
```



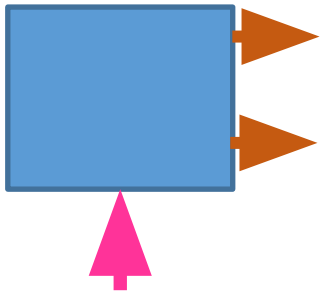
Multiplexer



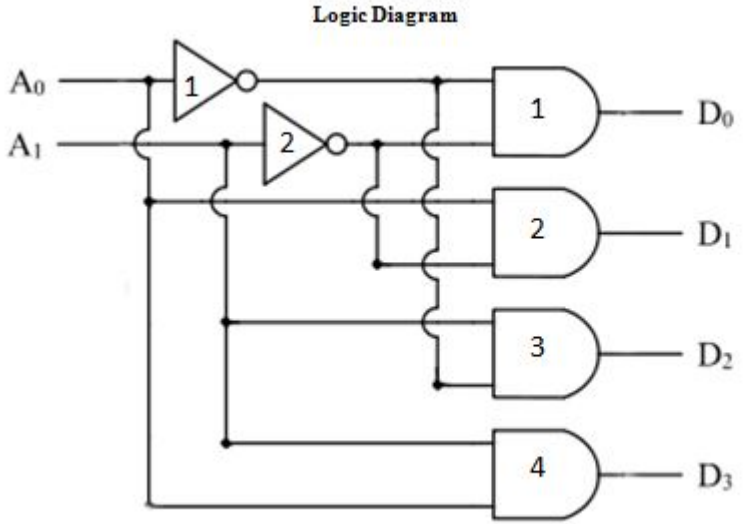
© Kanchan Manna; BITS-Pilani, Goa Campus, India.

How to map Algorithm's elements onto Architectural elements


```
if (sel)
   a = 10;
else
   b = 5;
```



Decoder



Truth Table

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Equations

$D_0 = \overline{A_1} \cdot \overline{A_0}$

$D_1 = \overline{A_1} \cdot A_0$

$D_2 = A_1 \cdot \overline{A_0}$

$D_3 = A_1 \cdot A_0$

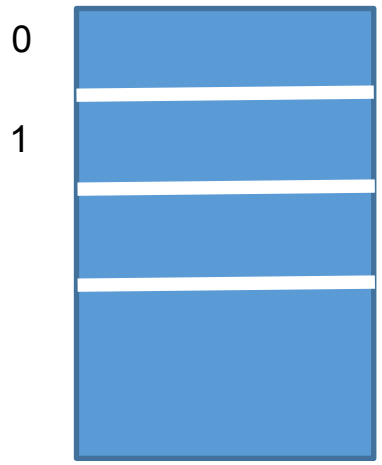
© Kanchan Manna; BITS-Pilani, Goa Campus, India.

How to map Algorithm's elements onto Architectural elements

int Mem[1024], Data;

Mem [10] = Data;

Data = Mem [20];



Mem

Write operation

```
switch (Addrs){

case LOC0:
      Mem[LOC0] = Data;
break;

case LOC1:
      Mem[LOC1] = Data;
break;

case LOC2:
      Mem[LOC2] = Data;
break;

case LOC3:
      Mem[LOC3] = Data;
break;

}
```

Decoder

Read operation

```
switch (Addrs){

case LOC0:
      Data_out = Mem[LOC0]
break;

case LOC1:
      Data_out = Mem[LOC1]
break;

case LOC2:
       Data_out = Mem[LOC2]
break;

case LOC3:
       Data_out = Mem[LOC3]
break;

}
```

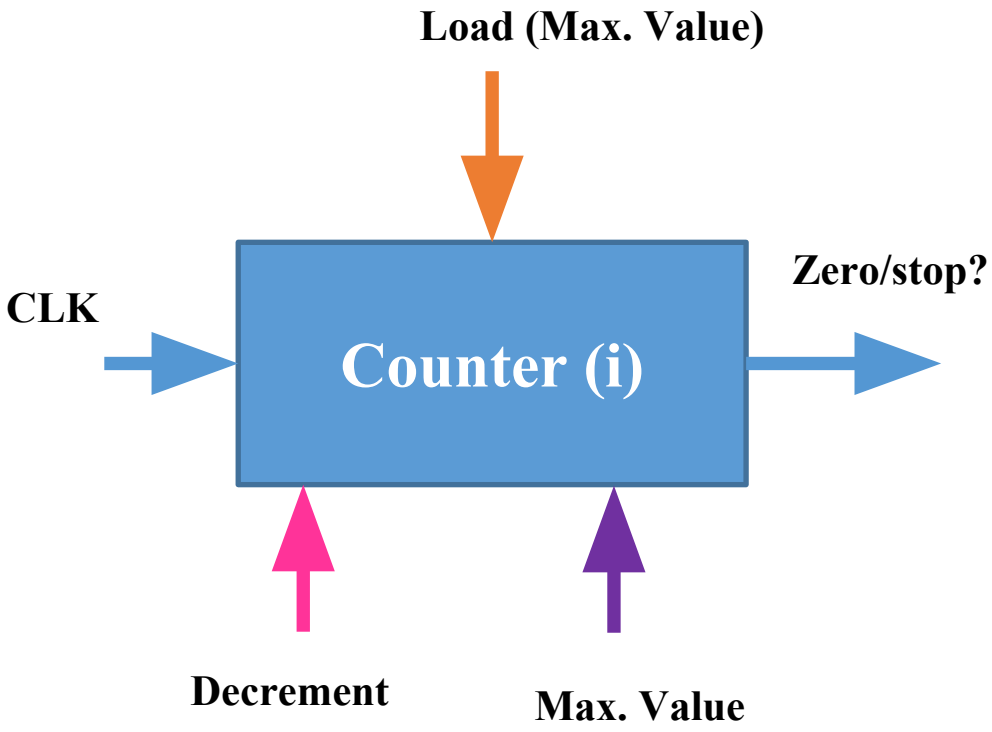
Multiplexer

How to map Algorithm's elements onto Architectural elements


```
int i;

for (i=10; i>0; i--){

}
```



Counter/Register

Counter Design Using D-F/F:
https://www.youtube.com/watch?v=ts4g_NUuHAc

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

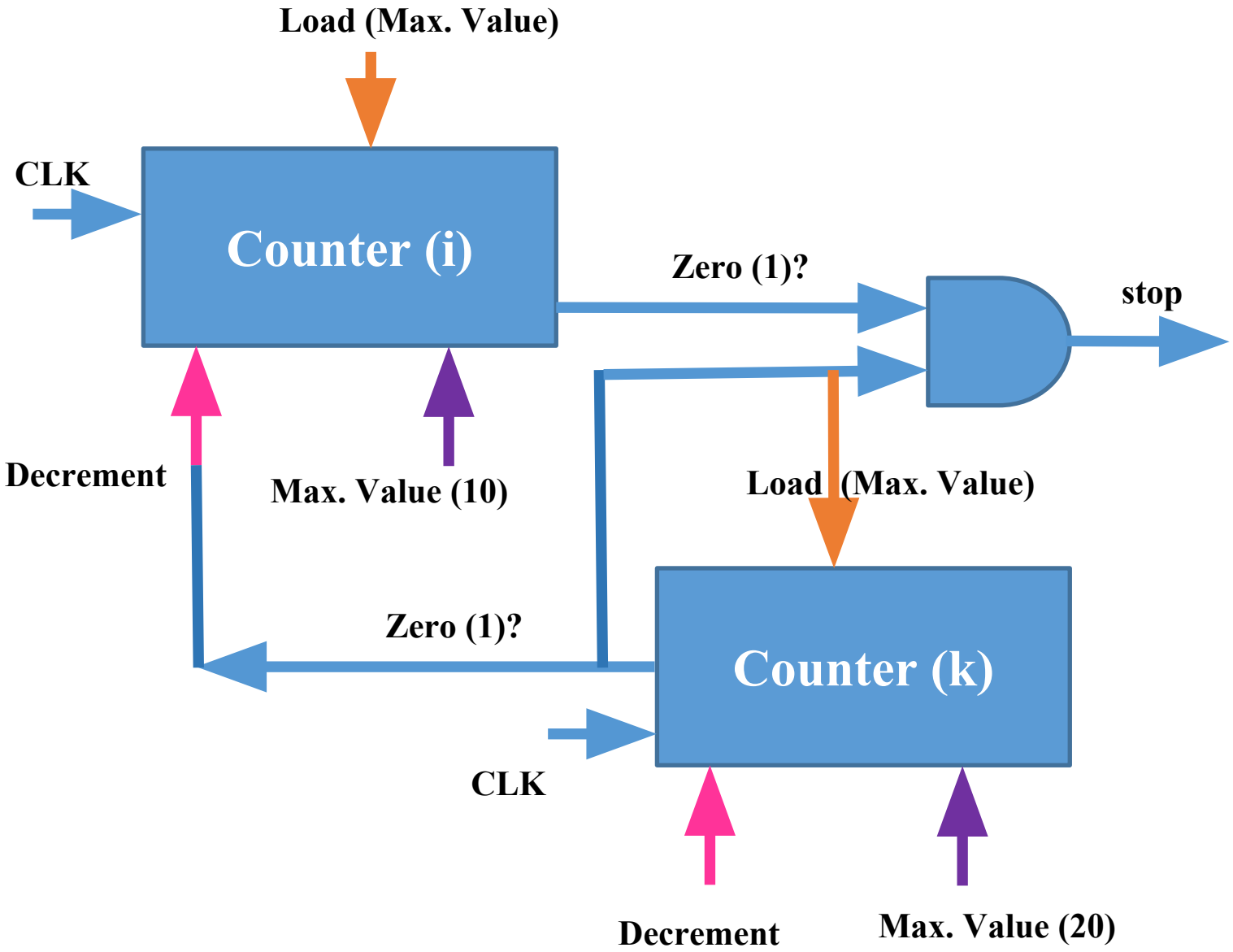
How to map Algorithm's elements onto Architectural elements


```
int i, k;

for (i=10; i>0; i--){
      for (k=20; k>0; k--){
       }
}
```

Zero = 1 if the content of the counter is 0.

Stop = 1 if both counters value is 0.

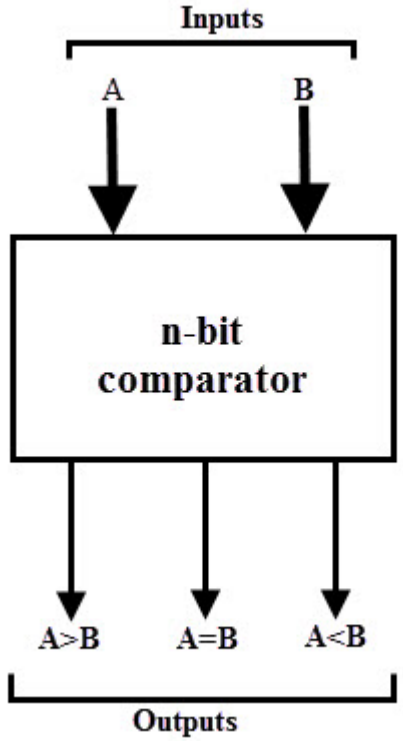


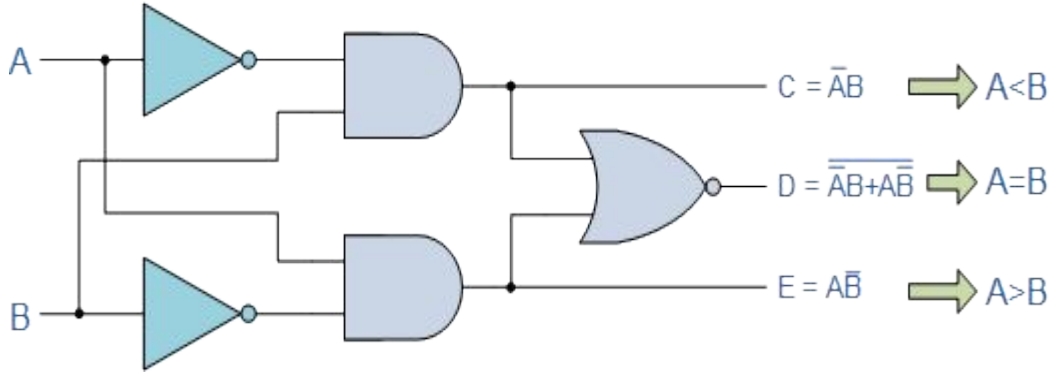
Counter Design Using D-F/F:
https://www.youtube.com/watch?v=ts4g_NUuHAc

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

How to map Algorithm's elements onto Architectural elements

Comparison





© Kanchan Manna; BITS-Pilani, Goa Campus, India.

A problem: Find the minimum and maximum number from a set of numbers

- Set of nos: 1, 2, -8, 0, 23, 11, -10
- Min = -10 and Max = 23
- Is there an algorithm to solve it automatically?

Input: set of n nos stored in A
Output: Min & Max
Min = infi
Max = -infi
Do scan i-th no.
    If Max < A [i] then
        Max = A [i]
    If Min > A [i] then
        Min = A [i]
Until i reaches to n
Stop

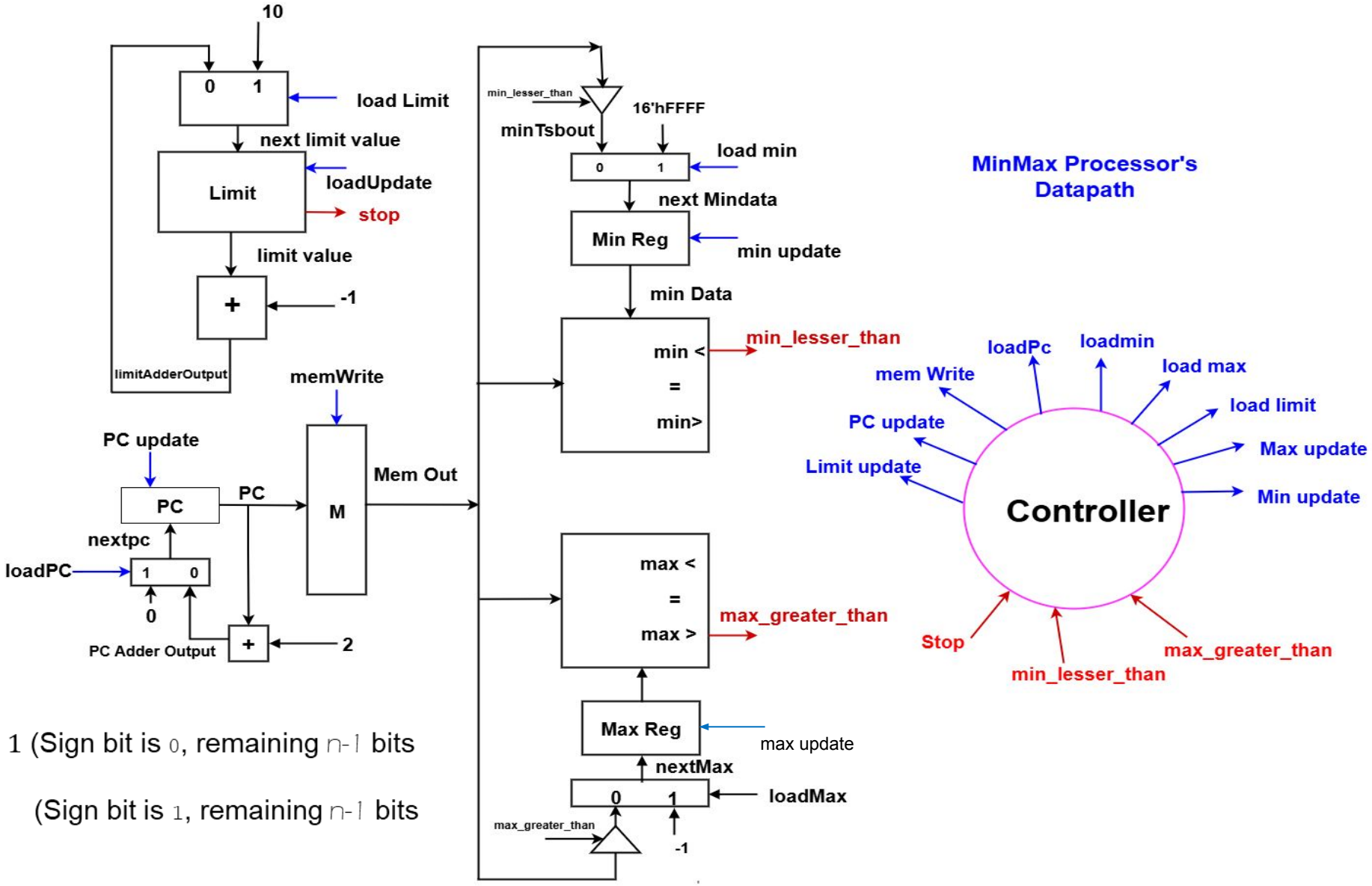
A problem: Find the minimum and maximum number from a set of numbers

- Is there an algorithm to solve it automatically?
- Yes
- Is there an architecture for that algorithm?

Input: set of n nos stored in A
Output: Min & Max
Min = ∞
Max = - ∞
Do scan i-th no.
    If Max < A [i] then
        Max = A [i]
    If Min > A [i] then
        Min = A [i]
Until i reaches to n
Stop

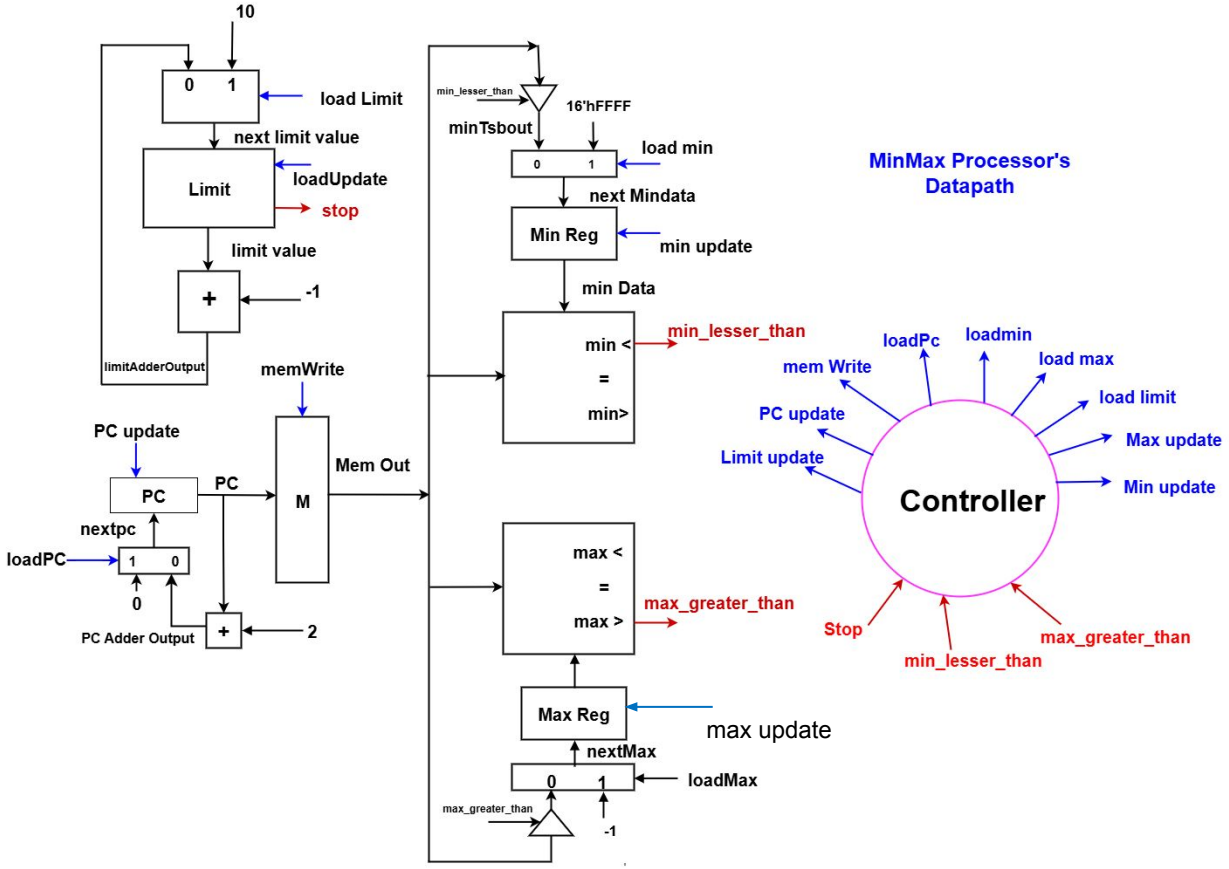
© Kanchan Manna; BITS-Pilani, Goa Campus, India.

MinMax Processor



- **Maximum Value:** $2^{n-1} - 1$ (Sign bit is 0, remaining n-1 bits are 1)
- **Minimum Value:** -2^{n-1} (Sign bit is 1, remaining n-1 bits are 0)

MinMax Processor



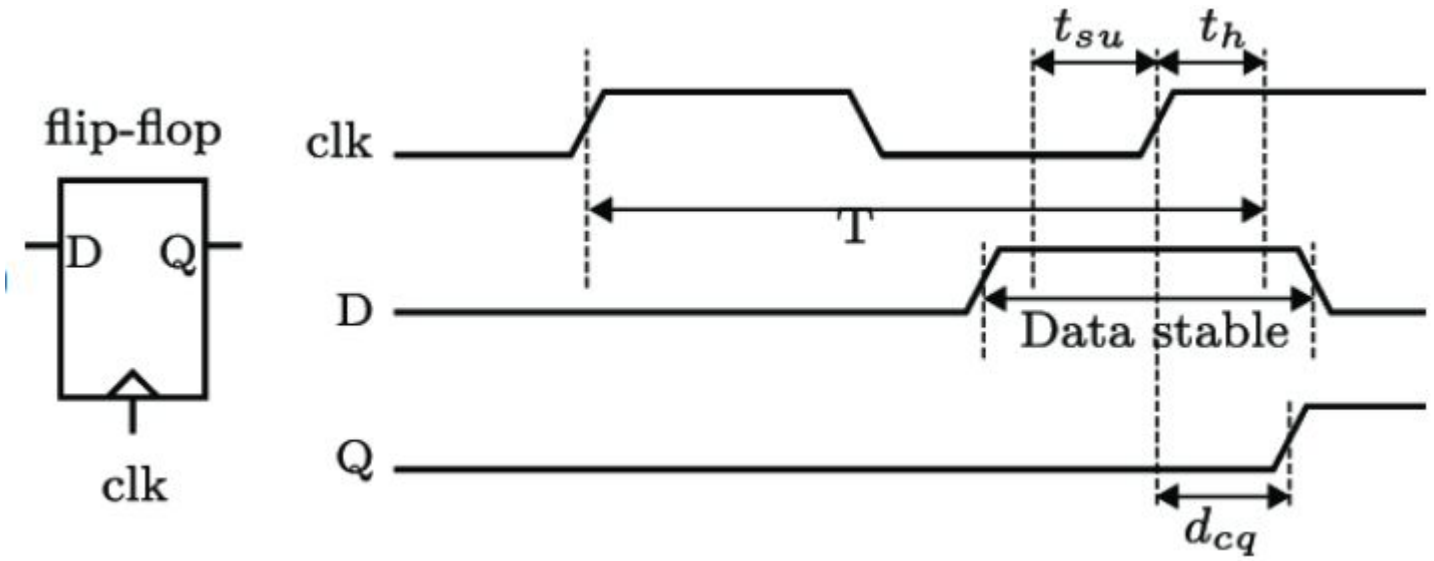
How does one measure the performance of this processor?

After giving inputs to it and time to get the output.

How does one include time in the processor?

Clock

Basic Elements: Clock

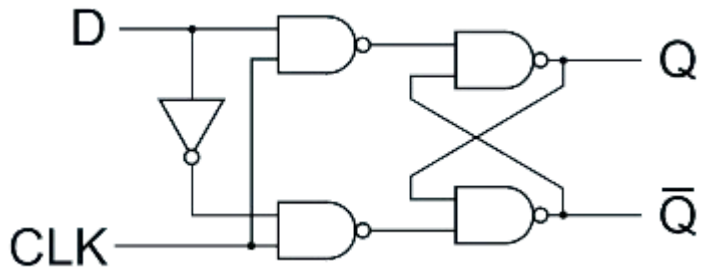


t_{su}: setup time

t_h: Hold time

t_{cq}: Clock-to-Q (q)

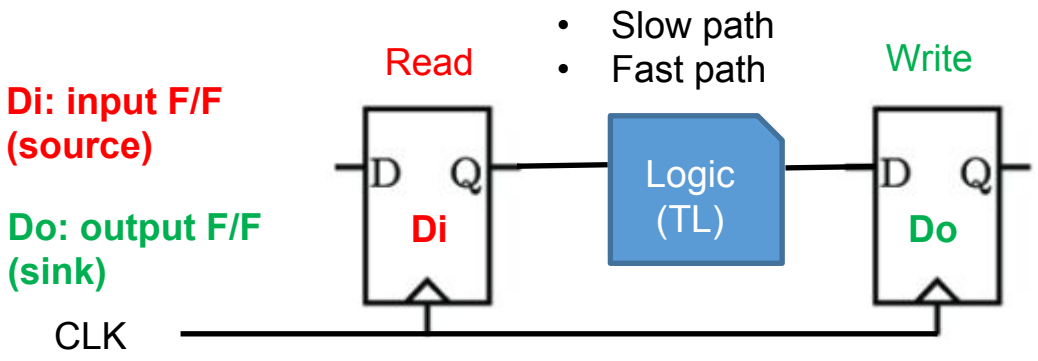
T: Clock (clk) Period



Setup and Hold time come from the Library.

Book- COD by P&H –ch-appendix-B

Data Setup time, Data Hold time, Clock Period calculation



t_{su}: Setup time t_h: Hold time

t_{cq}: Clock-to-Q (q)

T: Clock (clk) Period

TL: Time to compute Logic

Calculate all such input and output pairs of D-ff or Regs. and maintain eq. 1 and eq. 2. Find all T_i

Max (T_i) would be the clock period (T)

Book- COD by P&H –ch-appendix-B

Data launched at one flop (Di) should be captured at the next flop (Do) in the next clock edge. Data launched at one edge should be captured at the next active clock edge.

Let's consider the clock period as T

Data setup time of **Do** (t_{su_Do}) f/f must be

Clock period - t_{su_Do}

Clock period - t_{su_Do}> Data arrival time at **Do**

$$T - t_{su_Do} > t_{cq_{Di}} + TL$$

$$T > t_{cq_{Di}} + TL + t_{su_Do} \quad (1)$$

Hold the data in **Do** f/f so that **race conditions** won't happen

Data hold time of in f/f **Do** (t_{h_Do}) must be

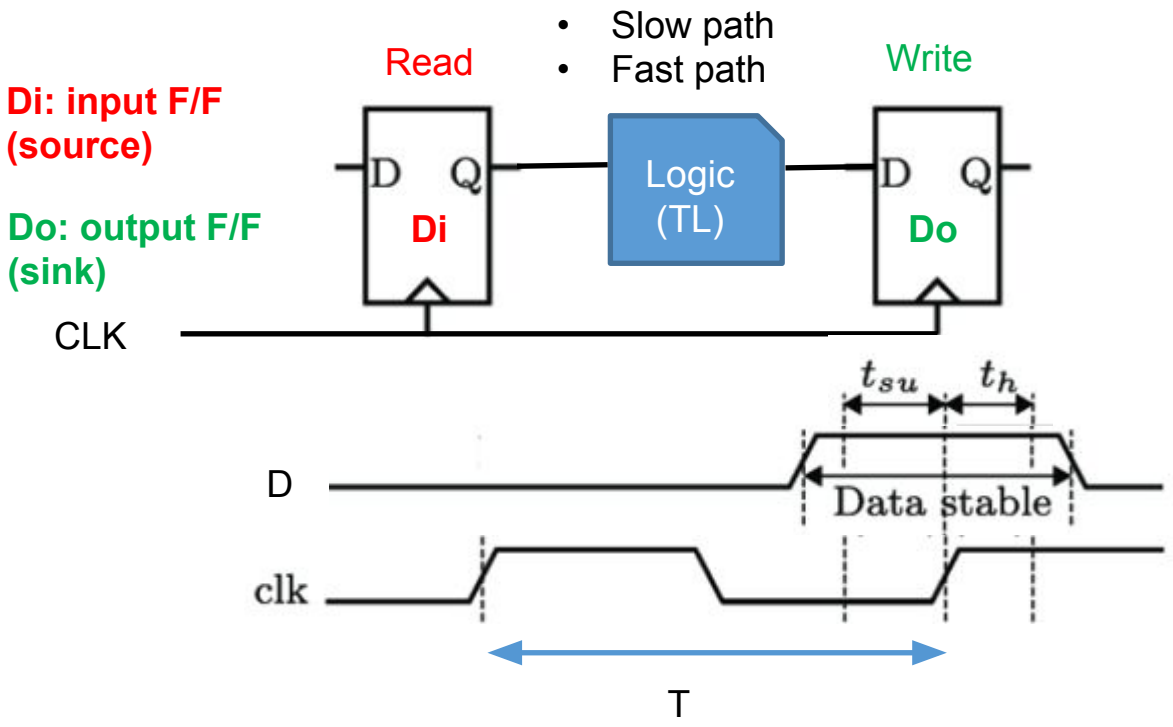
$$t_{cq_Di} + TL > t_{h_Do} \quad (2)$$

What if the previous data's hold time is t_{h_Do} is more than $t_{cq_Di} + TL$?

Clk is also a high-state at that time
Next data would be written in the previous clock

Data Setup time, Data Hold time, Clock period calculation

- Slow path
- Fast path



Let's consider the clock period as T

Data setup time of Do (t_{su_Do}) f/f must be

Clock period (T) - t_{su_Do}

Clock period (T) - t_{su_Do}> Data arrival time at Do

$$T - t_{su_Do} > t_{cq_{Di}} + TL$$

$$T > t_{cq_{Di}} + TL + t_{su_Do} \quad (1)$$

Hold the data in Do f/f so that **race conditions** won't happen

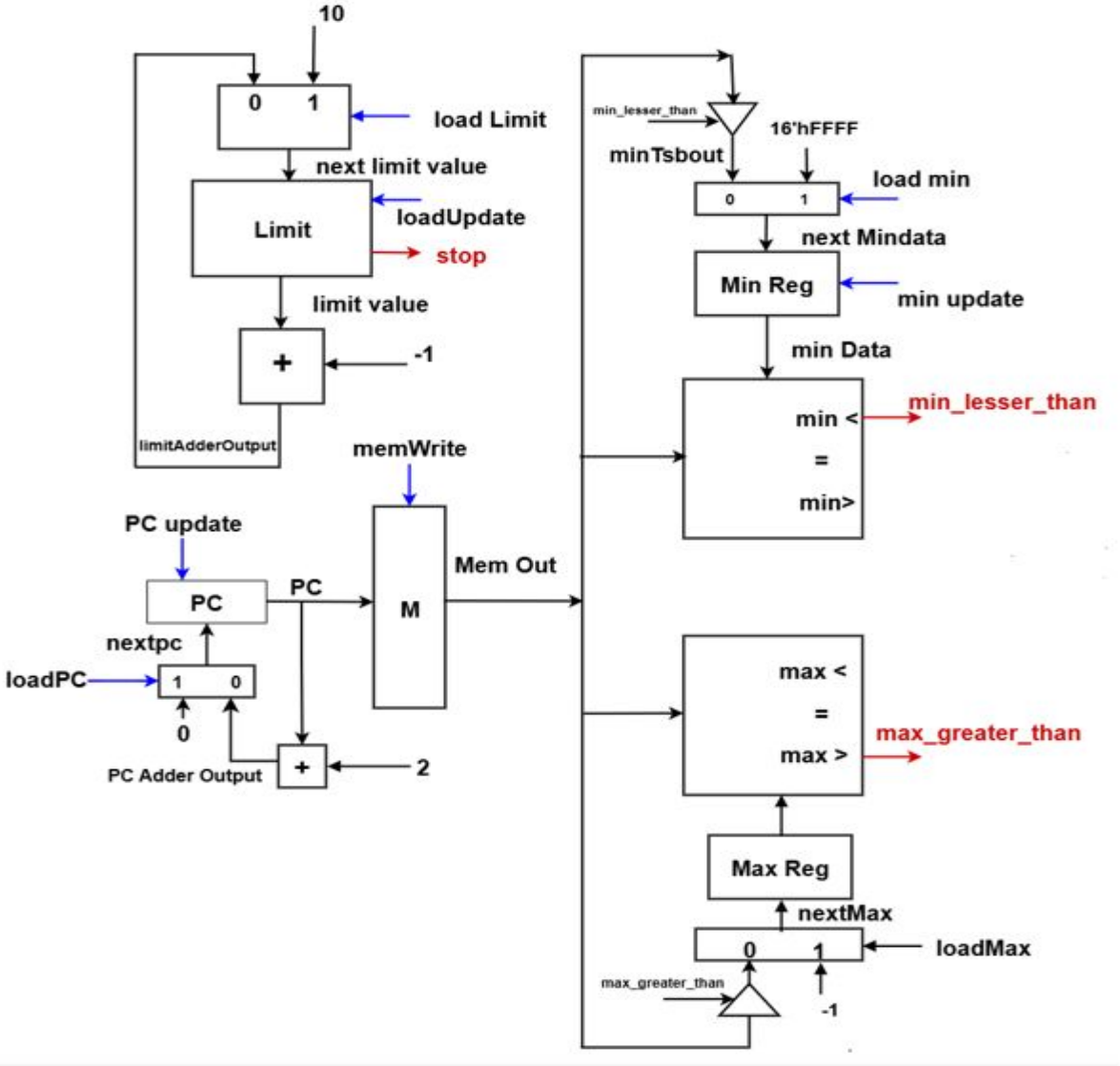
Data hold time of in f/f Do (t_{h_Do}) must be

$$t_{cq_Di} + TL > t_{h_Do} \quad (2)$$

What if the previous data's hold time is t_{h_Do} ?

If data hold time check is violated, data intended to be captured at the next edge will get captured at the same edge.

A problem: Find the minimum and maximum number from a set of numbers



Find clock period (T)

Time (Input_Reg to Output_Reg)

Time (Reg. PC/I to MaxReg)
Time (Reg. PC/i to MinReg)
Time (Reg. limit to limit)?

?

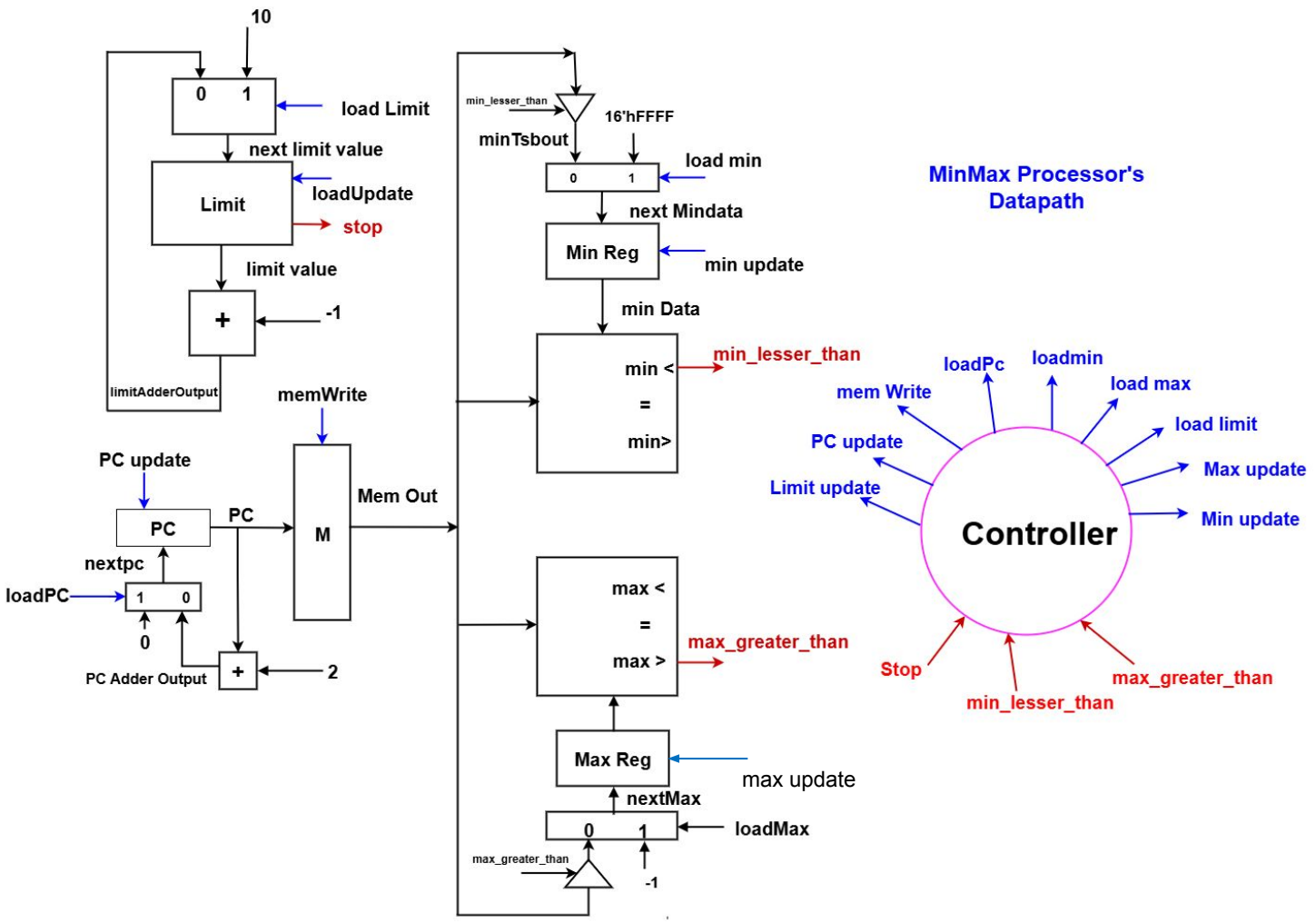
Find maximum Time from all possible Time values

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Can we execute other algorithms on this processor?

Special-purpose/Dedicated Processor
MinMax Processor

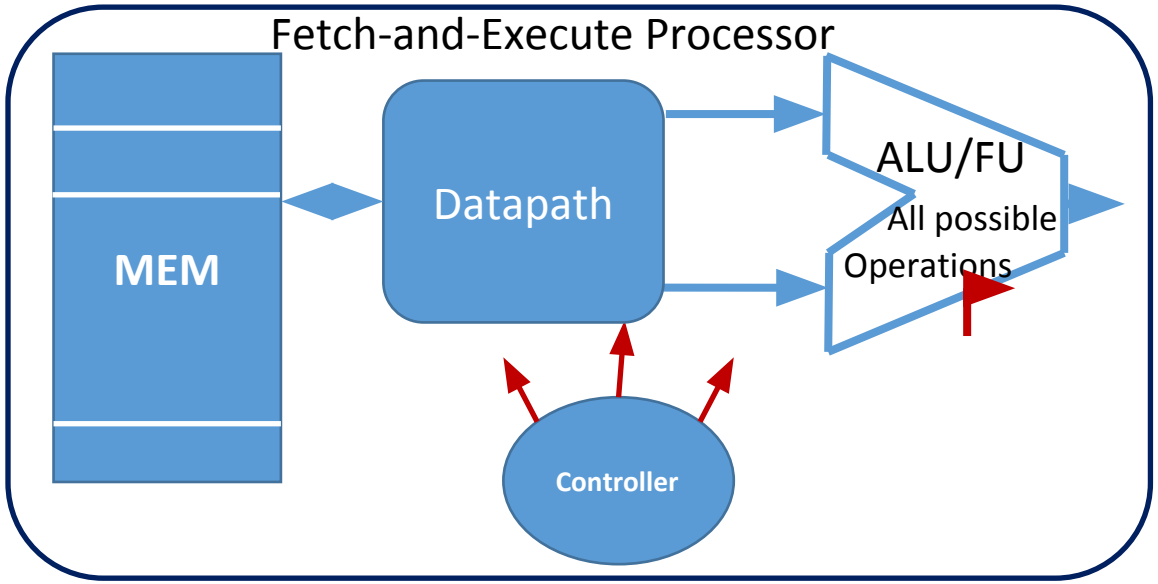
Input: set of n nos stored in A
Output: Min & Max
Min = ∞
Max = - ∞
Do scan i-th no.
    If Max < A [i] then
        Max = A [i]
    If Min > A [i] then
        Min = A [i]
Until i reaches to n
Stop



MinMax Processor

Necessity of General-purpose processor

- Is there an Algorithm which will execute or simulate other Algorithms?
  - The processor executes any algorithms
  - Programmable
  - Turing Model
  - Is there any **limitation** of such an Algorithm?
    - ***Halting problem:*** Can we have an Algorithm which takes other Algorithm as input and decides that whether given input Algorithm will halt/stop or not, **in general**?
    - Consider [*] such Algo. exists **A(P, D)**. Another Algo. **B(X):** loop-forever **if A (X, X)** = "Halt" **else** Halt. Next use **B(B),** it is unable to decide the answer. **A(P, I) doesn't exist.**
    - Used **Self-referential** structure
- What kind of Algorithm do we need for making the processor general purpose or programmable?
  - *Fetch-and-Execute* Algorithm
    - **Stored program (?) [*]**
    - Generalized Datapaths
    - Generalized Functional Unit
  - Proposed by Jhon von Neumann [*]



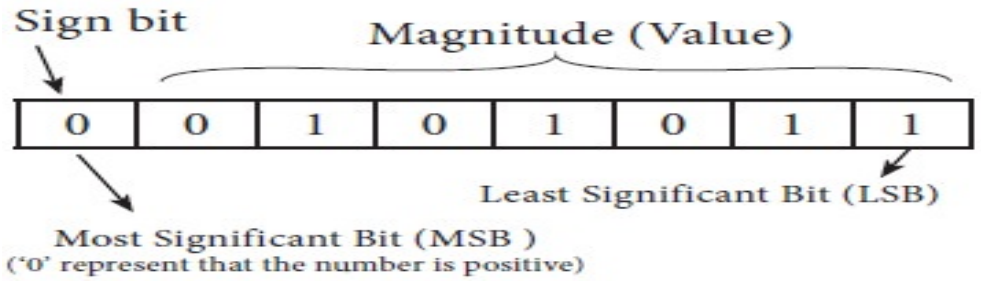
[*] An URL is embedded.

Fetch-and-Execute Algorithm

- What is to be fetched?
  - Program/instructions (Birth of the program or software)
    - Representation of instruction (opcode & operand)
    - Representation of number (sign and magnitude) [parallel reasoning]
- Where are the instructions coming from?
  - It's stored in storage/memory
- How do the instructions execute?
  - What kind of operation is it?
    - Decode the instruction
  - Execute the instruction

Fetch-and-Execute Algorithm

- How does one represent the instruction?
- Is there any similar problem we solved?

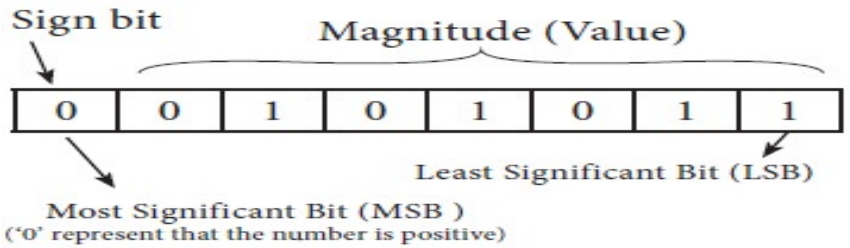


Number representation

- Representation of instruction
  - Instruction format

Fetch-and-Execute Algorithm

Number representation



- Representation of instruction
  - Instruction format

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

		6-bit	5-bit	5-bit	5-bit	5-bit	6-bit	
Instruction	**Format**	**OP**	**RS**	**RT**	**RD**	**Shamt**	**Funct**	**Address**
add	R	0	reg	reg	reg	0	32	NA
sub	R	0	reg	reg	reg	0	34	NA
addi	I	8	reg	reg	NA	NA	NA	Constant
lw	I	35	reg	reg	NA	NA	NA	Offset
Sw	I	43	reg	reg	NA	NA	NA	Offset
bne	I	5	reg	reg	NA	NA	NA	Offset

Microprocessor without **I**nterlocked **P**ipelined **S**tages

Fetch-and-Execute Algorithm

Inst. opcode	Opcode
LW	100010
SW	100011
BEQ	000100
R-type	000000
addi	001000
j	000010

Fetch-and-Execute Algorithm

- Bit wise and (&) and shift operations (<< and >>)

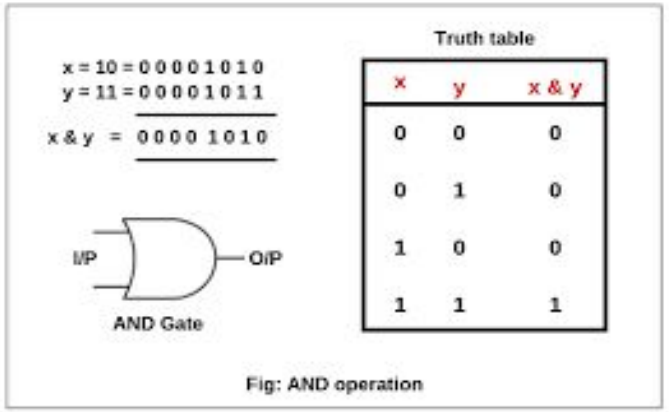
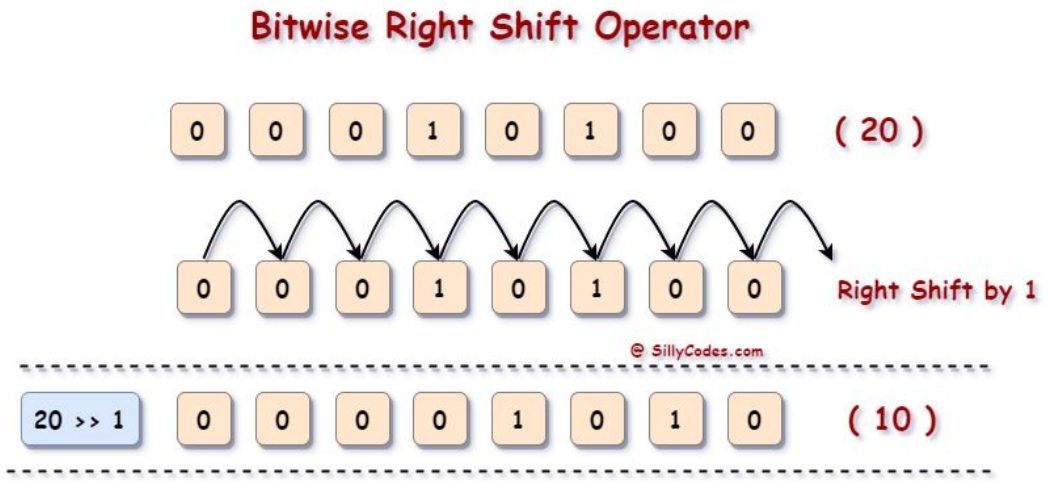


Fig: AND operation

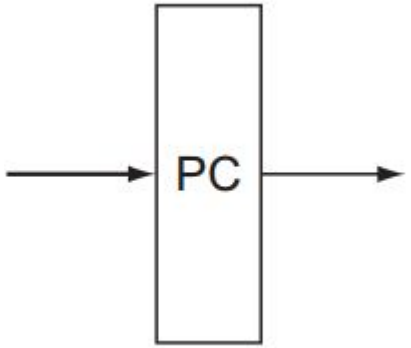
BITWISE OPERATORS

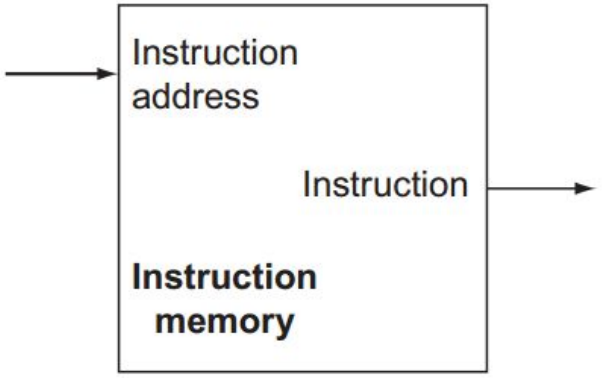
<< Shift Left

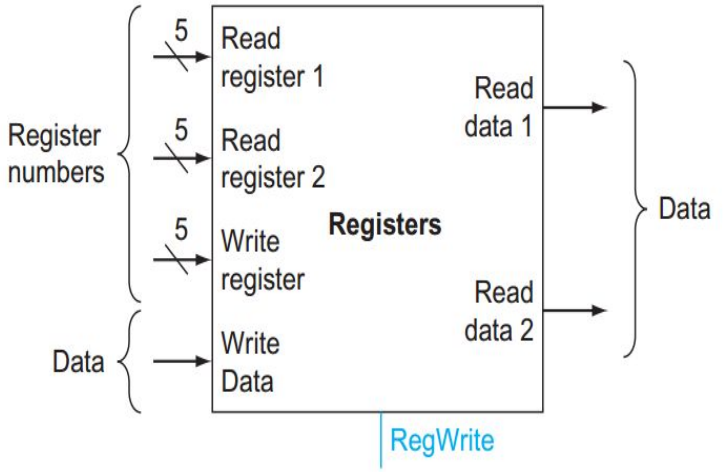
SYNTAX	BINARY FORM	VALUE
x = 7;	00000111	7
x=x<<1;	00001110	14
x=x<<3;	01110000	112
x=x<<2;	11000000	192

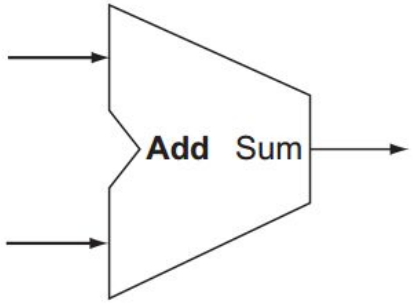


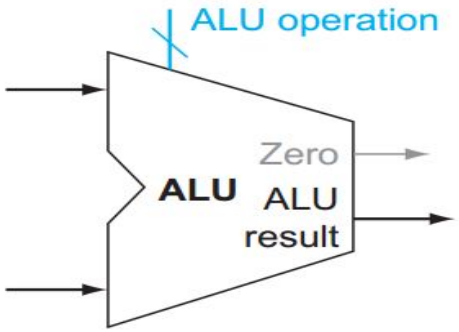
MIPS Processor/Generalized components

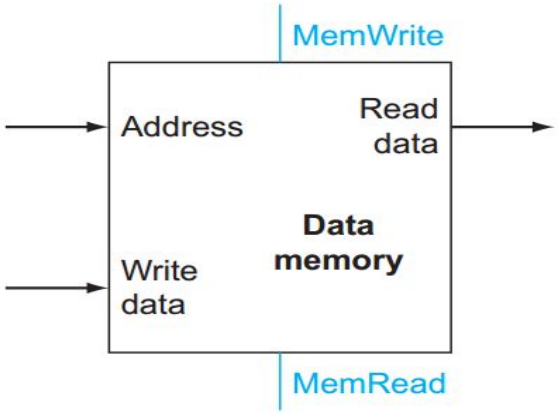


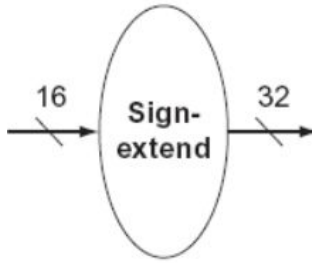












Fetch-and-Execute Algorithm


```
#define OPCODE 0b11111100000000000000000000000000
#define RS     0b00000011111000000000000000000000
#define RT     0b00000000000111110000000000000000
#define DST    0b00000000000000001111100000000000 //RD
#define OFFSET 0b00000000000000000111111111111111

int PC, IMM[1024], DMM[1024], RF[32];
Load(IMM, DMM);
Set PC with address 1st instruction which is stored in IMM;
while (1){
      switch((IMM[PC] & OPCODE) >>26){
          case R-type:
  RF[(IMM[PC] & DST) >> 11] = ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]); PC = PC + 4; break;
          case SW:
      DMM[ALU((IMM[PC] & RS)>>21, (IMM[PC] & OFFSET) ] = RF[(IMM[PC] & RT)>>16]; PC = PC + 4; break;
          case LW:
      RF[(IMM[PC] & RT) >>16]= DMM[ALU((IMM[PC] & RS) >>21, IMM[PC] & OFFSET) ]; PC = PC + 4; break;
          case BNE:
       IF (ZERO) PC = (PC + 4) + ((IMM[PC] & OFFSET) <<2); ELSE PC = PC + 4; break;
      }
}
```

Let's define ALU()

Can we have an Architecture for this Algorithm?

Fetch-and-Execute Algorithm for MIPS Microprocessor (32 bit)


```
#define OPCODE 0b11111100000000000000000000000000
#define RS     0b00000011111000000000000000000000
#define RT     0b00000000000111110000000000000000
#define RD     0b00000000000000001111100000000000
#define SHIFT  0b00000000000000000000011111000000
#define OFFSET 0b00000000000000001111111111111111

int PC, IMM[1024], DMM[1024], RF[32], ALUControl; bool ZERO;
Load(IMM, DMM);
Set PC with address 1st instruction which is stored in IMM;

  while (1){
    switch((IMM[PC] & OPCODE) >>26){
      case R-type:
            Set ALUControl; //ADD, SUB, AND, OR, etc
            RF[IMM[PC] & DST] = ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]); PC = PC + 4;
      case SW:
            Set ALUControl = 0b0010;
            DMM[ALU((IMM[PC] & RS)>>21, (IMM[PC] & OFFSET) ] = RF[(IMM[PC] & RT)>>16]; PC = PC + 4;
       case LW:
            Set ALUControl = 0b0010;
            RF[IMM[PC] & RT]= DMM[ALU((IMM[PC] & RS) >>21, IMM[PC] & OFFSET) ]; PC = PC + 4;
        case BNE:
            Set ALUControl = 0b0110;
            ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]);
            IF (ZERO ==1) PC = (PC + 4) + ((IMM[PC] & OFFSET) <<2); ELSE PC = PC + 4;
    }
}
```

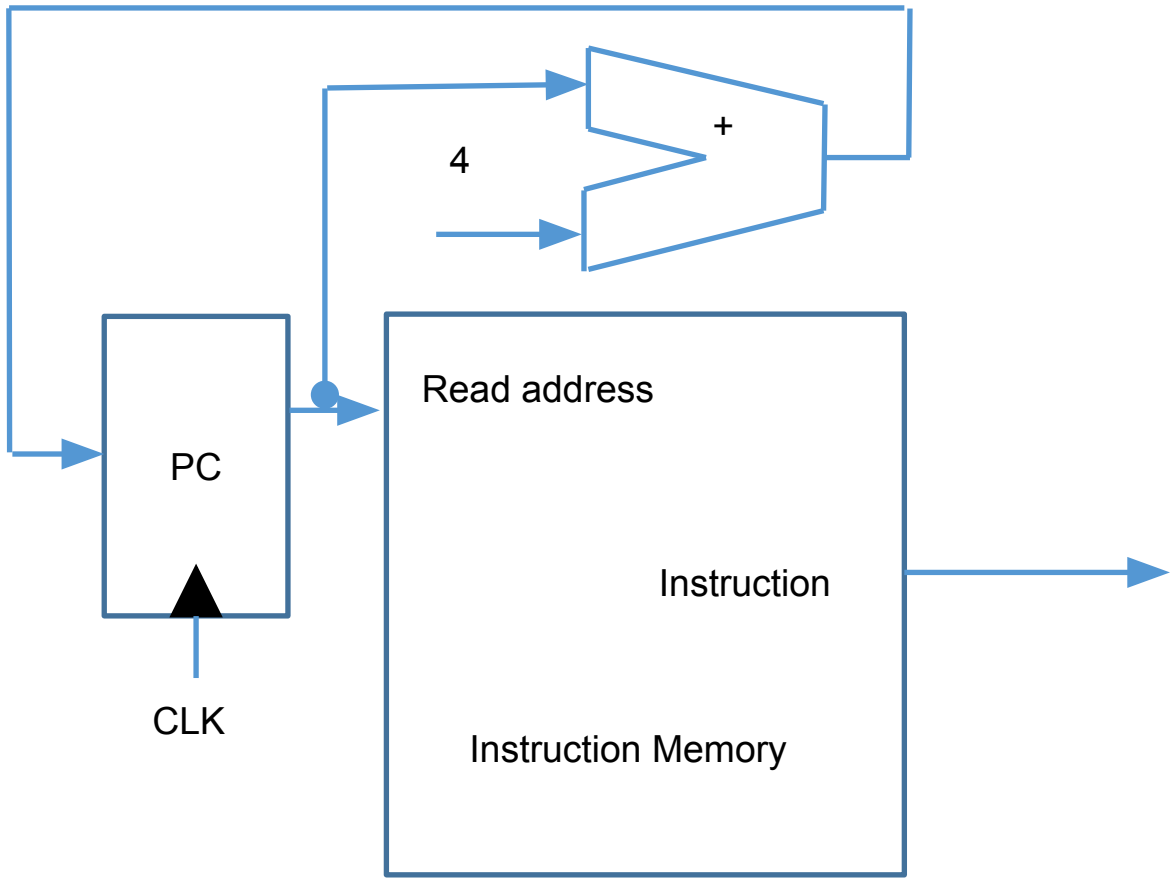
```
ALU(Src1, Src2){
      switch (ALUControl){
          case B-type: ZERO = (Src1- Src2) == 0 ? 1: 0;
                       break;
          case ADD: return (Src1 + Src2); break;
            . . .
      }
}
```

Need conversion for 16 bit offset to 32 bits

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path for **Fetch stage**

Why 4?



Clock Period (T-pc-to-pc)

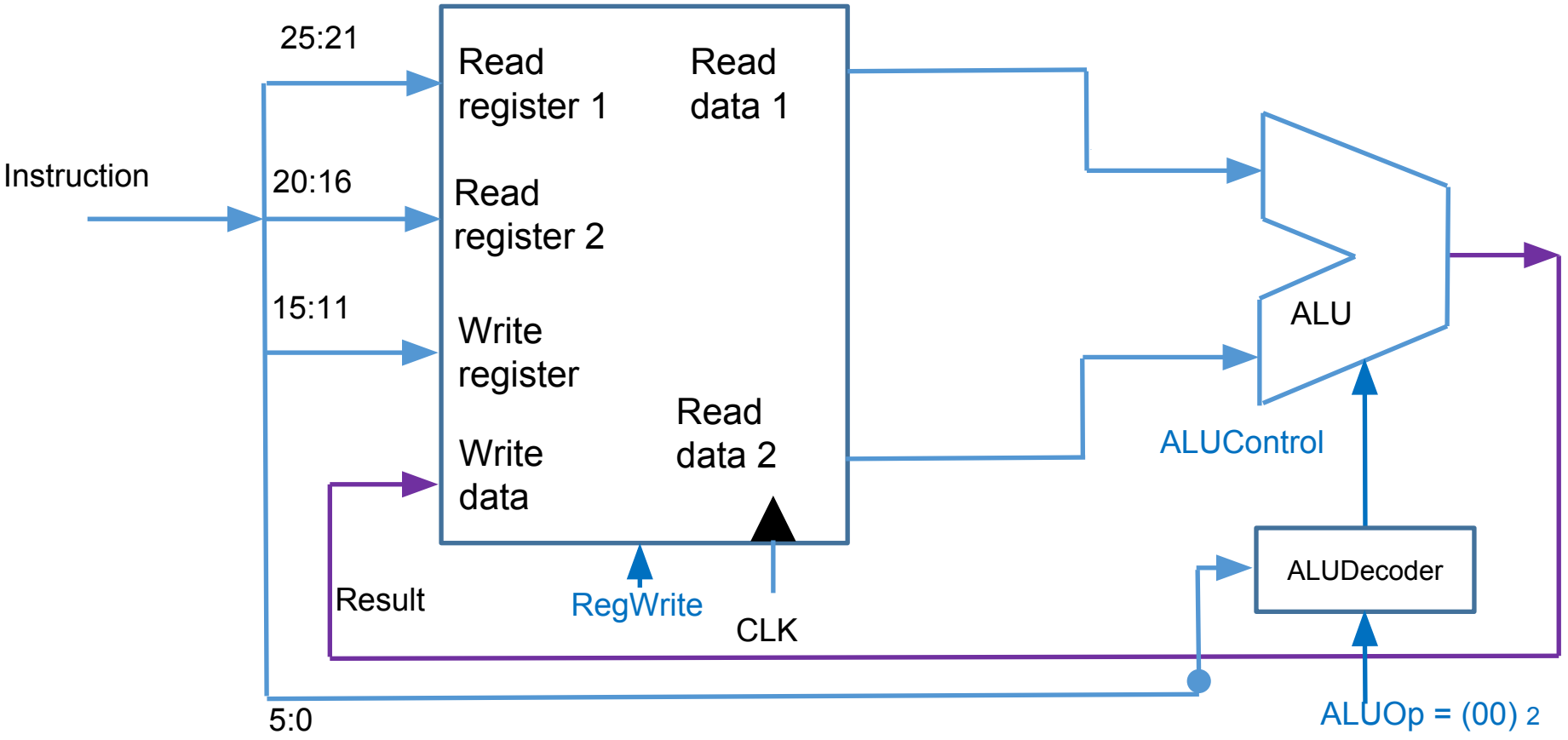
Source Reg is PC
Destination Reg is PC

IMM[PC]; PC = PC + 4

Analysis of data-path for **R-type instruction**

- ADD $S1, $S2, $S3 //$S1 ? $S2 + $S3

op	rs	rt	rd	shamt	funct
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Clock Period (T-pc-to-rf)

Source Reg is PC
Destination Reg is RF

Shamt=5'b0

ALU Slides

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path for **I-type instruction**

- LW $S1, offset[$S2] //$S1 ← DM[offset + $S2]

Is offset a physical address?

No. It is a relative address (here, relative with respective to Reg.)

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

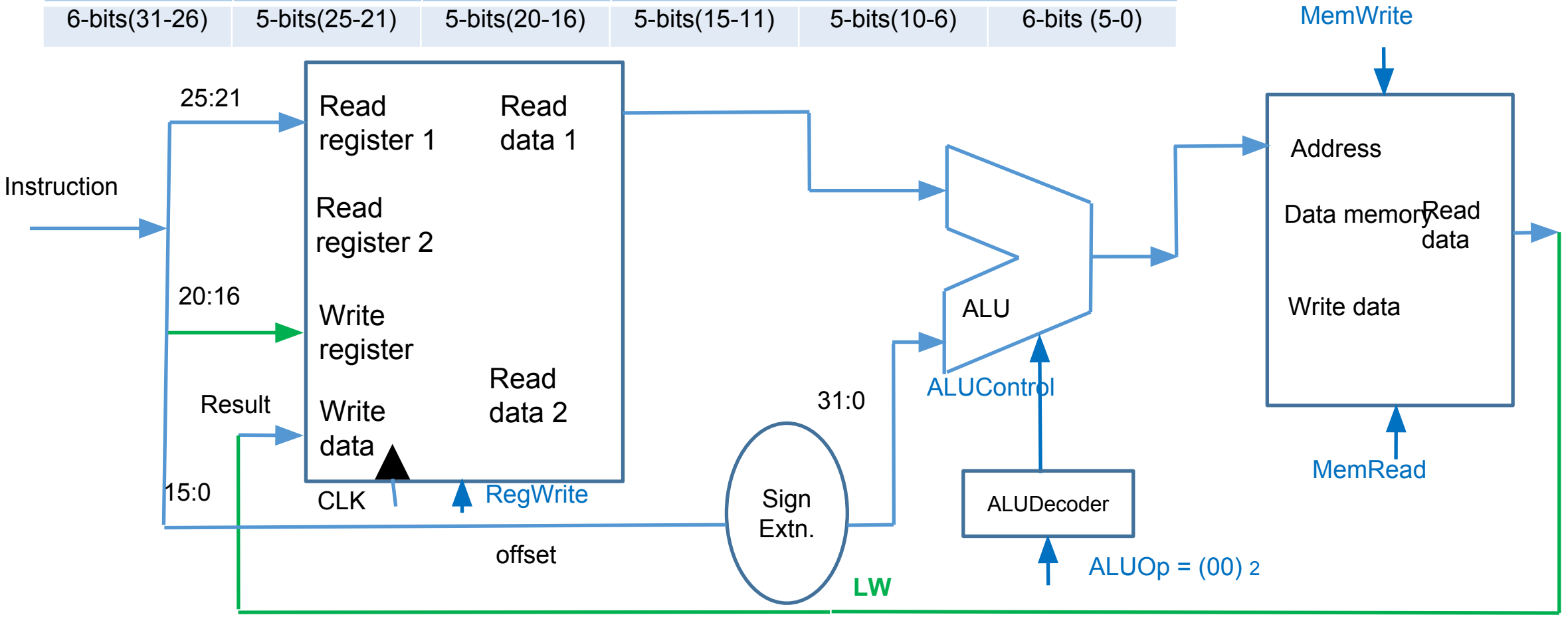
Analysis of data-path for **I-type instruction**

- LW $S1, offset[$S2] //$S1 ← DM[offset + $S2]

Is offset a physical address?

No. It is a relative address (here, relative with respective to Reg.)

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path for **I-type instruction**

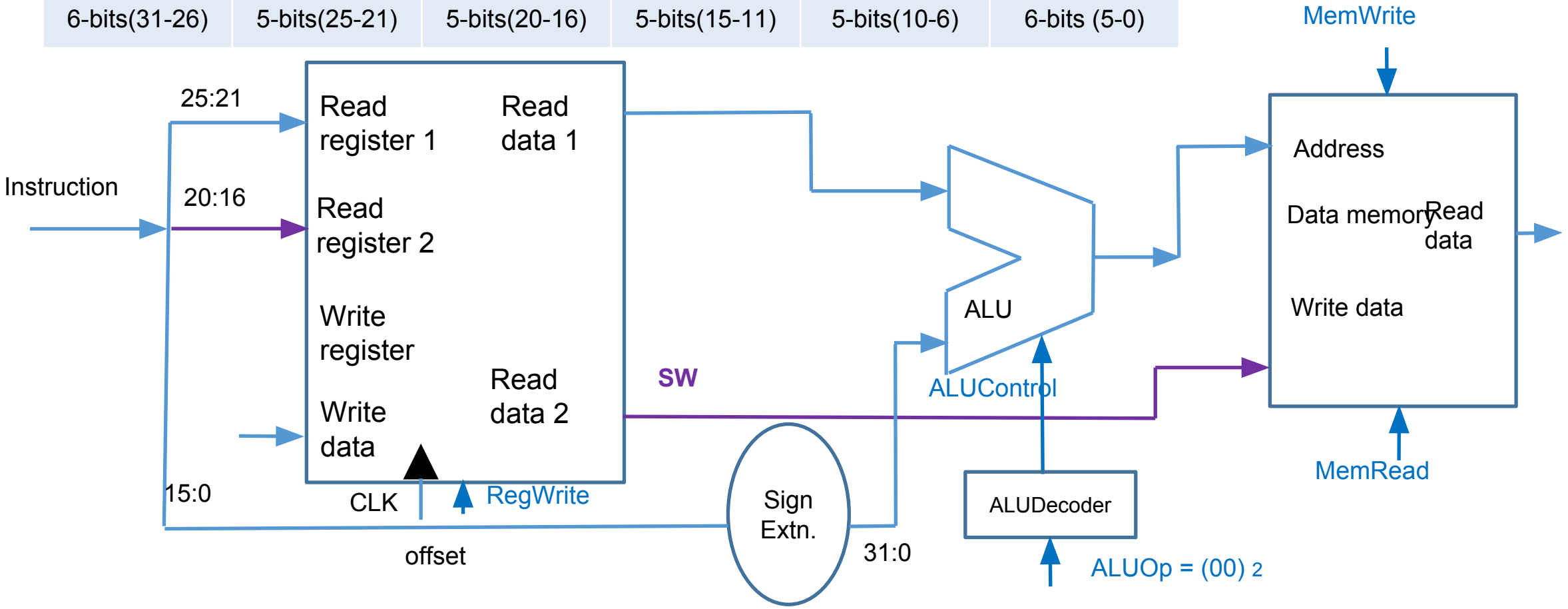
- SW $S1, offset[$S2] //DM[offset + $S2] ⍰ $S1

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path for **I-type instruction**

- SW $S1, offset[$S2] //DM[offset + $S2] ← $S1

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path for **I-type instruction**

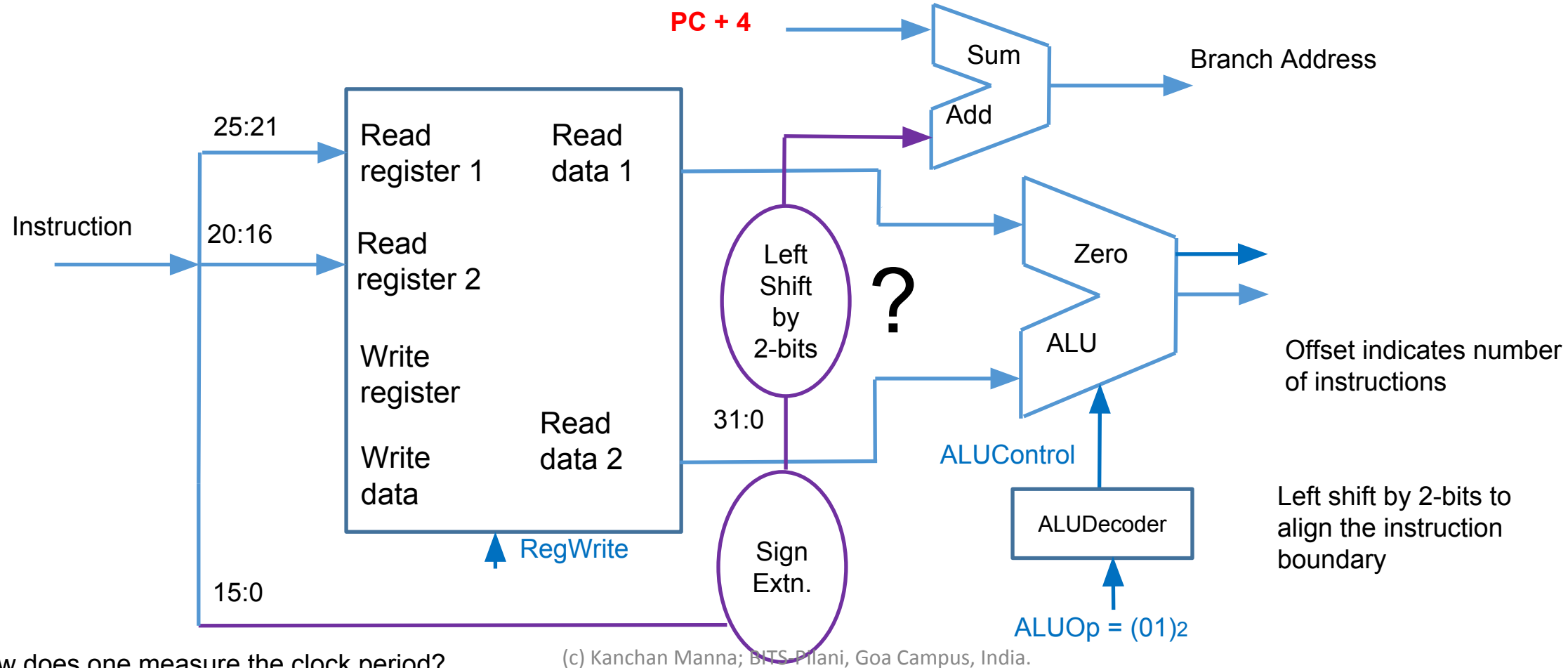
- BEQ $S1, $S2, offset //Jump to the offset no. of instr., when $S1 = $S2
- BNE $S1, $S2, offset //Jump to the offset no. of instr., when $S1 != $S2

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path for **I-type instruction**

- BEQ $S1, $S2, offset //Jump to the offset no. of instr., when $S1 = $S2
- BNE $S1, $S2, offset //Jump to the offset no. of instr., when $S1 != $S2

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path **I-type instruction**

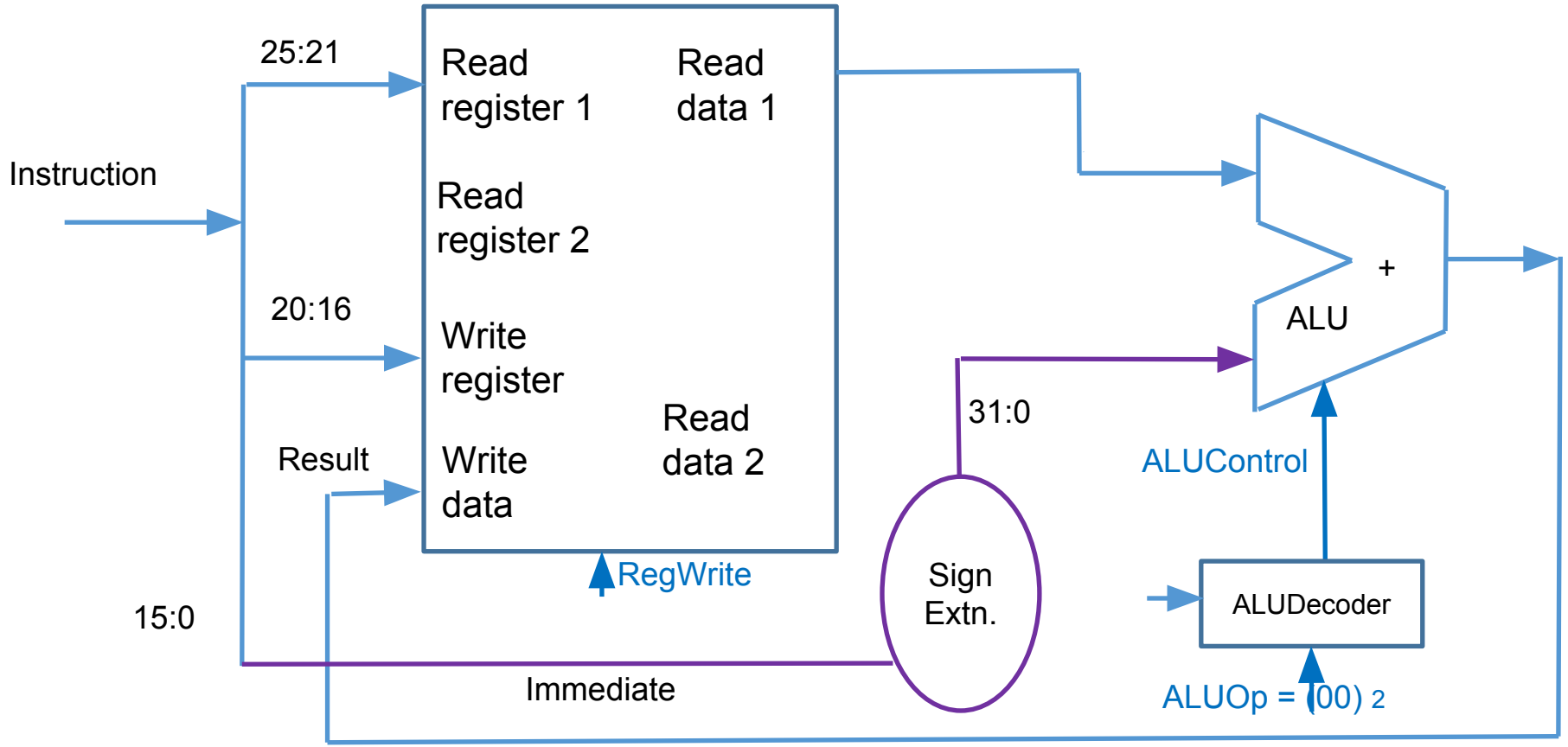
- ADDI $S1, $S2, -12 //$S1 ? $S2 + (-12)

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path **I-type instruction**

- ADDI $S1, $S2, -12 //$S1 ← $S2 + (-12)

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path **j-type instruction**

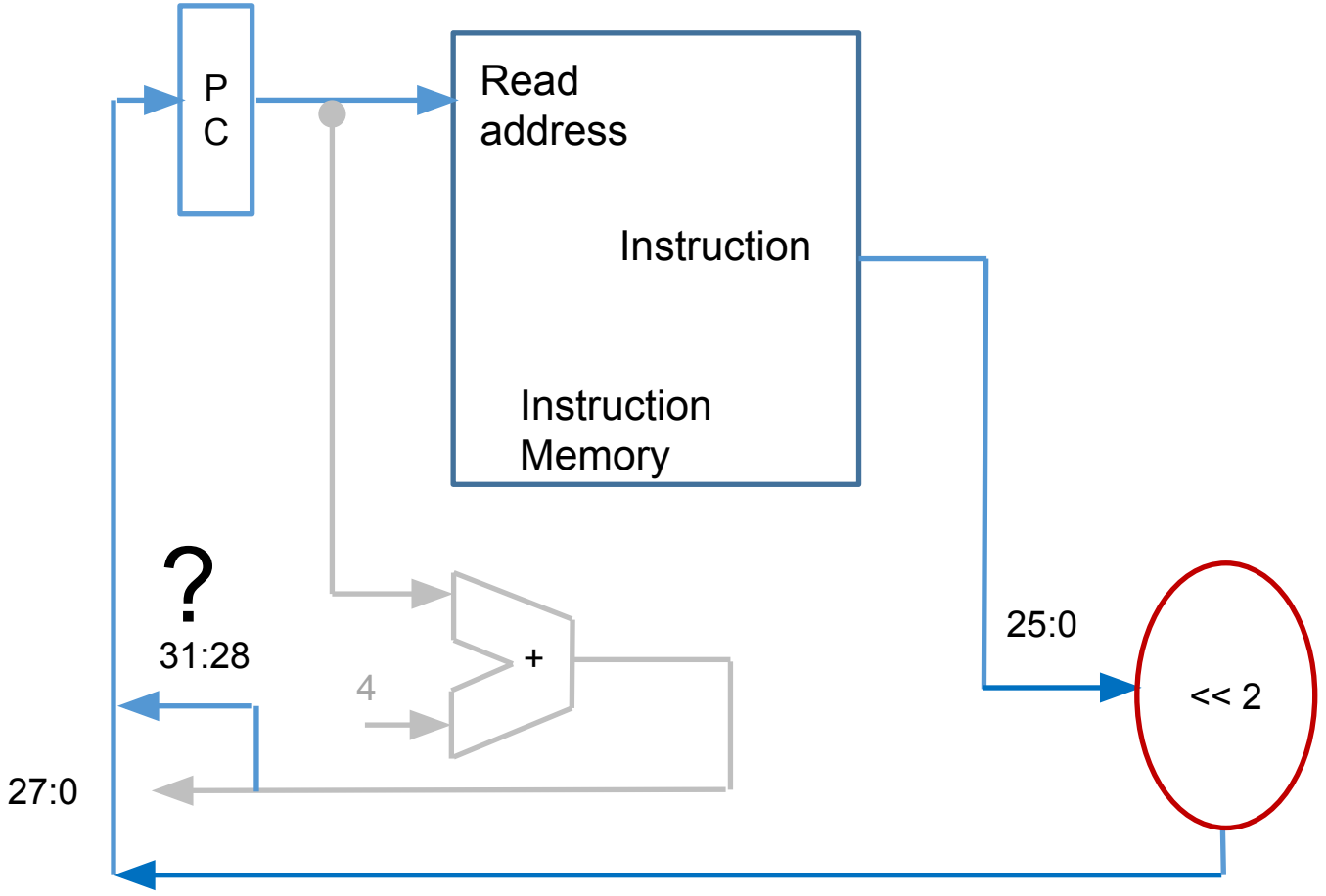
- J addrs //PC PC[31:28]addrs[27:0]

op	address				
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path **j-type instruction**

- J addrs //PC ? PC[31:28]addrs[27:0]

op	address				
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

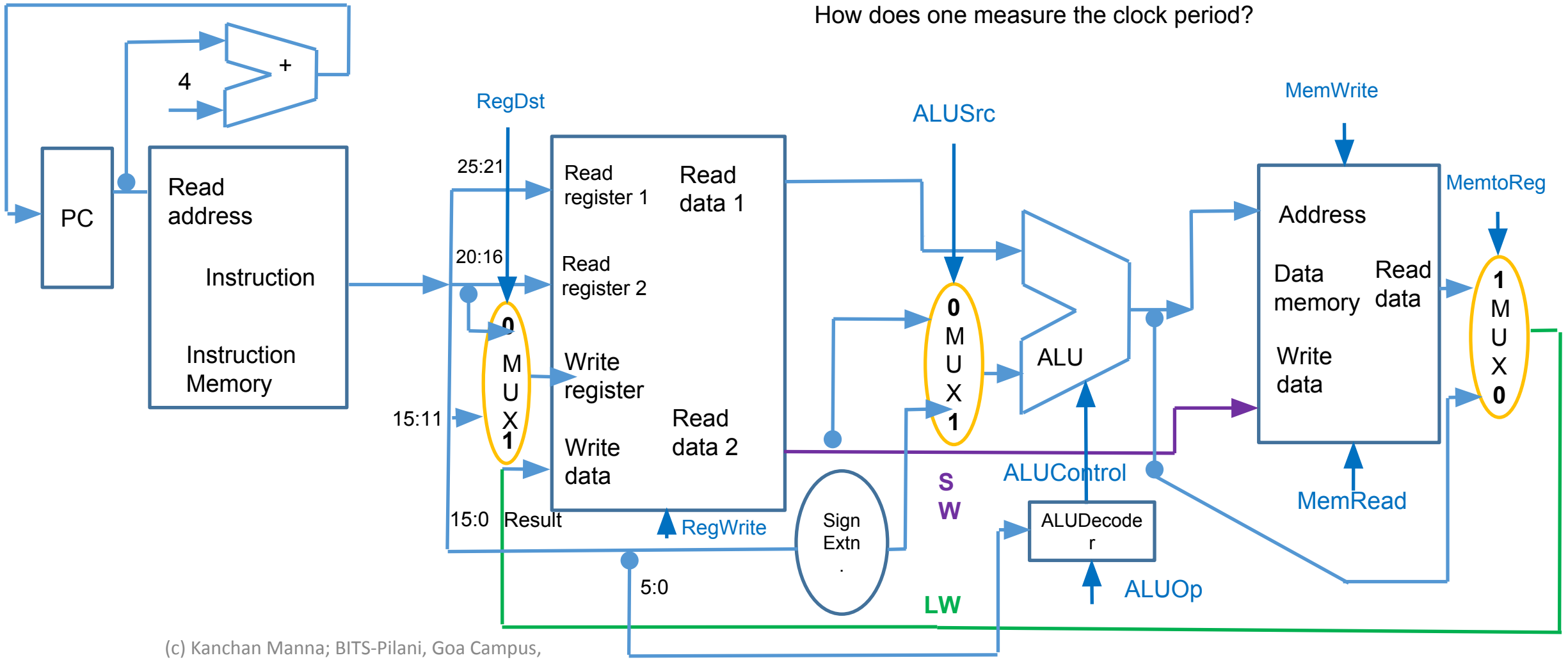
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Building Microprocessor

- Designed the individual datapath for
  - Instruction Fetch
  - R-type instructions
  - I-type instructions
  - J-type instructions
- How does one build the microprocessor with these instructions?
  - Merge the datapath for all the instructions type, including the fetch, How?
  - Using MUX

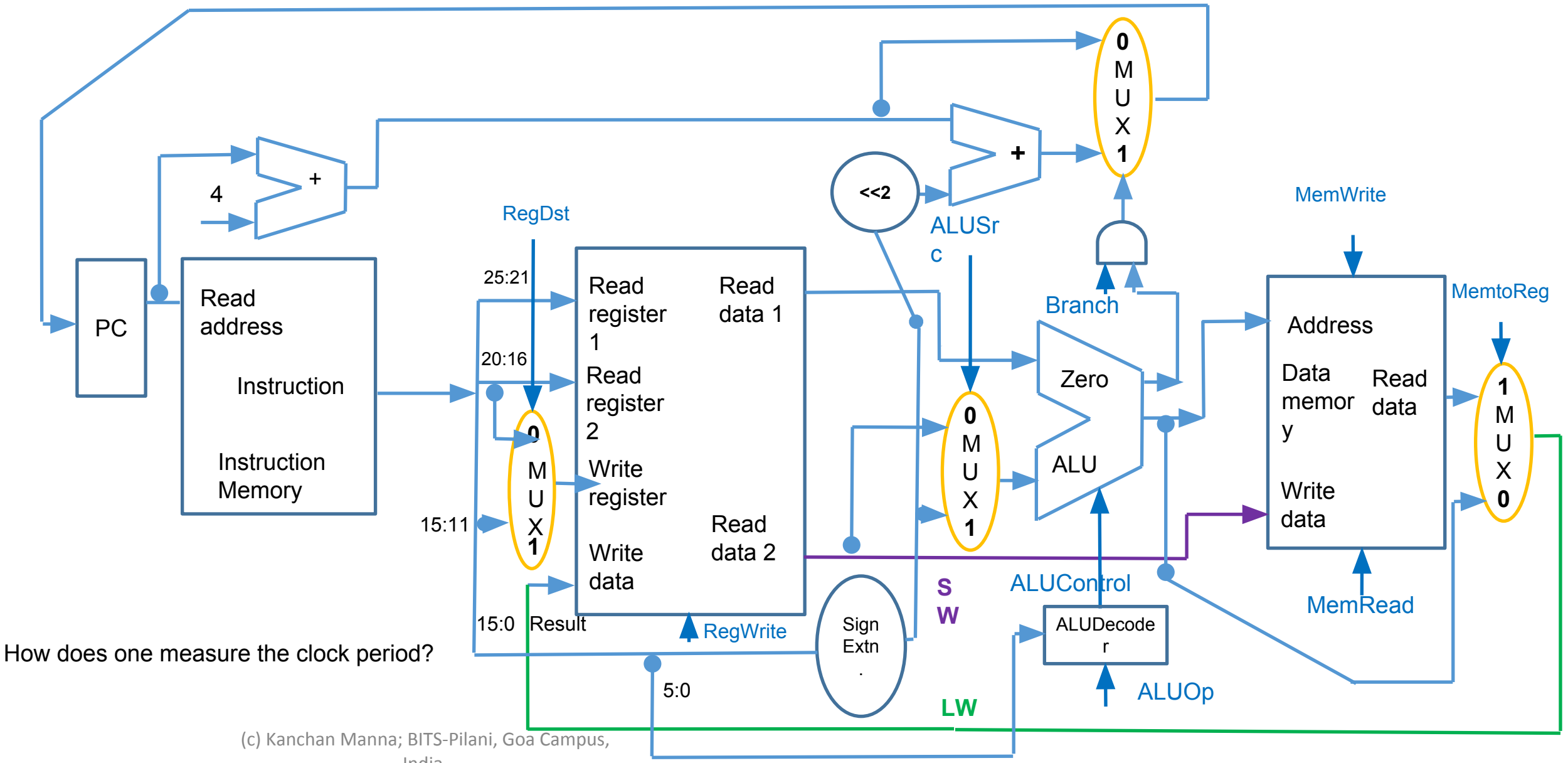
Combined Fetch cycle, R, and I-type data-path

- For ALU and write register, source of data, for the input, is more than one
- Insert MUX before such input signal and control the inputs through MUX-select line



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

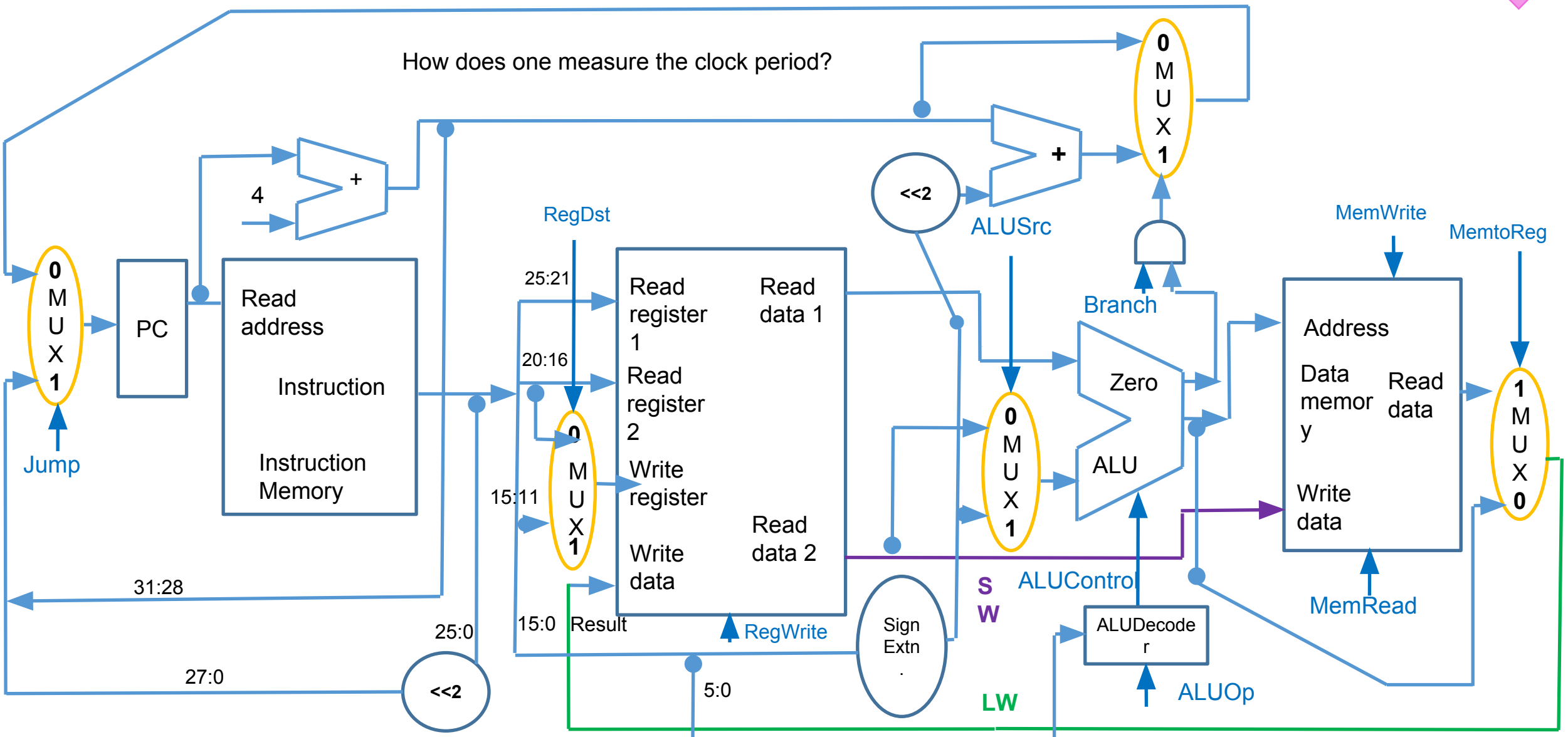
Combined Fetch cycle, R, I and Branch instr data-path



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Combined Fetch cycle, R, I and J-type data-path



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Identify the control signals

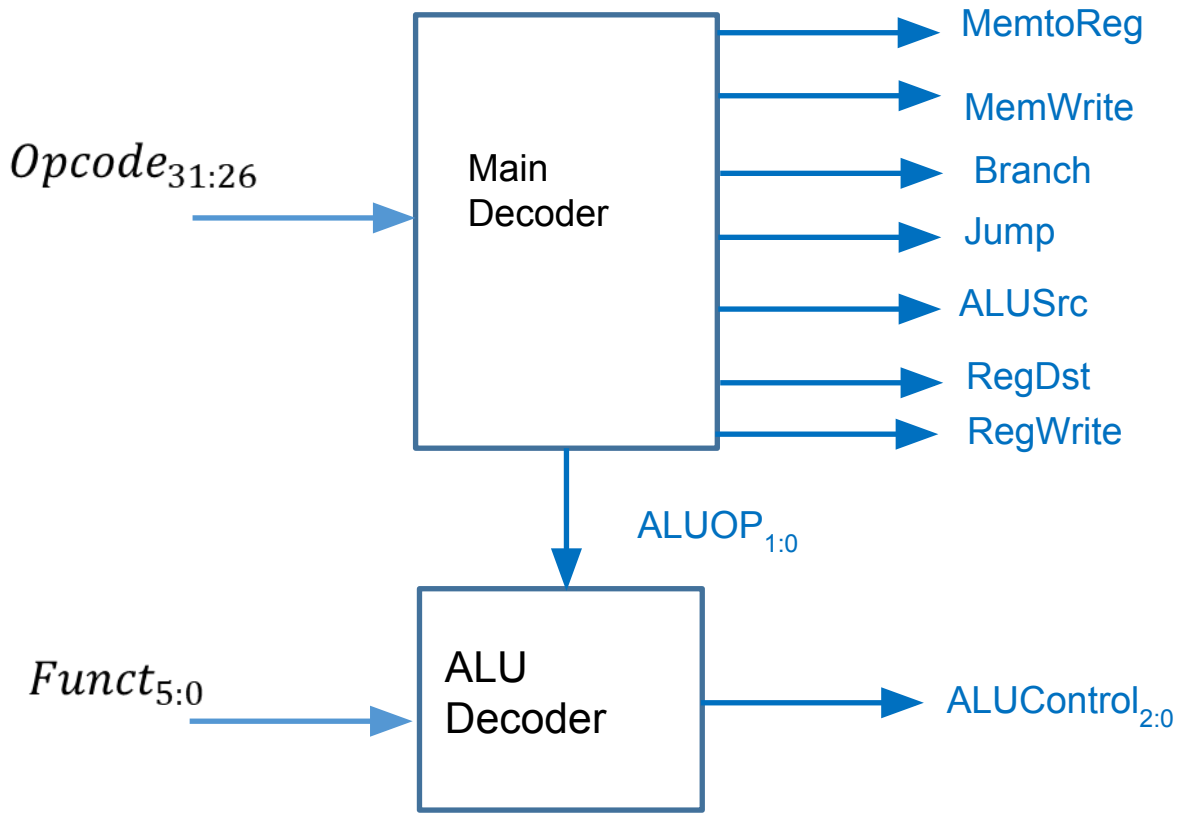
- Jump
- RegDst
- RegWrite
- ALUSrc
- Branch
- ALUOp
- MemRead
- MemWrite
- MemtoReg

ALUOp	Meaning
00	add
01	subtract
10	Look at *funct* field
11	n/a

How can we design a Controller?

Control Unit

Inputs



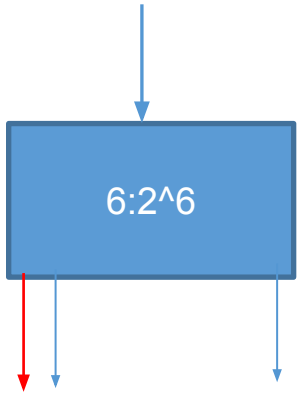
Outputs

ALUOp	Meaning
00	add
01	subtract
10	Look at *funct* field
11	n/a

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Generation of Controls: **Main decoder** truth table

op-code part [31:26]



Inputs to the control unit: op-code part [31:26] and funct part [5:0] of the instruction

Microprogrammed CU

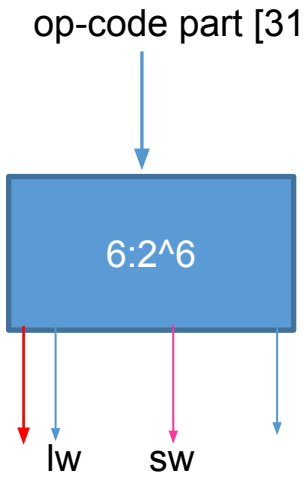
ALUOp	Meaning
00	add
01	subtract
10	Look at *funct* field
11	n/a

Output of the control unit:

Instr. (Input)	Jump	RegDst	RegWrite	ALUSrc	Branch	ALUOp1	ALUOp0	MemRead	MemWrite	MemtoReg
R-type	0	1	1	0	0	1	0	0	0	0
lw	0	0	1	1	0	0	0	1	0	1
sw	0	x	0	1	0	0	0	0	1	x
addi	0	0	1	1	0	0	0	0	0	0
B-type	0	x	0	0	1	0	1	0	0	x
J-type	1	x	0	x	x	x	x	0	0	x

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Generation of Controls: **Main decoder** truth table



Inputs to the control unit: op-code part [31:26] and funct part [5:0] of the instruction

Consider ALUSrc

What could be the C-expression?

```
If ( (lw) || (sw) || (addi) )

    ALUSrc = 1;
Else
    ALUSrc = 0;
```

How can we write it using Logical expression?

ALUSrc = (lw) + (sw) + (addi)

Output of the control unit:

Instr. (Input)	Jump	RegDst	RegWrite	ALUSrc
R-type	0	1	1	0
lw	0	0	1	1
sw	0	x	0	1
addi	0	0	1	1
B-type	0	x	0	0
J-type	1	x	0	x

Hardwired CU

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

ALU Operations

ALU control lines (3:0)	ALU Functions
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	set on less than

Generation of Control: **ALU decoder** truth table

ALUOp	ALU control lines (2:0)	Funct
00	010 (add)	X
01	110 (subtract)	X
1X	010 (add)	100000 (add)
1X	110 (subtract)	100010 (sub)
1X	000 (and)	100100 (and)
1X	001 (or)	100101 (or)
1X	111 (set less than)	101010 (slt)

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Generation of Controls

Inst. opcode	ALUOp	Instr. operation	Funct field	Desired ALU action	ALUControl
100010 (LW)	00	load word	xxxxxx	add	0010
100011 (SW)	00	store word	xxxxxx	add	0010
000100 (BEQ)	01	branch equal	xxxxxx	subtract	0110
000000 (R-type)	10	add	100000	add	0010
R-type	10	Subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111
001000 (addi)	00	Immediate	xxxxxx	add	xxxx
000010 (j)	xx	jump	xxxxxx	jump	xxxx

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Performance analysis implementation

- Execution time of a program is a metric
- $Execution\ Time = (\#instructions)\left(\frac{Cycles}{instruction}\right)\left(\frac{Seconds}{cycle}\right)$
- #instructions or length of a program depends on ISA
- Complicated Vs. Simple ISA
- The number of cycle per instruction (on an average) is called *CPI*
- Throughput = 1/CPI
- Assumption: an ideal memory model
- The number of seconds per cycle is the clock period (?)

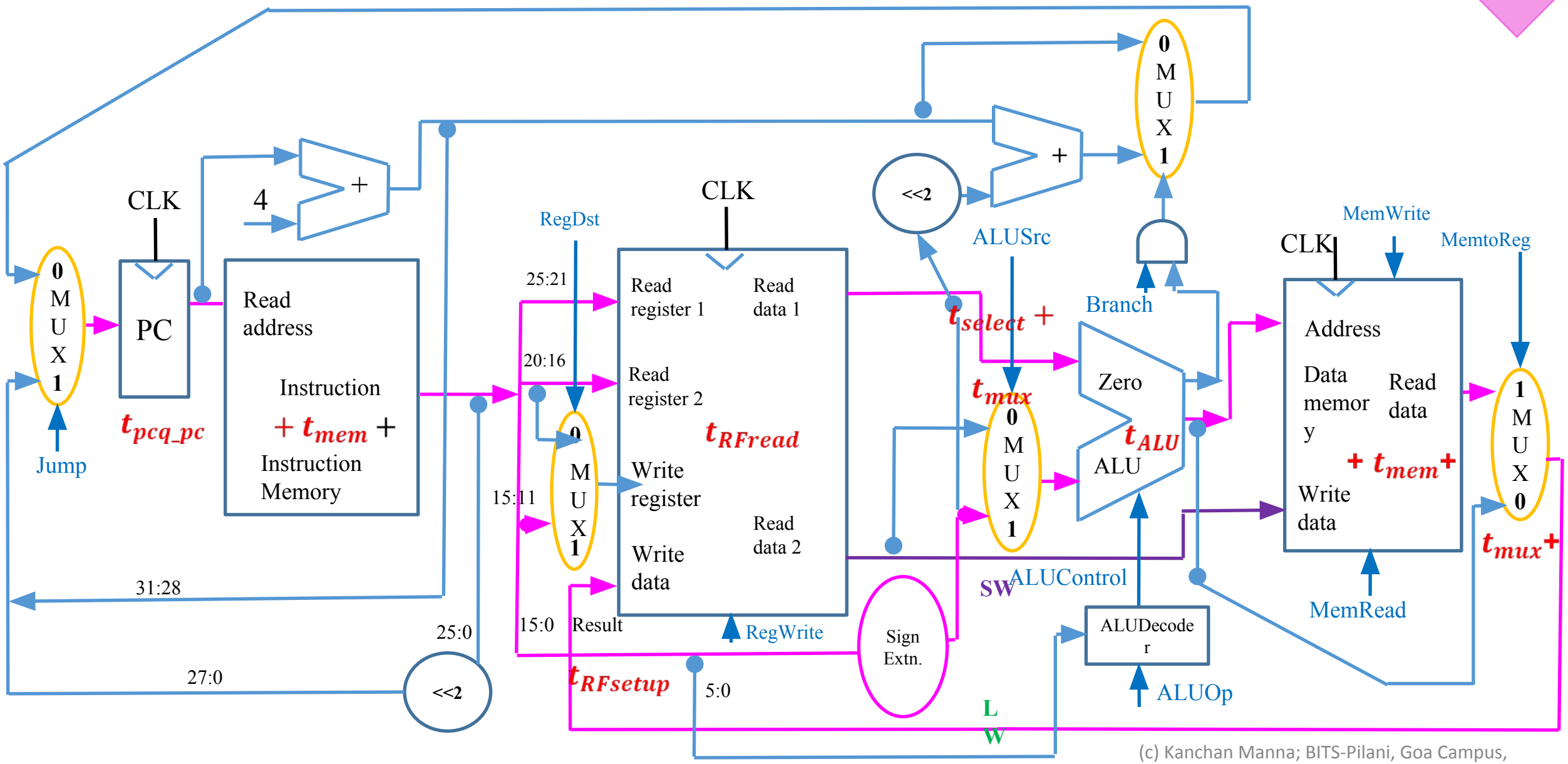
Performance analysis of implementation

Instr.	Resource Usage										
	PC	IMM	ADD_PC	RF	EXTN	ALU	<<2	ADD_B	DMM	RF (Write)	Total Resource Usage
ADD	√	√	√	√	×	√	×	×	×	√	6
BNE	√	√	√	√	√	√	√	√	×	×	8
J	√	√	√	×	×	×	×	×	×	×	3
SW	√	√	√	√	√	√	×	×	√	×	7
LW	√	√	√	√	√	√	×	×	√	√	8
ADDI	√	√	√	√	√	√	×	×	×	√	7

What could be maximum time to update the PC? Because updated PC will bring the next instruction in the datapath

Which of the instruction using maximum resource? Next consider the time taken by each resource to compute the functionality

Consider the pink lines for LW's path



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Performance analysis of Single-cycle implementation

- CPI = 1, for single-cycle implementation
- lw-instruction decides Critical path (T_c)
  - $T_c = t_{pcq_PC} + t_{mem} + \max\{t_{RFread}, t_{select} + t_{mux}\} + t_{ALU} + t_{mem} + t_{mux} + t_{RFwrite}$
- Register read takes longer time than mux selection
- $T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFwrite}$

Single-cycle implementation

- The previous design is called single-cycle implementation
- The instruction memory, register file and data memory are all **read combinationally**
  - What does it mean?
  - The new instruction appears to output of instruction memory after some propagation delay, if the address changes
- Operations are done on rising edge of the clock
- The single-cycle microarchitecture executes an entire instruction in one clock cycle
- Simple control unit (why?)
  - No next state is associated with it
  - Every operation is done in a clock cycle

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Performance analysis of Single-cycle implementation

- Need some quantity (or metric) for comparison of two design
- How does one measure the effectiveness of new design?

Performance analysis of Single-cycle implementation

- How to calculate Delay
- Delay: time between applying the Input and producing the output, another way to say time between two inputs, i.e., when we can update the PC
  - Input is reading an instruction from Memory
  - Output is producing the result by the read instruction
  - Next input is available when we update the PC by (PC + 4) or Branch address

Performance analysis of Single-cycle implementation

- XYZ-organization is going to build the single-cycle MIPS processor in a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compute the execution time for a program with 100 billion instructions.

Parameter	Delay (ps)
	30
	250
	20
	200
	25
	20

Performance analysis of Single-Cycle implementation

- XYZ-organization is contemplating building the single-cycle MIPS processor in a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compute the execution time for a program with 100 billion instructions.

Parameter	Delay (ps)
	30
	250
	150
	200
	25
	20

- $T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFSetup}$

  = 30 + 2(250) + 150 + 200 + 25 + 20 = **925 ps**

- The total execution time =

  (100 * 10^9 instrs.) * (1 cycle/instrs.) * (925 * 10^{-12} s/cycle)

  = 92.5 seconds

Other Benchmark: Standard Performance Evaluation Corporation (SPEC)

Princeton Application Repository for Shared-Memory Computers (PARSEC)

Stanford Parallel Applications for Shared-Memory (SPLASH-2) programs.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

A few more MIPS instructions

MIPS ISA & Load Byte (lb)

lui	rt, imm	Load Upper Imm.	rt = imm << 16
lb	rt, imm(rs)	Load Byte	rt = SignExt(M_1[rs + $imm_{\pm}$])
lbu	rt, imm(rs)	Load Byte Unsigned	rt = M_1[rs + $imm_{\pm}$] & 0xFF
lh	rt, imm(rs)	Load Half	rt = SignExt(M_2[rs + $imm_{\pm}$])
lhu	rt, imm(rs)	Load Half Unsigned	rt = M_2[rs + $imm_{\pm}$] & 0xFFFF
lw	rt, imm(rs)	Load Word	rt = M_4[rs + $imm_{\pm}$]
sb	rt, imm(rs)	Store Byte	M_1[rs + $imm_{\pm}$] = rt
sh	rt, imm(rs)	Store Half	M_2[rs + $imm_{\pm}$] = rt
sw	rt, imm(rs)	Store Word	M_4[rs + $imm_{\pm}$] = rt

MIPS ISA can be found:

https://www.scribd.com/document/358717972/mips-ref-pdf

MIPS ISA & Load Byte (lb)

lui	rt, imm	Load Upper Imm.	rt = imm << 16
lb	rt, imm(rs)	Load Byte	rt = SignExt(M_1[rs + $imm_{\pm}$])
lbu	rt, imm(rs)	Load Byte Unsigned	rt = M_1[rs + $imm_{\pm}$] & 0xFF
lh	rt, imm(rs)	Load Half	rt = SignExt(M_2[rs + $imm_{\pm}$])
lhu	rt, imm(rs)	Load Half Unsigned	rt = M_2[rs + $imm_{\pm}$] & 0xFFFF
lw	rt, imm(rs)	Load Word	rt = M_4[rs + $imm_{\pm}$]
sb	rt, imm(rs)	Store Byte	M_1[rs + $imm_{\pm}$] = rt
sh	rt, imm(rs)	Store Half	M_2[rs + $imm_{\pm}$] = rt
sw	rt, imm(rs)	Store Word	M_4[rs + $imm_{\pm}$] = rt

MIPS ISA can be found:

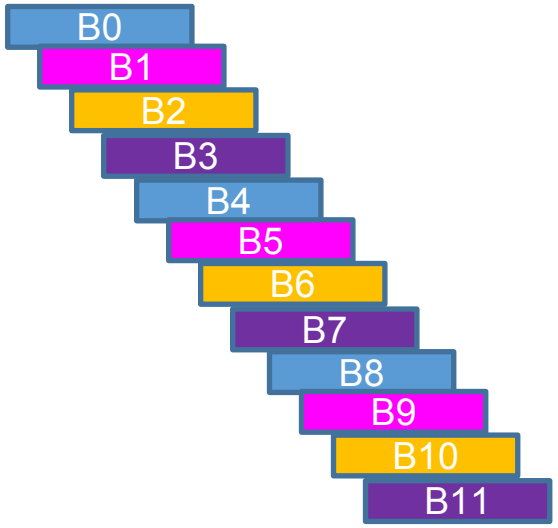
https://www.scribd.com/document/358717972/mips-ref-pdf

MIPS Reference Card (yumpu.com)

MIPS ISA & Load Byte (lb)

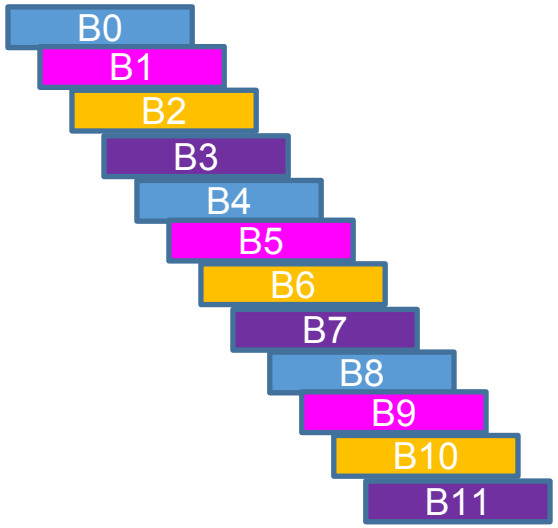
Name	Instr	Type	Operation
Load Byte	lb rt, imm(rs)	I	RF[rt] = Signextn(M[RF[rs]+signextn(Imm)](7:0)]
Load Byte Unsigned	lbu rt, imm(rs)	I	RF[rt] = {24'b0, M[RF[rs]+signextn(Imm)](7:0)]
Load Half-word	lh rt, imm(rs)	I	RF[rt] = Signextn(M[RF[rs]+signextn(Imm)](15:0)]
Load Half-word Unsigned	lbu rt, imm(rs)	I	RF[rt] = {16'b0, M[RF[rs]+signextn(Imm)](15:0)]

MIPS ISA & Load Byte (lb)



MIPS ISA & Store Byte (lb)

Name	instrn	Type	Operation
Store Byte	sb rt, imm(rs)	I	M[RF[rs]+signextn(Imm)](7:0) = RF[rt](7:0)
Store Half-word	sh rt, imm(rs)	I	M[RF[rs]+signextn(Imm)](15:0) = RF[rt](15:0)



Datapath and control?

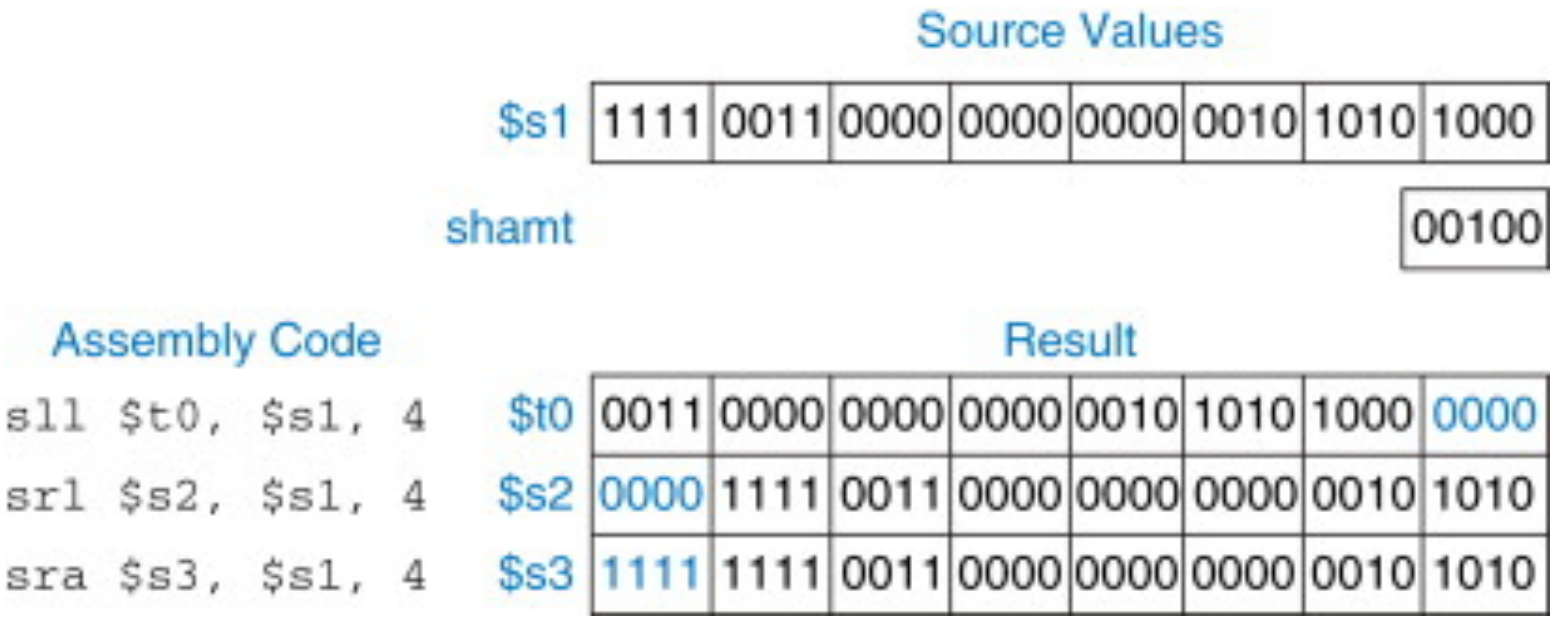
Shift instructions in MIPS

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
sll $t0, $s1, 4	0	0	17	8	4	0	000000	00000	10001	01000	00100	000000	(0x00114100)
srl $s2, $s1, 4	0	0	17	18	4	2	000000	00000	10001	10010	00100	000010	(0x00119102)
sra $s3, $s1, 4	0	0	17	19	4	3	000000	00000	10001	10011	00100	000011	(0x00119903)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

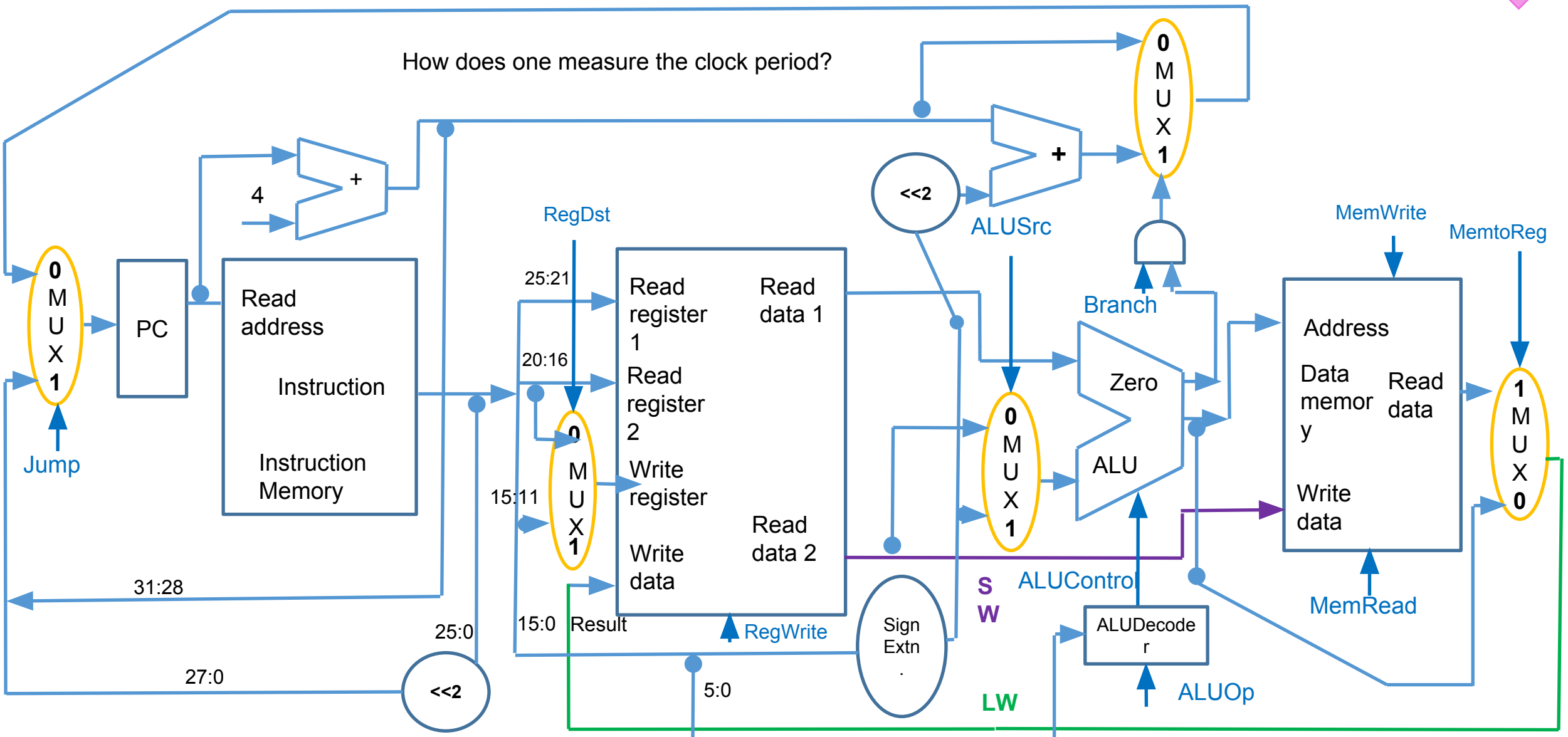
Shift instructions in MIPS

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
sllv $s3, $s1, $s2	0	18	17	19	0	4	000000	10010	10001	10011	00000	000100	(0x02519804)
srlv $s4, $s1, $s2	0	18	17	20	0	6	000000	10010	10001	10100	00000	000110	(0x0251A006)
srav $s5, $s1, $s2	0	18	17	21	0	7	000000	10010	10001	10101	00000	000111	(0x0251A807)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Shift instructions in MIPS



Combined Fetch cycle, R, I and J-type data-path



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Find the minimum and maximum number from a set of numbers

MIPS: Microprocessor without Interlocked Pipelined Stages

```
.data
array:      .word   1, 2, -8, 0, 23, 11, -10
array_size: .word   10
array_min:  .asciiz "\nMin: "
array_max:  .asciiz "\nMax: "
minE:          .word  999
maxE:          .word  -999

.text
main:
    la $a0, array
    lw $a1, array_size
    lw $t2, maxE # max
    lw $t3, minE # min
```

Find the minimum and maximum number from a set of numbers


```
loop_array:
        beq $a1, $zero, print_and_exit
        lw $t0, ($a0)
        bge $t0, $t3, not_min # if (current_element >= current_min) {don't modify min}
        move $t3, $t0
        not_min:
        ble $t0, $t2, not_max # if (current_element <= current_max) {don't modify max}
        move $t2, $t0
        not_max:
        addi $a1, $a1, -1
        addi $a0, $a0, 4
        j loop_array
```

Find the minimum and maximum number from a set of numbers


```
print_and_exit:
    # print minimum
    li $v0, 4 #for string
    la $a0, array_max
    syscall

    li $v0, 1 #for number
    move $a0, $t2
    syscall
```

```
# print maximum
    li $v0, 4
    la $a0, array_min
    syscall

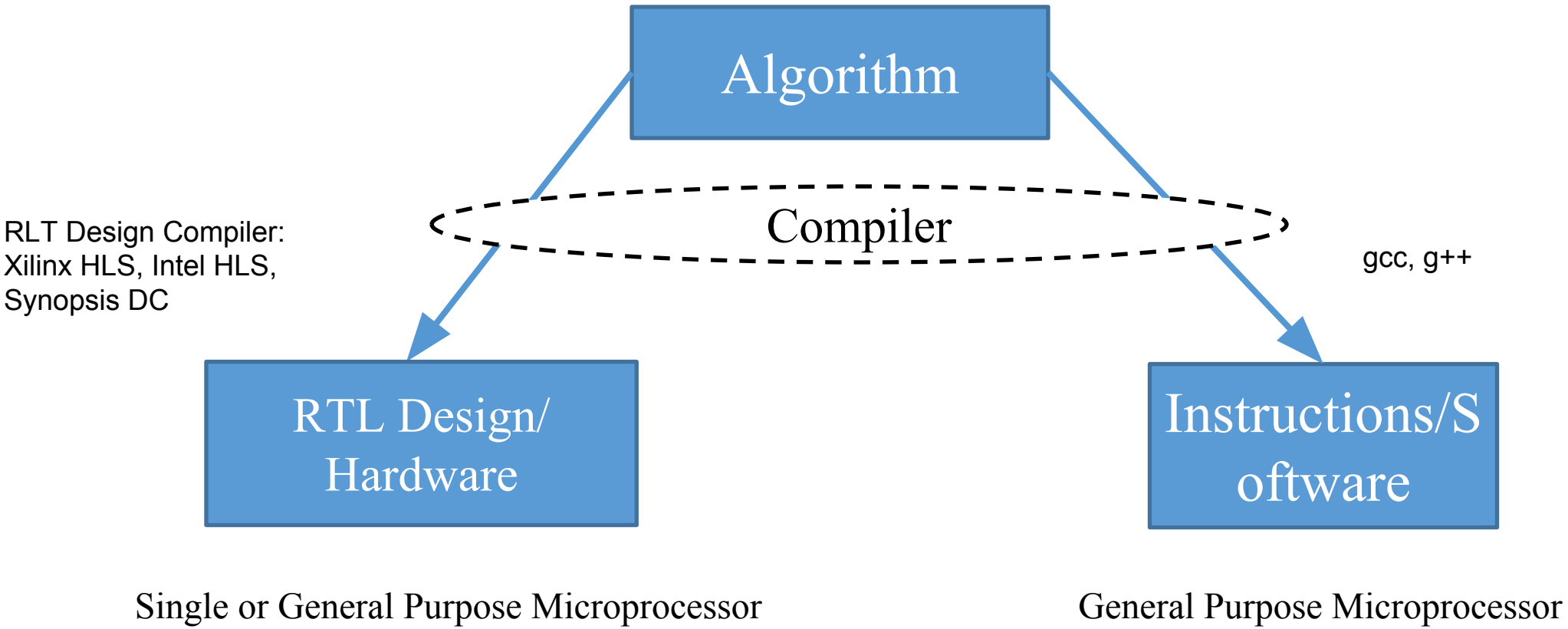
    li $v0, 1
    move $a0, $t3
    syscall

    # exit
    li $v0, 10
    syscall
```

Find the minimum and maximum number from a set of numbers


```
int main()
{
   int arr[10] = {1, 2, -8, 0, 23, 11, -10};
   int N = 10, i;
  int minE = 9999, maxE = -9999;
  // Traverse the given array
    for (i = 0; i < N; i++) {
        // If current element is smaller than minE then update it
        if (arr[i] < minE) {
            minE = arr[i];
        }
        // If current element is greater than maxE then update it
        if (arr[i] > maxE) {
            maxE = arr[i];
        }    } printf("The minimum element is %d", minE); printf("\n");
    printf("The maximum element is %d", maxE);
    return 0;
}
```

Algorithm and its Possible Architectures



How does the General-purpose micro-processor solve the problem?

Problem

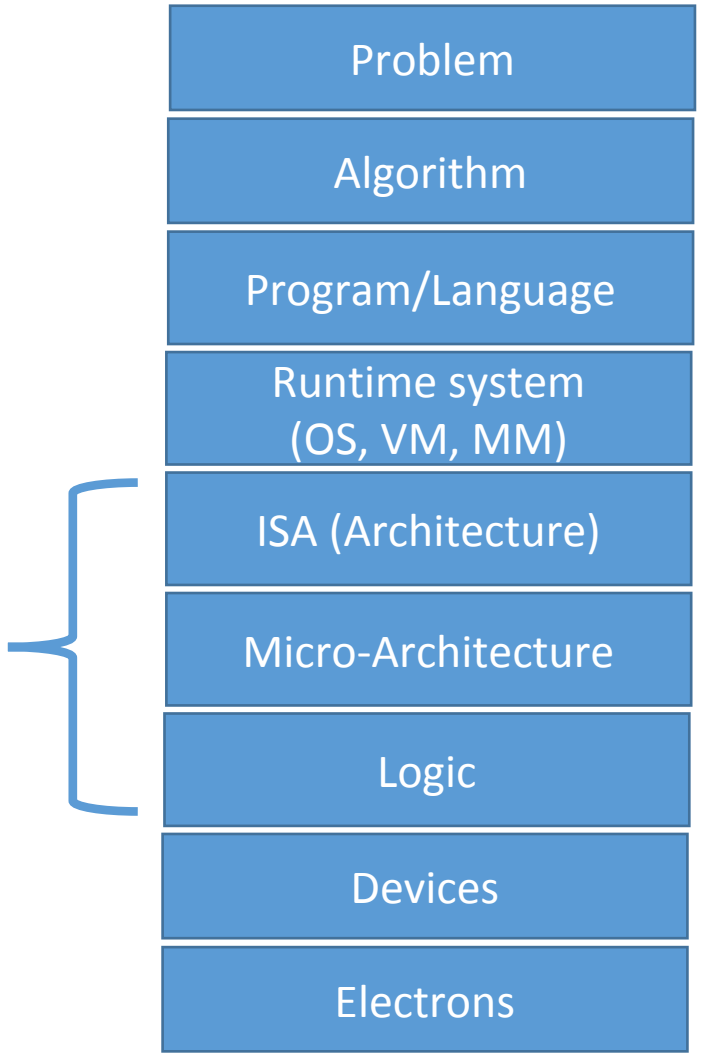
Beside problem it also take energy

How do we ensure problems are solved by electrons?

Electrons

Beside solutions it also produce heat

How does the General-purpose micro-processor solve the problem?



CS F342 focuses on

Beside problem it also take energy

How do we ensure problems are solved by electrons?

Beside solutions it also produce heat

Yale Patt, "Requirements, Bottlenecks, and Good Fortune- Agents for Microprocessor Evolution," *Proc. of the IEEE*, VOL. 89, NO. 11, NOV. 2001

What is the basic building block of a program?

- A C-program: min_max.c
- Perform: gcc –S min_max.c
- Assembly code: min_max.s
- The basic building block of a program is **instructions**
- What is the significance of instruction order?
- From a program's (compiler writer's) point of view the computer is the instruction set
- Instruction Set Architecture can be MIPS (32, 64), RISC-V (32, 64), 8085, x86 (refers to a 16/32-bit CPU of type 8086), x64 (refers to a 64-bit CPU of type 8086)
- All program use same set of instructions

What is the meaning of Computer Architecture?

- Architecture: Computational (dedicated or general-purpose) structure with respect to user (programmer, etc)
  - Use of minimal resources
  - Easily scalable
- For an example, a set of instruction can be the meaning of comp. arch. to a programmer
- A dedicated processor
- Algorithm has its physical structure

Two Very Important Ideas

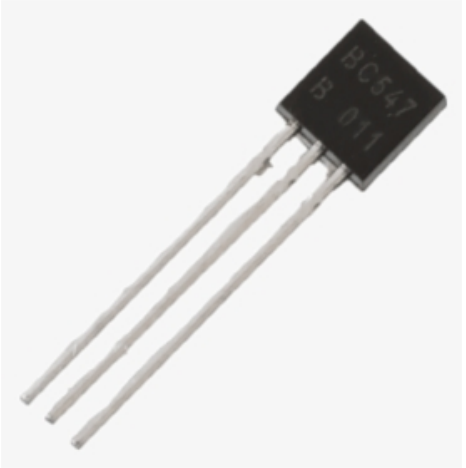
- **Idea 1:** All computers (the biggest and the smallest, the fastest and the slowest, the most expensive and the cheapest) are capable of computing exactly the same things if they are given enough time and enough memory.
- **Idea 2:** We describe our problems in English or some other language spoken by people. Yet the problems are solved by electrons running around inside the computer. It is necessary to transform our problem from the language of humans to the voltages that influence the flow of electrons.

History of computation

- Homework
- Go through the material on Gdrive, shared with you.
- https://drive.google.com/drive/folders/1JkTqBCwtP8o6-7YzjX8JfxbabKT8puzW
- Go through the order mentioned in the xlsx file
- Will ask the question in the next class

Changes in Computation

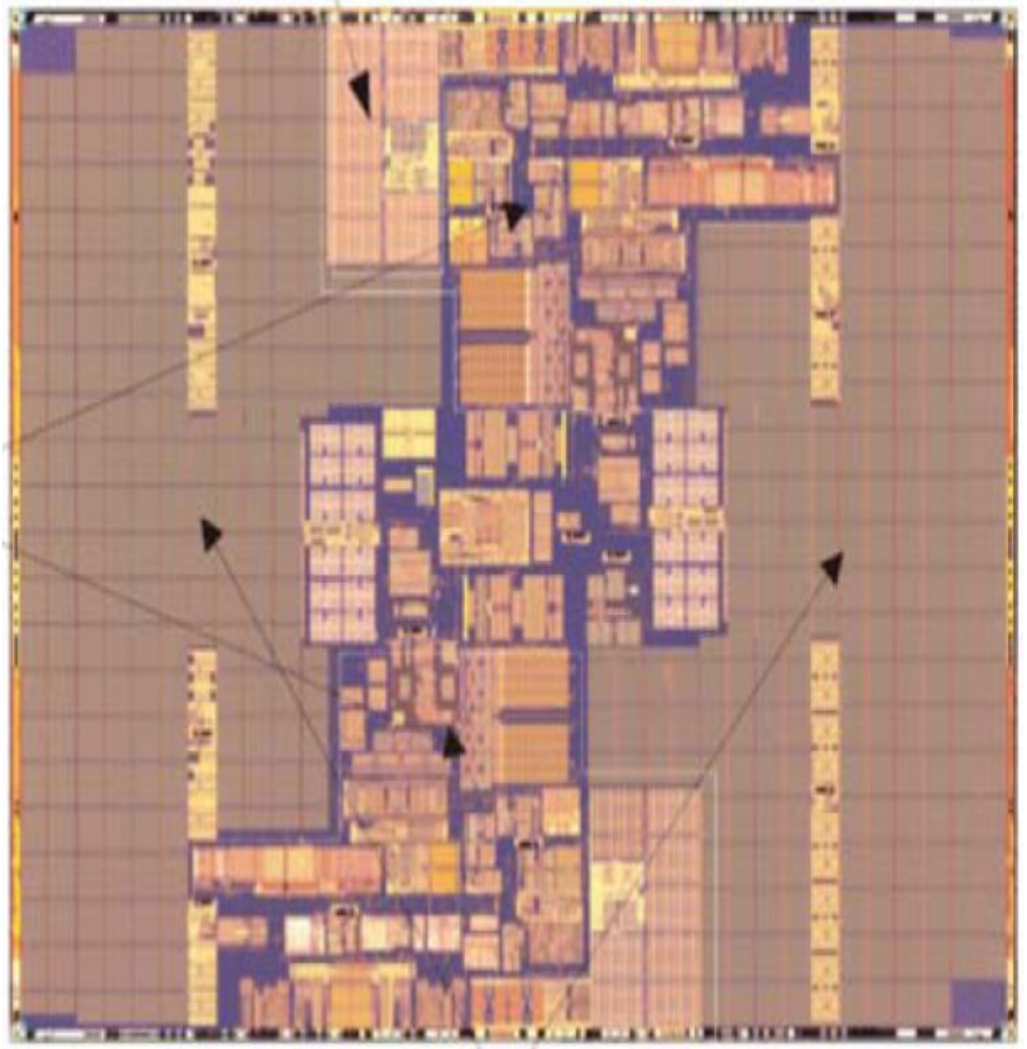
- Manual
- Mechanical
  - gears, chains, pulleys, and steam power
  - Punch cards
- Electro-mechanical
  - switches, relays
- Electrical
  - plugboards, vaccum tubes
  - later came DRUM memory, core memory, transistors and so on ...



http://www.computerhistory.org/timeline

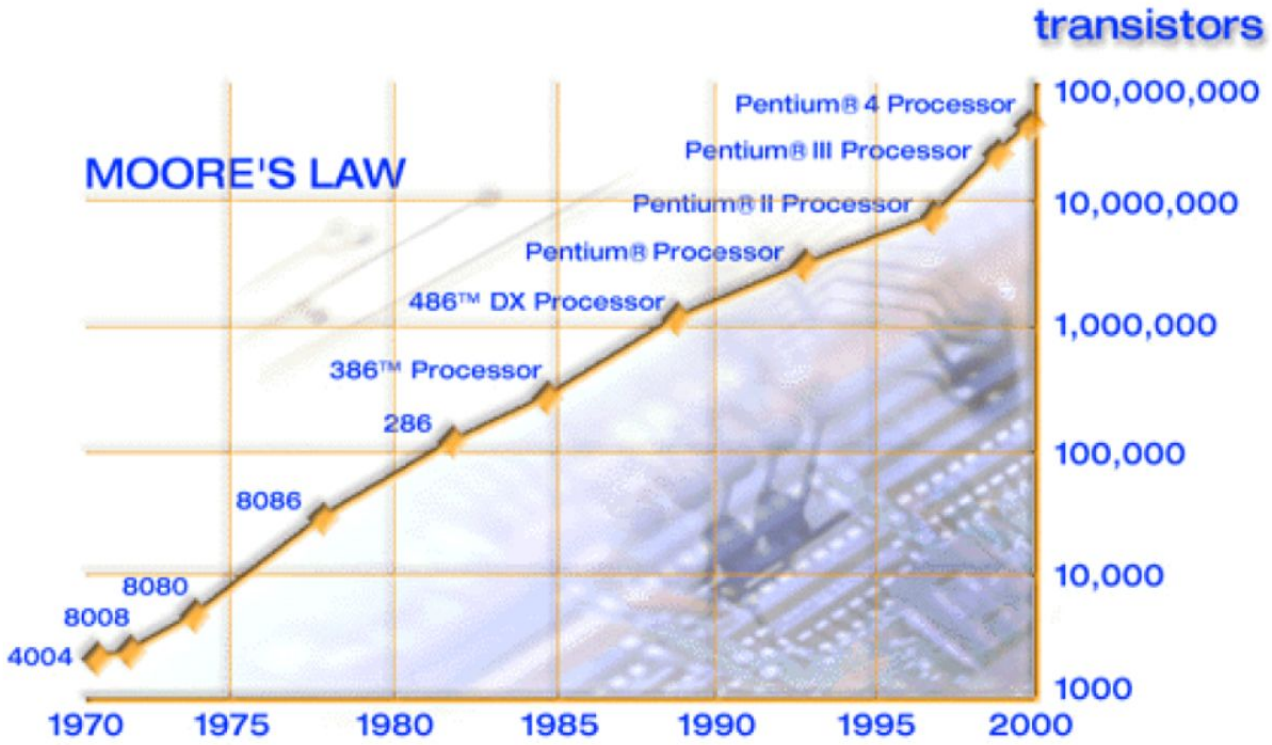
Computation on 2004

- 64-bit Itanium processor developed by Intel
- 1.7 billion transistors
- 1.7 GHz, issue up to 8 instructions per cycle
- 26 MByte of cache
- In ~30 years, about 100,000 fold growth in transistor count and performance

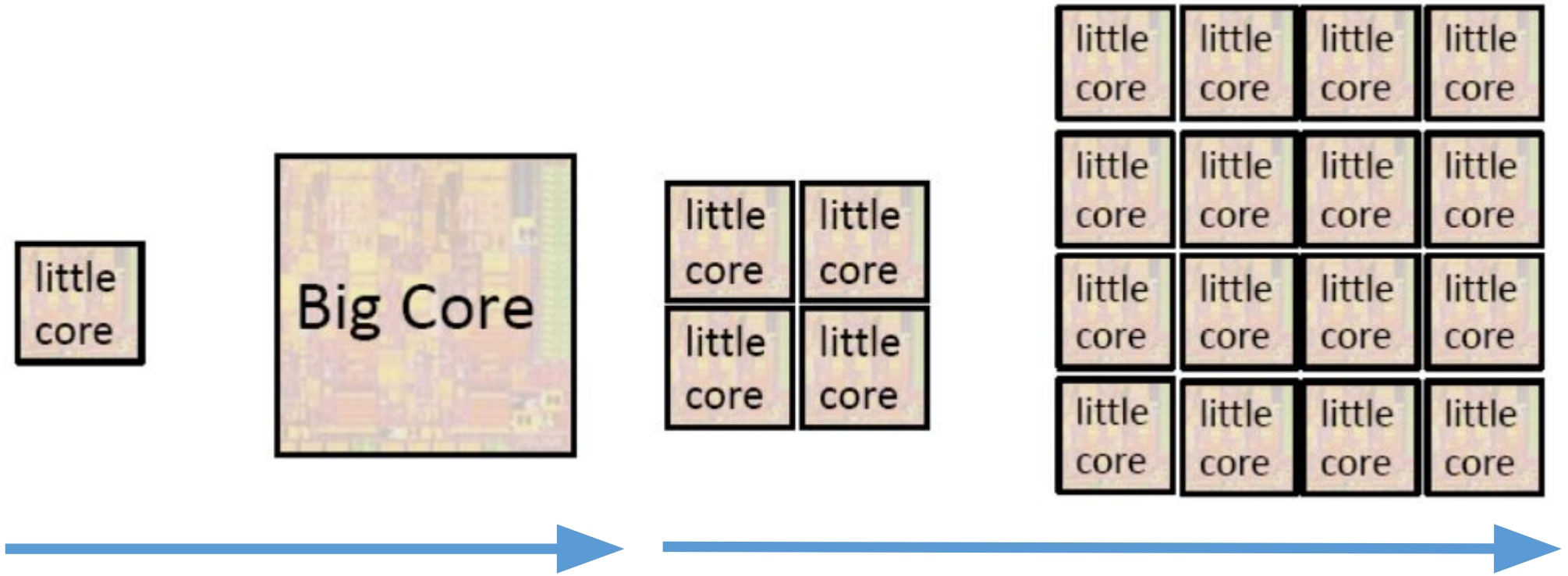


What Happened in Between

- **Moore's Law** refers to **Moore's** perception that the number of transistors on a microchip doubles every two years, though the cost of computers is halved.



Moore's Law Scaling with Cores

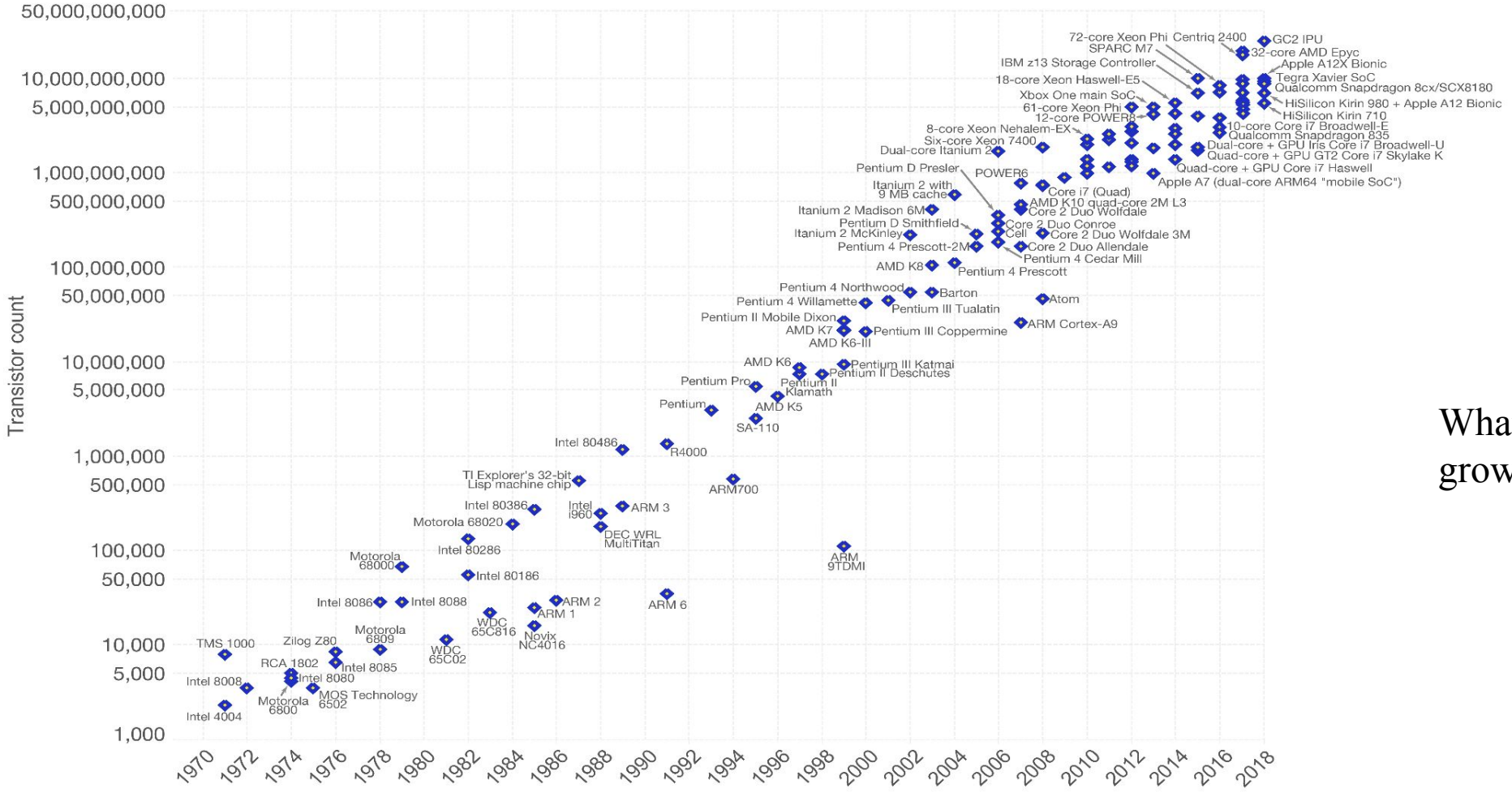


The Big picture

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.

Our World in Data

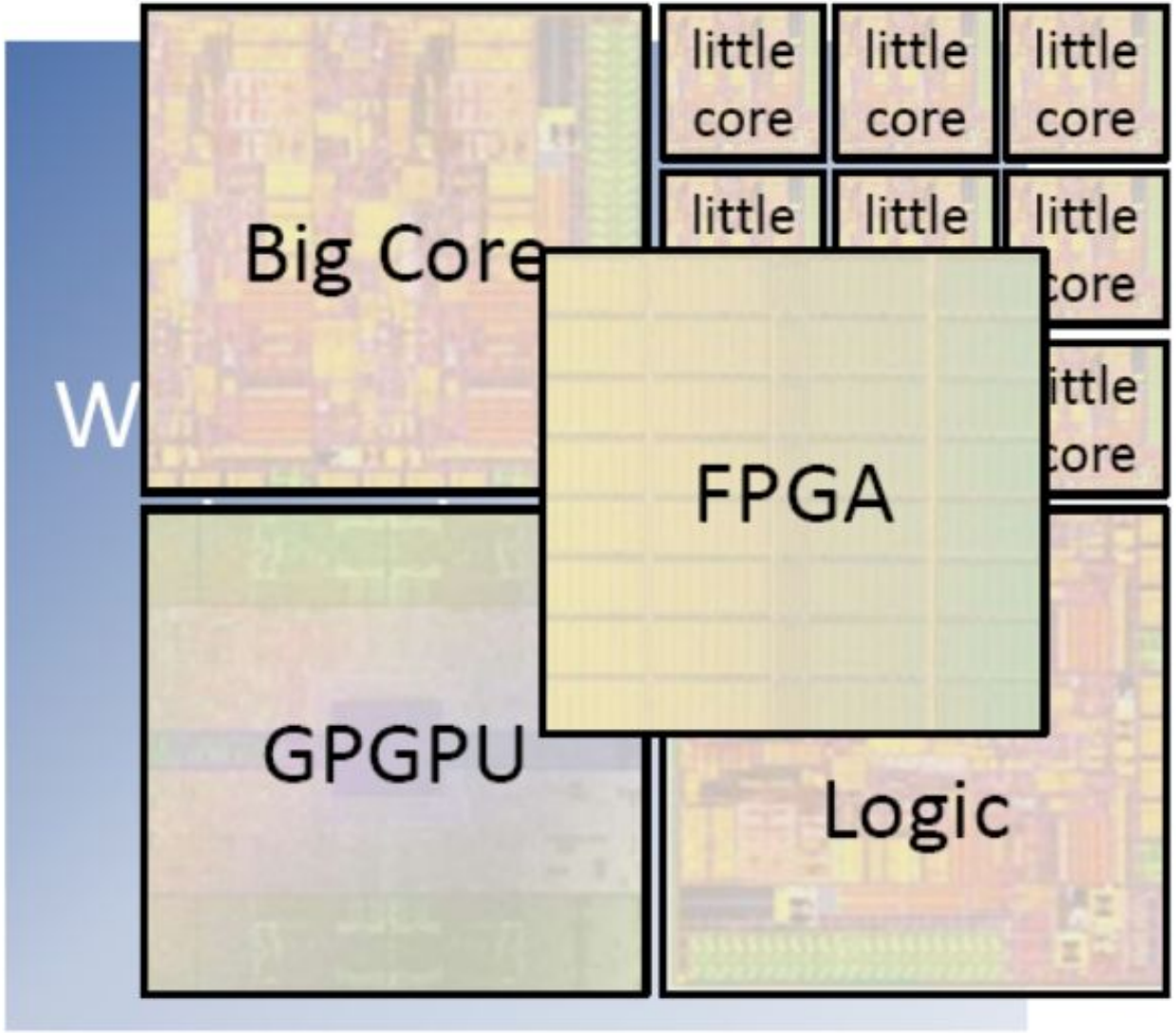


What kind of growth is it?

Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

Future is about



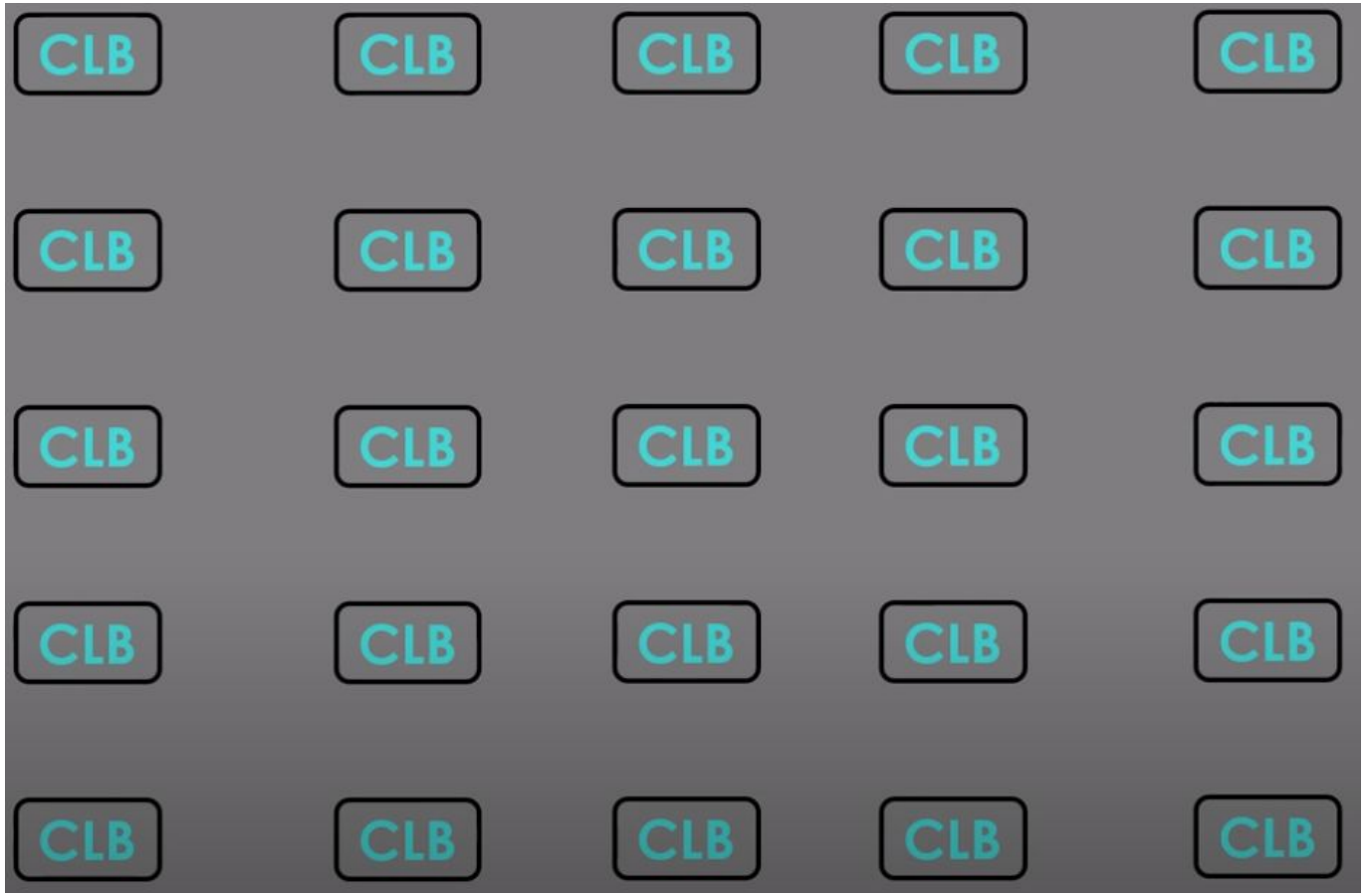
Quantum Computing

Carbon nanotube computer

- https://www.nature.com/articles/nature12502
- https://en.wikipedia.org/wiki/Carbon_nanotube_computer

FPGA

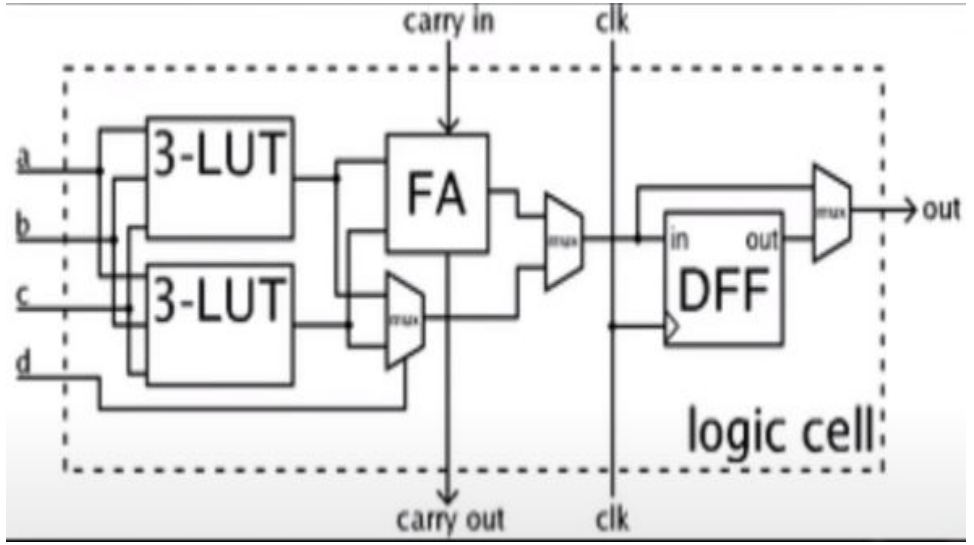
- Field Programmable Gate Array

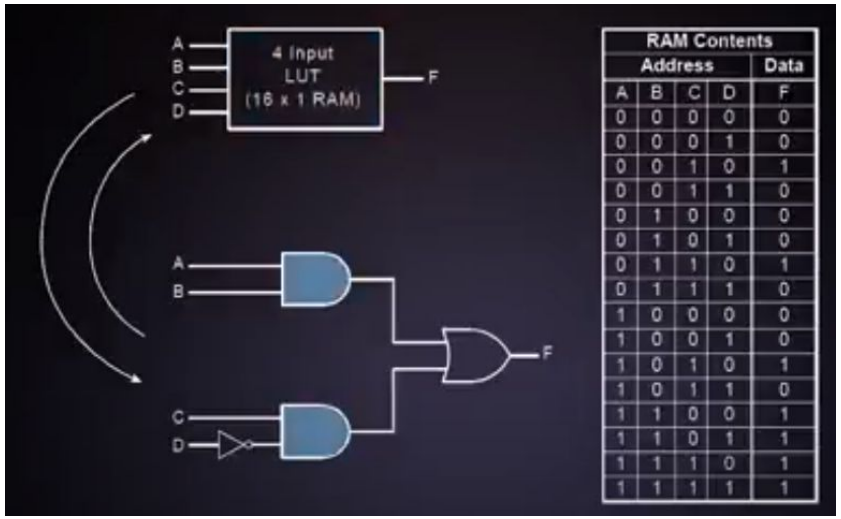


CLB: Configurable Logic block

https://www.youtube.com/watch?v=WY-F3knih7c
https://www.youtube.com/watch?v=K2LRXsKCW7w

Inside a CLB





4-inputs LUT

Quantum Computing

- https://awards.acm.org/about/2020-acm-prize
- https://www.ibm.com/quantum-computing/
- Google wants to build a useful quantum computer by 2029
  - https://www.theverge.com/2021/5/19/22443453/google-quantum-computer-2029-decade-commercial-useful-qubits-quantum-transistor
- Quantum Computing: Untangling the Hype (Talk at The Royal Institution)
  - https://www.youtube.com/watch?v=wE1OCXvaDtc

Homework

- Design a single-purpose processor for performing the only bubble sort.
- Design a general processor for a set of instructions:
  - ADD R1, R2, R3
  - SUB R1, R2, R3
  - LW R2, offset(R3)
  - SW R2, offset(R3)
  - BNE R1, R2, offset

  - Has a instruction and a data memory
  - Has 32 number of registers
  - Register R0 holds zero value only.

Summary

- Motivation for automated Computation
- Dedicated processor Vs. General-purpose processor
- Limitation of Algorithm
- Building block of a program
- Steps to solve a problem by a computer/laptop
- Changes in Computation

ALU Operations

ALU control lines (3:0)	ALU Functions
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	set on less than

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Generation of Control: **ALU decoder** truth table

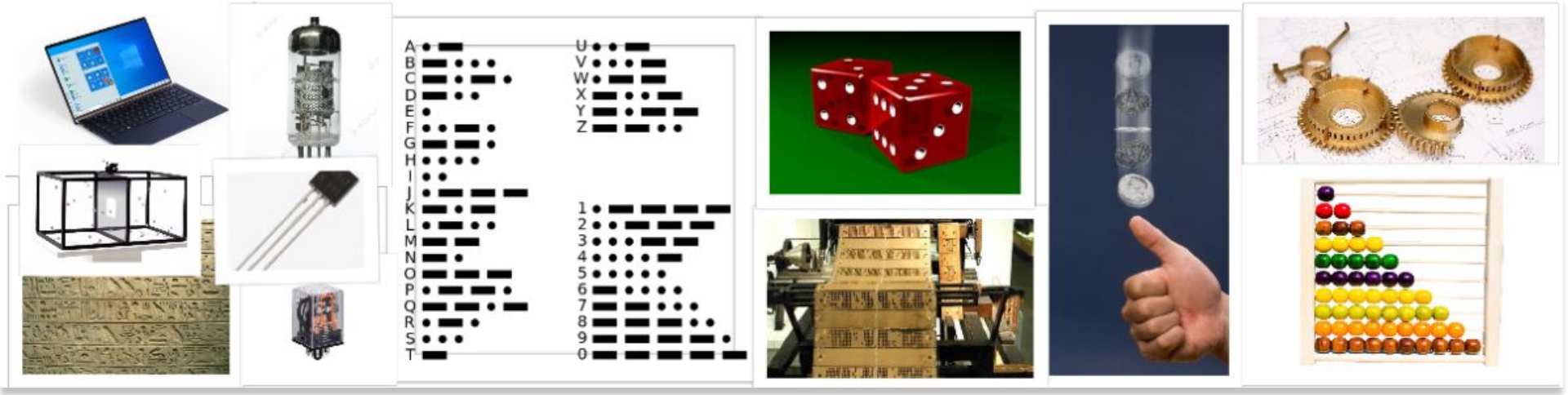
ALUOp	ALU control lines (2:0)	Funct
00	0010 (add)	X
01	0110 (subtract)	X
1X	0010 (add)	100000 (add)
1X	0110 (subtract)	100010 (sub)
1X	0000 (and)	100100 (and)
1X	0001 (or)	100101 (or)
1X	0111 (set less than)	101010 (slt)

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Generation of Controls

Inst. opcode	ALUOp	Instr. operation	Funct field	Desired ALU action	ALUControl
100010 (LW)	00	load word	xxxxxx	add	0010
100011 (SW)	00	store word	xxxxxx	add	0010
000100 (BEQ)	01	branch equal	xxxxxx	subtract	0110
000000 (R-type)	10	add	100000	add	0010
R-type	10	Subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111
001000 (addi)	00	Immediate	xxxxxx	add	xxxx
000010 (j)	xx	jump	xxxxxx	jump	xxxx

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.



Computer Architecture (CS F342)

MIPS ISA & MARS (a MIPS) Simulator

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

Book- COD by P&H –ch-appendix-A

Microprocessor without Interlocked Pipelined Stage

- 32 bit processor
- 32 registers
  - One register hold value 0 always
- Byte addressable
  - 4-bytes make a words
- 2^30 memory words
- Big-endian format used to represent
  - Instructions & Data
- Instruction & Data memory

Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Microprocessor without Interlocked Pipelined Stage

- MIPS processor
- Instruction Set Architecture (ISA)
  - R-type
  - I-type
  - J-type

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

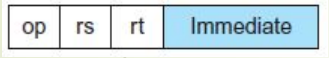
OP: Basic operation of the instruction, traditionally called the opcode.
rs: The first register source operand.
rt: The second register source operand.
rd: The register destination operand. It gets the result of the operation.
shamt: Shift amount.
funct: Function. This field, often called the *function code,* selects the specific variant of the operation in the op field.

Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

MIPS Addressing mode

Immediate addressing



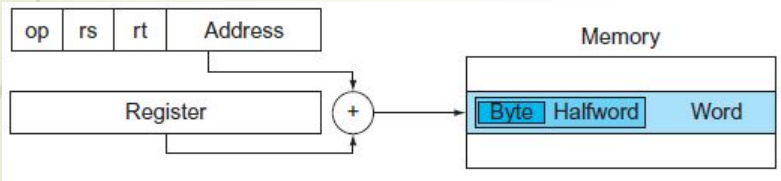
ADDI $S1, $S2, #-5

Register addressing



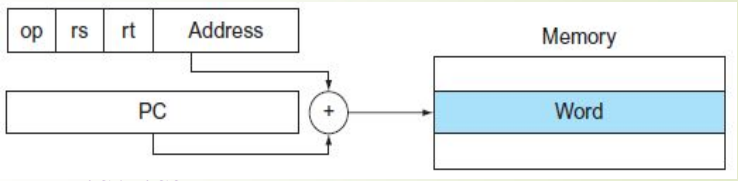
ADD $S1, $S2, $S3

Base addressing



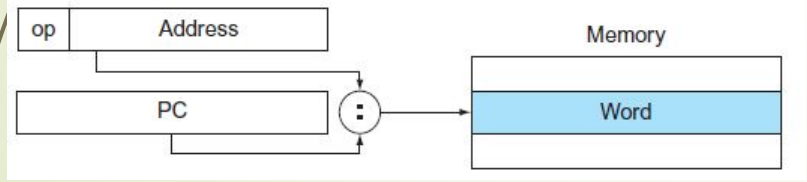
LD $S1, $S2, 5

PC-relative addressing



BNE $S1, $S2, 5

Pseudodirect addressing



J 200

Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

MIPS Operands

Name	Example	Comments
32 registers	`$s0-$s7, $t0-$t9, $zero, $a0-$a3, $v0-$v1, $gp, $fp, $sp, $ra, $at`	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register `$zero` always equals 0, and register `$at` is reserved by the assembler to handle large constants.
2^{30} memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Three register operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 – $s3	Three register operands
	add immediate	addi $s1,$s2,20	$s1 = $s2 + 20	Used to add constants
Data transfer	load word	lw $s1,20($s2)	$s1 = Memory[$s2 + 20]	Word from memory to register
	store word	sw $s1,20($s2)	Memory[$s2 + 20] = $s1	Word from register to memory
	load half	lh $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	load half unsigned	lhu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	store half	sh $s1,20($s2)	Memory[$s2 + 20] = $s1	Halfword register to memory
	load byte	lb $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	load byte unsigned	lbu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	store byte	sb $s1,20($s2)	Memory[$s2 + 20] = $s1	Byte from register to memory
	load linked word	ll $s1,20($s2)	$s1 = Memory[$s2 + 20]	Load word as 1st half of atomic swap
	store condition. word	sc $s1,20($s2)	Memory[$s2+20]=$s1;$s1=0 or 1	Store word as 2nd half of atomic swap
	load upper immed.	lui $s1,20	$s1 = 20 * 2^{16}	Loads constant in upper 16 bits
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,20	$s1 = $s2 & 20	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,20	$s1 = $s2 | 20	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq $s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne $s1,$s2,25	if ($s1!= $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for beq, bne
	set on less than unsigned	sltu $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than unsigned
	set less than immediate	slti $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant
	set less than immediate unsigned	sltiu $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr $ra	go to $ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4; go to 10000	For procedure call

https://www.scribd.com/document/358717972/mips-ref-pdf

MIPS Reference Card (yumpu.com)

Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

A few instructions

		6-bit	5-bit	5-bit	5-bit	5-bit	6-bit	
Instruction	**Format**	**OP**	**RS**	**RT**	**RD**	**Shamt**	**Funct**	**Address**
add	R	0	reg	reg	reg	0	32	NA
sub	R	0	reg	reg	reg	0	34	NA
addi	I	8	reg	reg	NA	NA	NA	Constant
lw	I	35	reg	reg	NA	NA	NA	Offset
Sw	I	43	reg	reg	NA	NA	NA	Offset
bne	I	5	reg	reg	NA	NA	NA	Offset

Can we write a MIPS program for generating 7-Fibonacci numbers, using these instructions, considering that first two Fibonacci numbers are stored at memory location ‘a’ & ‘b’?

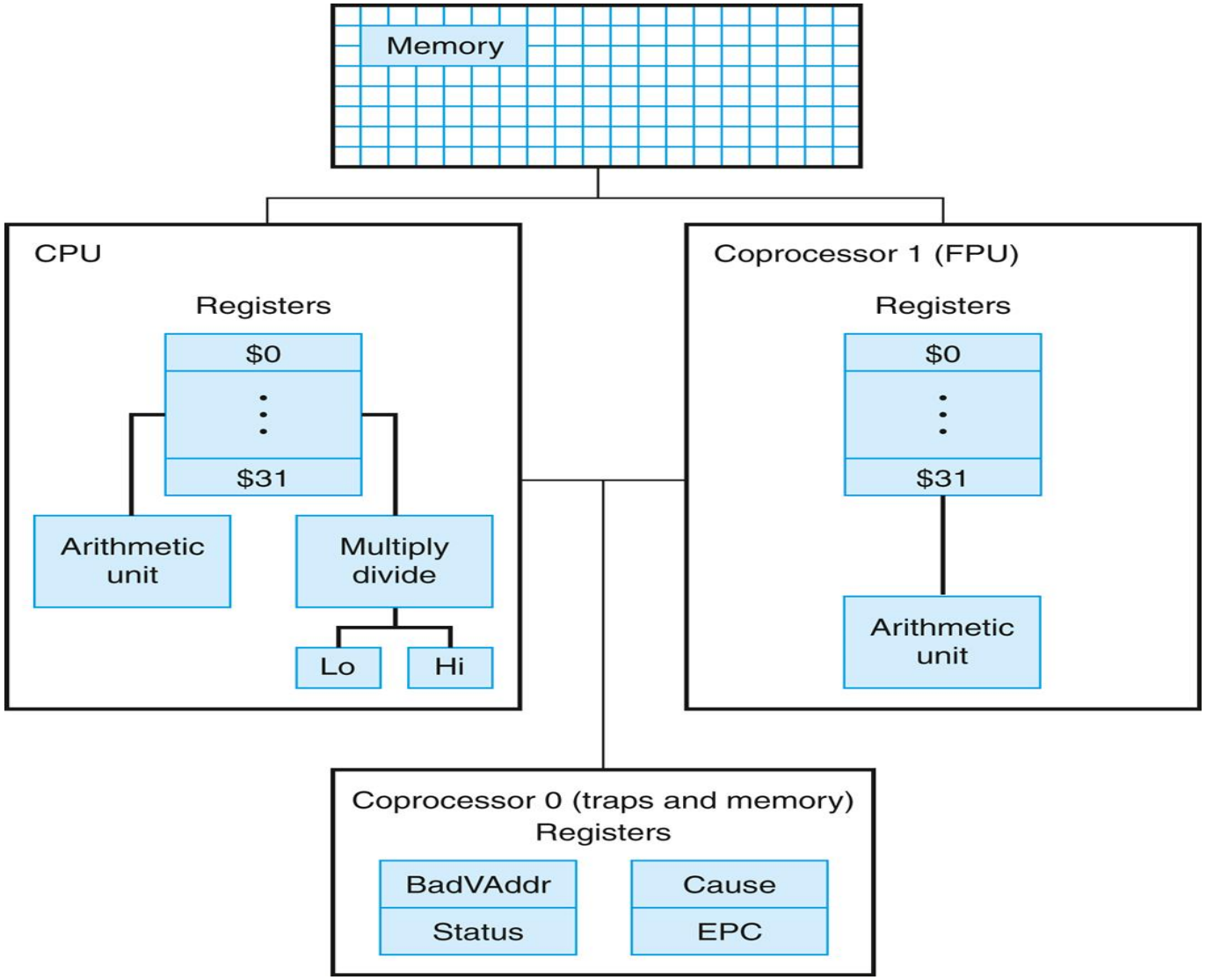
Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy, Chapter-2.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

MIPS ASM Programming

MARS- A MIPS Simulator

MIPS R2000



Book- COD by P&H –ch-appendix-A

MARS

- Install MARS
  https://courses.missouristate.edu/kenvollmar/mars/

- Documentation
  - book: appendix A9 (on CD)
    [https://pages.cs.wisc.edu/~larus/HP_AppA.pdf]
  - https://courses.missouristate.edu/KenVollmar/mars/tutorial.htm

- Tutorials:
  - https://www.youtube.com/playlist?list=PL5b07qlmA3P6zUdDf-o97ddfpvPFuNa5A
  - https://www.youtube.com/watch?v=d5amCbPUVYY&ab_channel=CSTutorials
  - https://www.it.uu.se/education/course/homepage/os/vt20/module-0/mips-and-mars/

- Alternative to MARS
  - SPIM: MIPS Assembler and Runtime Simulator
  - https://pages.cs.wisc.edu/~larus/spim.html

MARS in action


```
D:\CS G524 - ACA\2024\Labs\Demo-2-2-24\Mars4_5>java Mars h
MARS 4.5  Copyright 2003-2014 Pete Sanderson and Kenneth Vollmar

Usage:  Mars  [options] filename [additional filenames]
  Valid options (not case sensitive, separate by spaces) are:
      a   -- assemble only, do not simulate
  ae<n>   -- terminate MARS with integer exit code <n> if an assemble error occurs.
  ascii   -- display memory or register contents interpreted as ASCII codes.
      b   -- brief - do not display register/memory address along with contents
      d   -- display MARS debugging statements
```

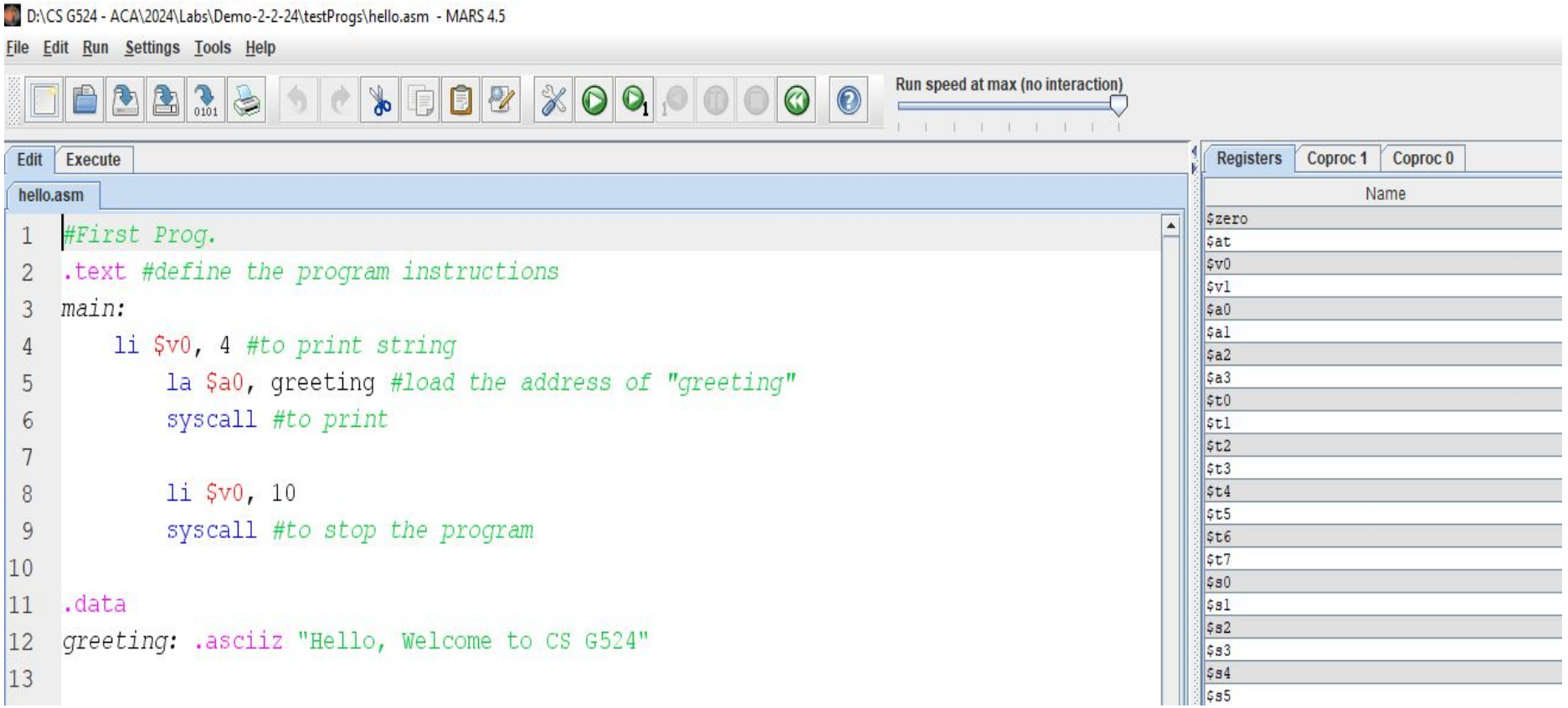


https://courses.missouristate.edu/kenvollmar/mars/Help/Help_4_1/MarsHelpCommand.html

Book- COD by P&H –ch-appendix-A

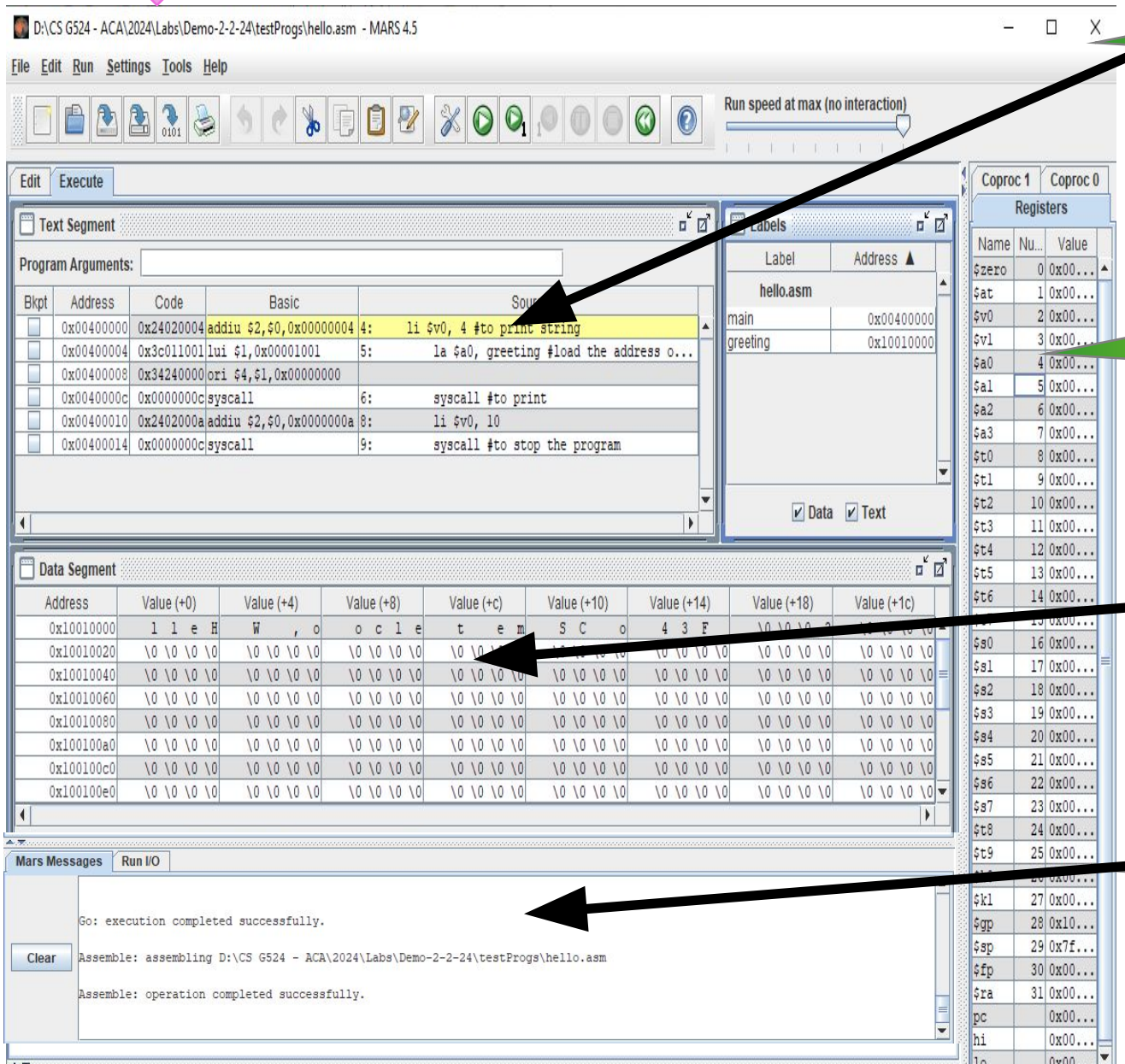
MARS in action


```
D:\CS G524 - ACA\2024\Labs\Demo-2-2-24\Mars4_5>java Mars
```



Book- COD by P&H –ch-appendix-A

MARS in action



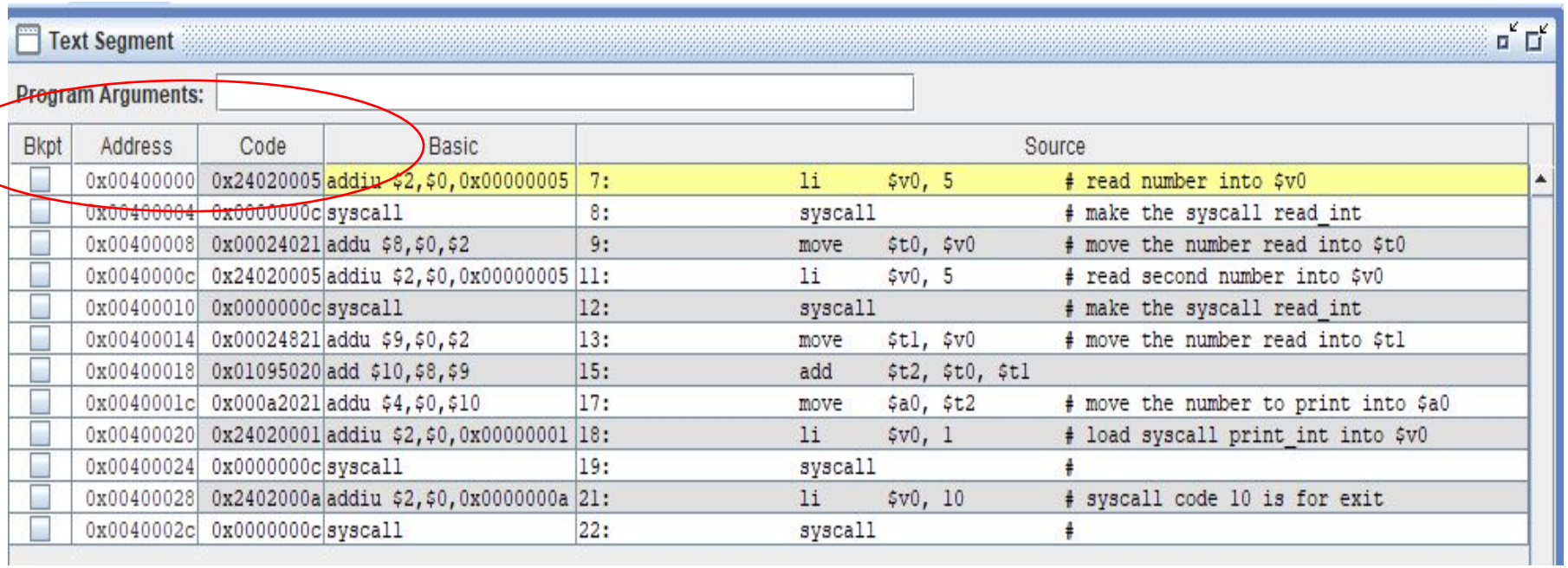
Program memory

MIPS registers

Data memory

Messages

MARS in action



Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	Source
	0x00400000	0x24020005	addiu $2,$0,0x00000005	7: li $v0, 5 # read number into $v0
	0x00400004	0x0000000c	syscall	8: syscall # make the syscall read_int
	0x00400008	0x00024021	addu $8,$0,$2	9: move $t0, $v0 # move the number read into $t0
	0x0040000c	0x24020005	addiu $2,$0,0x00000005	11: li $v0, 5 # read second number into $v0
	0x00400010	0x0000000c	syscall	12: syscall # make the syscall read_int
	0x00400014	0x00024821	addu $9,$0,$2	13: move $t1, $v0 # move the number read into $t1
	0x00400018	0x01095020	add $10,$8,$9	15: add $t2, $t0, $t1
	0x0040001c	0x000a2021	addu $4,$0,$10	17: move $a0, $t2 # move the number to print into $a0
	0x00400020	0x24020001	addiu $2,$0,0x00000001	18: li $v0, 1 # load syscall print_int into $v0
	0x00400024	0x0000000c	syscall	19: syscall #
	0x00400028	0x2402000a	addiu $2,$0,0x0000000a	21: li $v0, 10 # syscall code 10 is for exit
	0x0040002c	0x0000000c	syscall	22: syscall #

For MIPS, Program always starts 0x00400000

What could be the PC at the beginning?

$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400000

Book- COD by P&H –ch-appendix-A

MARS in action

Data Segment

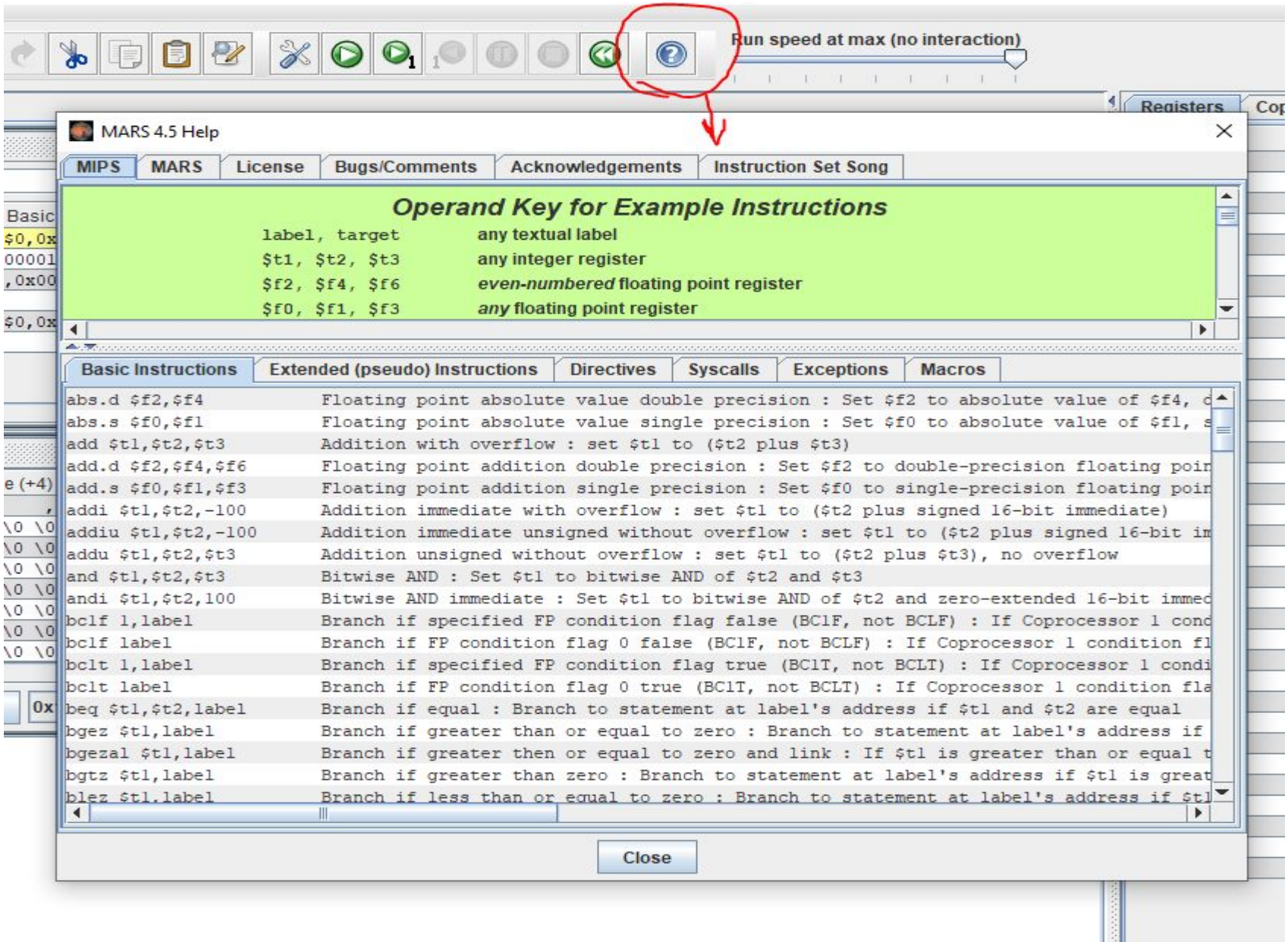
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010020	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010040	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010060	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010080	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x100100a0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x100100c0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x100100e0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

0x10010000 (.data) ☑ Hexadecimal Addresses ☑ Hexadecimal Values ☑ ASCII

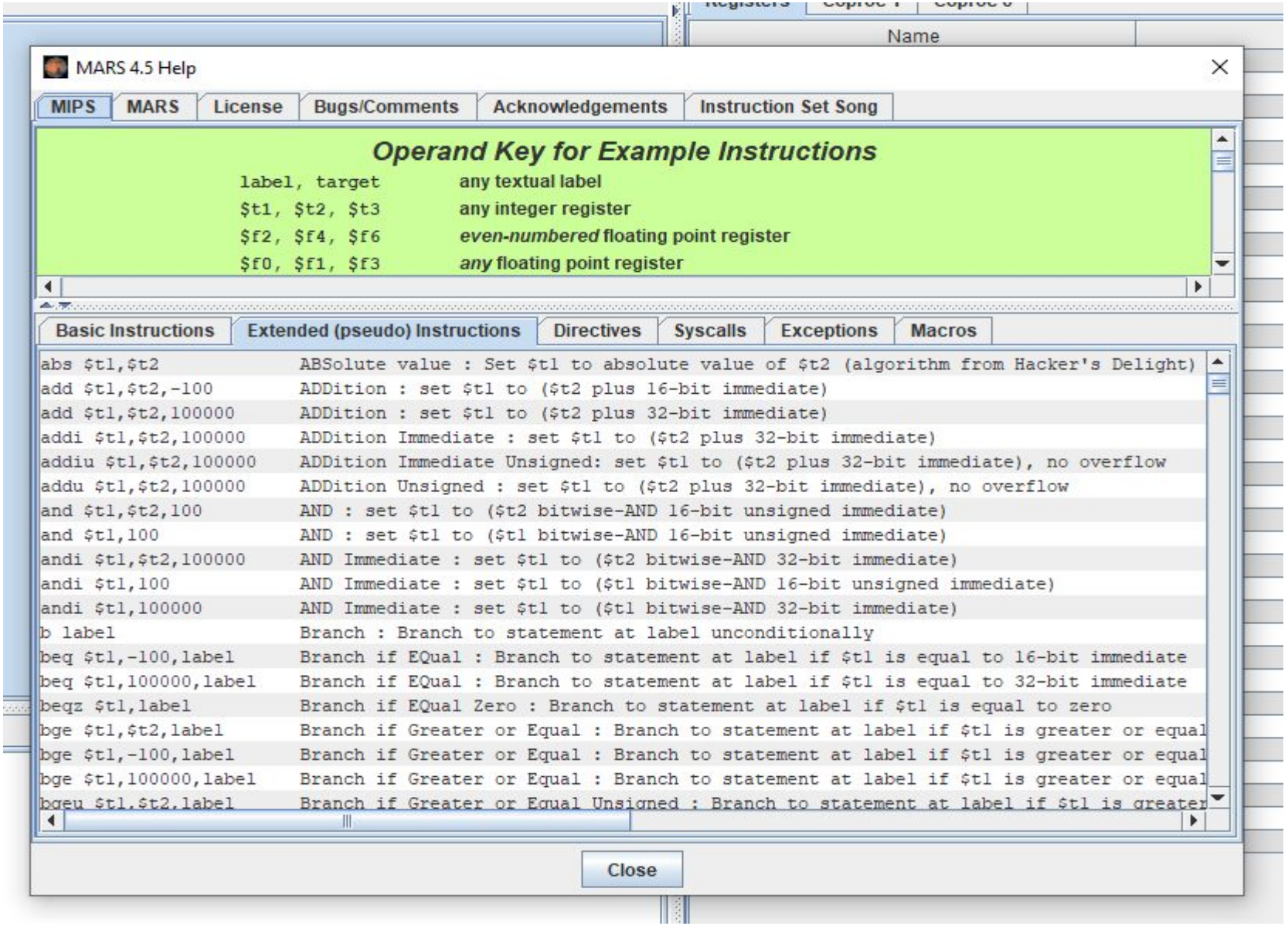
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400000

Book- COD by P&H –ch-appendix-A

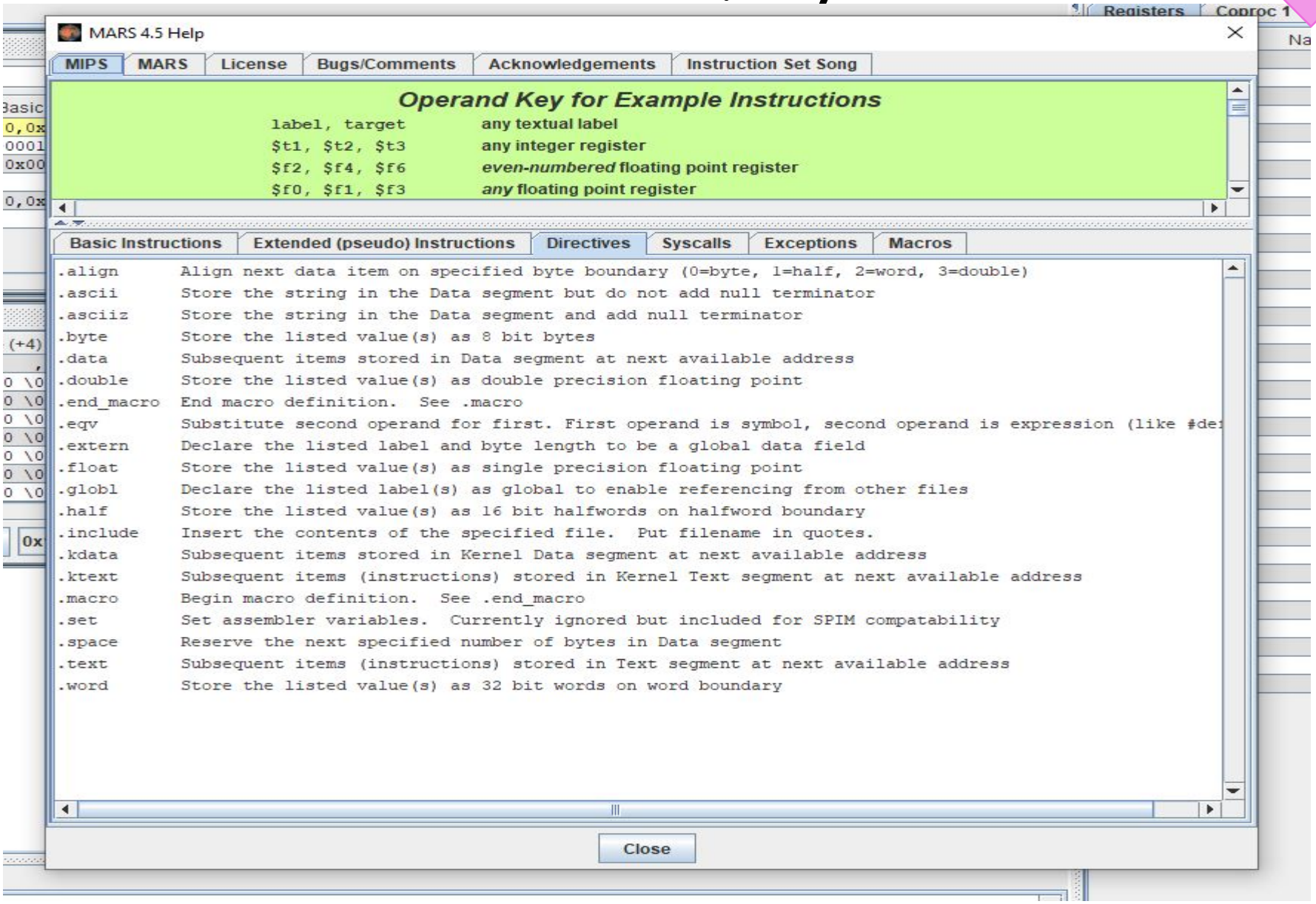
MARS/MIPS assembly directives



MARS/MIPS assembly directives



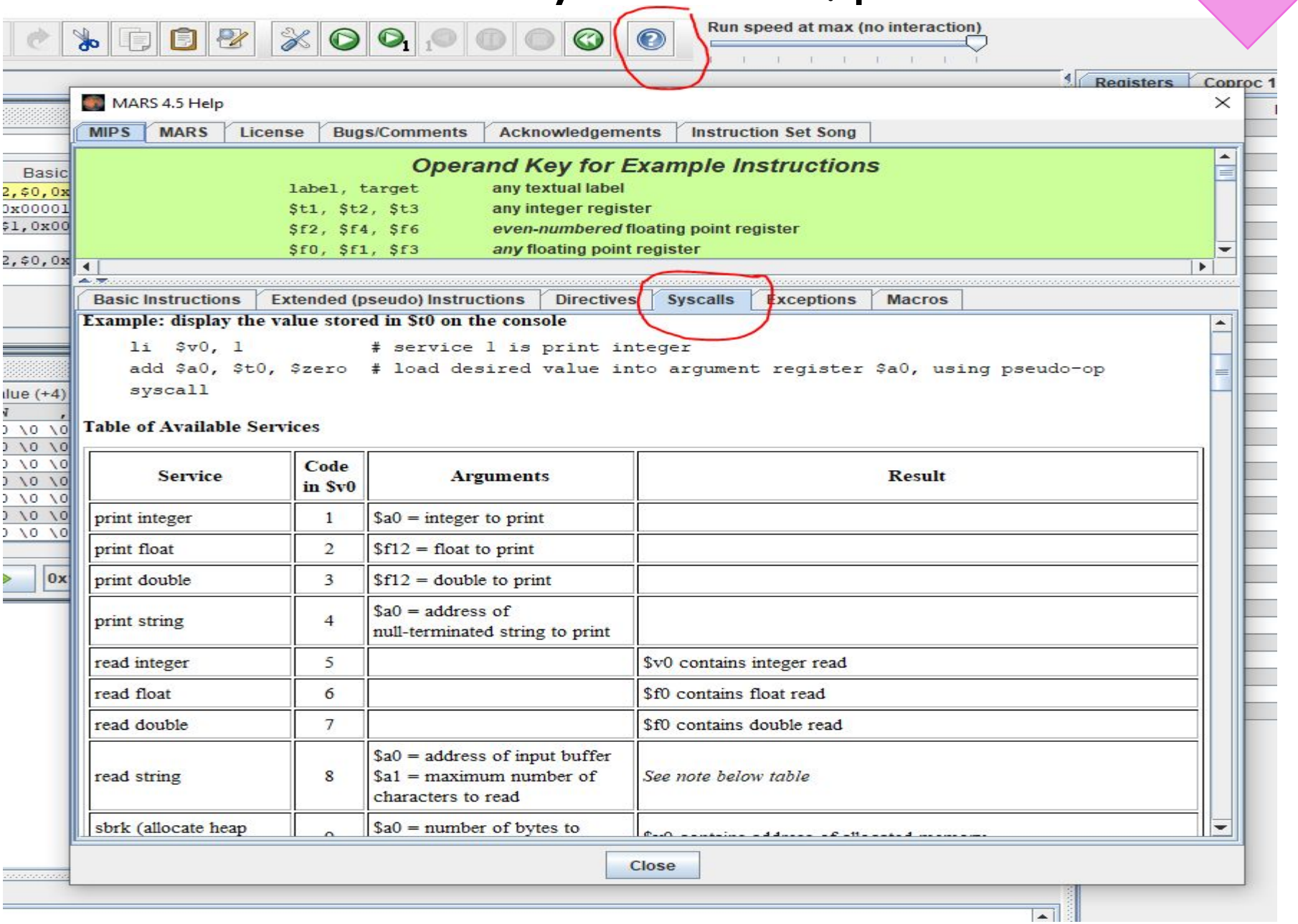
MARS/MIPS assembly directives



MARS syscall examples

Service	Trap code	Input	Output
print_int	$v0 = 1	$a0 = integer to print	prints $a0 to standard output
print_float	$v0 = 2	$f12 = float to print	prints $f12 to standard output
print_double	$v0 = 3	$f12 = double to print	prints $f12 to standard output
print_string	$v0 = 4	$a0 = address of first character	prints a character string to standard output
read_int	$v0 = 5		integer read from standard input placed in $v0
read_float	$v0 = 6		float read from standard input placed in $f0
read_double	$v0 = 7		double read from standard input placed in $f0
read_string	$v0 = 8	$a0 = address to place string, $a1 = max string length	reads standard input into address in $a0
sbrk	$v0 = 9	$a0 = number of bytes required	$v0= address of allocated memory Allocates memory from the heap
exit	$v0 = 10		
print_char	$v0 = 11	$a0 = character (low 8 bits)	
read_char	$v0 = 12		$v0 = character (no line feed) echoed
file_open	$v0 = 13	$a0 = full path (zero terminated string with no line feed), $a1 = flags, $a2 = UNIX octal file mode (0644 for rw-r--r--)	$v0 = file descriptor
file_read	$v0 = 14	$a0 = file descriptor, $a1 = buffer address, $a2 = amount to read in bytes	$v0 = amount of data in buffer from file (-1 = error, 0 = end of file)
file_write	$v0 = 15	$a0 = file descriptor, $a1 = buffer address, $a2 = amount to write in bytes	$v0 = amount of data in buffer to file (-1 = error, 0 = end of file)
file_close	$v0 = 16	$a0 = file descriptor	

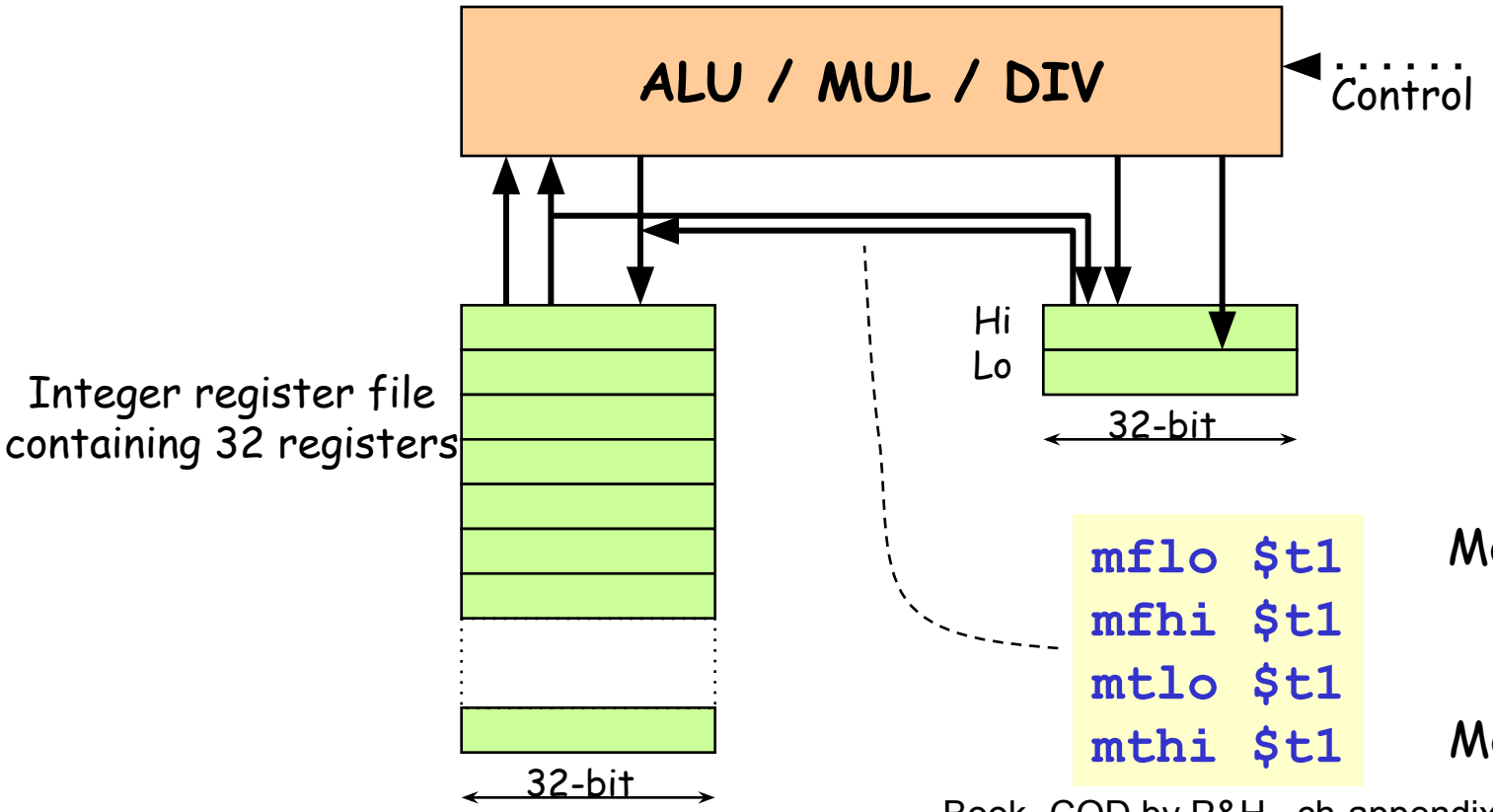
MARS syscall examples



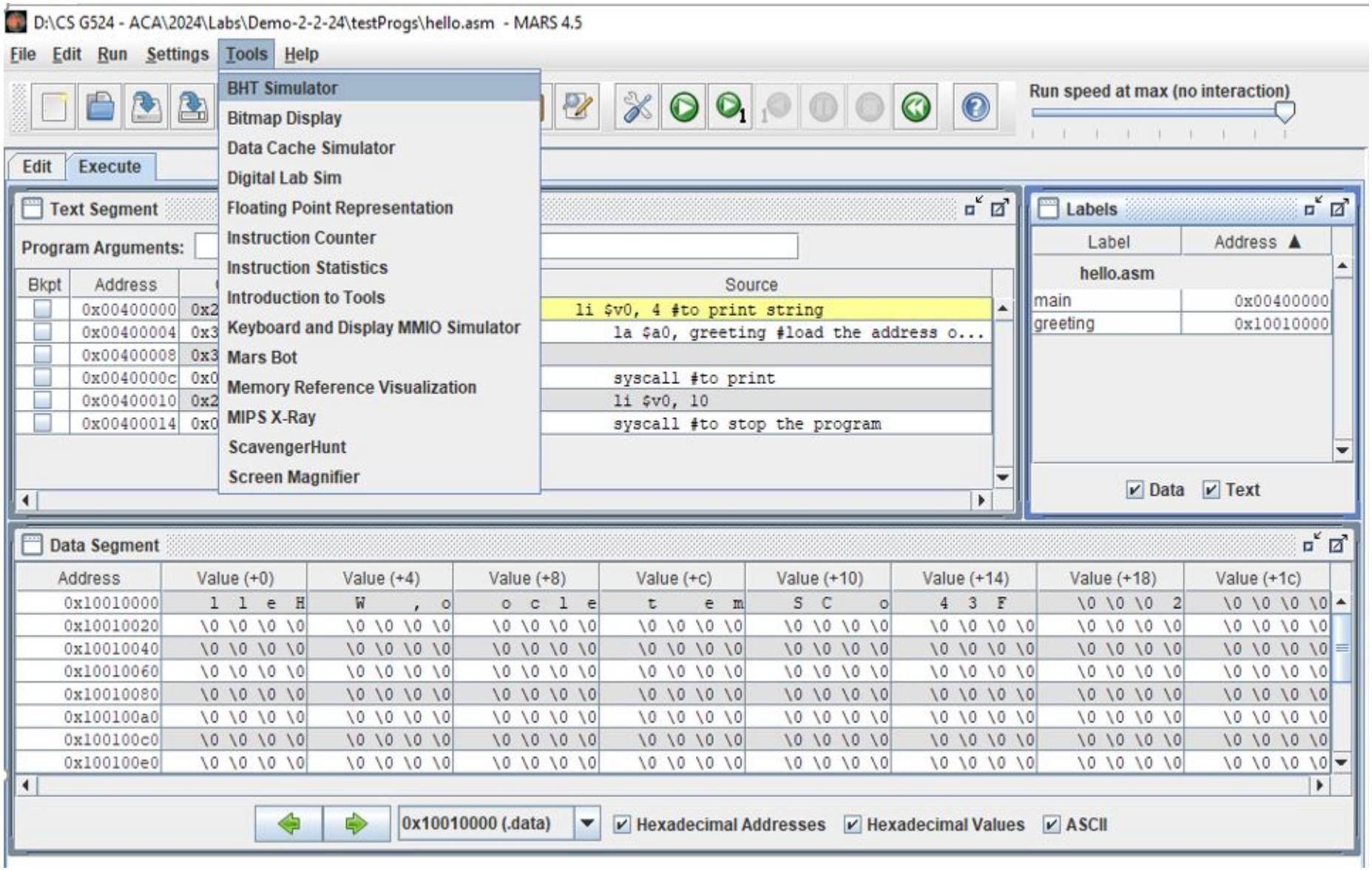
Strange MIPS instructions

- `div  $t1, $t2`
- `mult $t1, $t2`

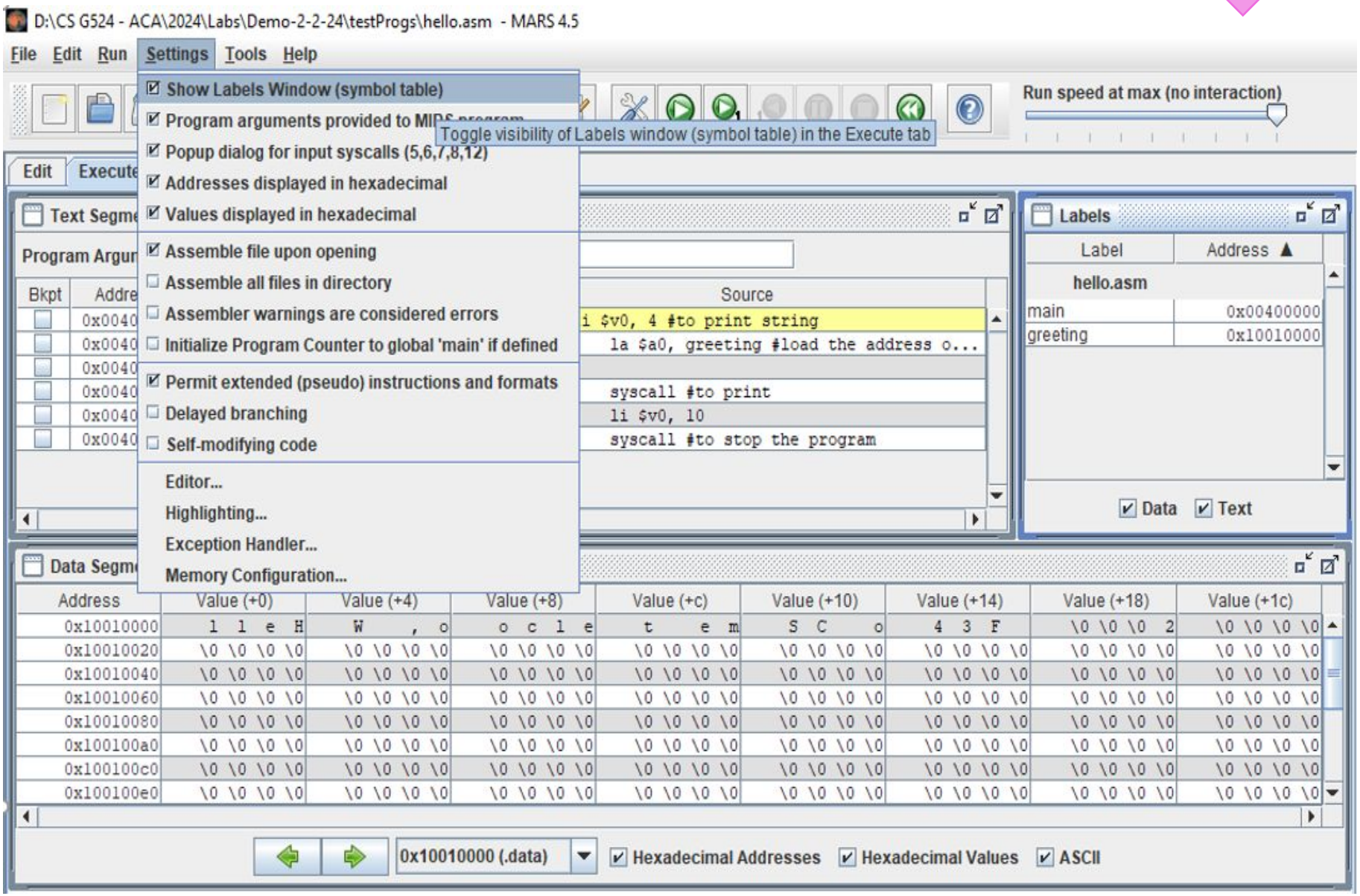
- These instructions seem to have no explicit destination register!



MARS in action: Tools menu



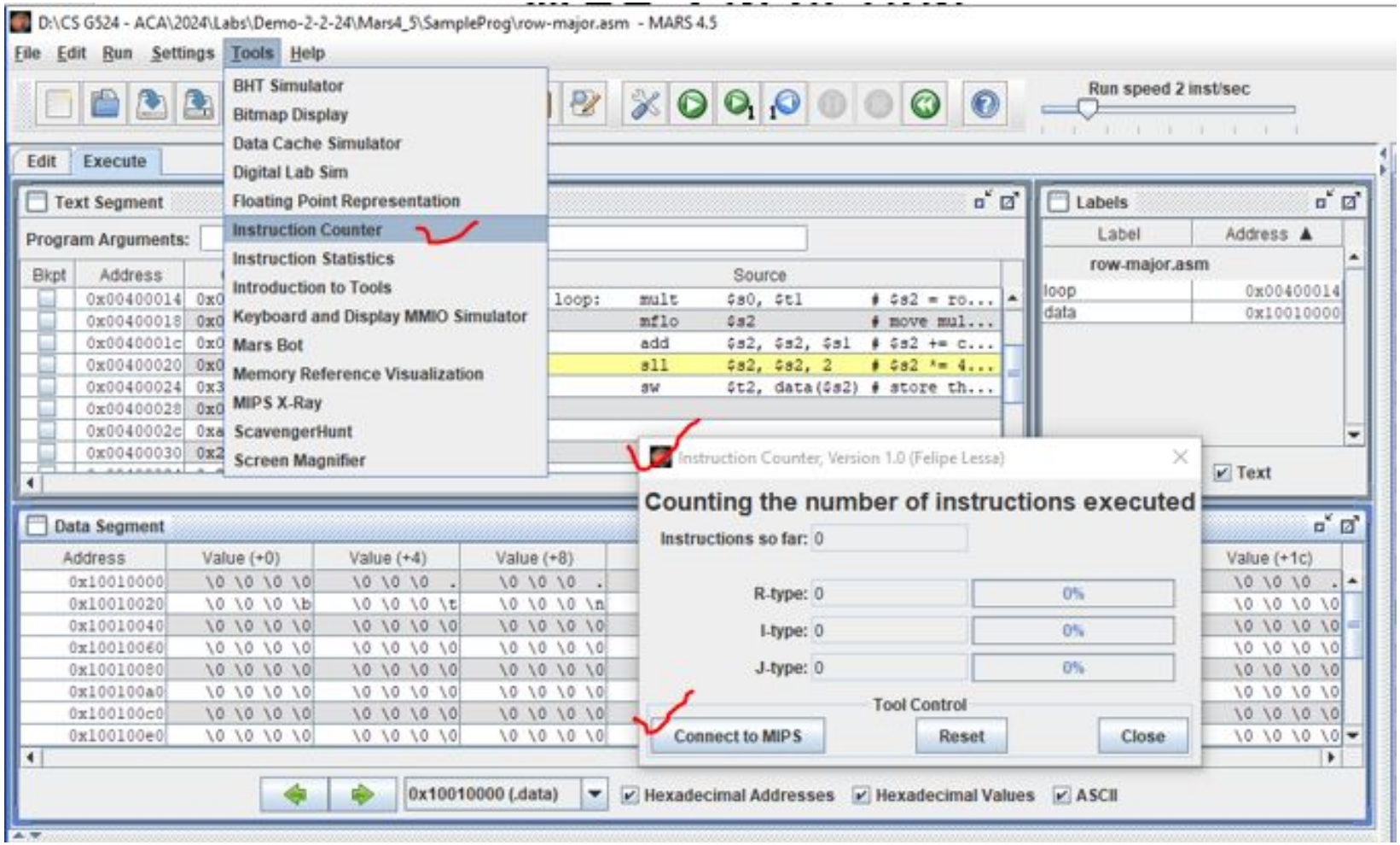
MARS in action: Settings menu



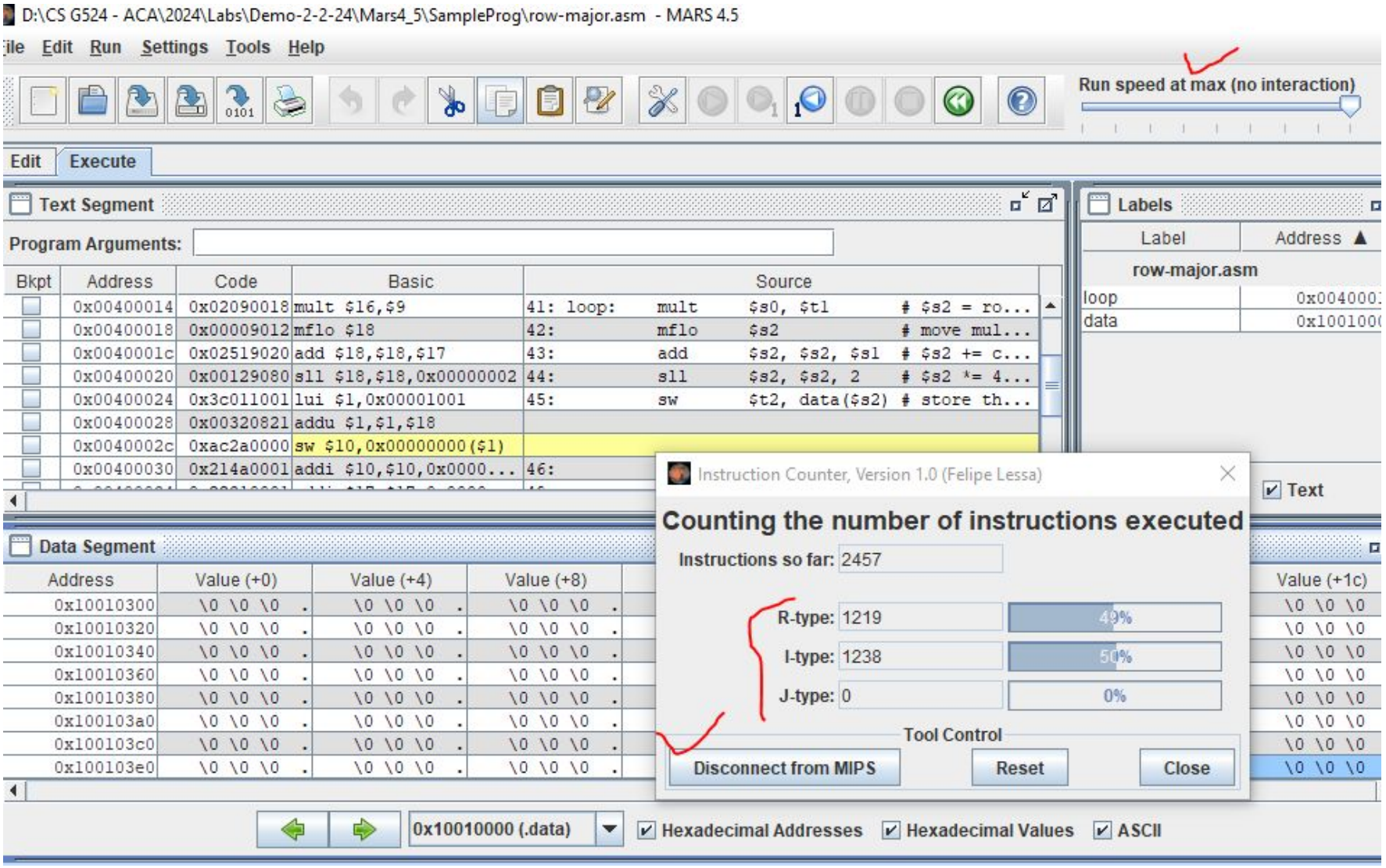
MARS in action



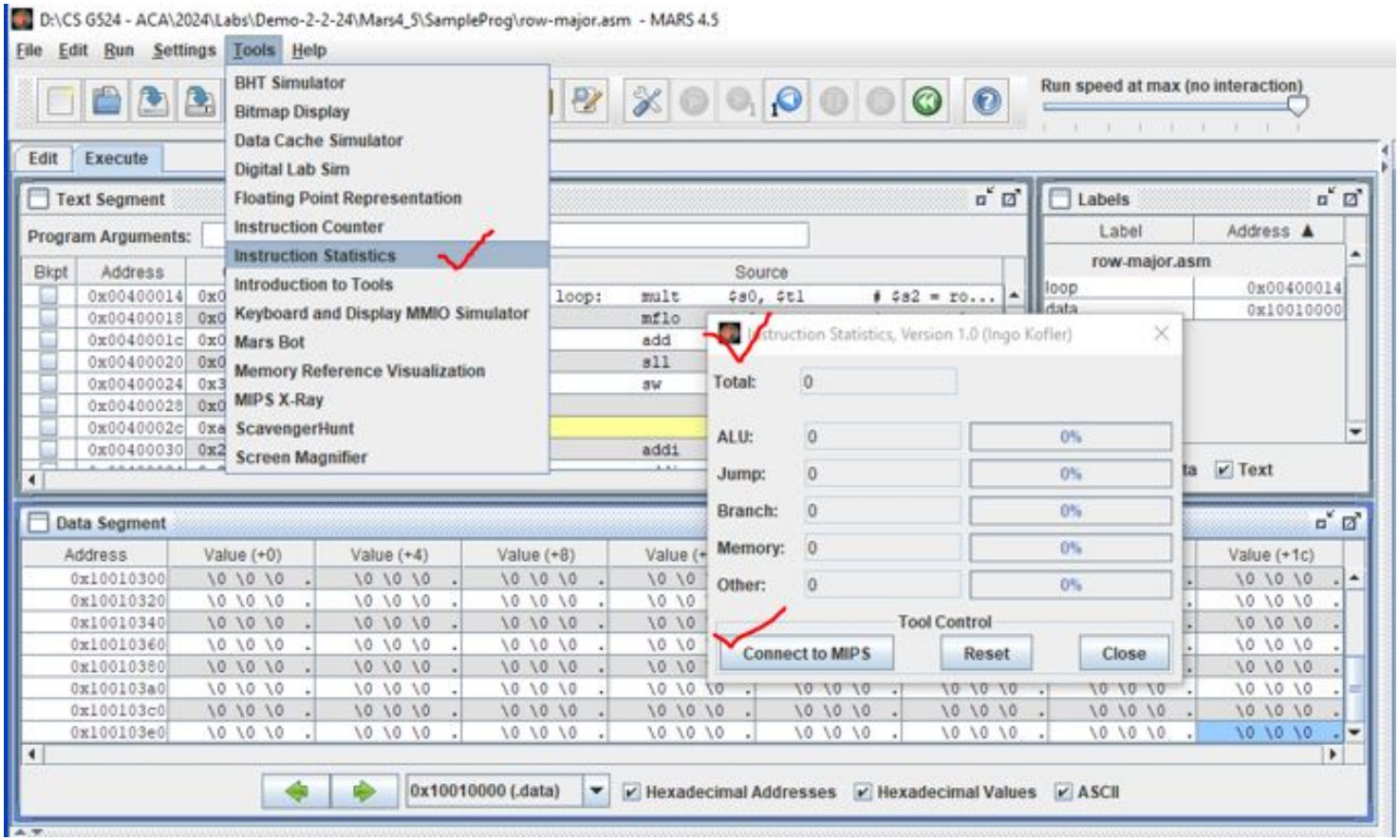
MARS in action: Count the number of instr executed



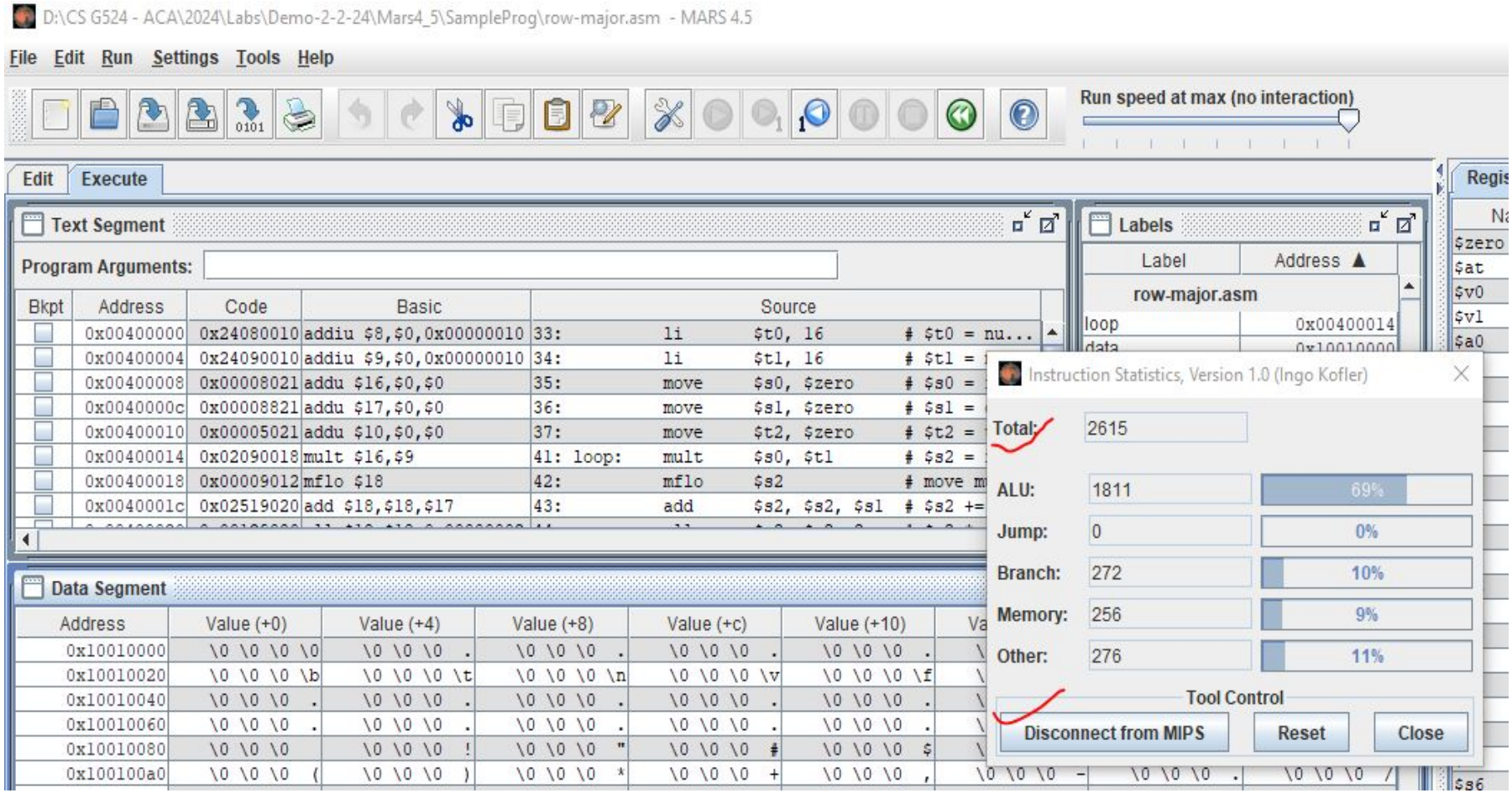
MARS in action: Count the number of instr executed



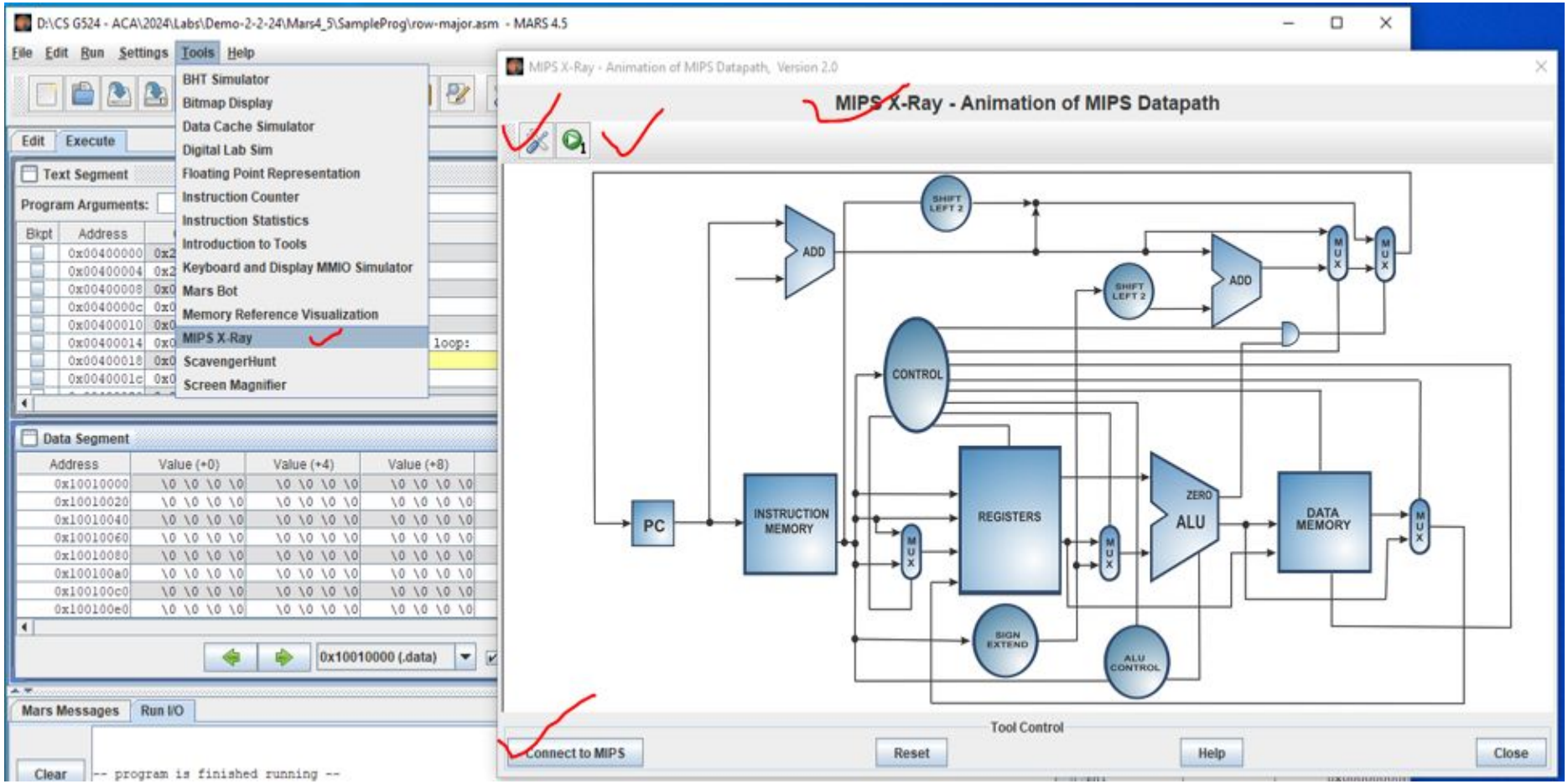
MARS in action: Instruction Statistics



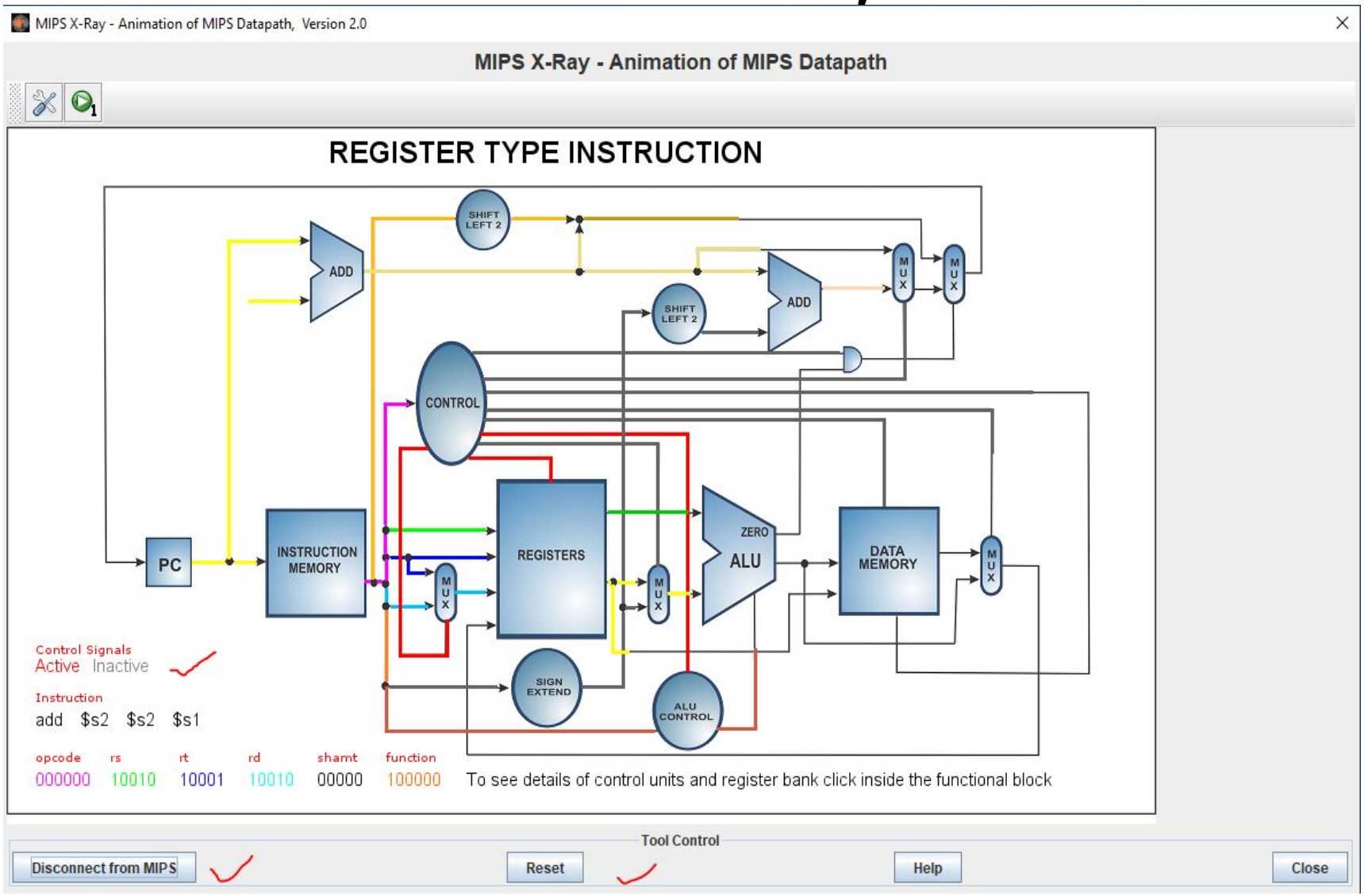
MARS in action: Instruction Statistics



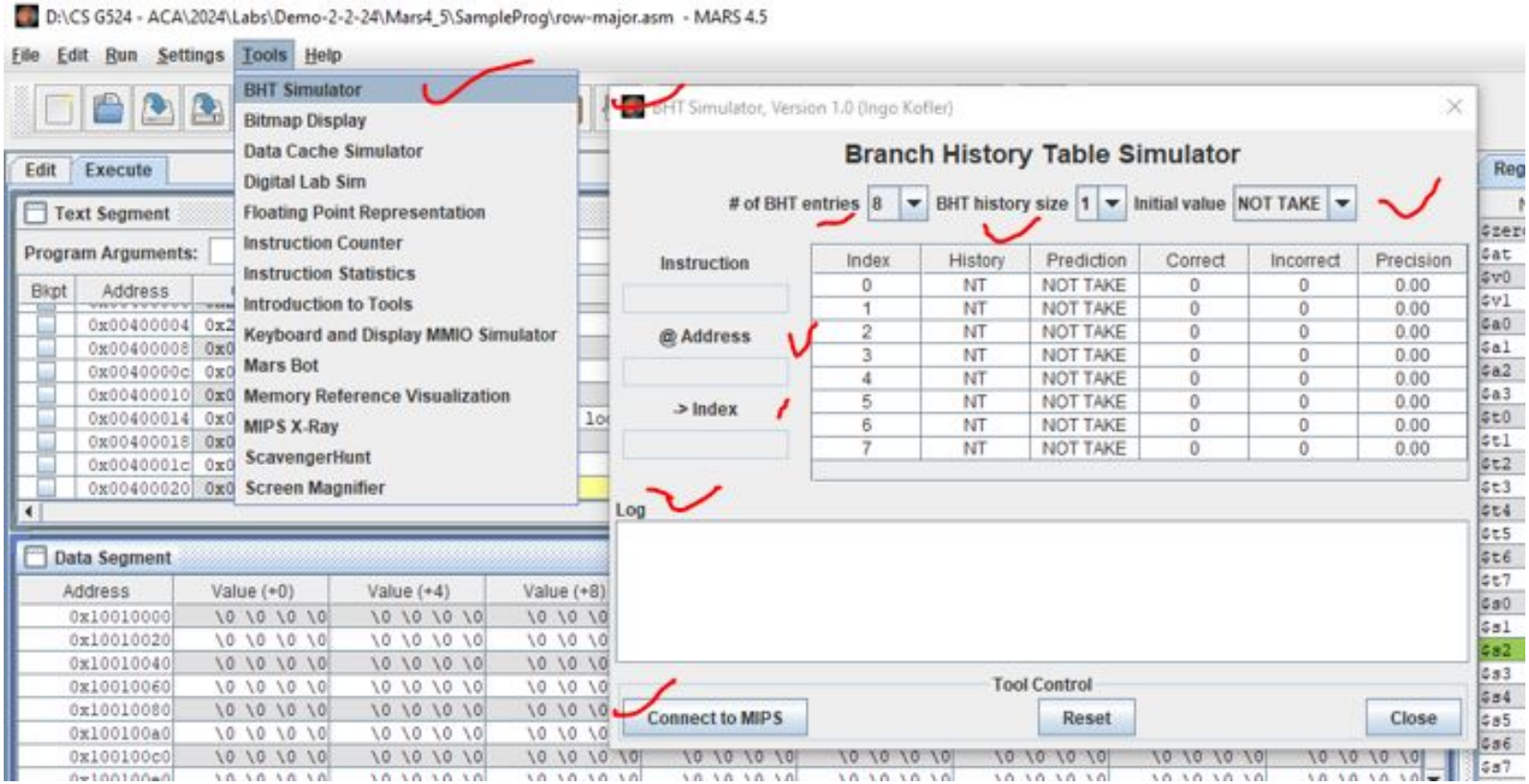
MARS in action: MIPS X-Ray



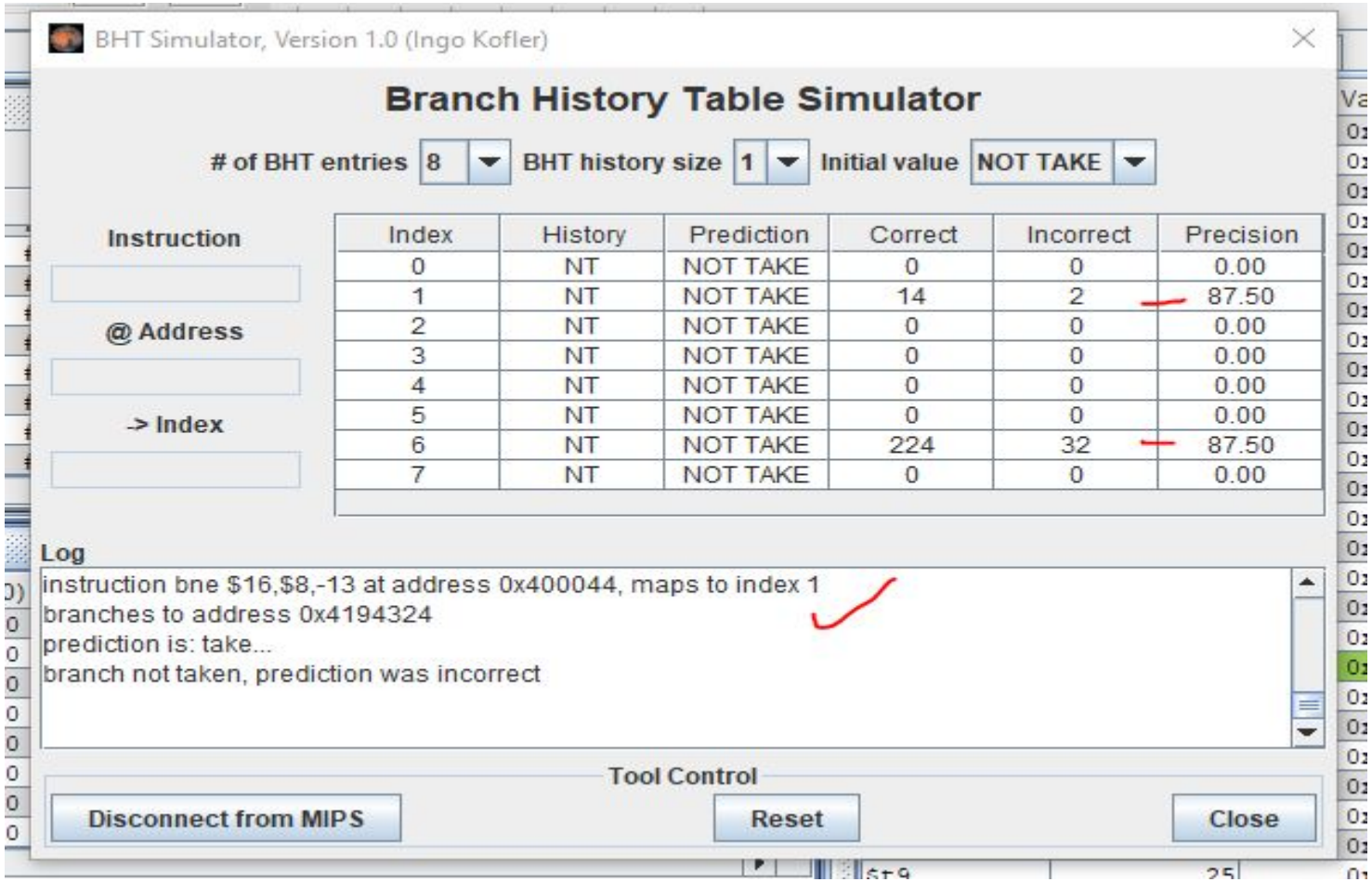
MARS in action: MIPS X-Ray



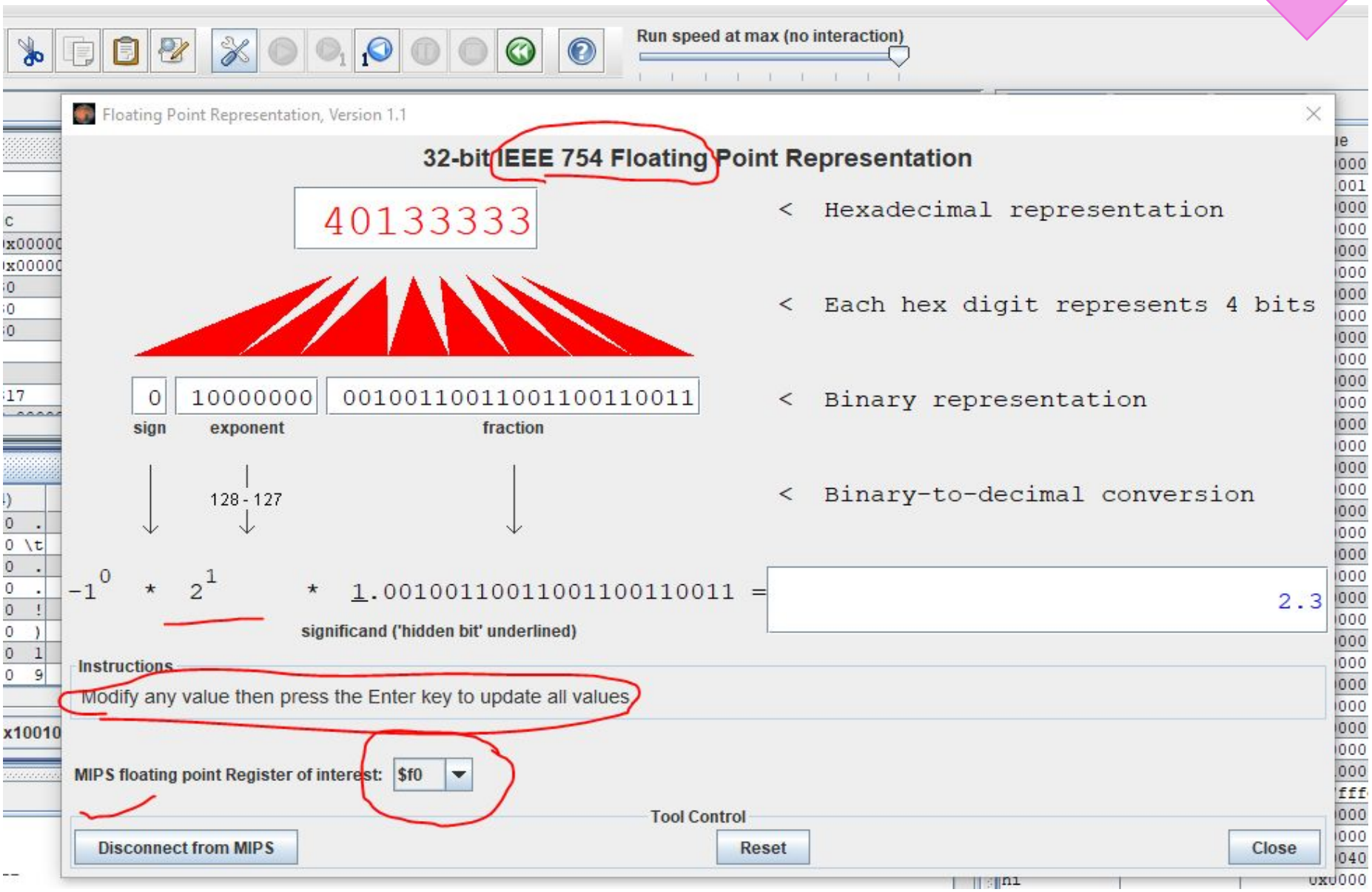
MARS in action: Branch History Table Simulator



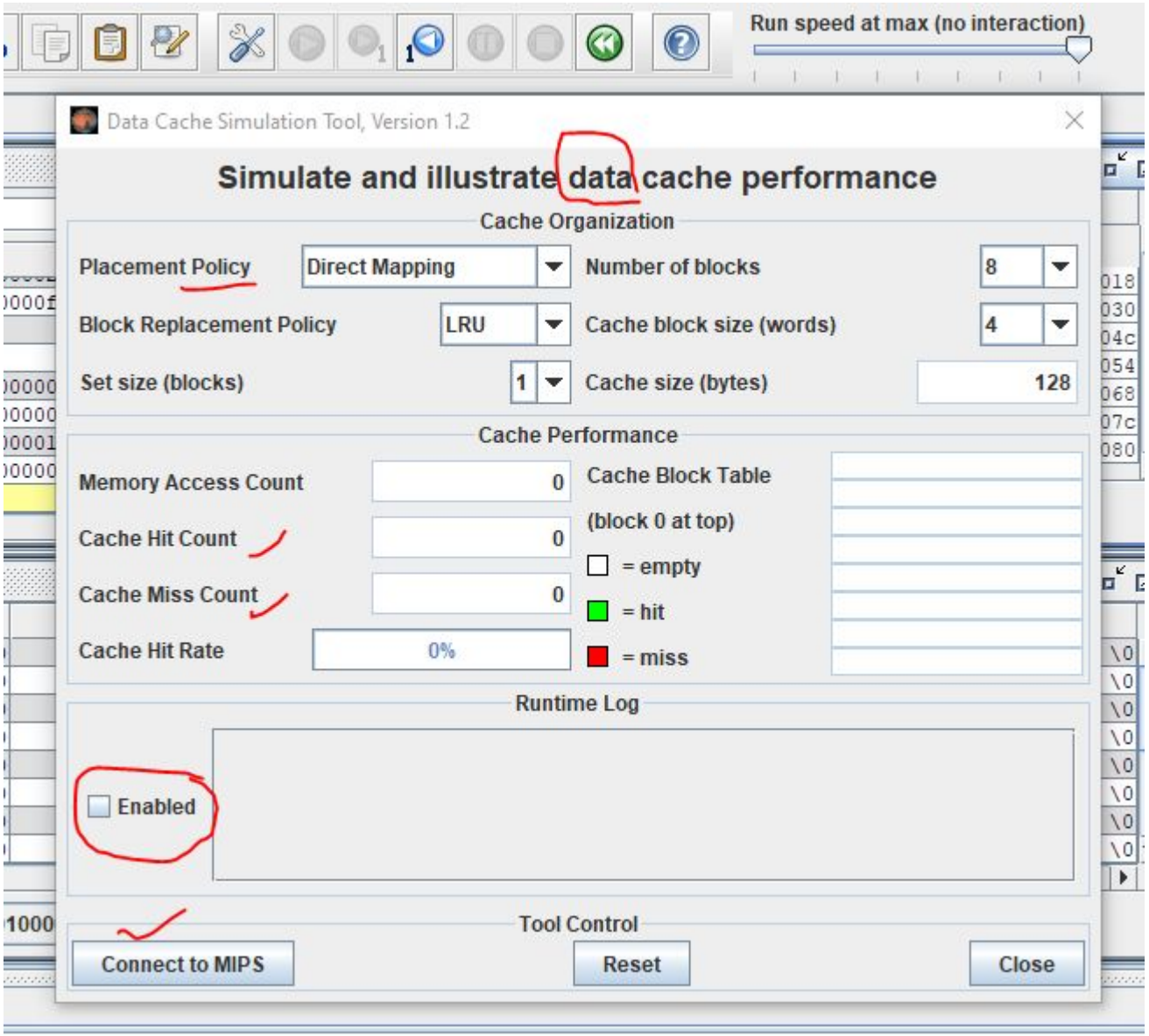
MARS in action: Branch History Table Simulator



MARS in action:



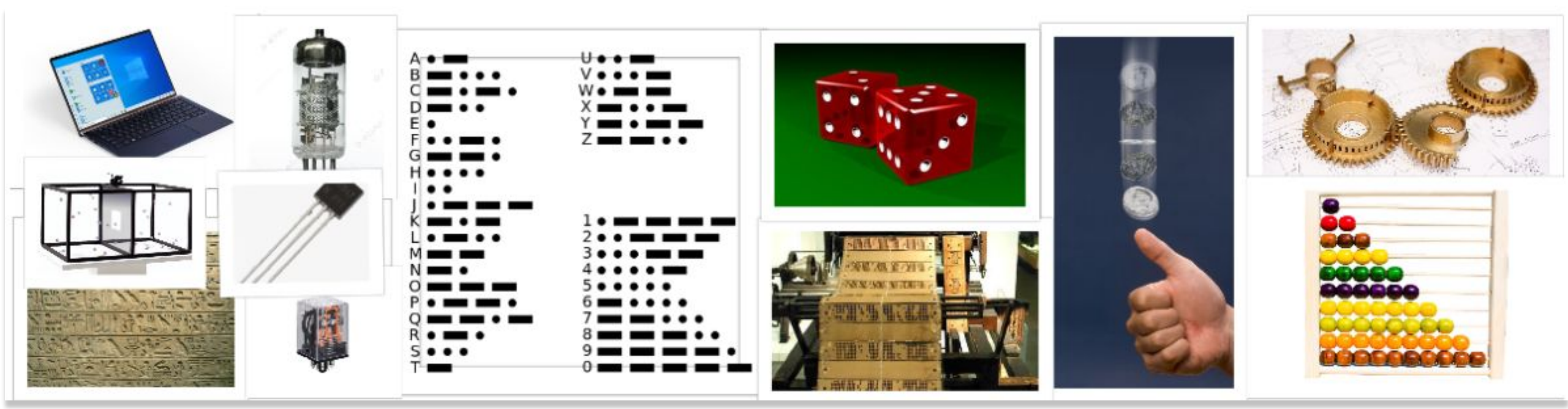
MARS in action:



MARS

- Documentation
  - book: appendix A9 (on CD) [https://pages.cs.wisc.edu/~larus/HP_AppA.pdf]
  - https://courses.missouristate.edu/KenVollmar/mars/tutorial.htm

- Tutorials:
  - https://www.youtube.com/playlist?list=PL5b07qlmA3P6zUdDf-o97ddfpvPFuNa5A
  - https://www.youtube.com/watch?v=d5amCbPUVYY&ab_channel=CSTutorials
  - https://www.it.uu.se/education/course/homepage/os/vt20/module-0/mips-and-mars/



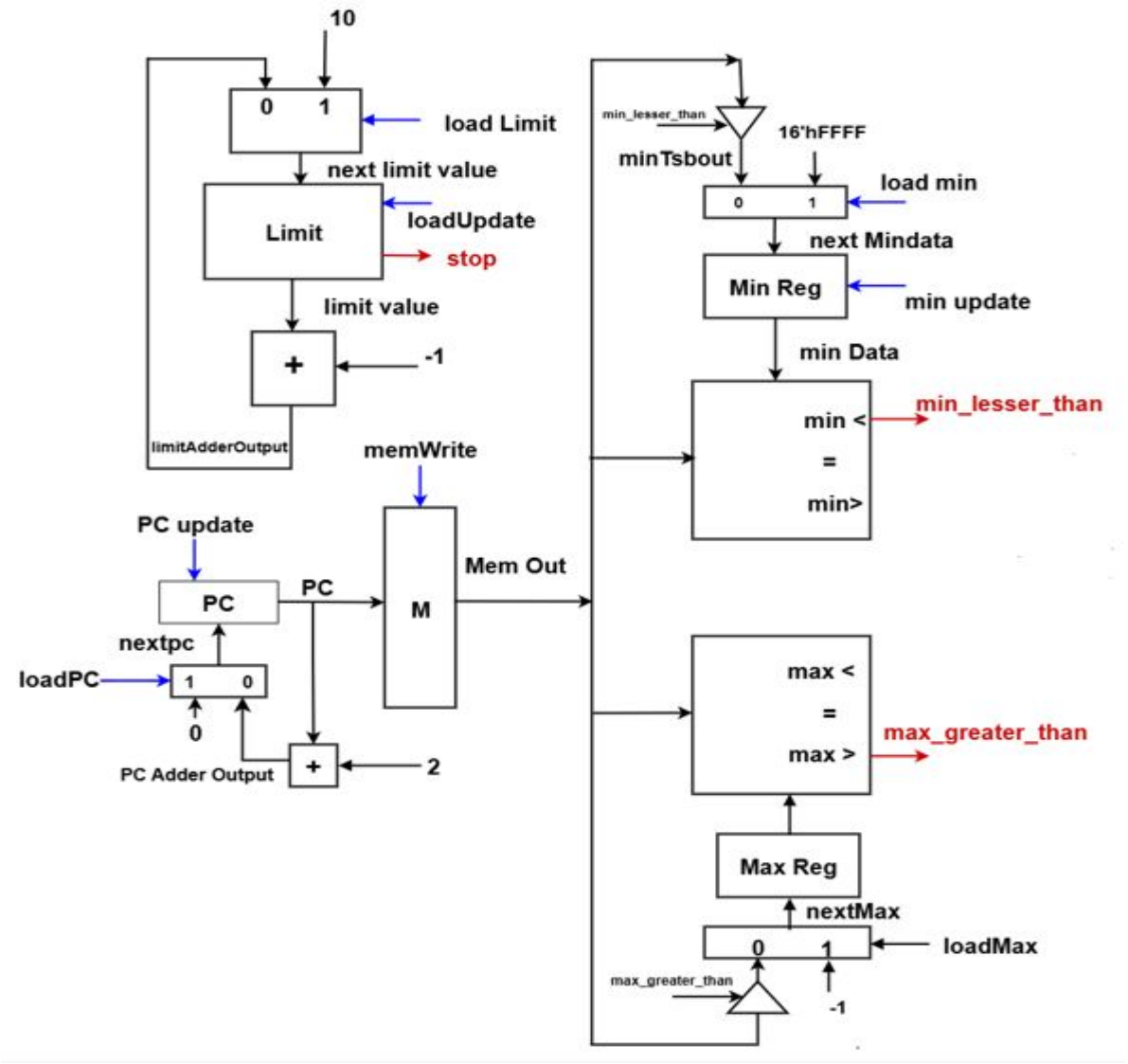
Computer Architecture (CS F342)

Design, Analysis, Optimization and Execution of Instructions

MinMax Processor

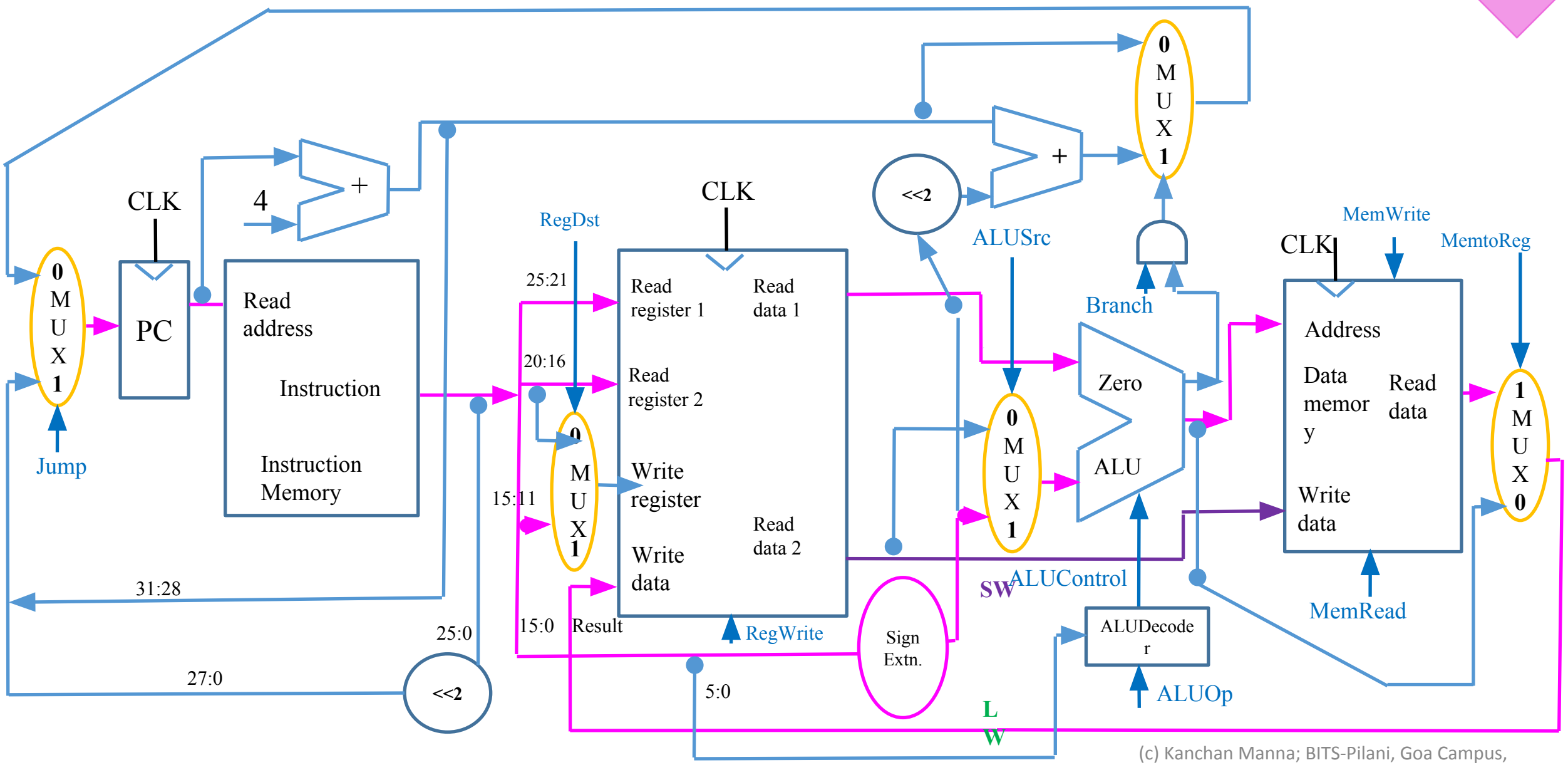
Special-purpose/Dedicated Processor
MinMax Processor

Input: set of n nos stored in A
Output: Min & Max
Min = ∞
Max = - ∞
Do scan i-th no.
    If Max < A [i] then
        Max = A [i]
    If Min > A [i] then
        Min = A [i]
Until i reaches to n
Stop



MinMax Processor

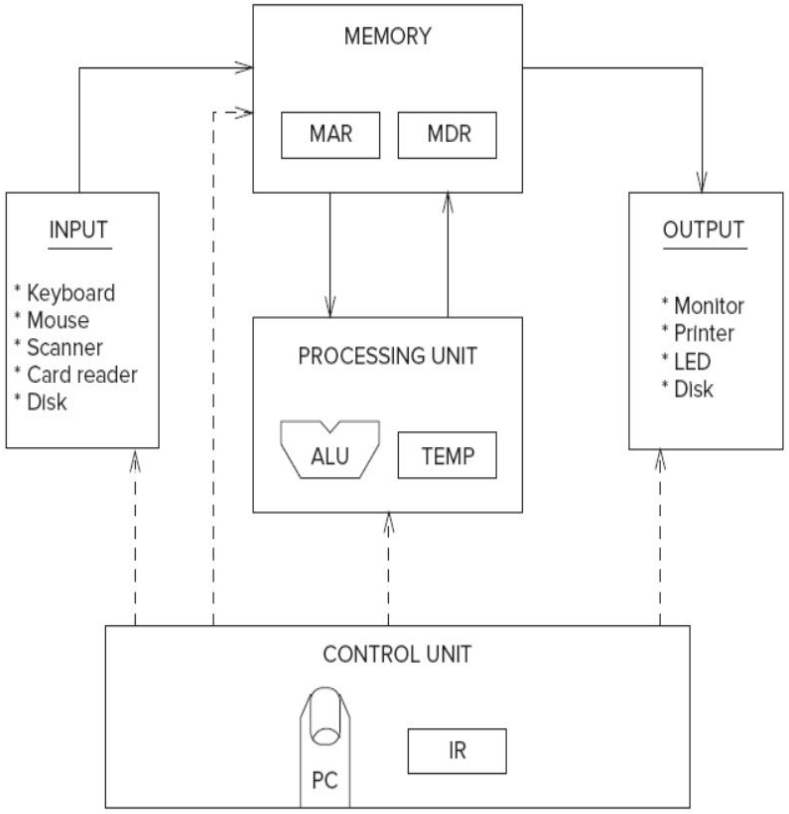
MIPS Microprocessor



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Von-Neumann or Princeton Model

- A fundamental model of a computer & all computers follow this model
- The model consists of 5-parts:
  - memory that stores instructions and data
  - a processing unit that contains an arithmetic logic unit and processor registers
  - input
  - output
  - a control unit that contains an instruction register and program counter
- The term von Neumann architecture has evolved to mean stored-program computer

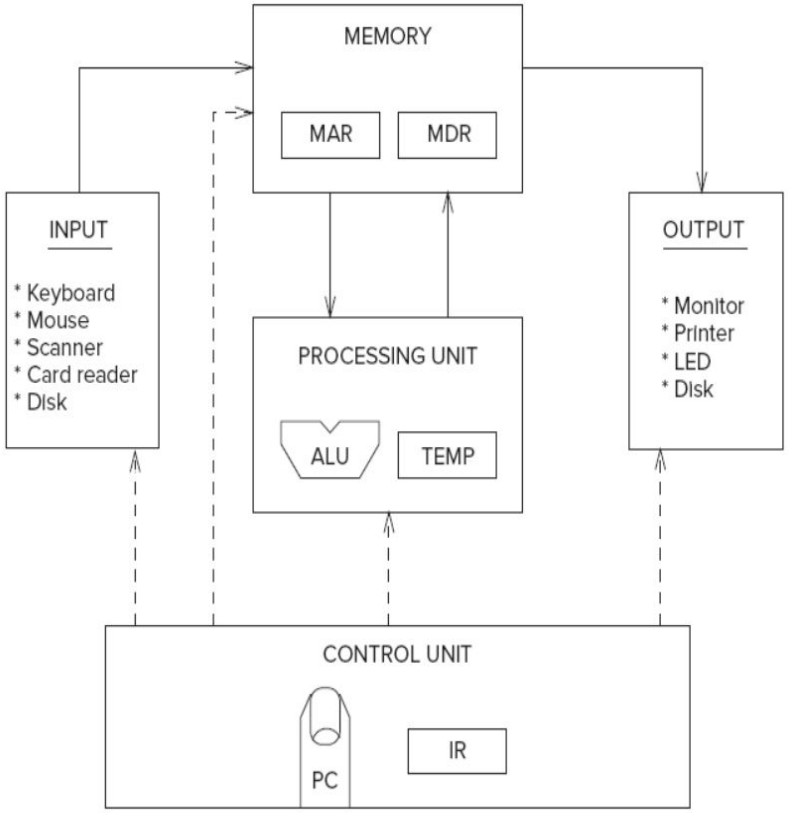


Book: Ch-4 of Introduction to computing systems from bits & gates to c/c++ & beyond by Yale Patt & Sanjay Patel

Von-Neumann or Princeton Model

- A von Neumann architecture stores program data and instruction data in same memory

- A computer with a Harvard architecture has separate memories for storing instructions (program) and data



The Instruction (executional) Cycle

- Clock cycle
- Instruction cycle

The Instruction Cycle (NOT the clock cycle)

- The six phases of the instruction cycle:
  - FETCH
  - DECODE
  - EVALUATE ADDRESS
  - FETCH OPERANDS
  - EXECUTE
  - STORE RESULT

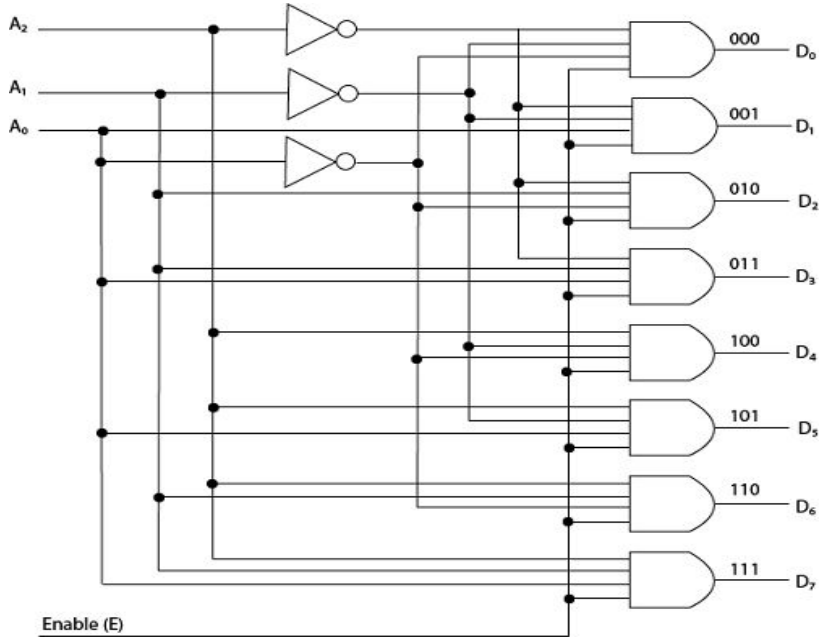
The Instruction Cycle (NOT the clock cycle)

- The FETCH phase takes the following steps:
- Step-1: Load the MAR with the contents of the PC, and simultaneously increment the PC.
- Step-2: Interrogate memory, resulting in the instruction being placed in the MDR.
- Step-3: Load the IR with the contents of the MDR.

Each of these steps is under the direction of the control unit. Time required to complete a step is called *machine cycle.*

The Instruction Cycle (NOT the clock cycle)

- The DECODE phase examines the instruction in order to figure out what the microarchitecture is being asked to do.
- How does one achieve that?
  - Using a decoder unit



The Instruction Cycle (NOT the clock cycle)

- The EVALUATE ADDRESS phase computes the address of the memory location that is needed to process the instruction.
- For an example, consider a scenario that the memory location of an instruction can be obtained by sign-extending bits [15:0] of the instruction to 32 bits and adding that value to the current contents of the PC. This calculation is performed during the EVALUATE ADDRESS phase.

The Instruction Cycle (NOT the clock cycle)

- In the FETCH OPERANDS phase, to obtain the source operands needed to process the instruction.
- For an example, instruction LD reads memory that resulted in the source operand being placed in MDR.

The Instruction Cycle (NOT the clock cycle)

- In the EXECUTE phase, carries out the execution of the instruction.
- In the ADD example, this phase consisted of the step of performing the addition in the ALU.

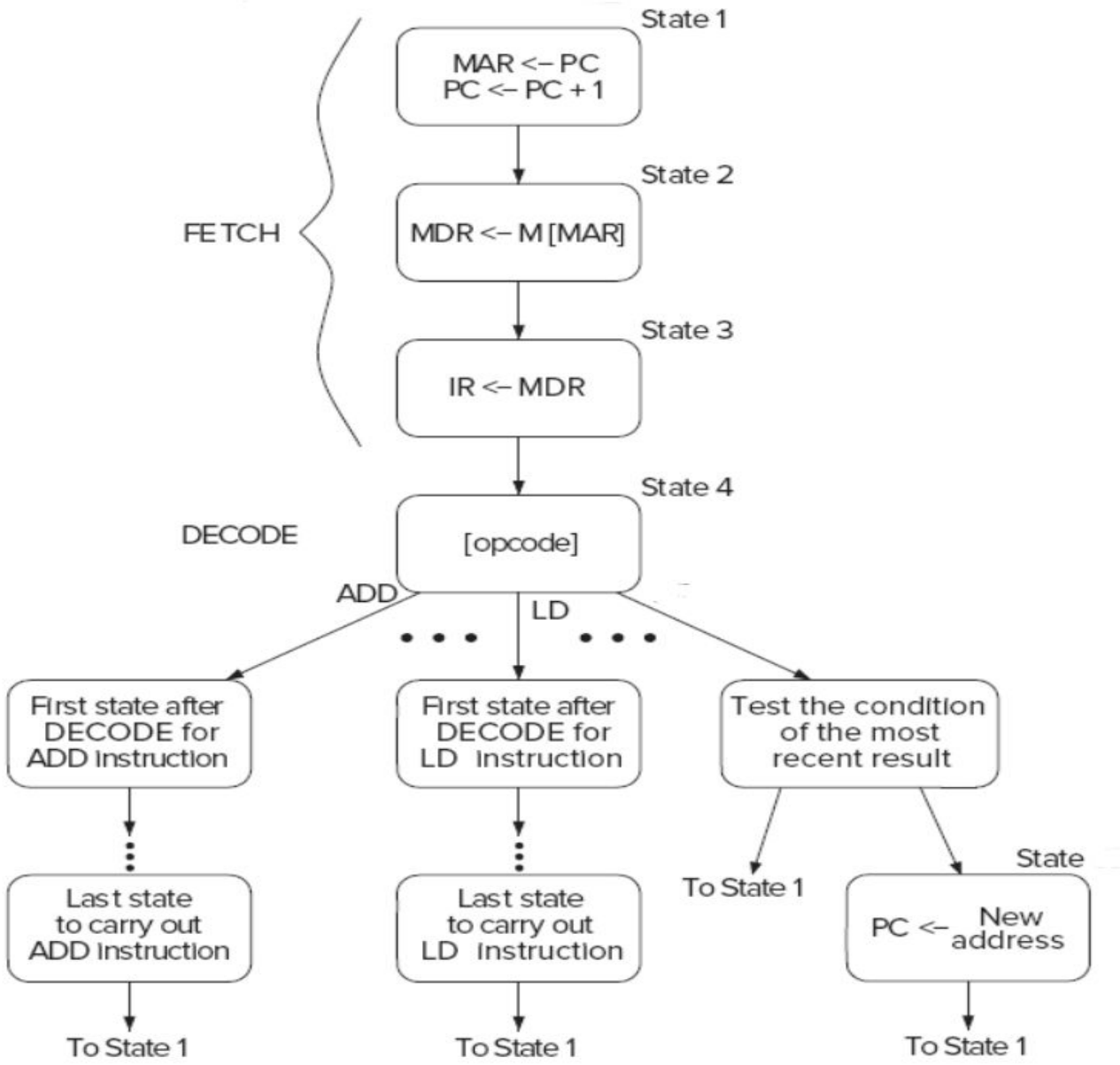
The Instruction Cycle (NOT the clock cycle)

- In the STORE RESULT phase, the result is written to its designated destination.
- The final phase of an instruction's execution.

The Instruction Cycle (NOT the clock cycle)

- Once the instruction cycle has been completed, the control unit begins the instruction cycle for the next instruction, starting from the top with the FETCH phase.
- Since the PC was updated during the previous instruction cycle, it contains at this point the address of the instruction stored in the next sequential memory location.
- Thus, the next sequential instruction is fetched next.
- Processing continues in this way until something breaks this sequential flow, or the program finishes execution.
- It is worth nothing again that although the instruction cycle consists of six phases, not all instructions require all six phases.

The Instruction Cycle (NOT the clock cycle)



Summary

- Building block of a program
- Von-Neumann Model
- Simple Memory model
- Instruction cycle
- Control unit
- Datapath



Computer Architecture (CS F342)

Design, Analysis, Execution and Optimization of Instructions

Reduced Instruction Set Computer (RISC): MIPS

Multicycle Datapath and Control unit

Book: COD by Patterson and Hennessy from Chapter 4 A

Fetch-and-Execute Algorithm for MIPS Microprocessor (32 bit)

```
#define OPCODE 0b11111100000000000000000000000000
#define RS     0b00000011111000000000000000000000
#define RT     0b00000000000111110000000000000000
#define RD     0b00000000000000001111100000000000
#define SHIFT  0b00000000000000000000011111000000
#define OFFSET 0b00000000000000001111111111111111

int PC, IMM[1024], DMM[1024], RF[32], ALUControl; bool ZERO;
Load(IMM, DMM);
Set PC with address 1st instruction which is stored in IMM;

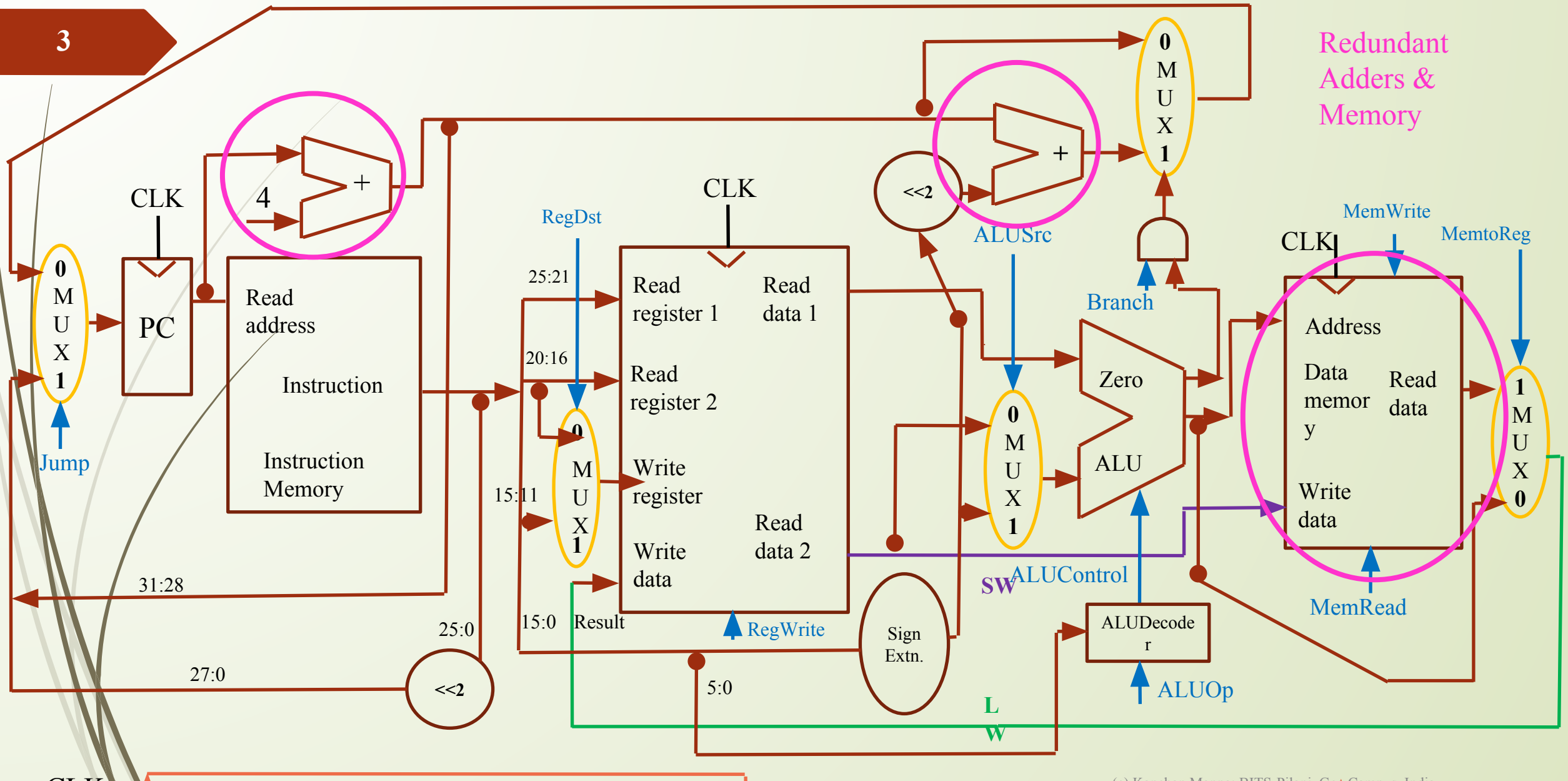
  while (1){
    switch((IMM[PC] & OPCODE) >>26){
      case R-type:
            Set ALUControl; //ADD, SUB, AND, OR, etc
            RF[(IMM[PC] & RD)>>11] = ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]); PC = PC + 4;
      case SW-type:
            Set ALUControl = 0b0010;
            DMM[ALU((IMM[PC] & RS)>>21, (IMM[PC] & OFFSET) ] = RF[(IMM[PC] & RT)>>16]; PC = PC + 4;
       case LW-type:
            Set ALUControl = 0b0010;
            RF[IMM[PC] & RT]= DMM[ALU((IMM[PC] & RS) >>21, IMM[PC] & OFFSET) ]; PC = PC + 4;
        case B-type:
            Set ALUControl = 0b0110;
            ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]);
            IF (ZERO ==1) PC = (PC + 4) + ((IMM[PC] & OFFSET) <<2); ELSE PC = PC + 4;
    }
  }
```

```
ALU(Src1, Src2){
      switch (ALUControl){
          case B-type: ZERO = (Src1- Src2) == 0 ? 1: 0;
                       break;
          case ADD: return (Src1 + Src2); break;
            …
      }
}
```

Need conversion for 16 bit offset to 32 bits

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Single-cycle datapath



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

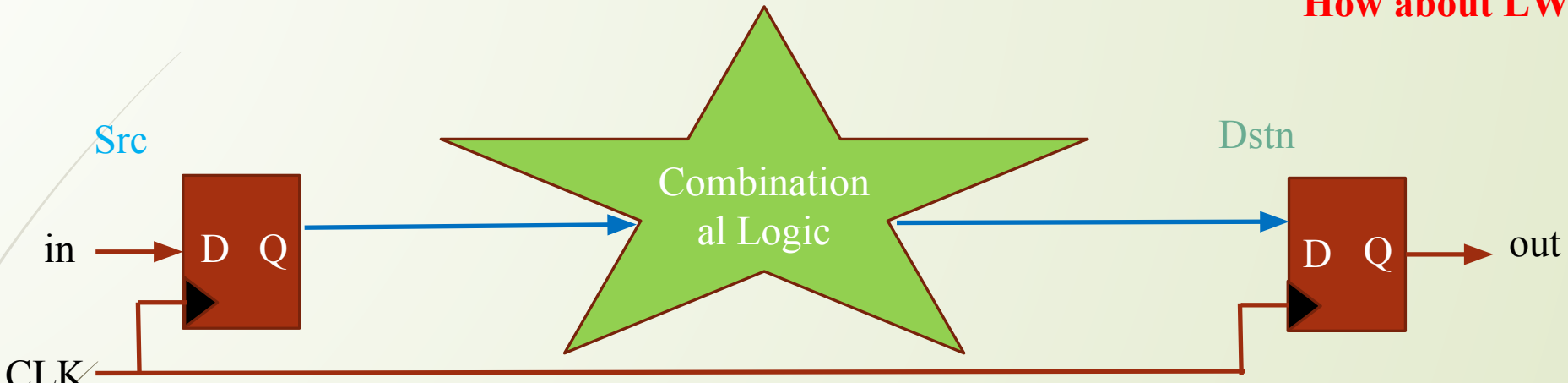
Performance analysis

- The execution time of a program is a metric
  - $Execution\ Time = (\#instructions)\left(\frac{Cycles}{instruction}\right)\left(\frac{Seconds}{cycle}\right)$
- #instructions or length of a program depends on ISA
- How did we measure the clock period?
- Which instruction was dominating the clock period?
- **How can we reduce the clock period or improve the frequency (f=1/T)?**

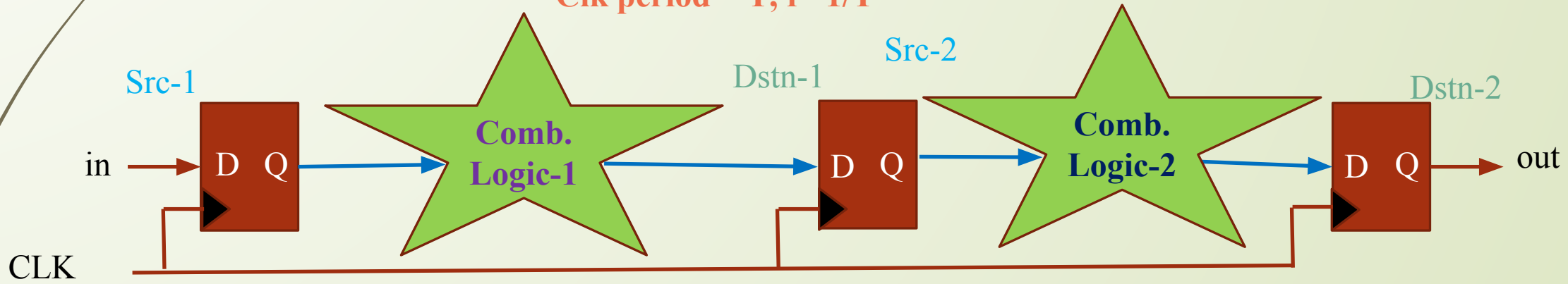
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Improvement on clock period

How about LW?



Clk period = T, f =1/T



T =Max{ (src-1, dstn-1), (src-2, dstn-2) }

Clk period = T/2, f = 2/T

This is not the pipeline structure

Combinational Logic = Combinational Logic-1 U Combinational Logic-2

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Placement of Registers/ Partition the logic

- How does one place the registers in the datapath?
- T = Max{ (src-1, dstn-1), (src-2, dstn-2), …}
- What Max{ } is telling us?
- Each partition can take almost equal time for computation

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Problems of Single-cycle Datapath Design

- Single-cycle design works well but inefficient design
- Clock length (worst-case delay) is same for all instructions
- It is not a **balanced** design
- CPI is 1
- Use more resources:
  - Adder
  - Memory
- Necessity of balanced datapath design technique by focusing on common-case design & analysis principle

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Balanced datapath design: Multi-cycle approach

- Instruction execution can be broken down to smaller steps
- Simple instruction can complete the execution earlier than the complex instructions
- One can design the multi-cycle datapath as similar in single-cycle
  - Connecting architectural elements with **the storage** using combinational logic
  - Next, design the controller

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Balanced datapath design: Multi-cycle approach

- The key difference is
  - Controller produces different signals on different steps/states
  - A finite state machine (FSM) approach

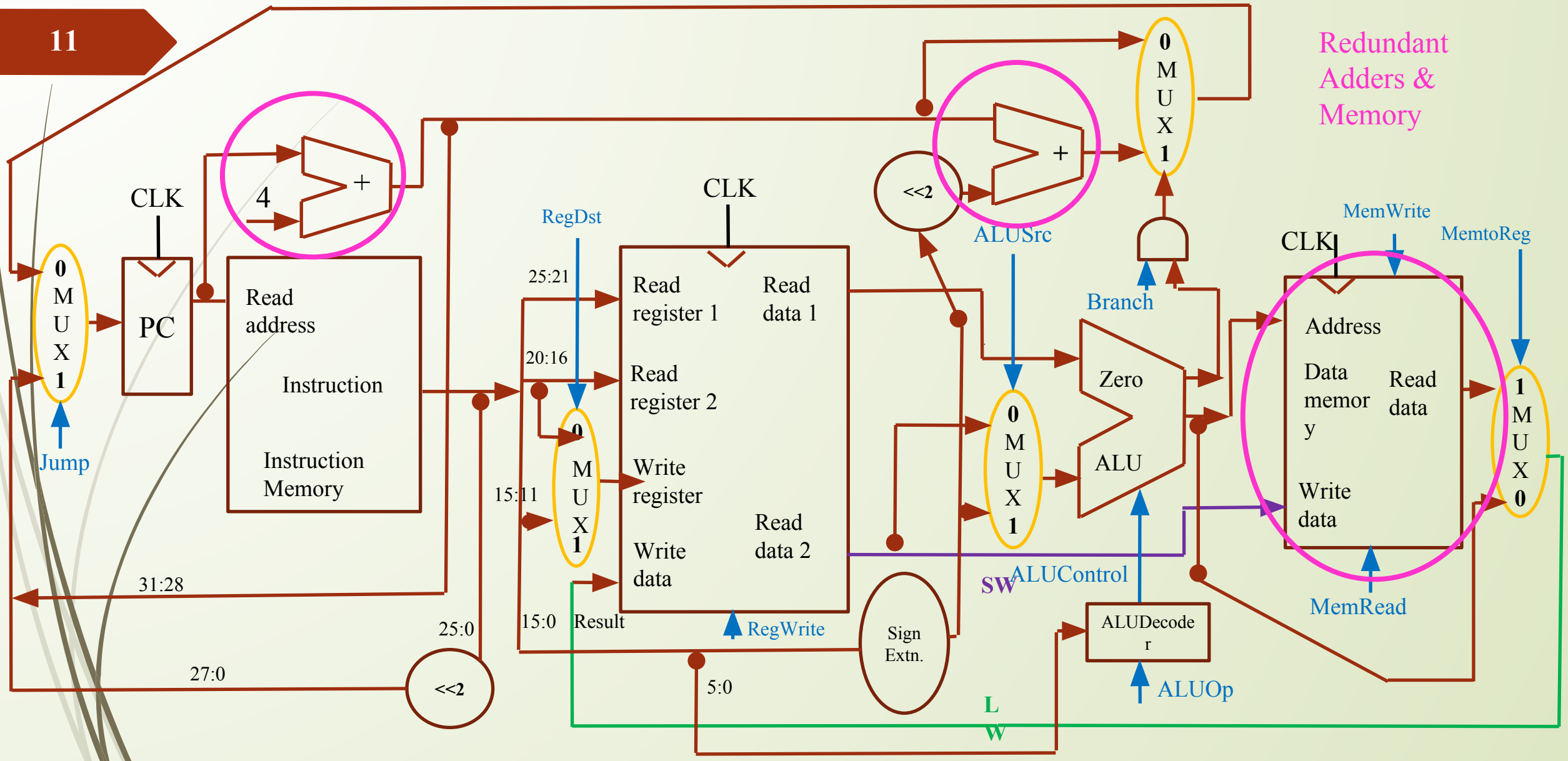
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Balanced datapath design: Multi-cycle approach

- Combined the instruction and data memory
- Remove the redundant adders
- Incorporate the **non-architectural state/storage** elements
  - Not visible to the programmer

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Single-cycle datapath



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Balanced datapath design: Multi-cycle approach

- Remove the redundant adders & memory
- Where do we place this operations?
- How does one control such operations?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Fetch-and-Execute Algorithm for MIPS Microprocessor (32 bit)

```
#define OPCODE 0b11111100000000000000000000000000
#define RS     0b00000011111000000000000000000000
#define RT     0b00000000000111110000000000000000
#define RD     0b00000000000000001111100000000000
#define SHIFT  0b00000000000000000000011111000000
#define OFFSET 0b00000000000000001111111111111111

int PC, IMM[1024], DMM[1024], RF[32], ALUControl; bool ZERO;
Load(IMM, DMM);
Set PC with address 1st instruction which is stored in IMM;

  while (1){
    switch((IMM[PC] & OPCODE) >>26){
      case R-type:
            Set ALUControl; //ADD, SUB, AND, OR, etc
            RF[(IMM[PC] & RD)>>11] = ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]); PC = PC + 4;
      case SW-type:
            Set ALUControl = 0b0010;
            DMM[ALU((IMM[PC] & RS)>>21, (IMM[PC] & OFFSET) ] = RF[(IMM[PC] & RT)>>16]; PC = PC + 4;
       case LW-type:
            Set ALUControl = 0b0010;
            RF[IMM[PC] & RT]= DMM[ALU((IMM[PC] & RS) >>21, IMM[PC] & OFFSET) ]; PC = PC + 4;
        case B-type:
            Set ALUControl = 0b0110;
            ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]);
            IF (ZERO ==1) PC = (PC + 4) + ((IMM[PC] & OFFSET) <<2); ELSE PC = PC + 4;
    }
  }
```

```
ALU(Src1, Src2){
      switch (ALUControl){
          case B-type: ZERO = (Src1- Src2) == 0 ? 1: 0;
                       break;
          case ADD: return (Src1 + Src2); break;
            …
      }
}
```

Need conversion for 16 bit offset to 32 bits

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Key points: Resource Sharing leads to Multi-Cycle Approach

- Datapath components are shared
  - Functional units
  - Memory
  - Interconnect
- Balanced datapath design

Book: COD by Patterson and Hennessy from Chapter **4 A**

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Fetch-and-Execute Algorithm for MIPS Microprocessor (32 bit)

```
#define OPCODE 0b11111100000000000000000000000000
#define RS     0b00000011111000000000000000000000
#define RT     0b00000000000111110000000000000000
#define RD     0b00000000000000001111100000000000
#define SHIFT  0b00000000000000000000011111000000
#define OFFSET 0b00000000000000001111111111111111

int PC, MM[1024],  RF[32], ALUControl; bool ZERO;
int IR, A, B, ALUOut, MDR; //non-architectural elements
Load(MM);
Set PC with address 1st instruction which is stored in IMM;
   while (1){ IR = MM[PC]; ALUControl = 0b0010;PC = ALU(PC, 4);
      A = RF[(IR & RS)>>21]; B = RF[(IR & RT) >>16];
      switch((IR & OPCODE) >>26){
         case R-type:
                Set ALUControl; //ADD, SUB, AND, OR, etc
                ALUOut = ALU(A, B); RF[(IR & RD)>>11] = ALUOut;
         case LW-type:
                ALUControl = 0b0010;
                ALUOut = ALU(A, (IR & OFFSET)); MDR = MM[ALUOut];   RF[(IR & RT)>>16]= MDR;
         case SW-type:
                ALUControl = 0b0010;
                ALUOut = ALU(A, (IR & OFFSET)); MM[ALUOut] = B;
          case B-type: ALUControl = 0b0010;
                ALUOut = ALU(PC, (IR & OFFSET) <<2);
                ALUControl = 0b0110; ALU(A, B);
                IF (ZERO ==1) {PC = ALUOut;}
      }
   }
```

```
ALU(Src1, Src2){
        switch (ALUControl){
           case B-type: ZERO = (Src1- Src2) == 0 ? 1: 0; break;
           case ADD: return (Src1 + Src2);
                …
        }
}
```

Need conversion for 16 bit offset to 32 bits

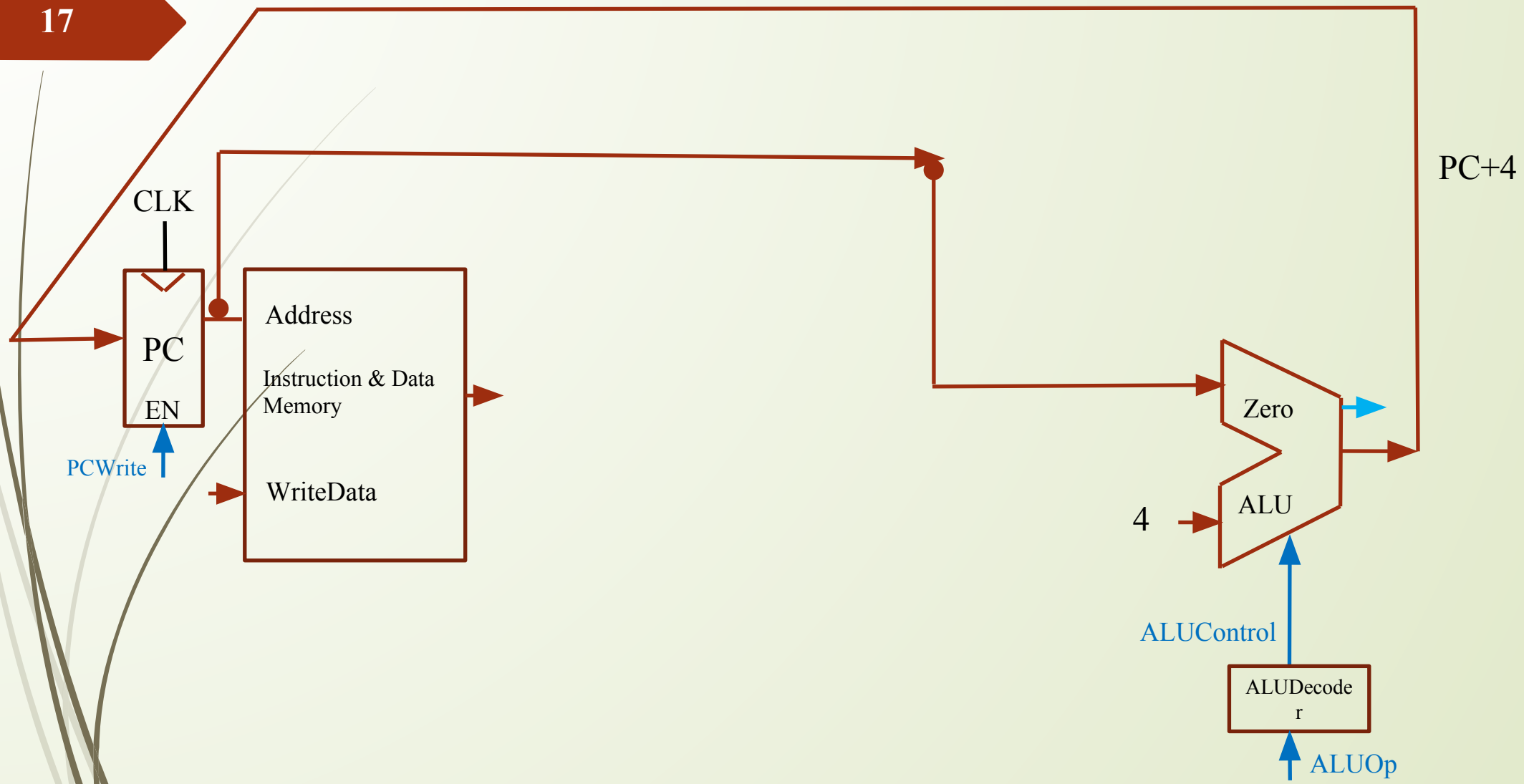
Multi-Cycle Approach

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Single memory unit, **Fetch stage** and datapath for reusing the ALU

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Single memory unit, Fetch stage and datapath for reusing the ALU



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Balanced datapath design: Multi-cycle approach

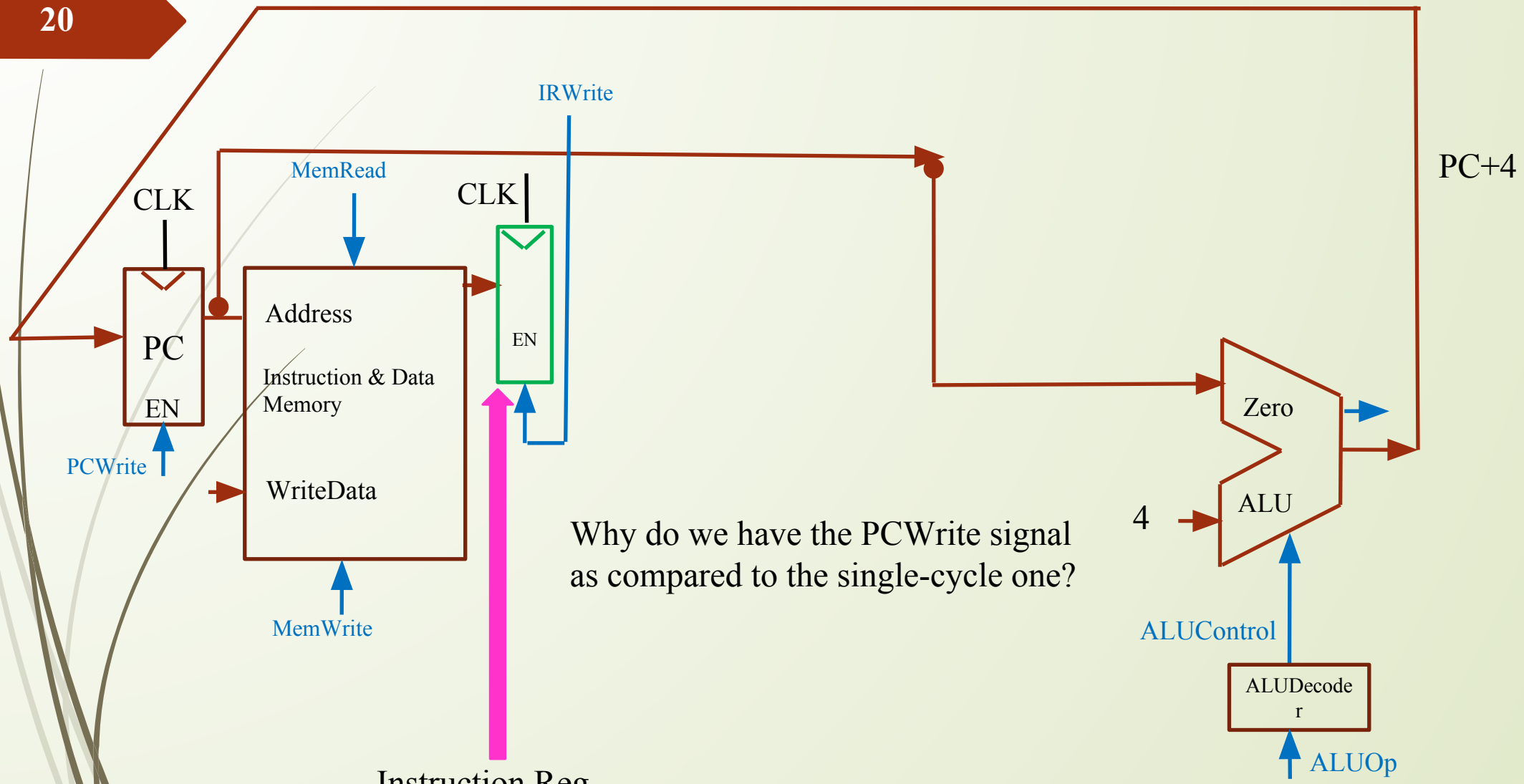
- Combined the instruction and data memory
- Remove the redundant adders
- Incorporate the **non-architectural state/storage** elements

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Fetch stage and **Non-architectural** elements Instruction register

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Fetch stage and **Non-architectural** elements Instruction register



Why do we have the PCWrite signal as compared to the single-cycle one?

1: IR = M[PC]; PC = PC + 4

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

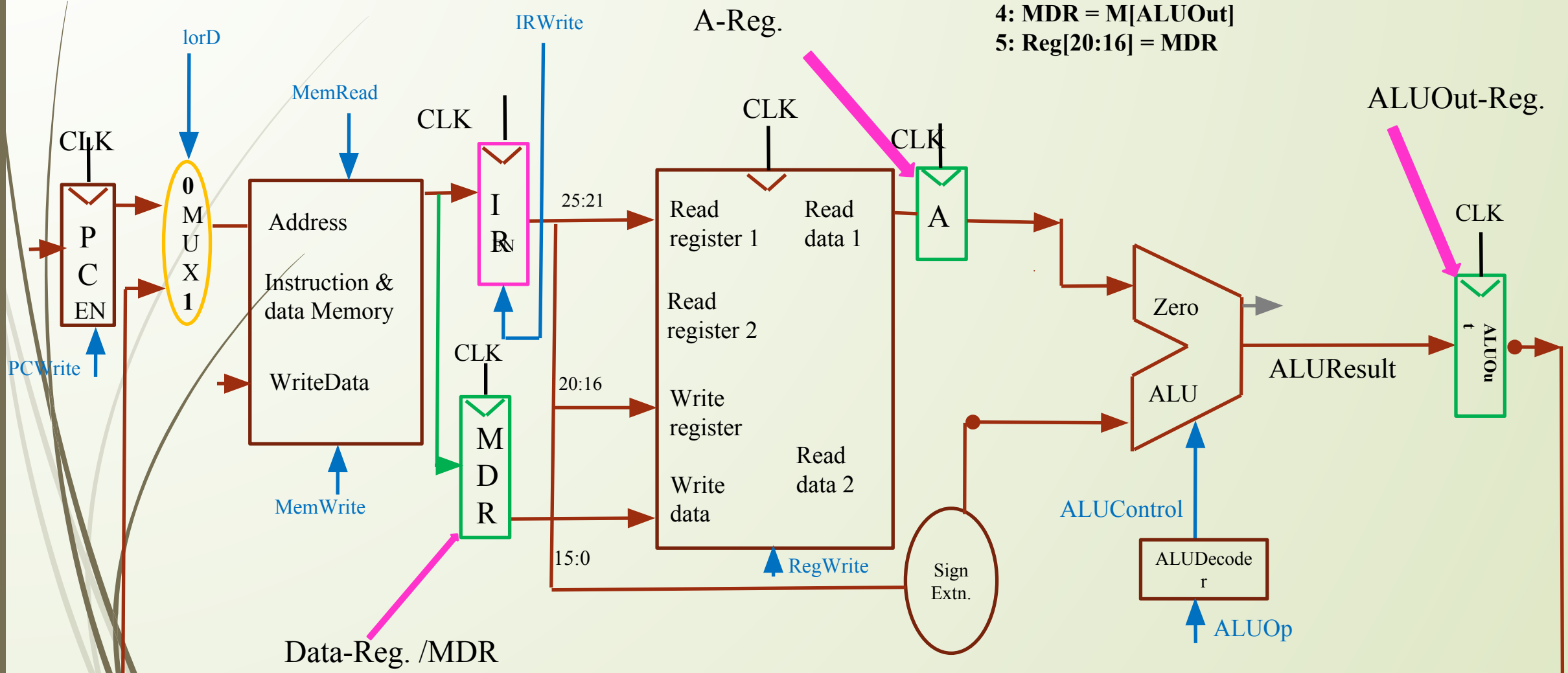
lw-instr. and **non-architectural elements**: A, Data and ALUOut-register

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21];
3: ALUOut = A + SignExtn(Imm)
4: MDR = M[ALUOut]
5: Reg[20:16] = MDR

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

lw-instr. and **non-architectural elements**: A, Data and ALUOut-register

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21];
3: ALUOut = A + SignExtn(Imm)
4: MDR = M[ALUOut]
5: Reg[20:16] = MDR



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

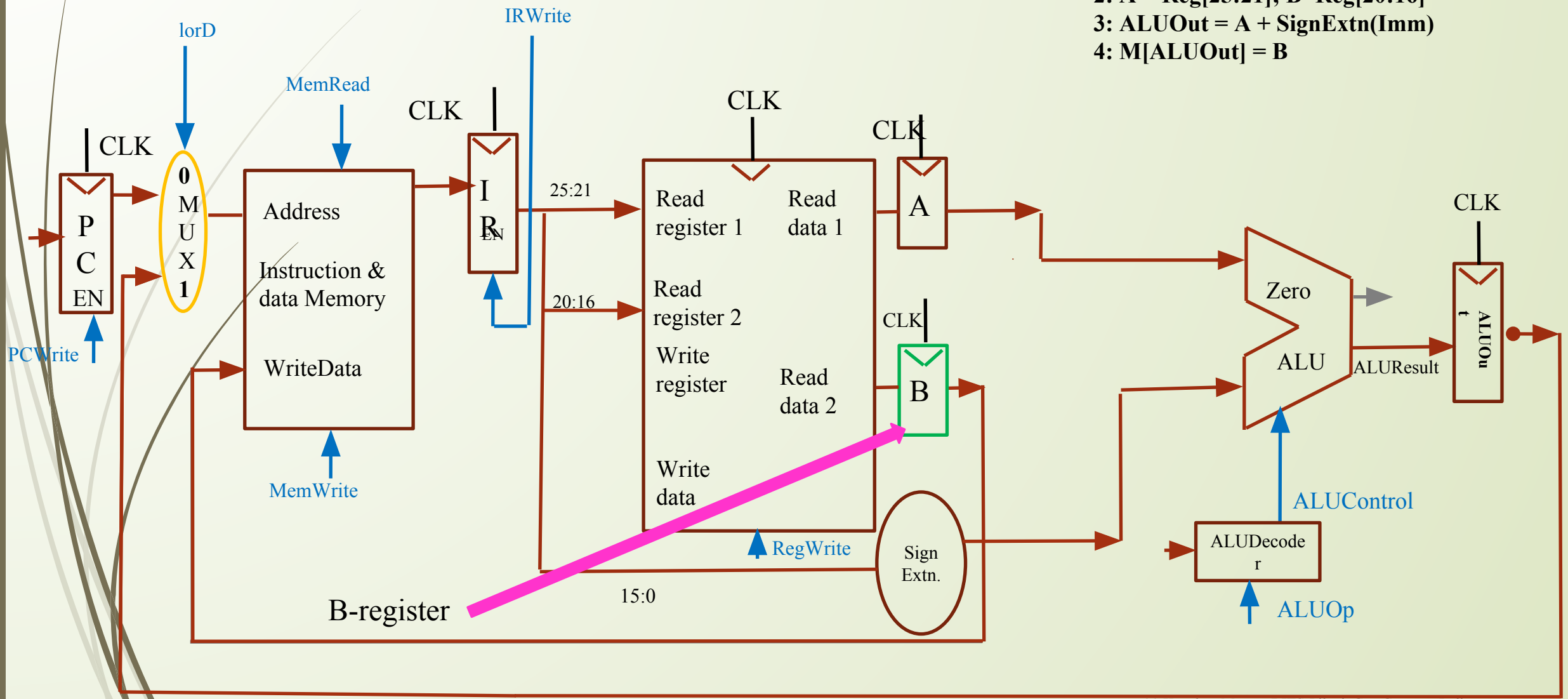
sw-instr and non-architectural elements: B-register

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21]; B=Reg[20:16]
3: ALUOut = A + SignExtn(Imm)
4: M[ALUOut] = B

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

sw-instr and non-architectural elements: B-register

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21]; B=Reg[20:16]
3: ALUOut = A + SignExtn(Imm)
4: M[ALUOut] = B



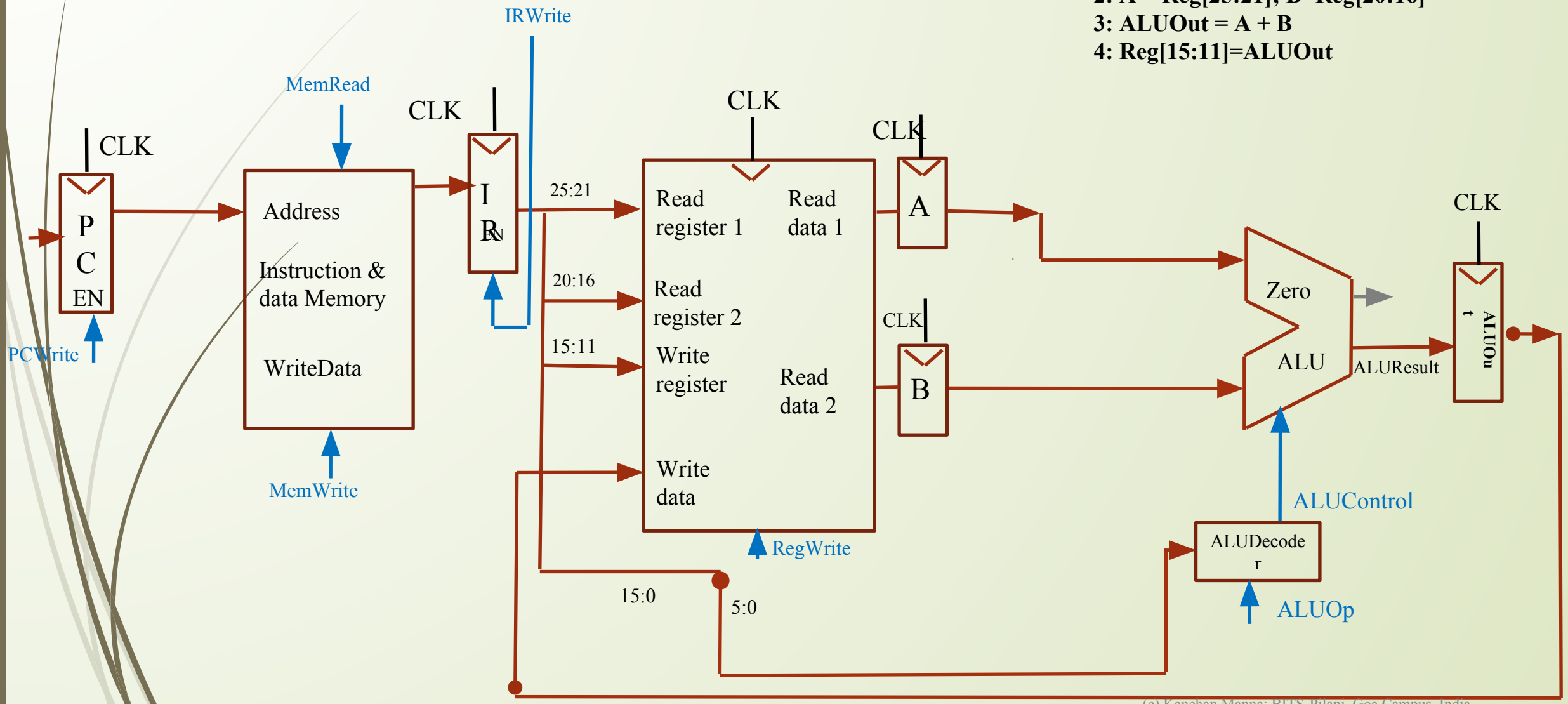
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

R-type instruction

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21]; B=Reg[20:16]
3: ALUOut = A + B
4: Reg[15:11]=ALUOut

R-type instruction

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21]; B=Reg[20:16]
3: ALUOut = A + B
4: Reg[15:11]=ALUOut



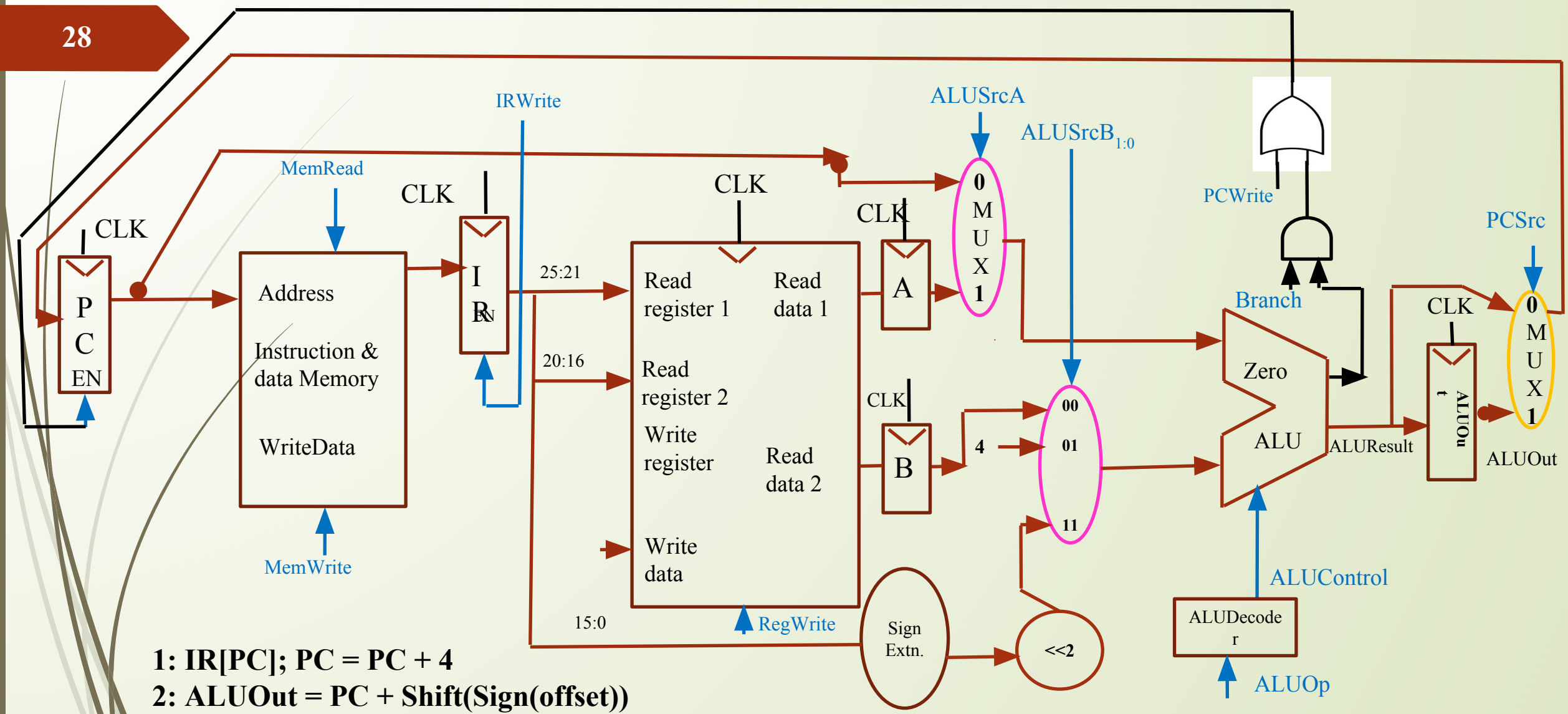
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

BEQ R1, R2, offset; if true, PC =PC+4+offset
else PC = PC+4

1: IR[PC]; PC = PC + 4
2: ALUOut = PC + Shift(Sign(offset))
3: PC = ALUOut if Zero == 1 //A-B

Fetch stage and B(R)-type instruction

BEQ R1, R2, offset; if true, PC =PC+4+offset
else PC = PC+4



1: IR[PC]; PC = PC + 4
2: ALUOut = PC + Shift(Sign(offset))
3: PC = ALUOut if Zero == 1 //A-B

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

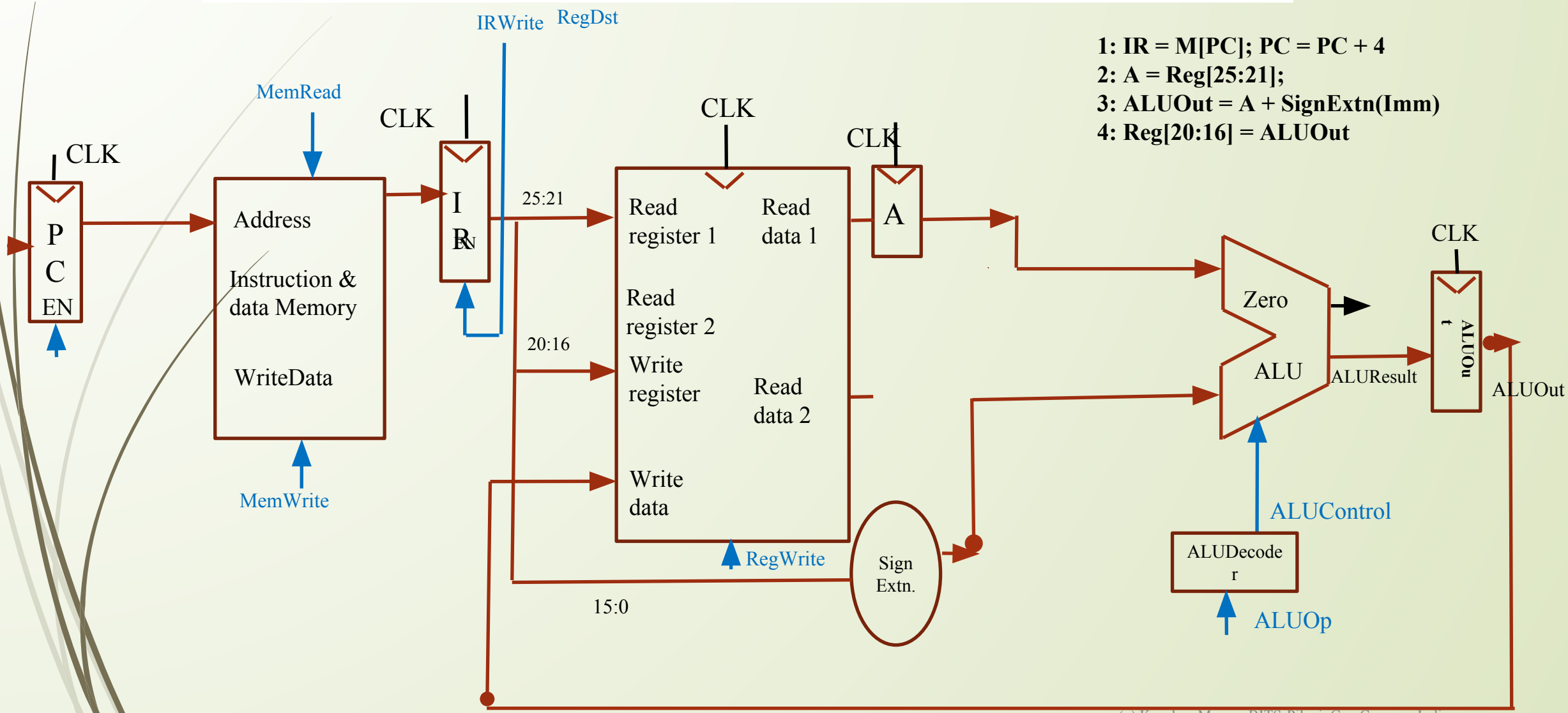
I-type instruction: ADDI

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

1: IR = M[PC]; PC = PC + 4
2: A = Reg[25:21];
3: ALUOut = A + SignExtn(Imm)
4: Reg[20:16] = ALUOut

I-type instruction: ADDI

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

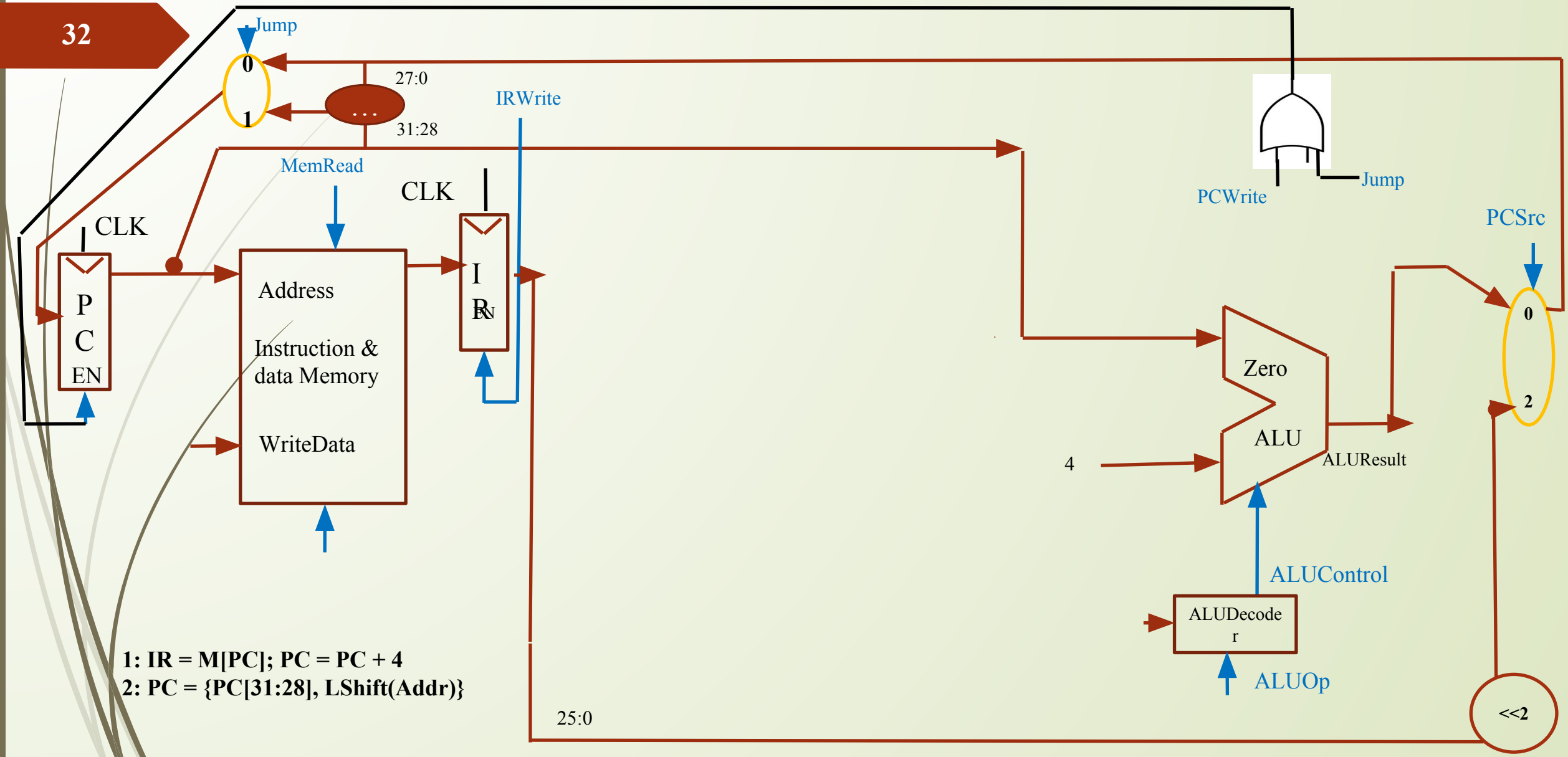
Fetch stage and Jump instruction

PC □ PC+4[31:28] addrs[27:0]

1: IR = M[PC]; PC = PC + 4
2: PC = {PC[31:28], LShift(Addr)}

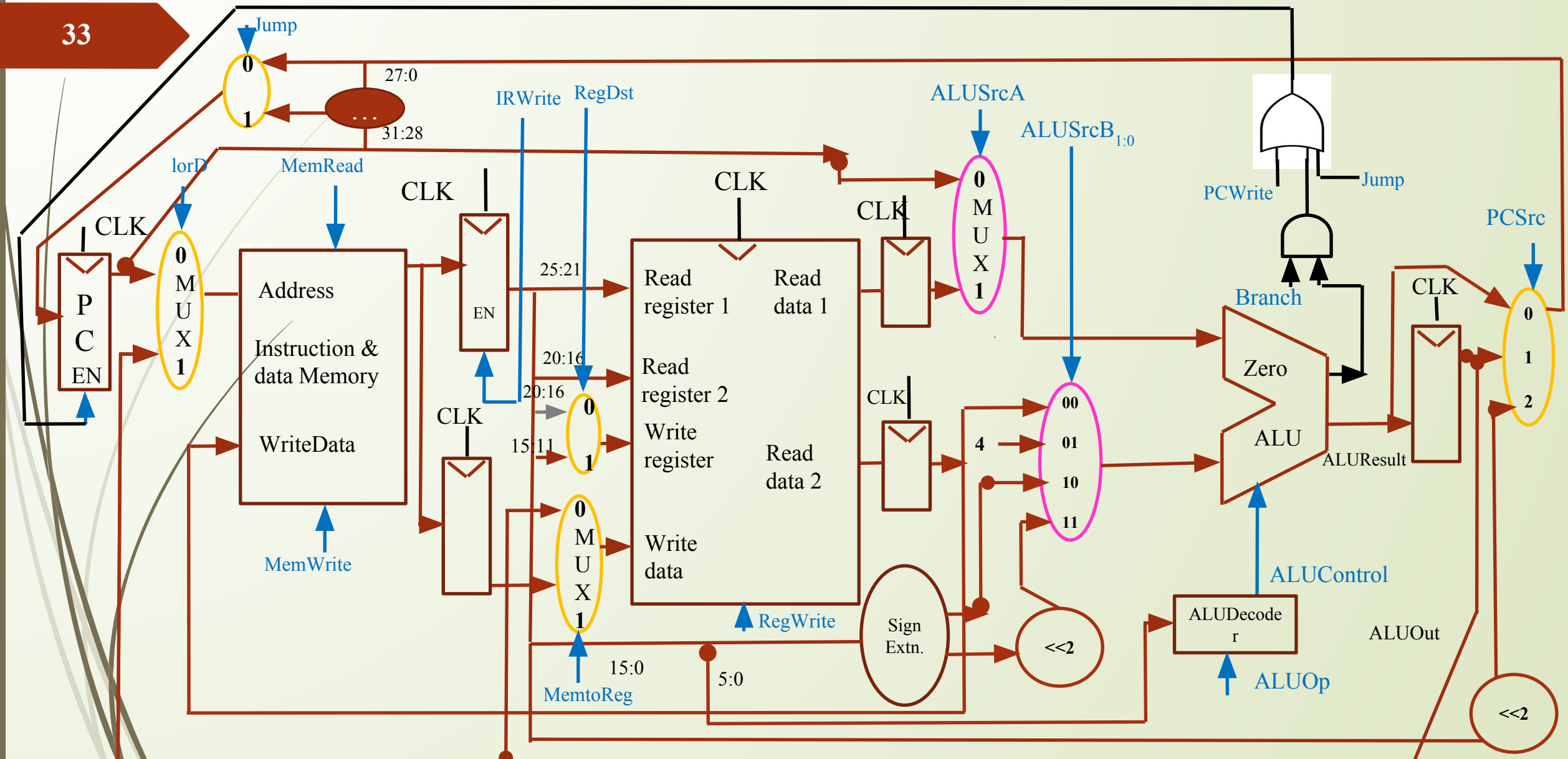
Fetch stage and Jump instruction

PC □ PC+4[31:28] addrs[27:0]



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Combined Datapath of all types of instructions



(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Control signals

Truth Table?

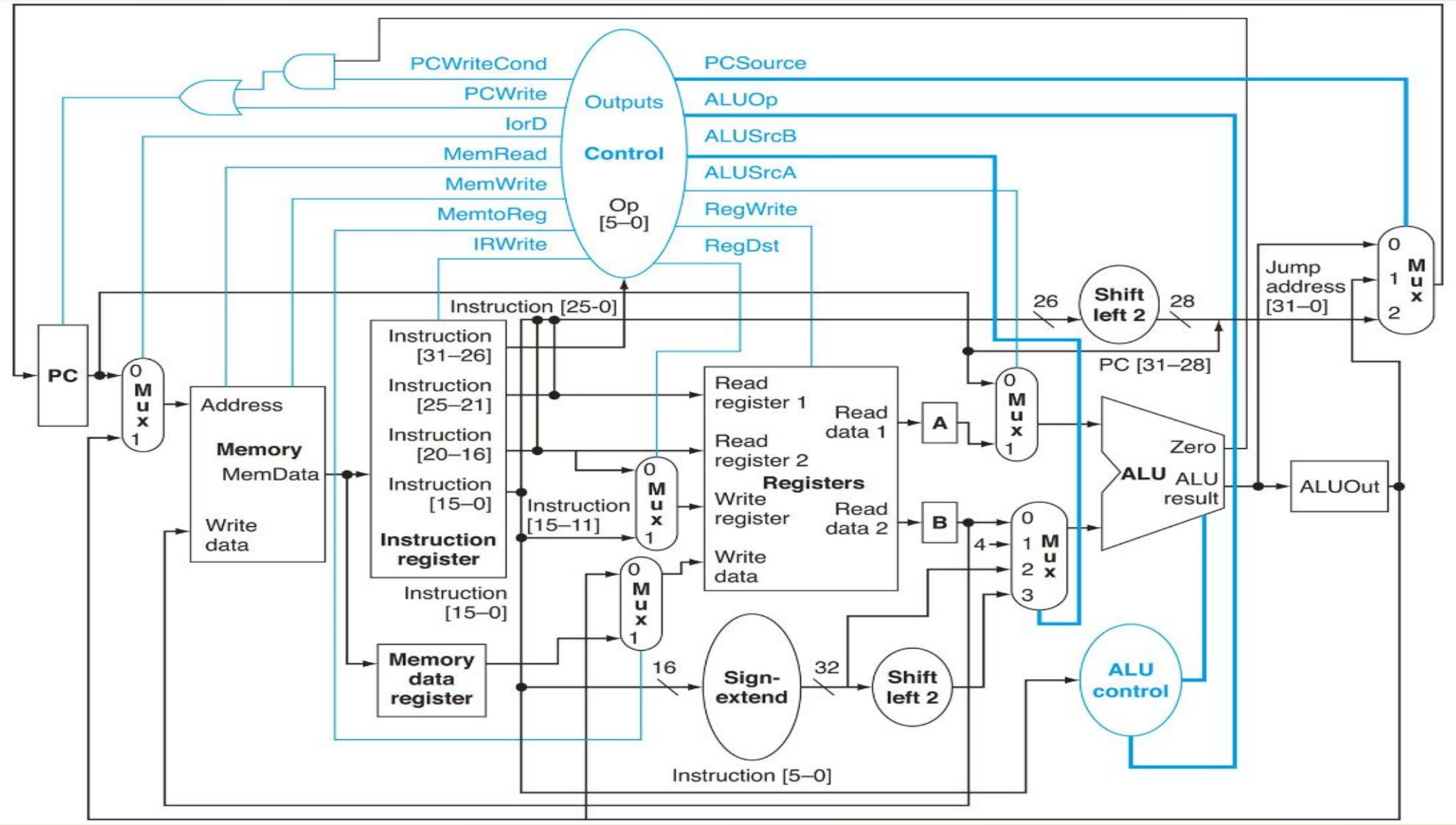
- IorD
- Jump
- Memwrite
- MemRead
- IRWrite
- RegDst
- MemtoReg
- RegWrite
- ALUSrcA
- ALUSrcR1:0
- PCWrite
- Branch
- PCSrc
- ALUOp
- ALUControl

15 Control signals

- States

How many bits are needed to represent a state?

Multicycle model



Chapter 4 A of COD by P&H

Summery of Steps taken to execute any instruction class

Step name	Action for R-type instructions	Action for memory reference instructions	Action for branches	Action for jumps
Instruction fetch	IR <= Memory[PC] PC <= PC + 4			
Instruction decode/register fetch	A <= Reg [IR[25:21]] B <= Reg [IR[20:16]] ALUOut <= PC + (sign-extend (IR[15:0]) << 2)			
Execution, address computation, branch/jump completion	ALUOut <= A op B	ALUOut <= A + sign-extend (IR[15:0])	if (A == B) PC <= ALUOut	PC <= {PC [31:28], (IR[25:0]],2'b00)}
Memory access or R-type completion	Reg [IR[15:11]] <= ALUOut	Load: MDR <= Memory[ALUOut] or Store: Memory [ALUOut] <= B		
Memory read completion		Load: Reg[IR[20:16]] <= MDR		

Fetch stage

Machine state	Operation	Control signals	Next state
T0	InsR □ M[PC]; PC □PC+4	IorD=0, IRWrite=1, MemRead=1 ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite=1, Jump=0	T1

Decode stage

Machine state	Operation	Control signals	Next state
T1	(PC+4) + Lshift(SigExtn(offset)) A = Reg[25:21] B = Reg[20:16]	ALUSrcA=0, $ALUSrcB_{1:0}$ = 11, ALUOp=00	x

Useful for BRZ instruction

LW type instruction

Machine state	Operation	Control signals	Next State
T2	A + sigEx(offset)	ALUSrcA=1, $ALUSrcB_{1:0}$ = 10, ALUOp=00	T3
T3	MDR □M[A+sigEx(off)]	IorD=1 , MemRead=1	T4
T4	RF[dest] □MDR	RegDst=0, MemtoReg=1, RegWrite=1	T0

SW type instruction

Machine state	Operation	Control signals	Next state
T2	A + sigEx(offset)	ALUSrcA=1, $ALUSrcB_{1:0}$ = 10, ALUOp=00	T5
T5	M[A+sigEx(offset)] □ B	IorD=1, MemWrite=1	T0

R-type instruction

Machine state	Operation	Control signals	Next state
T6	A Op B	ALUSrcA=1, $ALUSrcB_{1:0}$ = 00, ALUOp=00	T7
T7	RF[dstn] □ A Op B	RegDst=1, MemtoReg=0, RegWrite=1	T0

B-type instruction

Machine state	Operation	Control signals	Next State
T1	(PC+4) + SigExtn(offset) A = Reg[25:21] B = Reg[20:16]	ALUSrcA=0, $ALUSrcB_{1:0}$ = 11, ALUOp=00, PCSrc=01	T8
T8	A-B	ALUSrcA=1, $ALUSrcB_{1:0}$ = 00, ALUOp=01, Branch=1	T0

Decoding stage

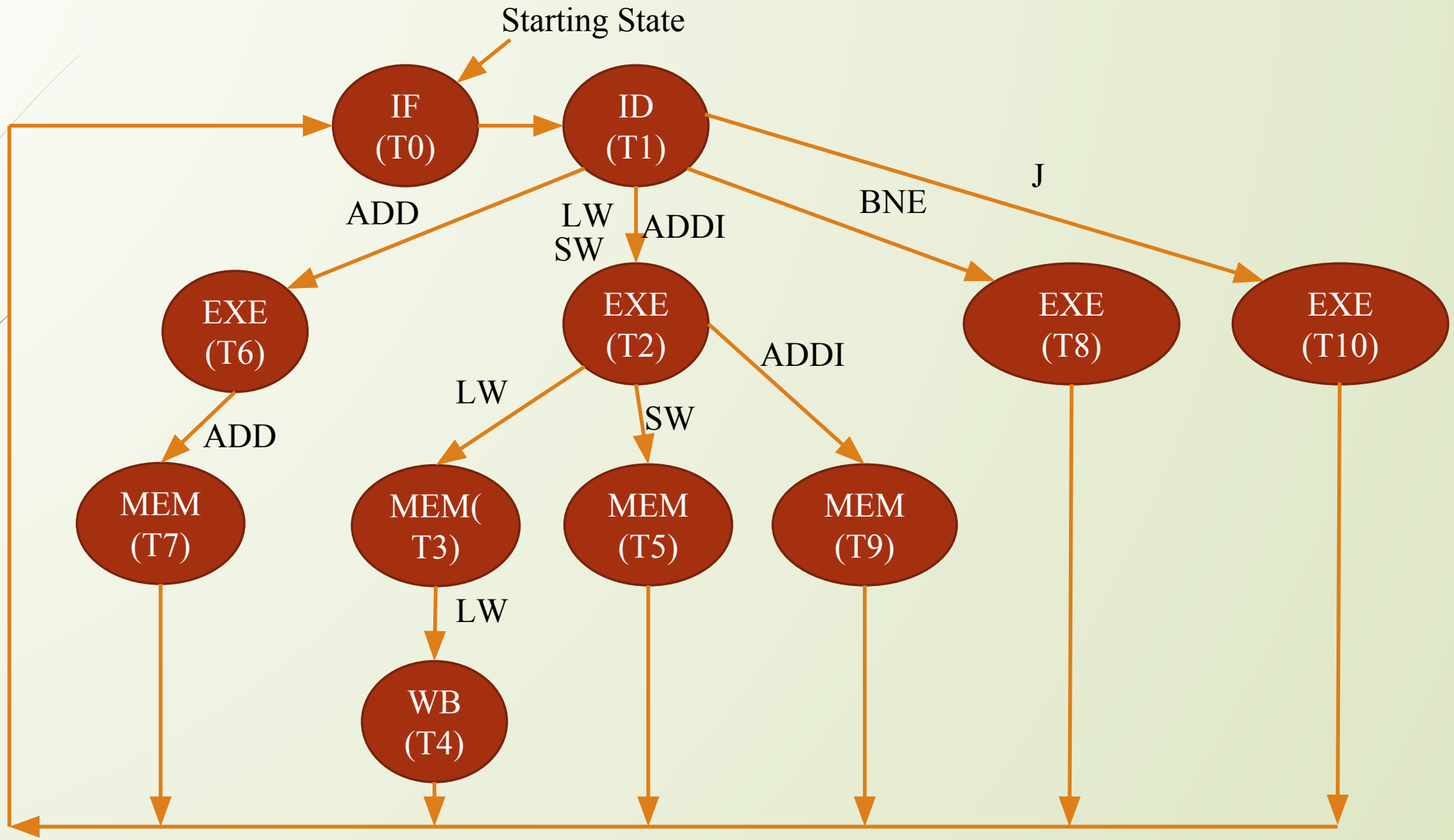
ADDI instruction

Machine state	Operation	Control signals	Next state
T2	A OP SigExtn(offset)	ALUSrcA=1, $ALUSrcB_{1:0}$ = 10, ALUOp=00	T9
T9	RF[Destn]□ A OP SigExtn(offset)	RegDst=0, MemtoReg=0	T0

Jump instruction

Machine state	Operation	Control signals	Next state
T10	A OP SigExtn(offset)	Jump=1, PCSrc=10	T0

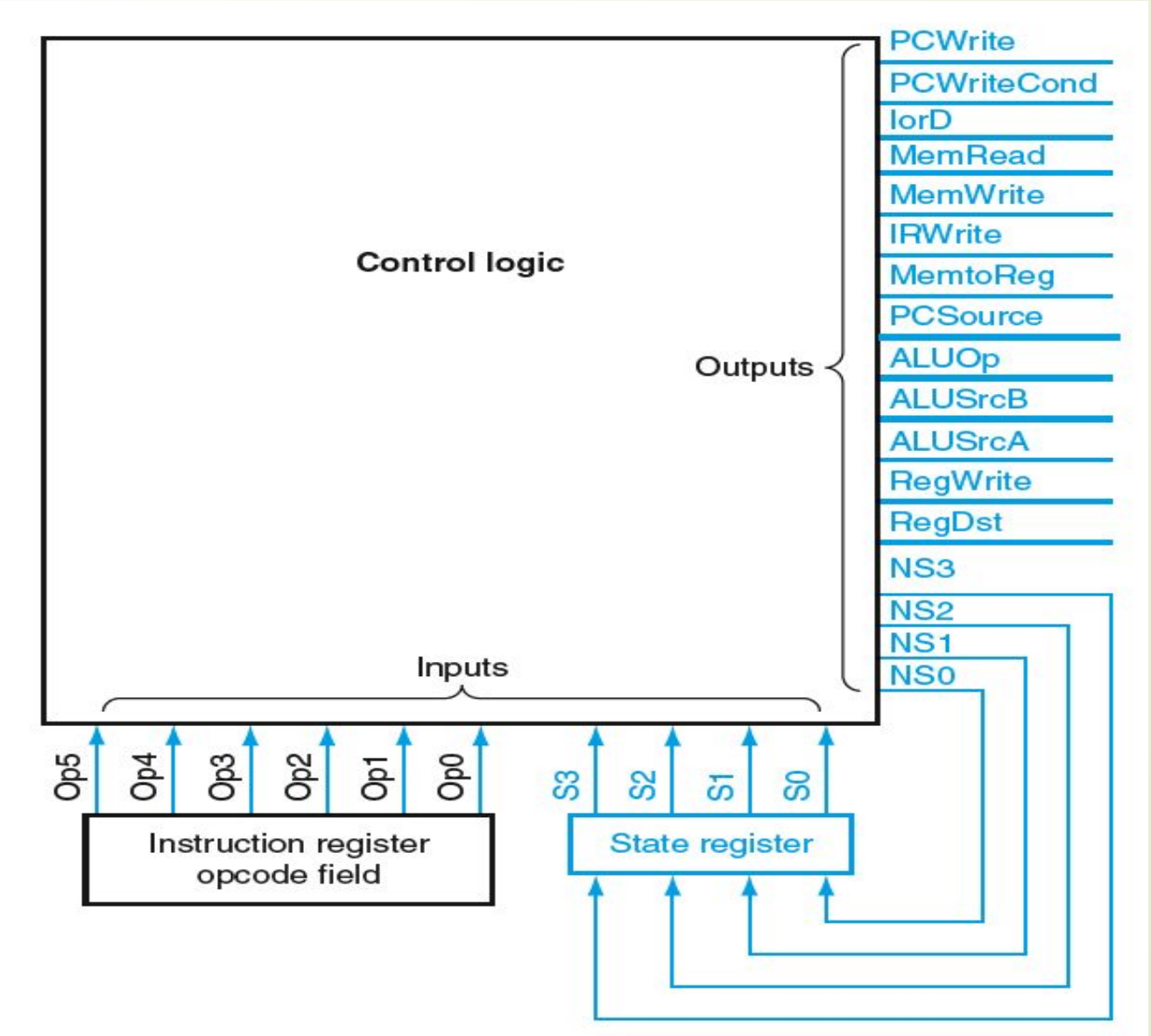
Controller States' information: Graphical Representation



Clock-cycle needed for the instructions

Instructions	Clock-cycle
LW	5
SW	4
R-type	4
BEQ	3
ADDI	4
J	3

(Finite state) Control Unit



Book-P&H – ch-appx-D

Fetch stage

Machine state (Inputs)	Operation	Control signals (Outputs)	Next state (Outputs)
T0	InsR □ M[PC]; PC □PC+4	IorD=0, IRWrite=1, MemRead=1 ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite=1, Jump=0	T1

Microprogram based control unit

Address	IorD	Jump	Memwrite	MemRead	IRWrite	RegDst	MemtoReg	RegWrite	ALUSrcA	ALUSrcB_1:0	PCWrite	Branch	PCSrc	ALUOp	NextAddress	Mode		States
0	0	0	0	1	1	x	x	0	0	01	1	0	00	00	1	0	Fetch	T0
1	x	0	0	0	0	x	x	0	0	11	0	0	00	00	xxxx	1	Decode	T1
2	x	0	0	0	0	x	x	0	1	10	0	0	00	00	3	0		T2
3	1	0	0	1	0	x	x	0	x	xx	0	0	00	xx	4	0	LW	T3
4	x	0	0	0	0	0	1	1	x	xx	0	0	00	xx	0	0		T4
5	x	0	0	0	0	x	x	0	1	10	0	0	00	00	6	0	SW	T2
6	1	0	1	0	0	x	x	0	x	xx	0	0	00	xx	0	0		T3
7	x	0	0	0	0	x	x	0	1	00	0	0	00	00	8	0	ADD	T2
8	x	0	0	0	0	1	0	1	x	xx	0	0	00	xx	0	0		T3
9	x	0	0	0	0	x	x	0	1	10	0	0	00	00	10	0	ADDI	T2
10	x	0	0	0	0	0	0	0	x	xx	0	0	00	xx	0	0		T3
11	x	1	0	0	0	x	x	0	x	xx	0	0	10	xx	0	0	J	T2
12	x	0	0	0	0	x	x	0	1	00	0	1	01	01	0	0	BEQ	T2

Can we reduce the bits used for NextAddress field?

Controller's Truth (state) Table

CF: Control Field: 15 Signals or 17 bits

NA: Next Address: 4 bit

Decoding Table	
Opcode	Address
LW	2
SW	5
ADD	7
ADDI	9
J	11
BEQ	12

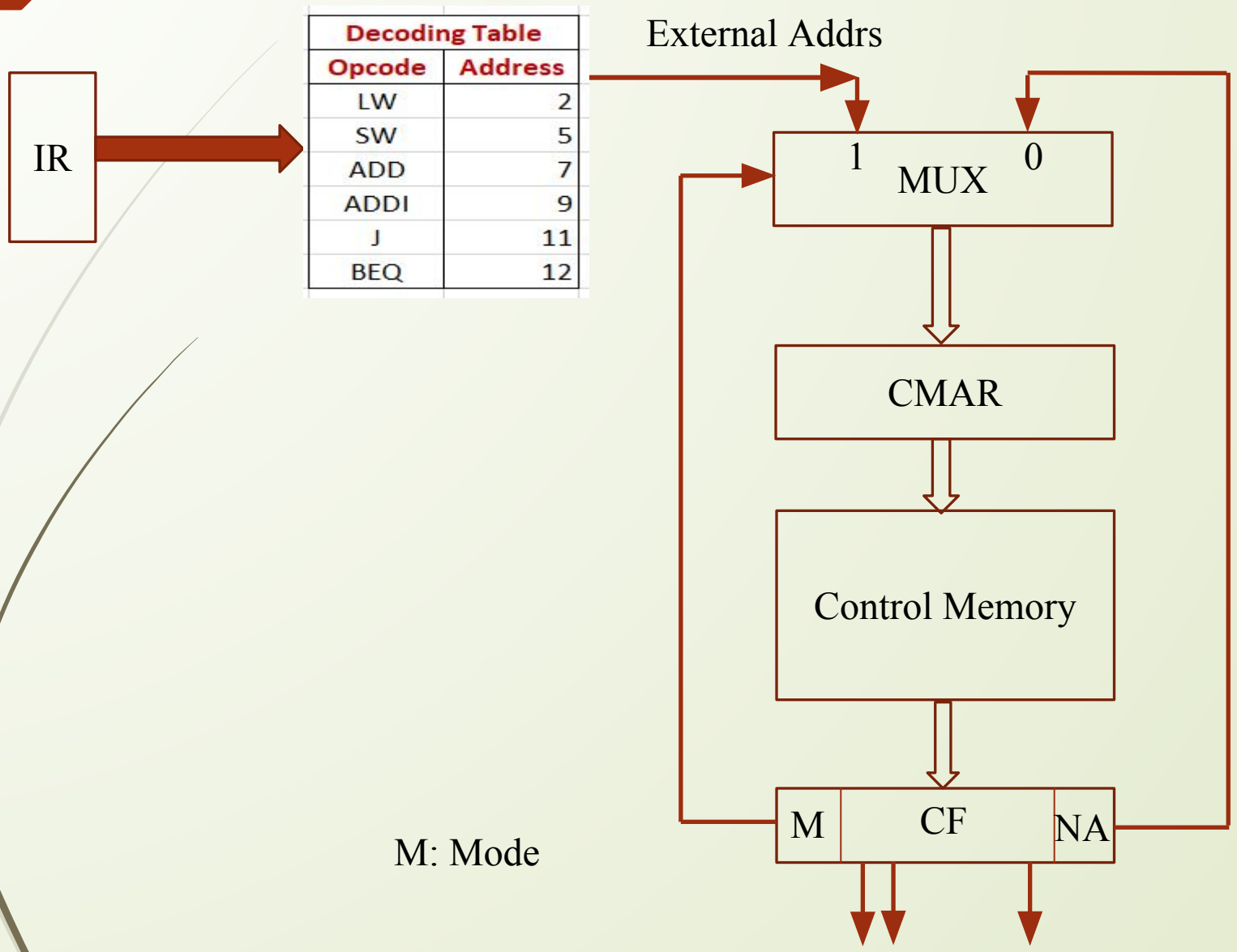
Book-Hamacher-ch5
P&H – ch-appx-C

For better understanding, we have represented the value in NextAddress and the Address as decimal.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Microprogrammed Control

- Organization of microprogrammed control unit



Microprogram sequencer: all units, except control memory.

CF: Control Field

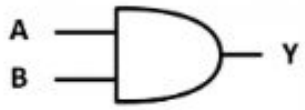
NA: Next Address

Book-Hamacher-ch5
P&H – ch-appx-C

Hardwired based control unit

Address	IorD	Jump	Memwrite	IRWrite	RegDst	MemtoReg	RegWrite	ALUSrcA	ALUSrcB_1:0	PCWrite	Branch	PCSrc	ALUOp	NextAddress	Mode		States
0	0	0	0	1	x	x	0	0	01	1	0	00	00	1	0	Fetch	T0
1	x	0	0	0	x	x	0	0	11	0	0	00	00	xxxx	1	Decode	T1
2	x	0	0	0	x	x	0	1	10	0	0	00	00	3	0		T2
3	1	0	0	0	x	x	0	x	xx	0	0	00	xx	4	0	LW	T3
4	x	0	0	0	0	1	1	x	xx	0	0	00	xx	0	0		T4
5	x	0	0	0	x	x	0	1	10	0	0	00	00	6	0		T2
6	1	0	1	0	x	x	0	x	xx	0	0	00	xx	0	0	SW	T3
7	x	0	0	0	x	x	0	1	00	0	0	00	00	8	0		T2
8	x	0	0	0	1	0	1	x	xx	0	0	00	xx	0	0	ADD	T3
9	x	0	0	0	x	x	0	1	10	0	0	00	00	10	0		T2
10	x	0	0	0	0	0	0	x	xx	0	0	00	xx	0	0	ADDI	T3
11	x	1	0	0	x	x	0	x	xx	0	0	10	xx	0	0	J	T2
12	x	0	0	0	x	x	0	1	00	0	1	01	01	0	0	BEQ	T2

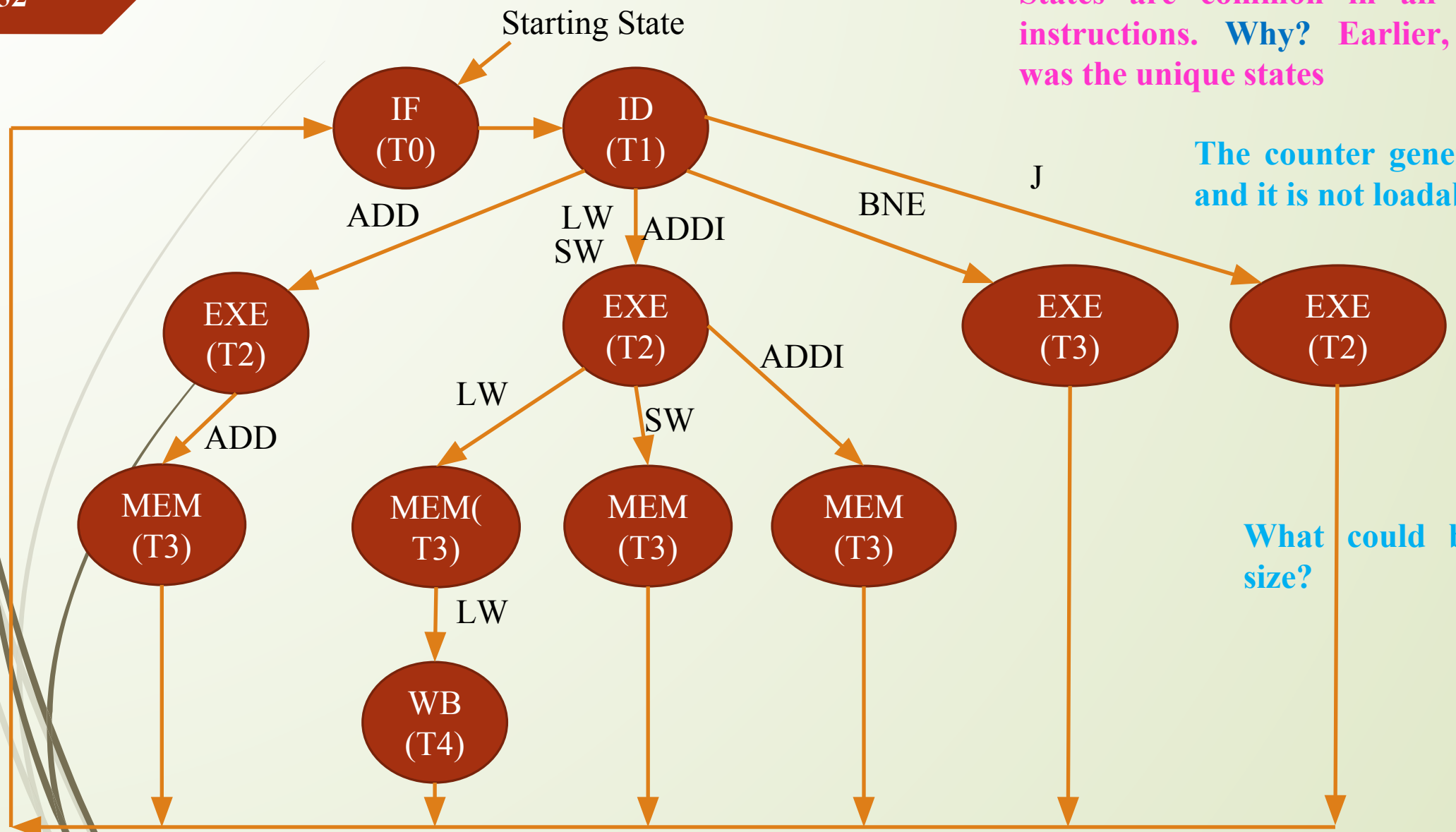
Inputs		Output
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



$Y = A.B$

Decoding Table	
Opcode	Address
LW	2
SW	5
ADD	7
ADDI	9
J	11
BEQ	12

Controller States' information for Hardwired sequential CU

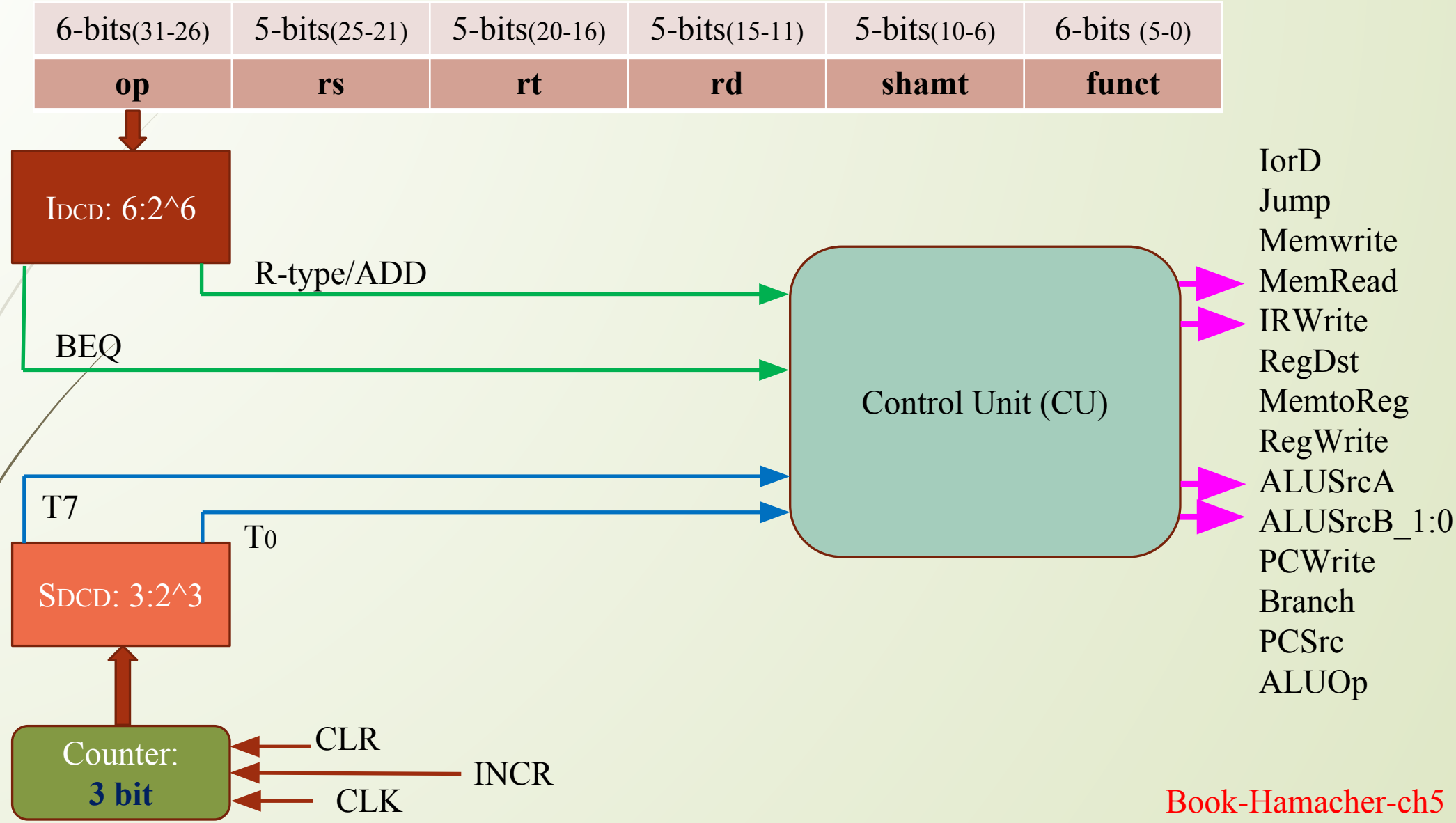


States are common in all the instructions. Why? Earlier, it was the unique states

The counter generates the states and it is not loadable.

What could be the counter size?

Hardwired based control unit



Book-Hamacher-ch5
P&H – ch-appx-C

Hardwired based control unit

IorD = T0’ + T3.LW + T3.SW +
Jump = T2.J
Memwrite = T3.SW
MemRead=T0’+
IRWrite =
RegDst =
MemtoReg =
RegWrite =
ALUSrcA =
ALUSrcB_1:0 =
PCWrite =
Branch =
PCSrc =
ALUOp = {T0’,T0’}+{T1’,T1’}+{(T2.LW)’, (T2.LW)’}
INCR = T0 + T1 + T2.LW + T3.LW + T2.SW
CLR = T4.LW + T3.SW +

Logical Expression of Control Signals

Two approaches

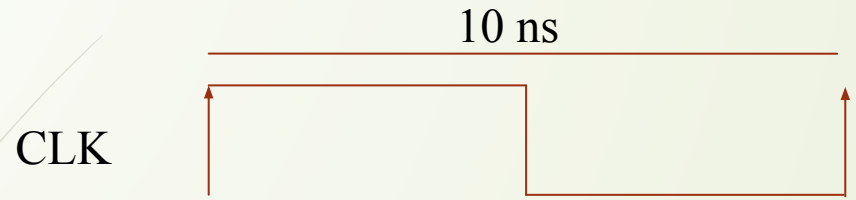
- All the ALU operations (ALUOp) similar as in Single-cycle approach
- Generate the control signals using:
  - Hardwired-based approach
  - Microprogrammed-based approach
- Inputs of the control unit similar as in Single-cycle approach

Comparison between Single and Multi-cycle datapath

- Single cycle
  - No non-architectural component
  - Every instruction takes one clock cycle (CPI=1)
- Multi-cycle
  - Non-architectural elements/register
    - Instruction, data (memory), A, B, ALU-output
  - Instructions take different unit of clock cycle
    - Average CPI (>1)

Comparison between Single and Multi-cycle Control unit

Single Cycle

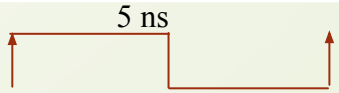


ALUOp	Meaning
00	add
01	subtract
10	Look at *funct* field
11	n/a

Instr.	Jump	RegDst	RegWrite	ALUSrc	Branch	ALUOp1	ALUOp0	MemRead	MemWrite	MemtoReg
R-type	0	1	1	0	0	1	0	0	0	0
lw	**0**	**0**	**1**	**1**	**0**	**0**	**0**	**1**	**0**	**1**
sw	0	x	0	1	0	0	0	0	1	x
addi	0	0	1	1	0	0	0	0	0	0
B-type	0	x	0	0	1	0	1	0	0	x
J-type	1	x	0	x	x	x	x	0	0	x

Multi-Cycle

LW's FSM



Machine state	Operation	Control signals	Next State
T2	A + sigEx(offset)	ALUSrcA=1, $ALUSrcB_{1:0}$ = 10, ALUOp=00	T3
T3	Data □M[A+sigEx(off)]	IorD=1	T4
T4	RF[dest] □Data	RegDst=0, MemtoReg=1, RegWrite=1	T0

Performance Analysis

- The program consists of approximately 25% loads, 10% stores, 11% branches, 2% jumps, and 52% R-type instructions. Determine the average CPI for this program.

Performance Analysis

- The program consists of approximately 25% loads, 10% stores, 11% branches, 2% jumps, and 52% R-type instructions. Determine the average CPI for this program.
- The average CPI is the sum over each instruction of the CPI for that instruction multiplied by the fraction of the time that instruction is used
- **Average CPI** = (0.11 + 0.02)(3) + (0.52 + 0.10)(4) + (0.25)(5) = 4.12
- For Single-cycle approach, Avg. CPI is 1

Performance Analysis

- Each cycle involved **one** ALU operation, memory access, or register file access
- Each instruction is using only one stage at any time
- Assumptions:
  - the register file is faster than the memory and
  - writing memory is faster than reading memory
- Datapath has two possible critical paths:
- $T_c = t_{clk2q} + t_{mux} + \max\{t_{mux} + t_{ALU}, t_{mem}\} + t_{setup}$

Begin of each stage

Delay elements

Setup time for each stage

t_{clk2q} = clock-to-Q D-ff

Performance Analysis

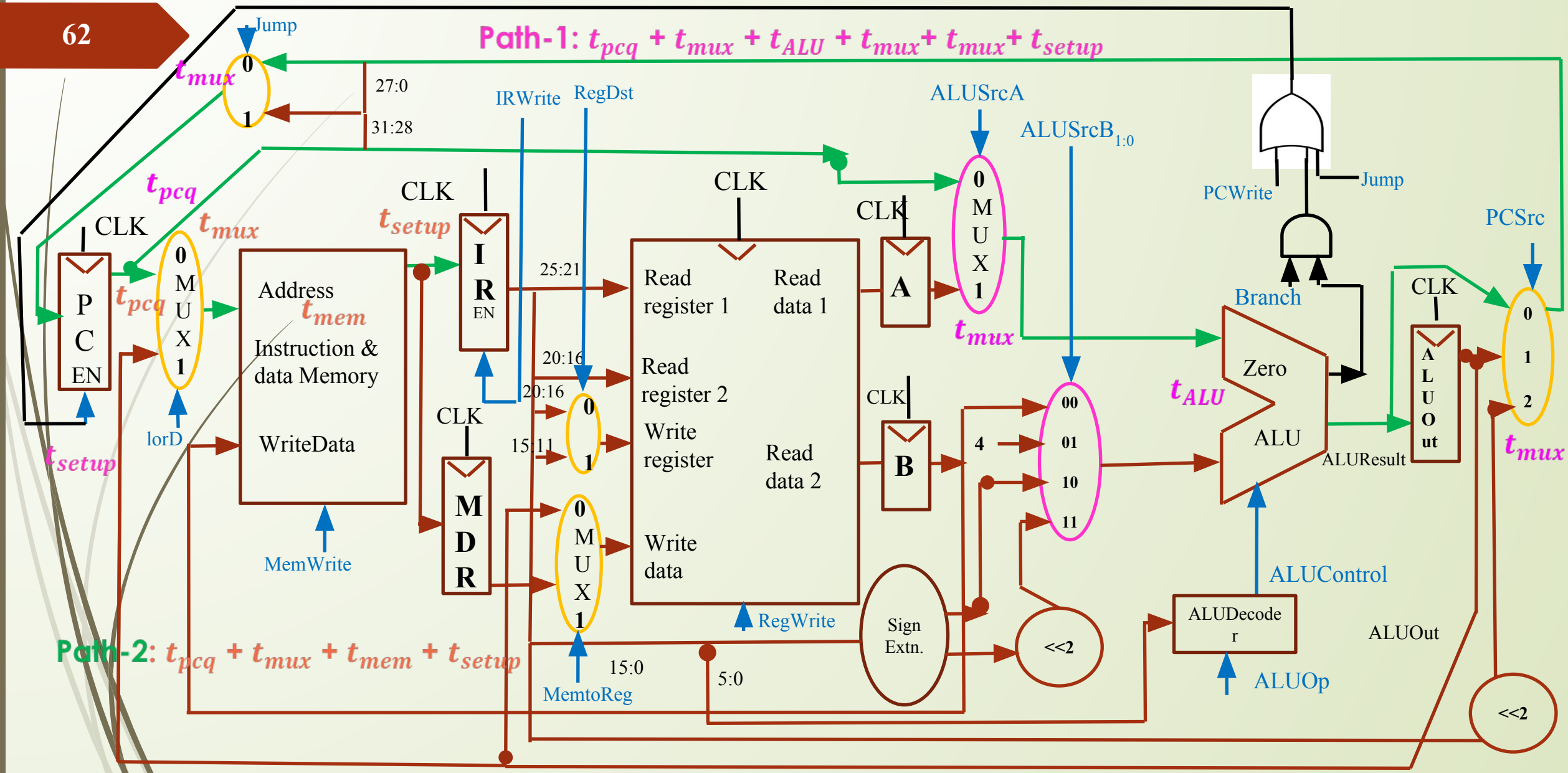
- Each cycle involved **one** ALU operation, memory access, or register file access
- Each instruction is using only one stage at any time
- Assumptions:
  - the register file is faster than the memory and
  - writing memory is faster than reading memory
- Datapath has two possible critical paths:
- $T_c = t_{pcq} + t_{mux} + \max\{t_{ALU} + t_{mux} + t_{mux}, t_{mem}\} + t_{setup}$

Path-1: $t_{pcq} + t_{mux} + t_{ALU} + t_{mux} + t_{mux} + t_{setup}$

Path-2: $t_{pcq} + t_{mux} + t_{mem} + t_{setup}$

Combined Datapath of all types of instructions

Green line decides the Critical paths



Performance Analysis

- XYZ-organization is contemplating building the multi-cycle MIPS processor instead of the single-cycle processor. For both designs, the organization plans on using a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compare each processor's execution time for a program with 100 billion instructions

Parameter	Delay (ps)
	30
	250
	20
	200
	25
	20

Performance Analysis of Multi-Cycle Implementation

- XYZ-organization is contemplating building the multi-cycle MIPS processor instead of the single-cycle processor. For both designs, the organization plans on using a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compare each processor's execution time for a program with 100 billion instructions

Parameter	Delay (ps)
	30
	250
	20
	200
	25
	20

- $T_c = t_{pcq_PC} + t_{mux} + \max\{t_{ALU} + t_{mux} + t_{mux}, t_{mem}\} + t_{setup}$
- T_c = 30 + 25 + 250 + 20 = **325 ps**
- Execution time =

  (100 * 10^9 instrs.) * (**4.12** cycle/instrs.) * (325 * 10^{-12} s/cycle)

  = 133.9 seconds

Performance Analysis: A Comparison

- For multi-cycle, T_c = 350 ps and CPI = 4.12
- For single-cycle, T_c = 925 ps and CPI = 1
- For multi-cycle, execution time = 133.9 seconds
- For single-cycle, execution time = 92.5 seconds
- This example shows multi-cycle processor is slow than the single-cycle processor; why is it so?
  - Sequencing overhead: 30 (clk-Q) + 20 (t_{setup})
- Multi-cycle processor is less expensive
- It has 5-nonarchitectural elements

Parameter	Delay (ps)
	30
	250
	20
	200
	25
	20

Multicycle MinMax processor

Simple processor

Homeworks

- How many cycles are required to run the following program on the multicycle MIPS-processor? What is the CPI of this program?

```
Addi $s1, $s2, 5
Sub $t0, $t1, $t2
Lw $t3, 15($s1)
Sw $t5, 72($t0)
Or  $t2, $s4, $s5
```

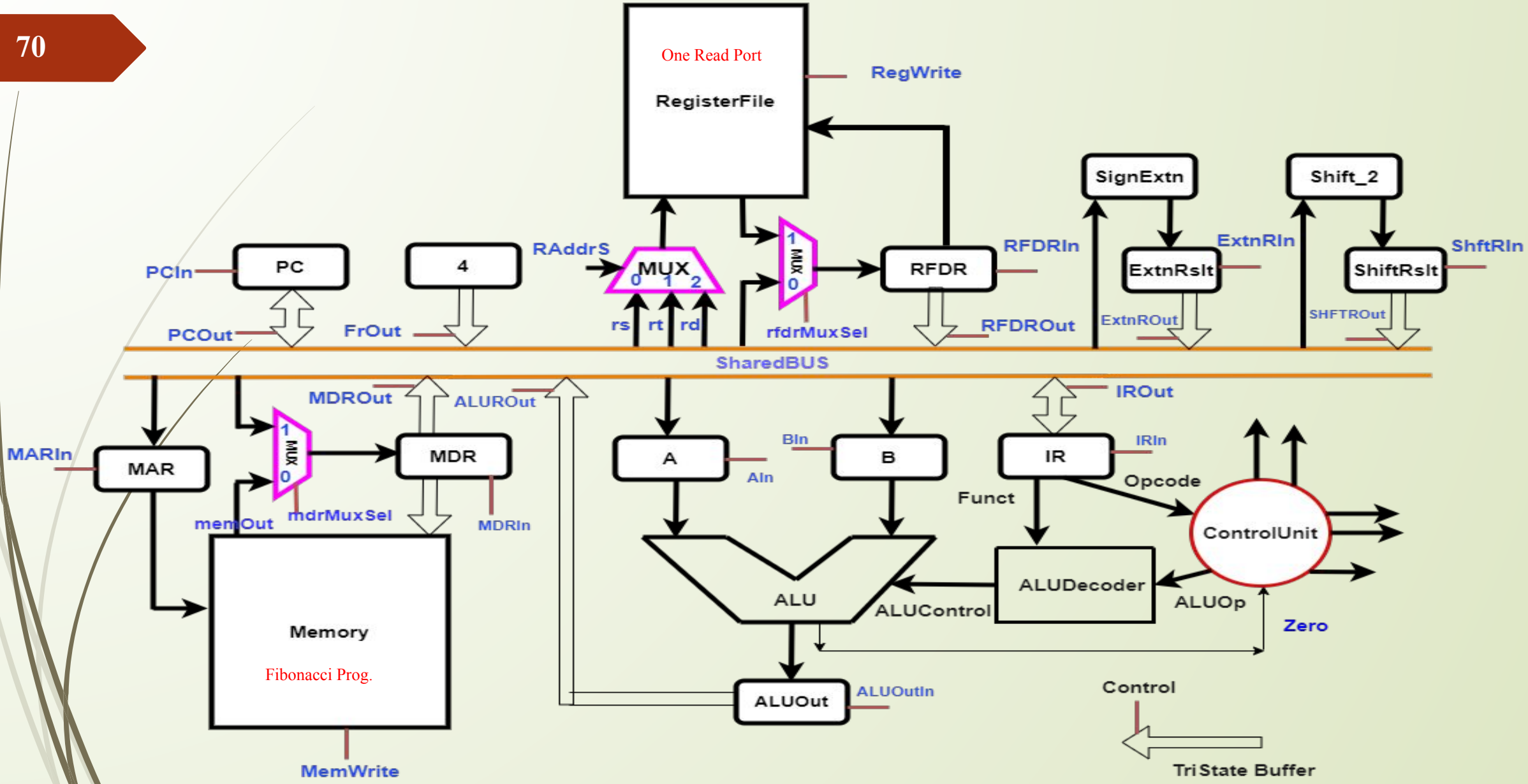
Homework

- Write CPP programs which can simulate the behaviors of the following instructions at **functional level** for MIPS-processor with **single cycle** and **multi-cycle** datapath:
  - R-type: ADD
  - M-type: LW & SW
  - B-type: BEQ
  - I-type: ADDI
  - J-type: J
- Write a CPP program for microprogramme-based CU for MIPS ISA

Processor Design With Constraints on Interconnection

Will it be a Single-Cycle or Multi-Cycle Processor?

Design this 32 bits MIPS Processor: LW, SW, ADDI, ADD, BNE



Design this 32 bits MIPS Processor: LW, SW, ADDI, ADD, BNE

Fetch Stage

Machine state	Operation	Control signals	Next state
T0	MAR □ PC A □ PC	PCOut=1, MARIn=1 Ain=1	T1
T1	MDR □ Mem[MAR] B□4	mdrMuxSel=0, MDRIn=1, FrOut=1, Bin=1, ALUOp=00	T2
T2	ALUOut □A+B	ALUOutIn=1	T3
T3	PC □ ALUOut	PCIn=1, ALUROut=1	T4
T4	IR □ MDR	MDROut=1, IRIn=1	xxx

Design this 32 bits MIPS Processor: LW, SW, ADDI, ADD, BNE

ADD

Machine state	Operation	Control signals	Next state

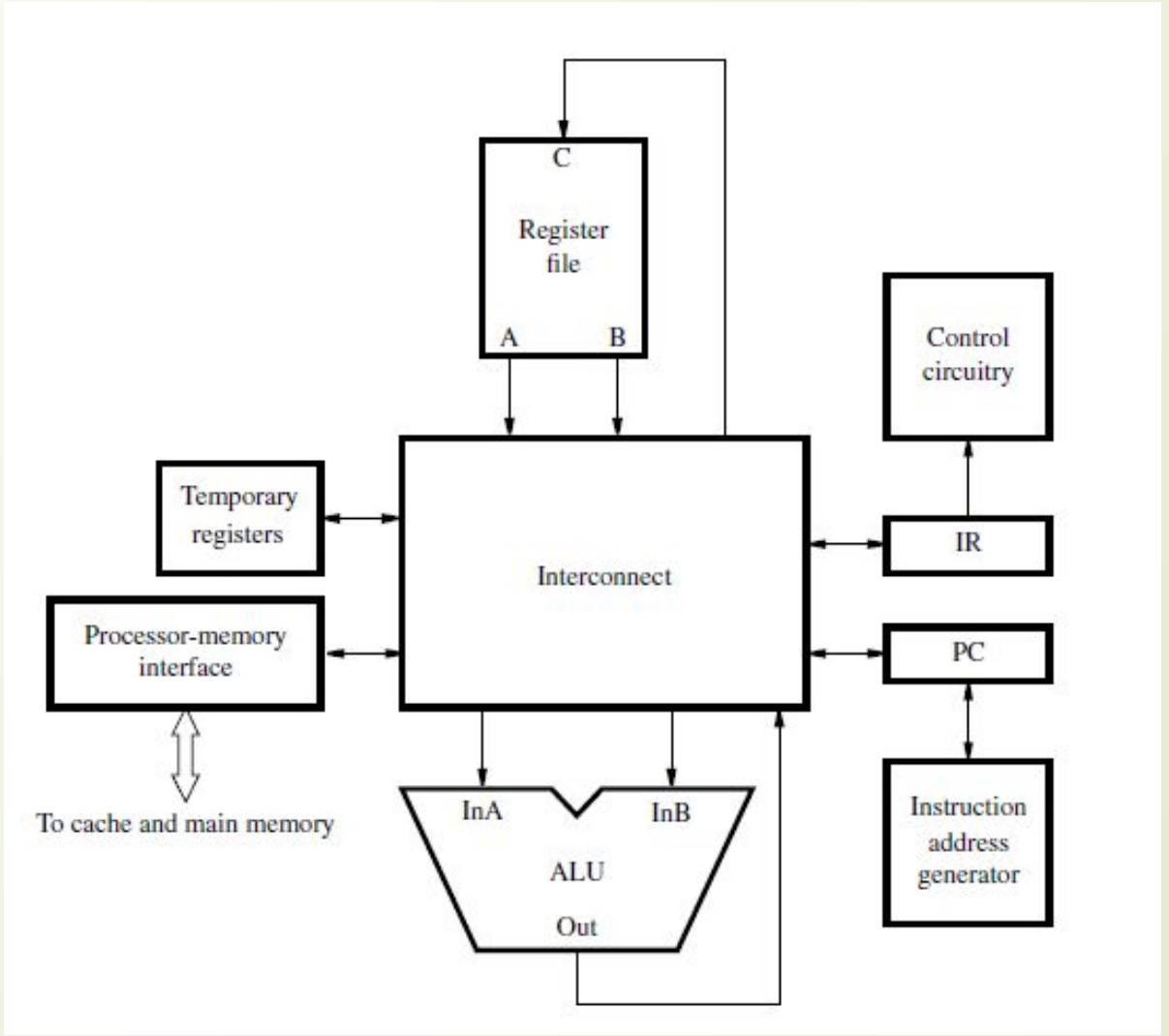
Homework: Design this 32 bits MIPS Processor: LW, SW, ADDI, ADD, BNE

All other Instructions

No of states: T0 to T29

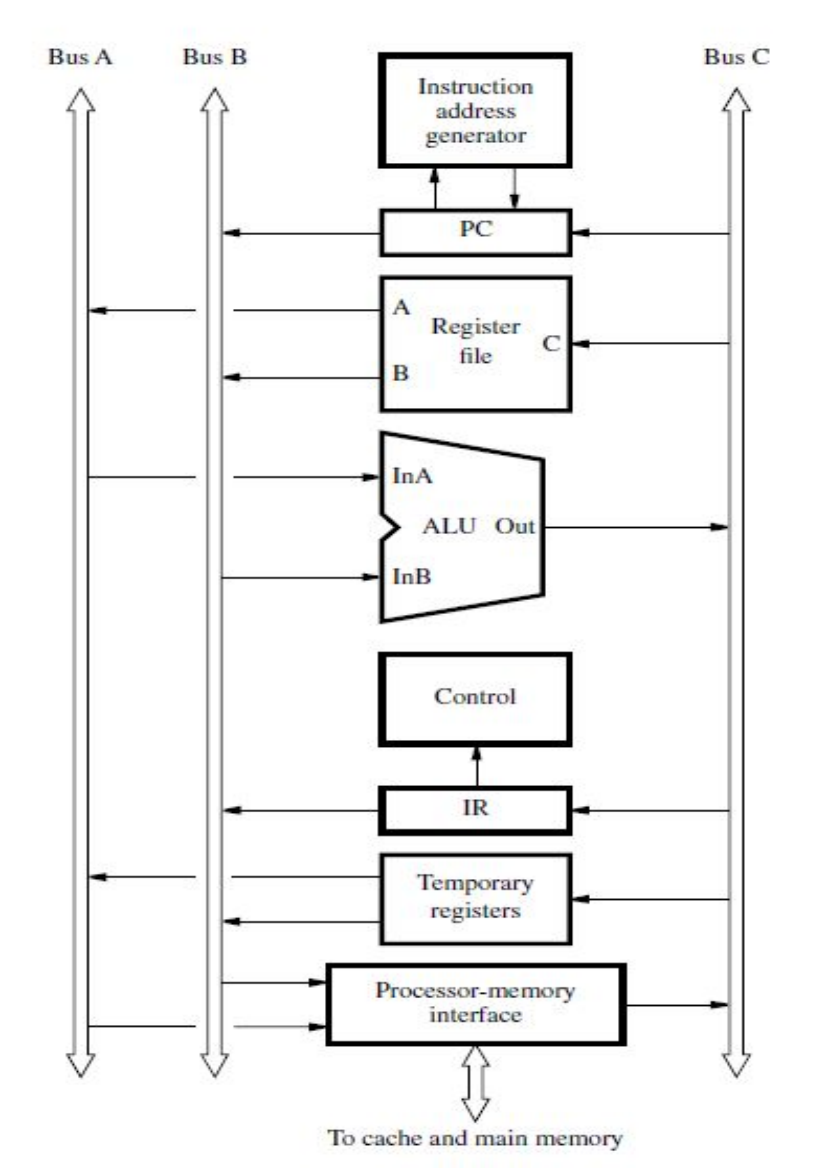
Machine state	Operation	Control signals	Next state

Organization of a CISC-style processor.



Book-Hamacher-ch5

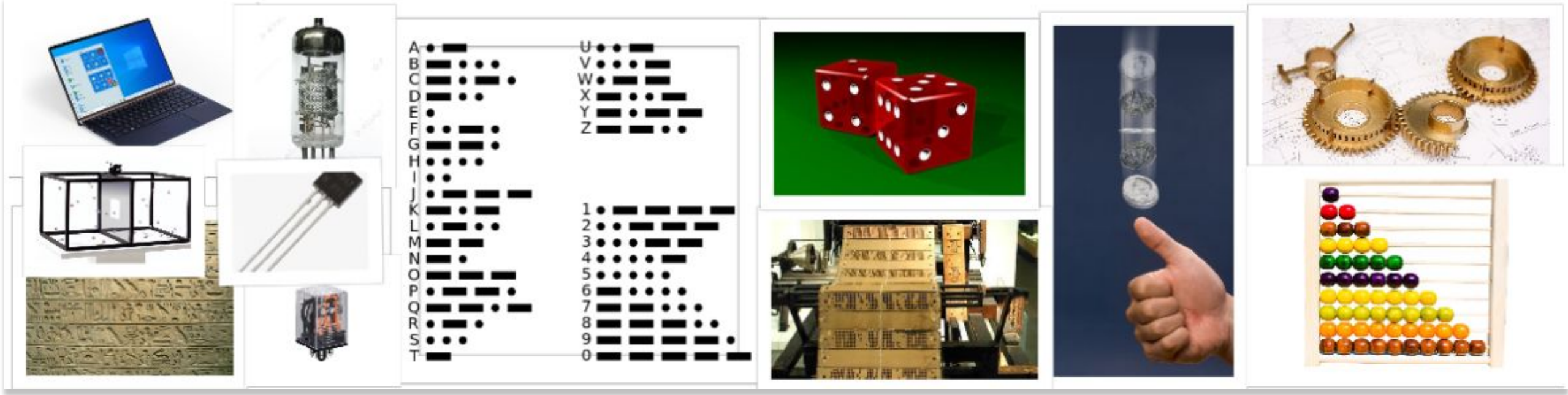
Three-bus CISC-style processor organization.



Book-Hamacher-ch5

Summary

- Disadvantages of Single-cycle processor
- Necessity of balanced design approach and multi-cycle processor
- Datapath design of multi-cycle processor
- Design of Controls for multi-cycle processor
- Performance analysis of multi-cycle processor
- Comparison between single-cycle and multi-cycle processor
- Disadvantages of multi-cycle processor



Computer Architecture

Design, Analysis, Execution and Optimization of Instructions

Datapath & CU for Pipelined Microprocessor: MIPS

Objectives

- Design the processor such that
  - Clock period (T) should be lesser than a single-cycle processor [similar to a multi-cycle one]
  - CPI should be 1

Comparison of Single & Multi-Cycle MIPS Processor

Performance analysis of Single-Cycle implementation

75

- XYZ-organization is contemplating building the single-cycle MIPS processor in a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compute the execution time for a program with 100 billion instructions.
- $T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFSetup}$
  = 30 + 2(250) + 150 + 200 + 25 + 20 = **925 ps**
- The total execution time =
  (100 * 10^9 instrs.) * (1 cycle/instrs.) * (925 * 10^{-12} s/cycle)
  = 92.5 seconds

Parameter	Delay (ps)
t_{pcq_PC}	30
t_{mem}	250
t_{RFread}	150
t_{ALU}	200
t_{mux}	25
$t_{RFSetup}$	20

Other Benchmark: Standard Performance Evaluation Corporation (SPEC)
Princeton Application Repository for Shared-Memory Computers (PARSEC)
Stanford Parallel Applications for Shared-Memory (SPLASH-2) programs.

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Performance Analysis of Multi-Cycle Implementation

64

- XYZ-organization is contemplating building the multi-cycle MIPS processor instead of the single-cycle processor. For both designs, the organization plans on using a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compare each processor's execution time for a program with 100 billion instructions
- $T_c = t_{pcq_PC} + t_{mux} + \max\{t_{ALU} + t_{mux} + t_{mux}, t_{mem}\} + t_{setup}$
- T_c = 30 + 25 + 250 + 20 = **325 ps**
- Execution time =
  (100 * 10^9 instrs.) * (**4.12** cycle/instrs.) * (325 * 10^{-12} s/cycle)
  = 133.9 seconds

Parameter	Delay (ps)
t_{pcq_PC}	30
t_{mem}	250
t_{RFread}	20
t_{ALU}	200
t_{mux}	25
t_{setup}	20

Problems of Multi-cycle Processor

- The fundamental problem
  - Split the slowest instruction, *lw*, 5-steps
  - The processor's clock cycle time does not improve 5-times, 185 ps
- The steps take unequal length of time
- Only one stage is busy and the remaining stages are idle
- 5-non-architectural registers and an additional multiplexer

Instructions	Multi-cycle (Clock-cycle)	Single-cycle (Clock-cycle)
LW	5	1
SW	4	1
R-type	4	1
BEQ	3	1
ADDI	4	1
J	3	1
CPI	**> 1**	**1**
CLK Period: T	**Lesser than Single-cycle**	**More than Multi-Cycle**

Single Cycle: Non-shared FUs, **CPI =1 or IPC=1**, clock period (T_{single})= slowest instr. in ISA

Multi-cycle: Shared FUs, CPI > 1 or IPC <1, **clock period**: $T_{multi} < T_{single}$

Can we have a microprocessor like: IPC=1 & clock period [< Tmulti < Tsingle]?

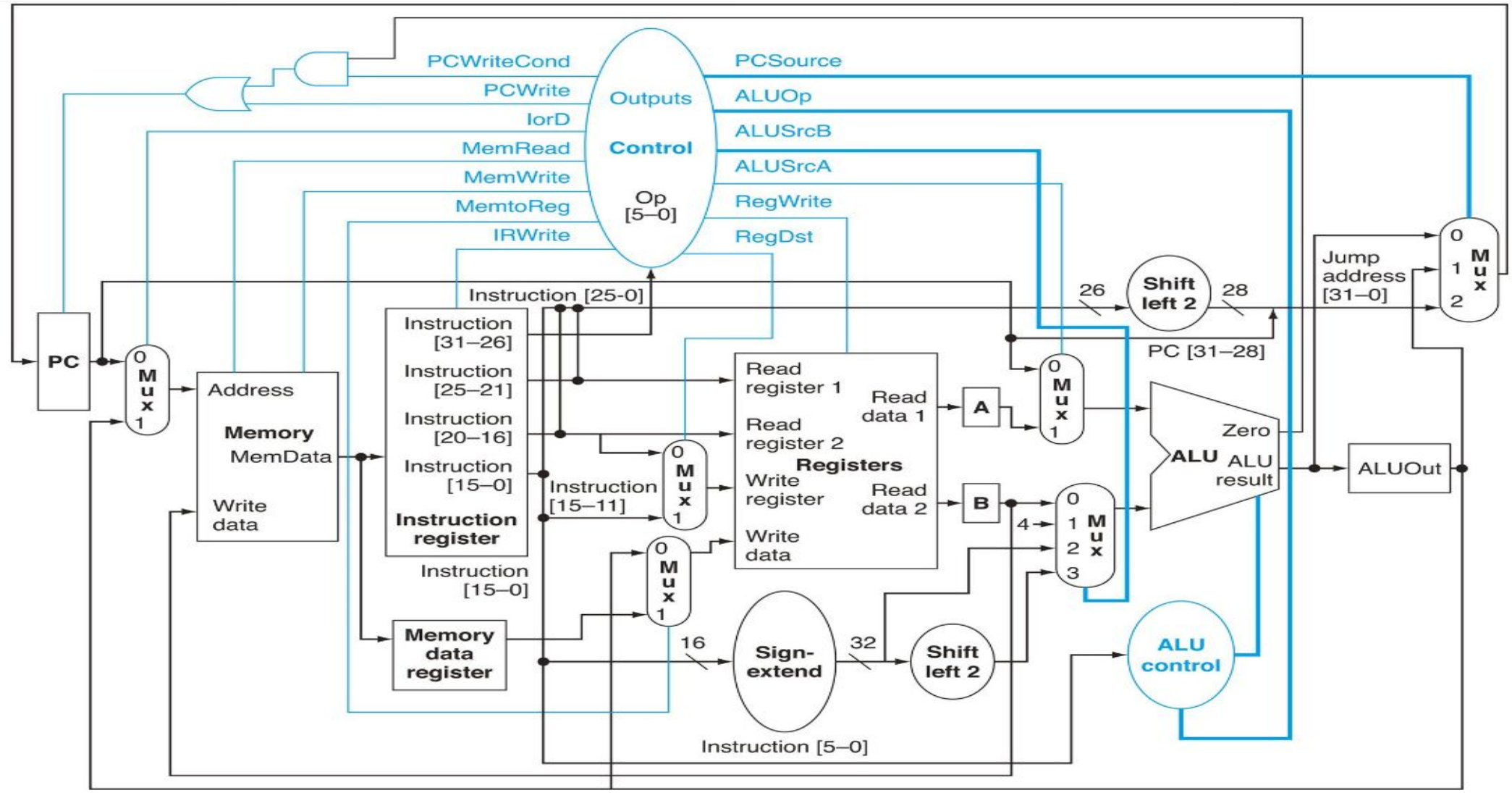
Cycles Per Instruction (CPI)

Program Execution time: #instr. x CPI x Clk (T)

Instructions Per Cycle/Seconds (IPC) = 1/CPI

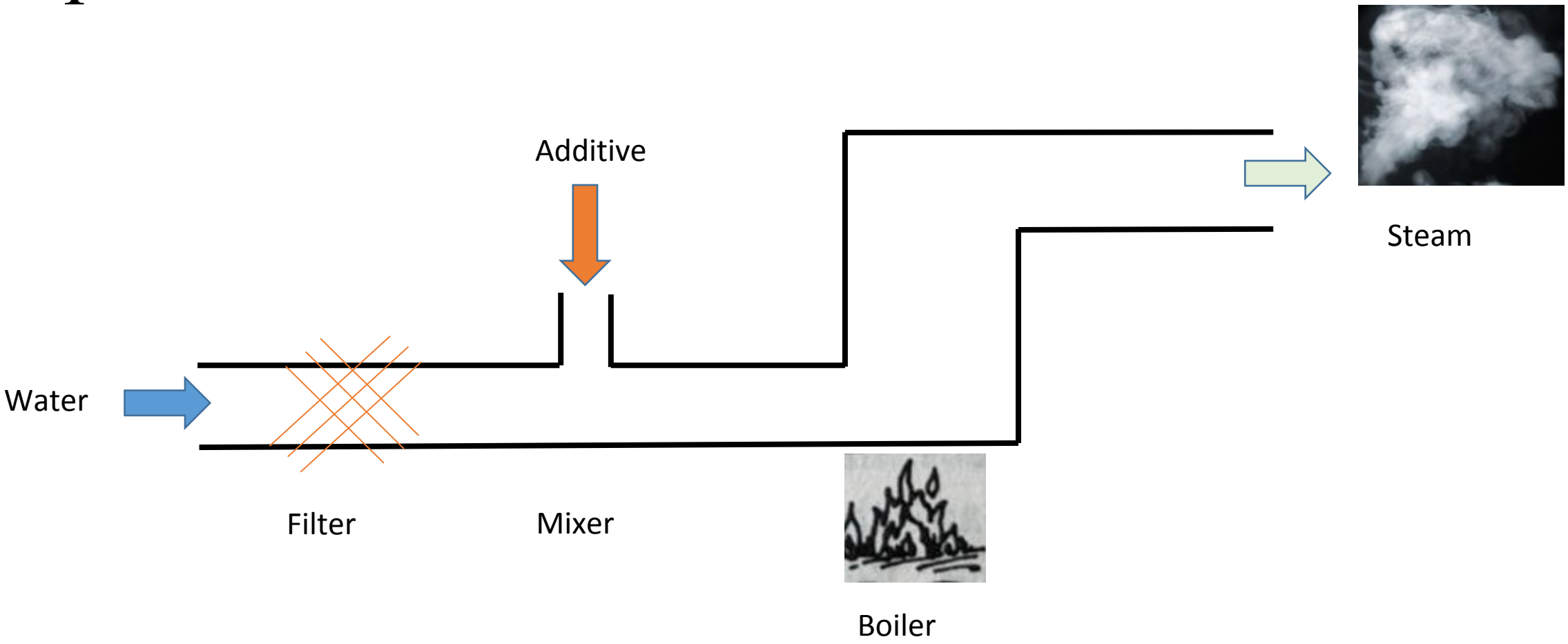
Problems of Multi-cycle Processor

Only one stage is busy and remaining stages are idle at anytime

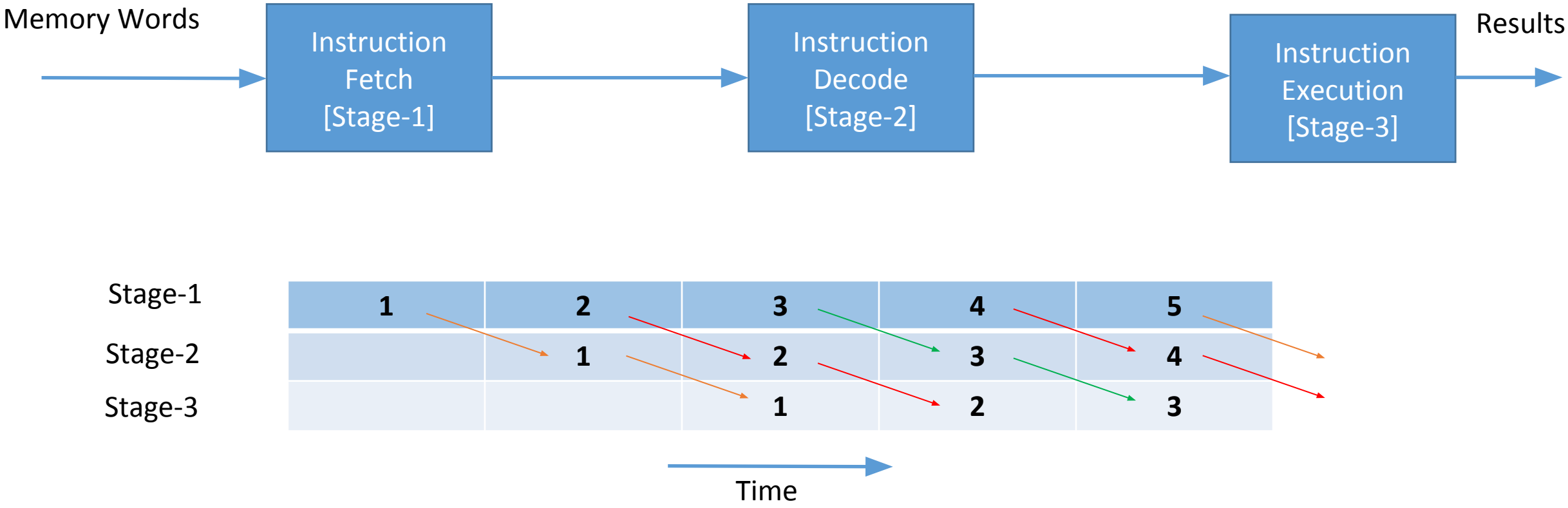


Book- P&H-COD

Pipeline in a Chemical Plant



Pipeline in the Instruction Execution



What is the difference in this Analogical or Parallel reasoning?

Pipelined MIPS-based processor

- Partitioning the Instruction Executional cycle (function)
  - Subfunctions
  - Input of one subfunction TOTALLY comes from output of previous subfunctions
  - Other than inputs & outputs, there are no interrelationships between subfunctions
  - Hardware may be developed (stage) to execute each subfunction
  - Each hardware units' evaluations are usually approximately equal

Pipelined MIPS-based processor

- Powerful way to improve the *throughput*
- Divide the single-cycle implementation
  - Fetch
  - Decode
  - Execute
  - Memory
  - Writeback
- A commercial MIPS processor: R2000/R3000

- Partitioning the Instruction Executional cycle (function)
  - Subfunctions
  - Input of one subfunction TOTALLY comes from output of previous subfunctions
  - Other than inputs & outputs, there are no interrelationships between subfunctions
  - Hardware may be developed (stage) to execute each subfunction
  - Each hardware units' evaluations are usually approximately equal

Latency of each instructions is unchanged, but throughput is ideally 5-times better

Pipelined MIPS-based processor

- Stage elements
  - Reading & writing the memory
  - Register file
  - ALU operation
- Each stage takes almost same amount of time
  - Consists of one element

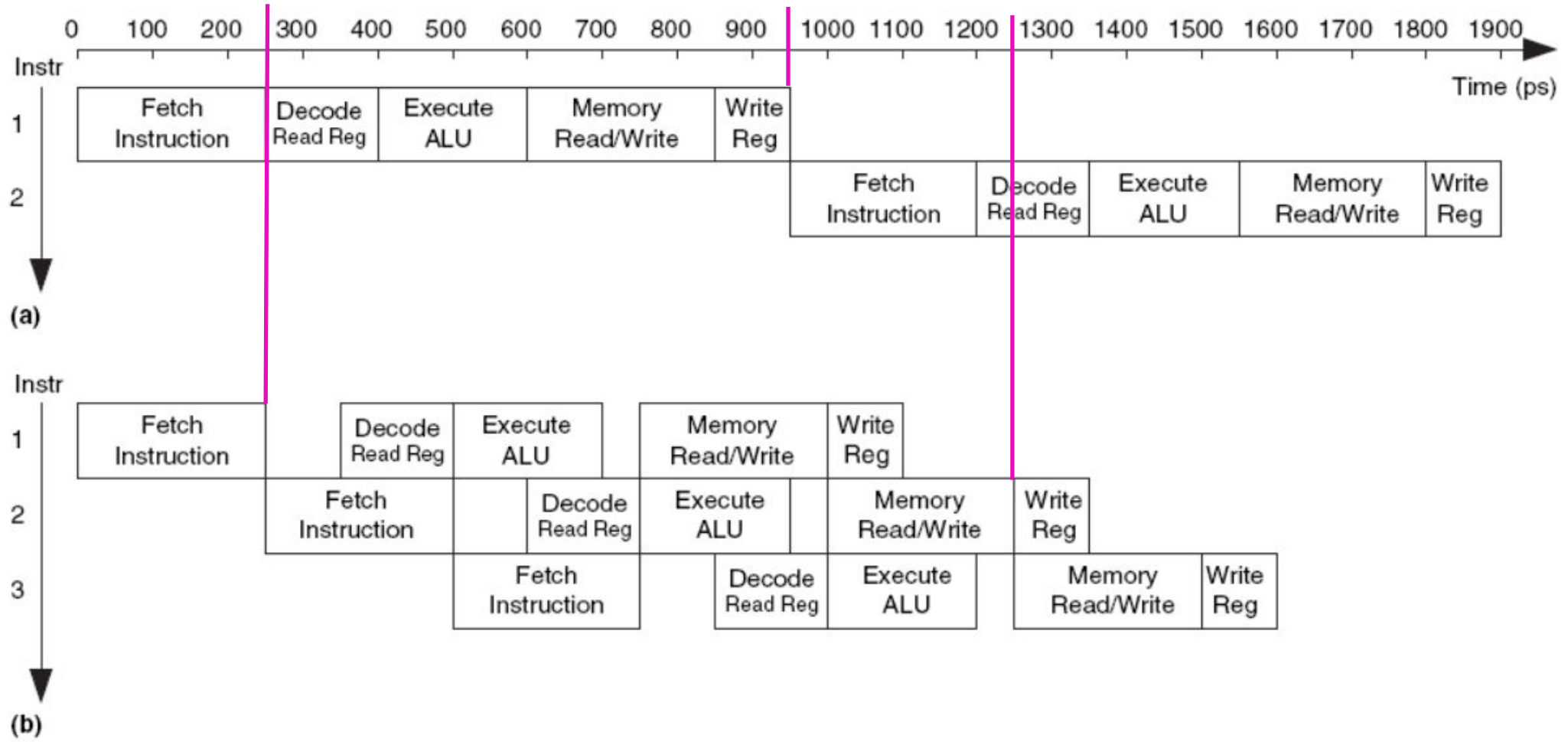
Comparison of timing diagram

- Delay of the elements

Element	Parameter	Delay (ps)
Register clk-to-Q	T_{pcq}	30
Register setup	T_{setup}	20
Multiplexer	T_{mux}	25
ALU	T_{ALU}	200
Memory read	T_{mem}	250
Register file read	t_{RFread}	150
Register file write	t_{RWrite}	100
Register file setup	$t_{RFsetup}$	20

Comparison of timing diagram

- Delay of MUX & register is not included



Timing diagram of (a) single-cycle processor (b) pipelined processor

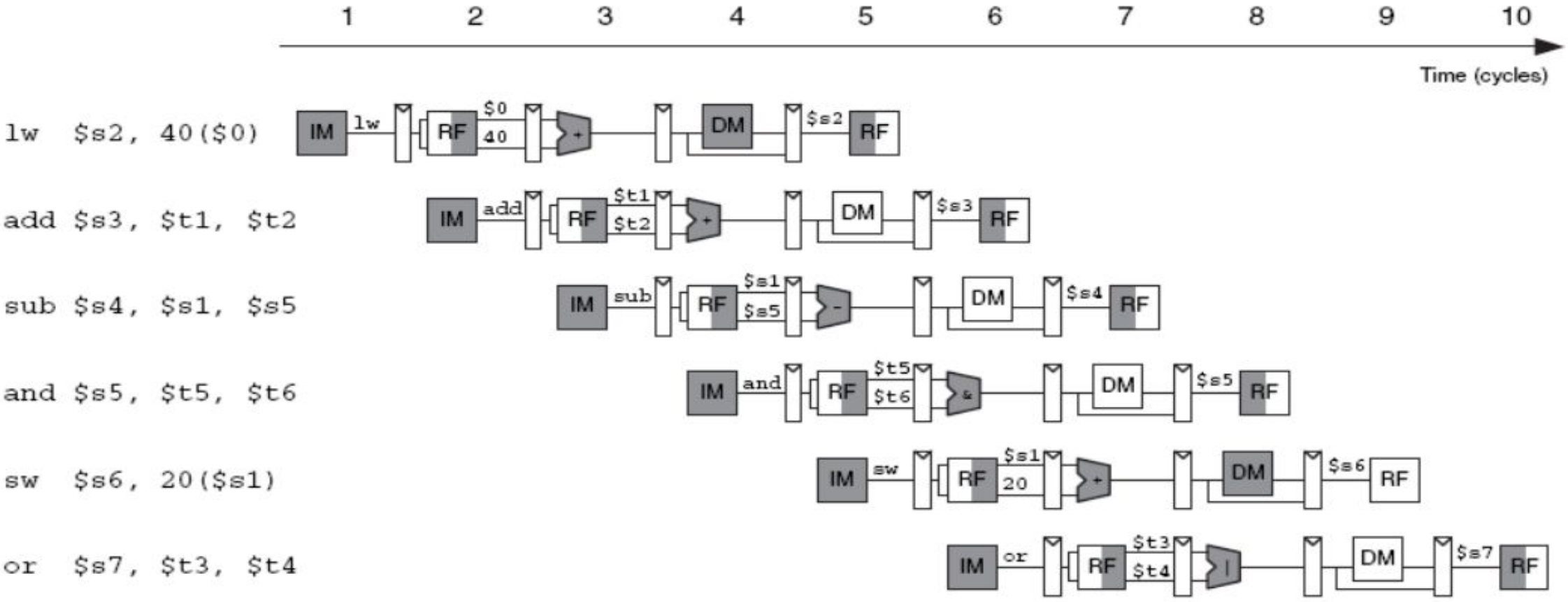
Book- P&H-COD

Comparison of timings

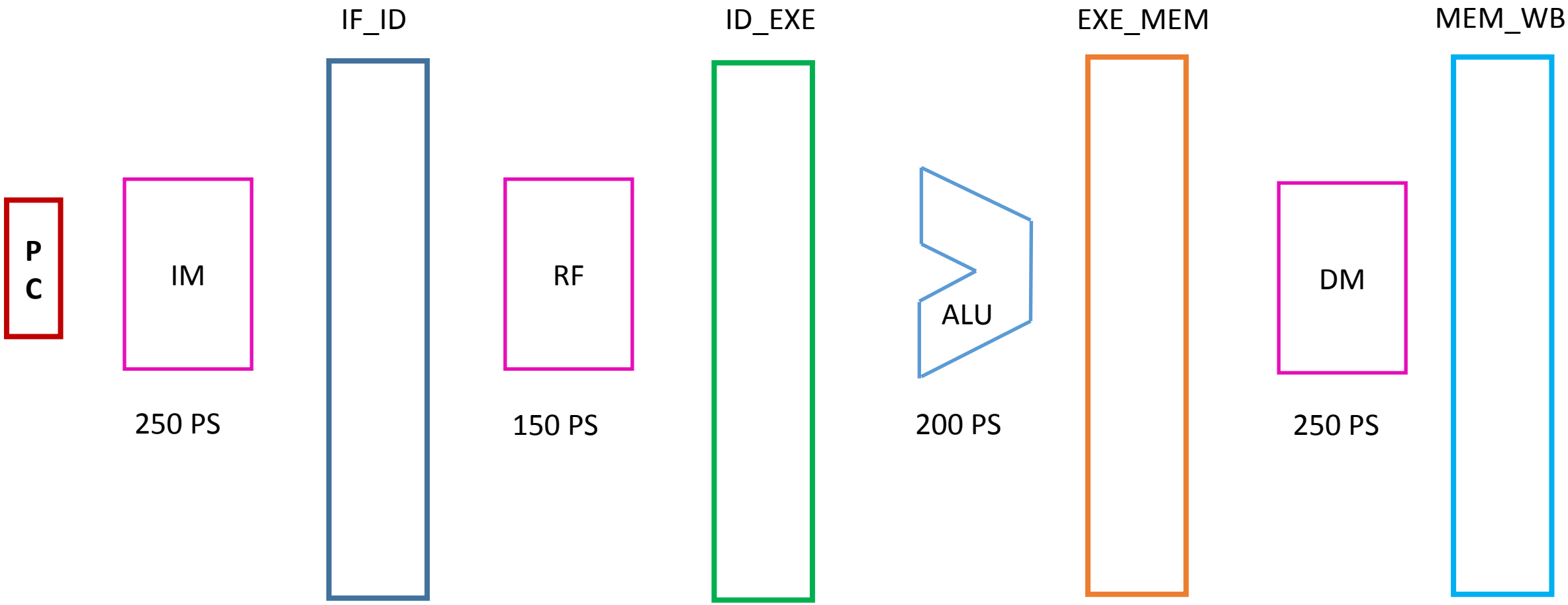
- **Single-cycle processor**
  - Instruction latency is 950 ps
  - Throughput 1 instruction per 950 ps
  - 1.05 billions instruction per second
- **Pipelined processor**
  - Length of pipeline stage is 250 ps (mem. access)
  - Instruction latency is 5*250 = 1250 ps
  - Throughput 1 instruction per 250 ps
  - 4 billions instructions per seconds

A view of pipeline in operation

- Resource utilization



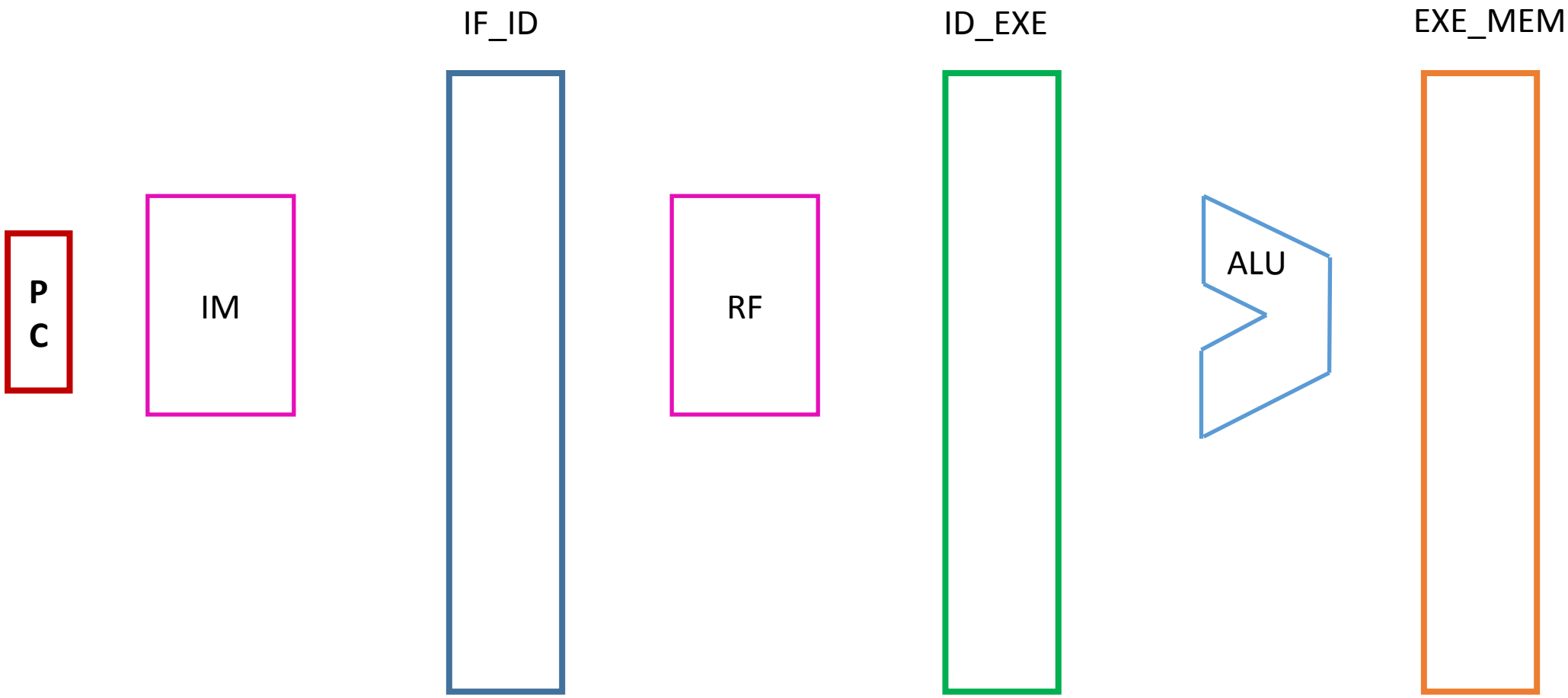
Delay elements and stage registers



Delay values are taken from the previous table.

Datapath for R-type: ADD R1, R2, R3

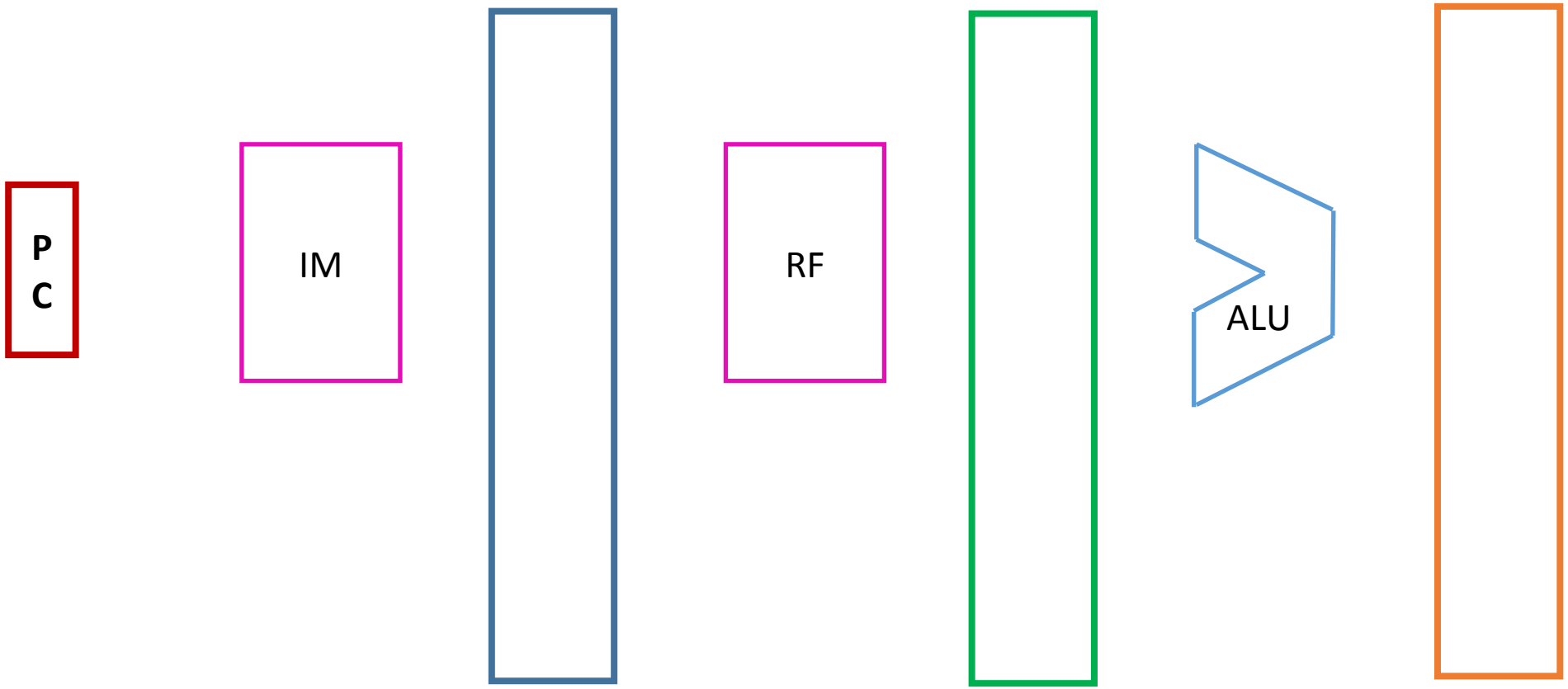
op	rs	rt	rd	shamt	funct
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Datapath for B-type: BEQ R1, R3, offset

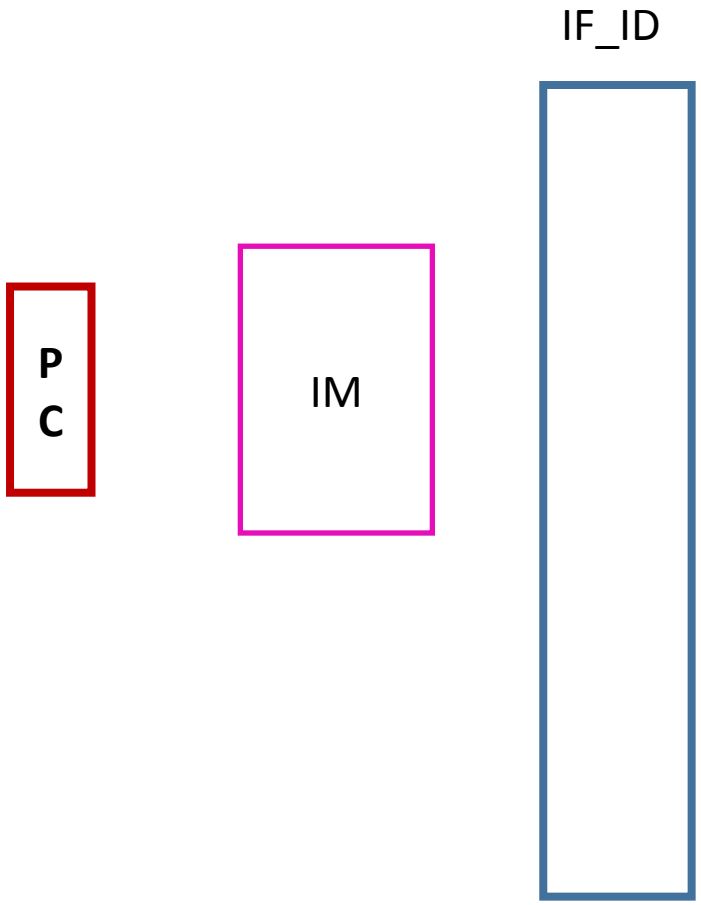
op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

IF_ID ID_EXE EXE_MEM



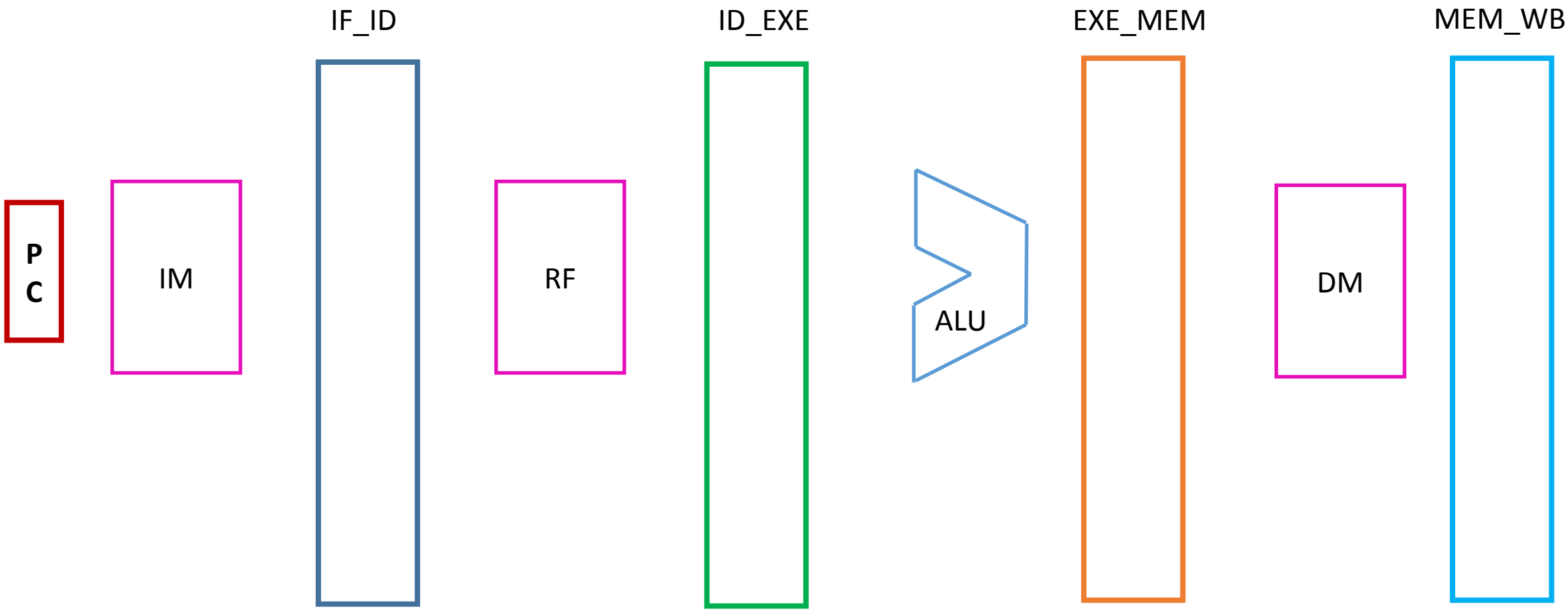
Datapath for J-type: J Offset

op	address				
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



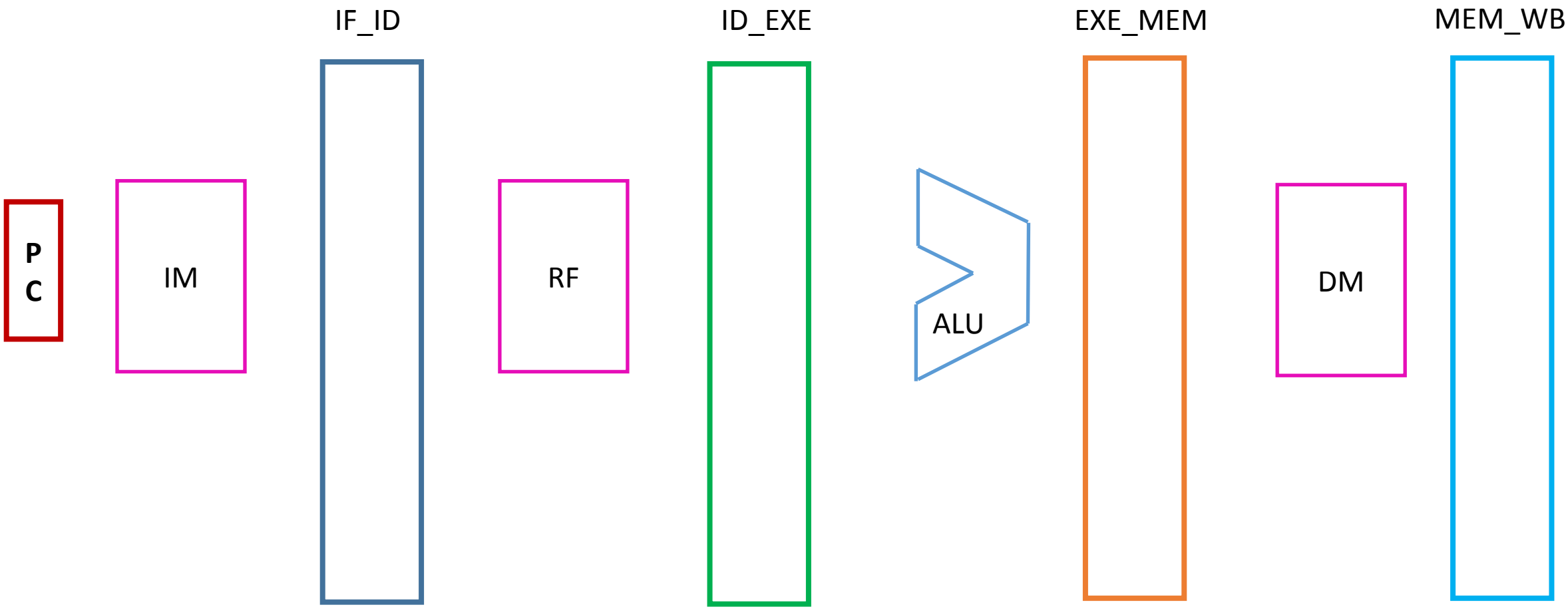
Datapath for I-type: LW R1, #5(R3)

op	rs	rt	Offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



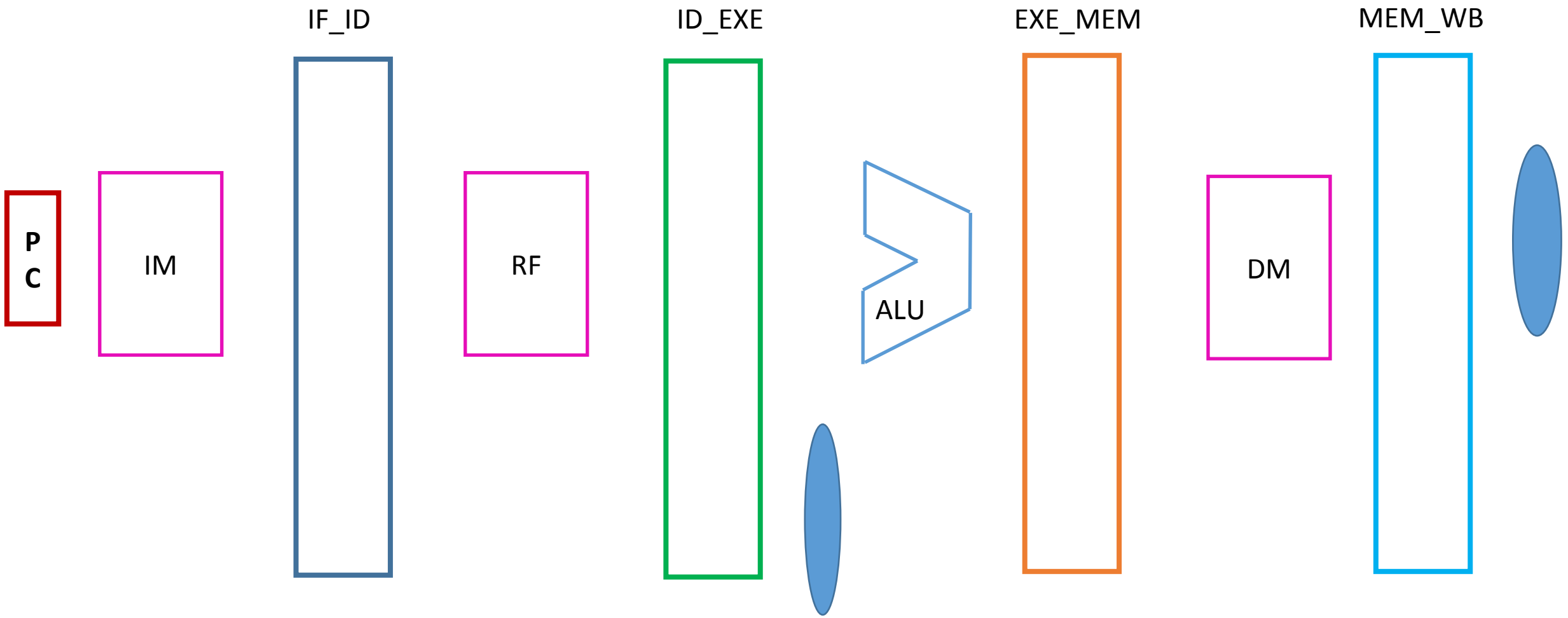
Datapath for I-type: SW R1, #5(R3)

op	rs	rt	Offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Datapath for LW R1, #5(R3) & R-type

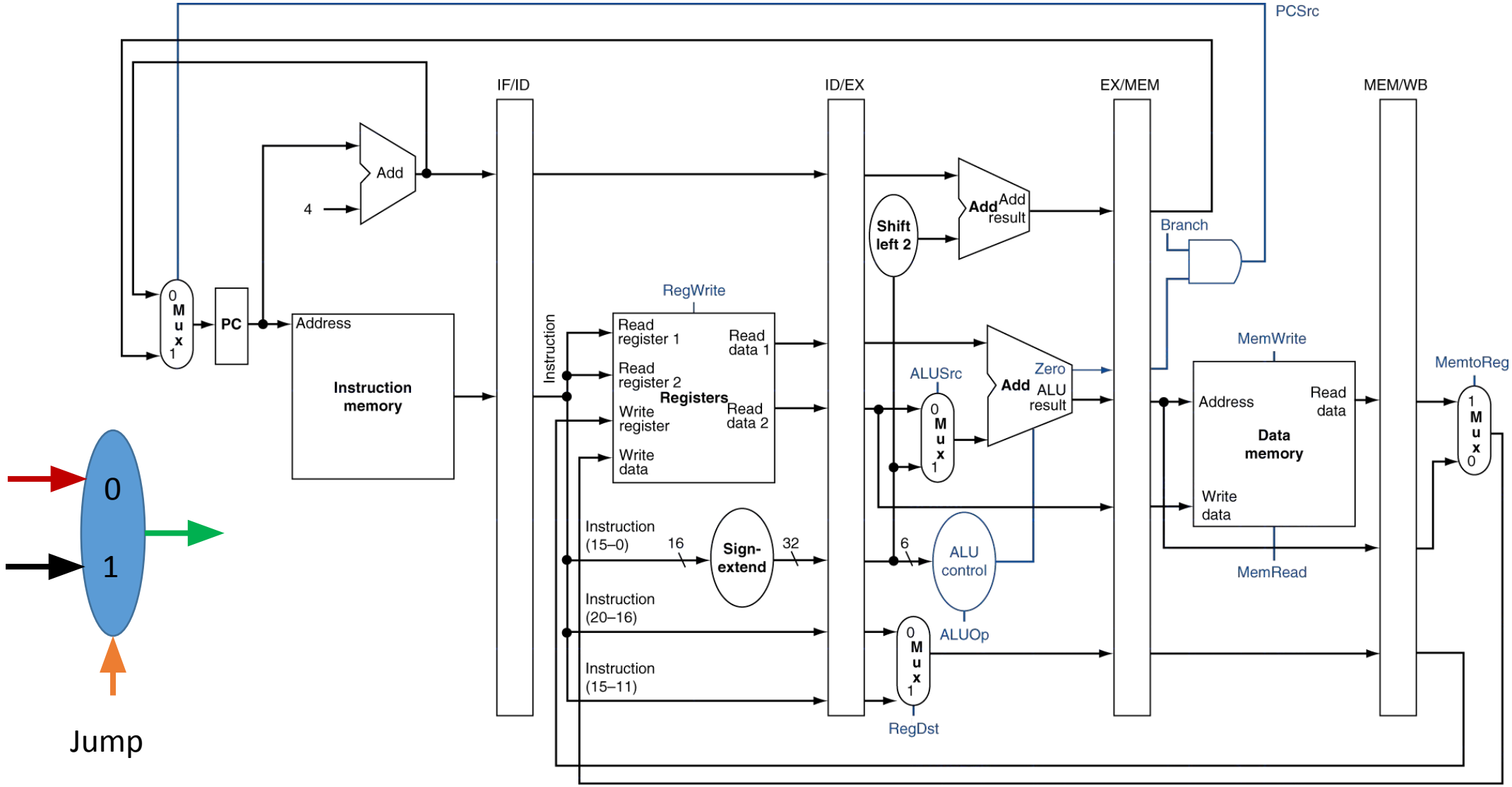
op	rs	rt	rd	shamt	funct
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Combined Datapath

- Stages
  - Insert multiplexer
- Stage registers
  - Union of registers added for each instruction

Combined Datapath



Jump

Book- P&H-COD

Control unit for Pipelined MIPS processor

- Identify the control signals
  - Jump
  - RegDst
  - RegWrite
  - ALUSrc
  - Branch
  - ALUOp
  - MemRead
  - MemWrite
  - MemtoReg

Fetch	Decode	Execute	Memory	Write Back
jump		ALUSrc	Branch	MemtoReg
		ALUOp	MemRead	RegWrite
		RegDst	MemWrite	

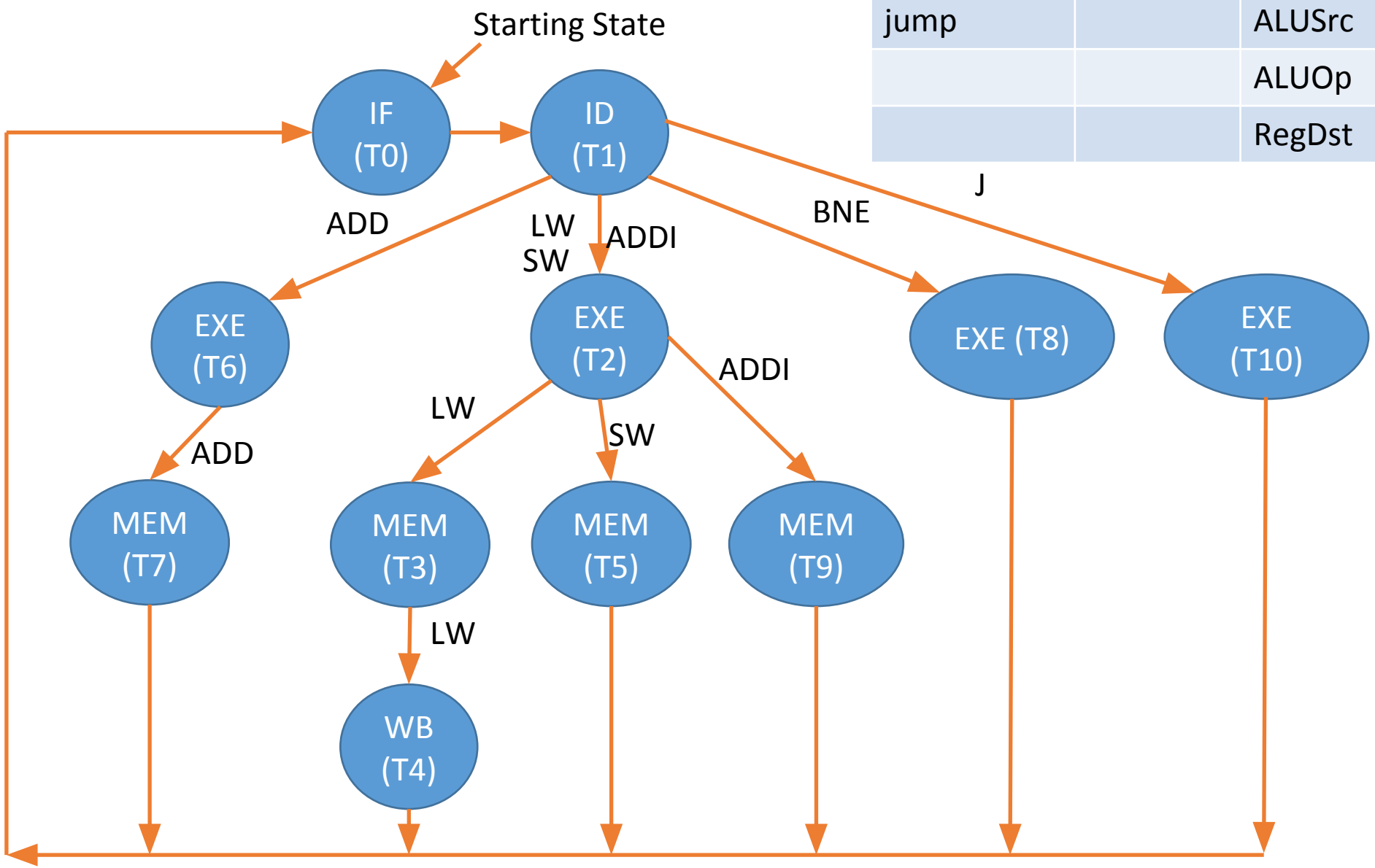
- How does one generate the control signals?
  - We want to add a minarals liquid with the water flowing through a pipeline after every 1 KM
  - How?

Control generation: Single-cycle

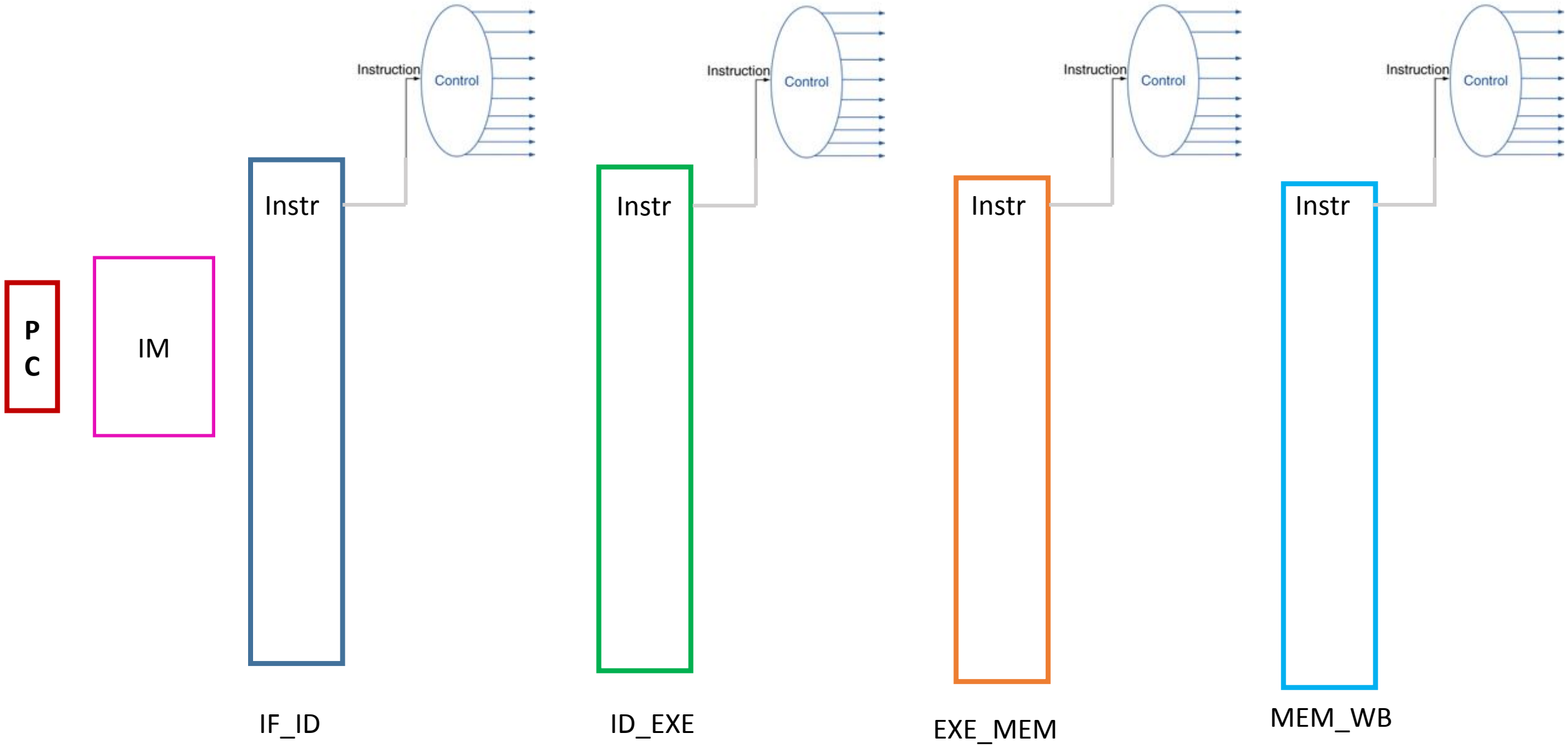
Instr.	Jump	RegDst	RegWrite	ALUSrc	Branch	ALUOp1	ALUOp0	MemRead	MemWrite	MemtoReg
R-type	0	1	1	0	0	1	0	0	0	0
lw	0	0	1	1	0	0	0	1	0	1
sw	0	x	0	1	0	0	0	0	1	x
addi	0	0	1	1	0	0	0	0	0	0
B-type	0	x	0	0	1	0	1	0	0	x
J-type	1	x	0	x	x	x	x	0	0	x

Control generation: Multi-cycle

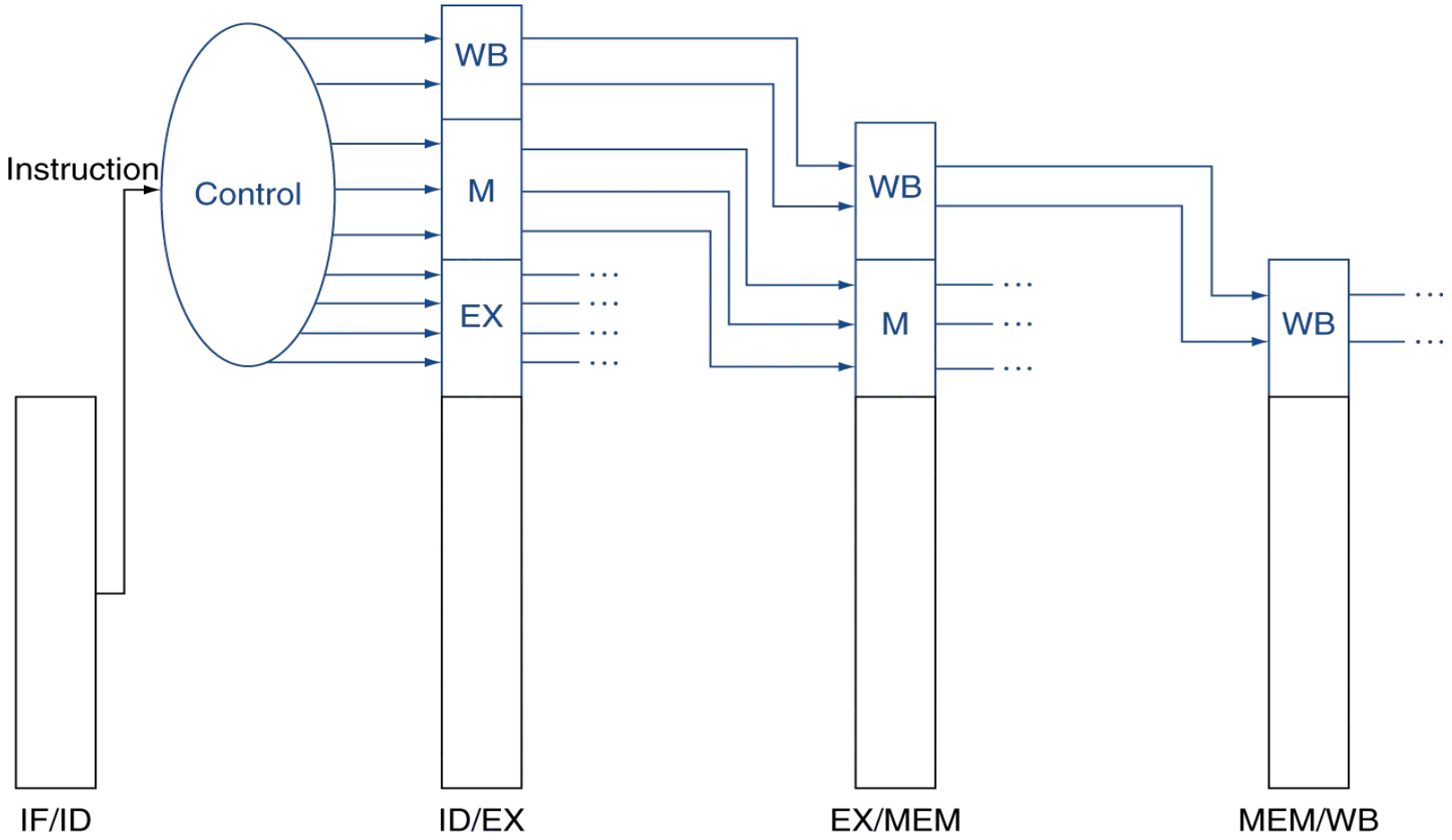
Fetch	Decode	Execute	Memory	Write Back
jump		ALUSrc	Branch	MemtoReg
		ALUOp	MemRead	RegWrite
		RegDst	MemWrite	



Control generation: Strategy - 1



Control generation: Strategy - 2



Book- P&H-COD

Control unit for Pipelined MIPS processor

Instr	Execution/Address Calc stage control lines					Memory access stage control lines			Write-back control lines	
Instr	**Jump**	**RegDst**	**ALUOp1**	**ALUOp0**	**ALUSrc**	**Branch**	**MemRead**	**MemWrite**	**RegWrite**	**MemtoReg**
R-format	0	1	1	0	0	0	0	0	1	0
lw	0	0	0	0	1	0	1	0	1	1
sw	0	x	0	0	1	0	1	0	0	x
beq	0	x	0	1	0	1	0	0	0	x

Single cycle MIPS processor

Instr	Jump	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R-format	0	1	1	0	0	0	0	0	1	0
lw	0	0	0	0	1	0	1	0	1	1
sw	0	x	0	0	1	0	1	0	0	x
beq	0	x	0	1	0	1	0	0	0	x

Control unit for Pipelined MIPS processor

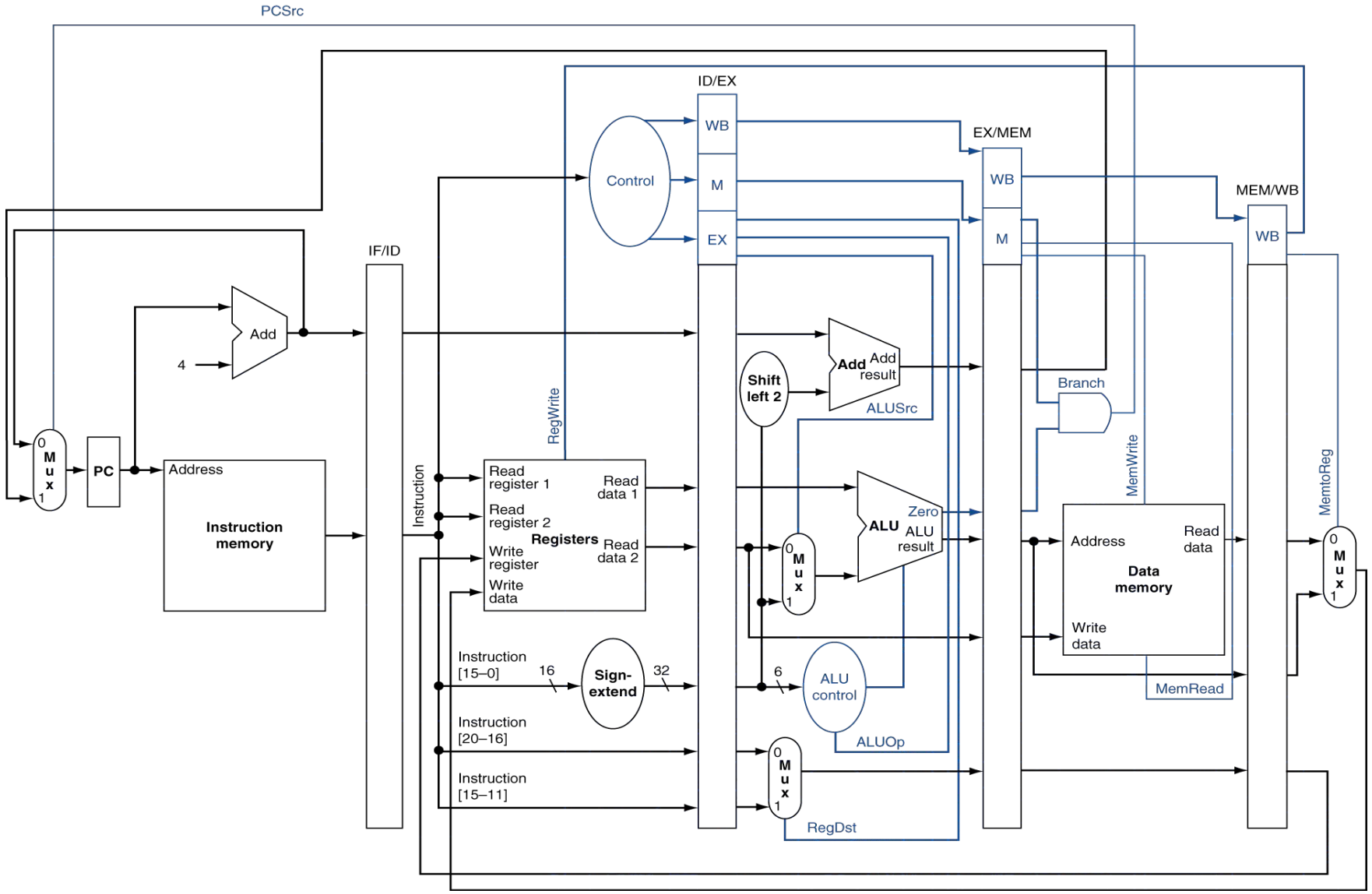
- How to generate such control signals?
  - Settings the 10 control lines in each stage for each instruction
  - Simplest way is same as in single cycle
  - Most the controls can be generated at the same time or decoding stage

- How to manage the control signals generated for i-th instruction and control signal will be generated for (i+1)-th instructions?
  - Erroneous control signals can be generated

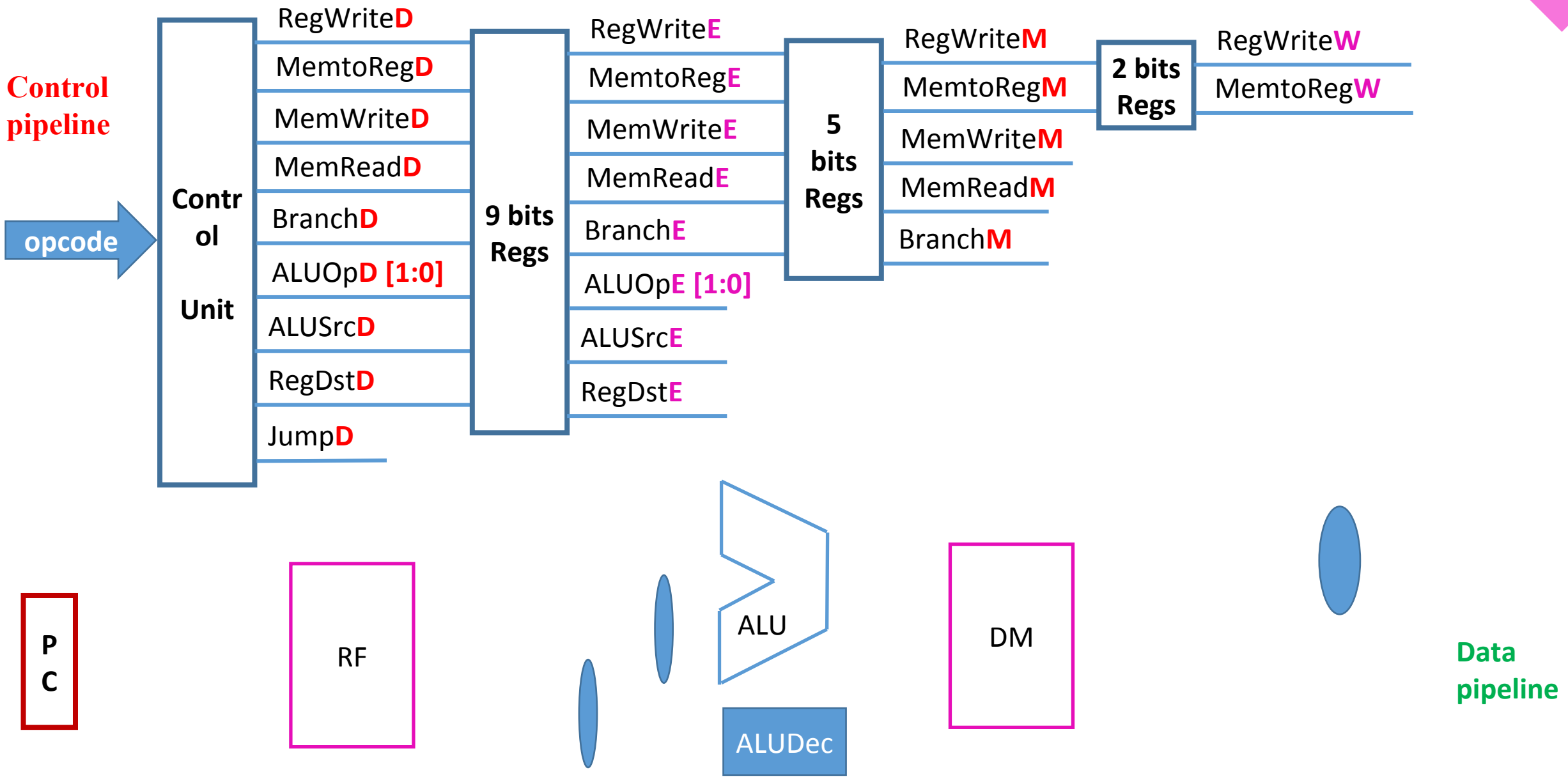
Control unit for Pipelined MIPS processor

- How to manage the control signals generated for i-th instruction and control signal will be generated for (i+1)-th instructions?
  - Erroneous control signals can be generated
  - Extension of the pipeline registers for storing the control signals' values

Pipelined Datapath & Control



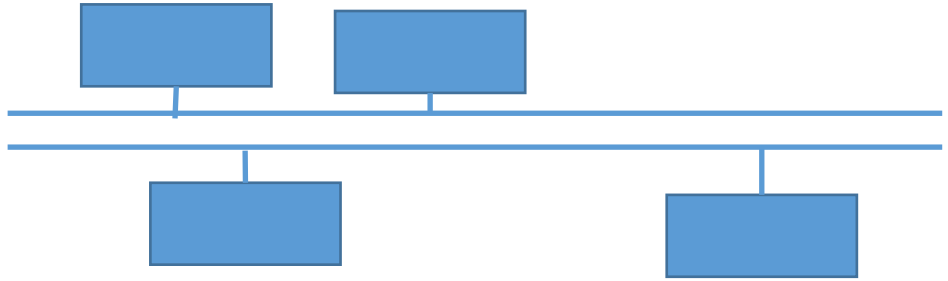
Pipelined Control Signals

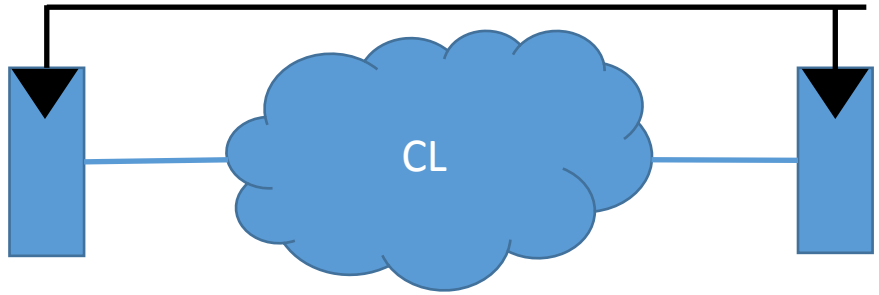


Designing Instruction Sets for Pipelining

- MIPS's instructions are same length
- X86's instructions vary 1 byte to 15 byte, is pipelining challenging ??
- MIPS has a few addressing modes
- Memory operands only appear in loads or stores in MIPS
- Operand are aligned in memory

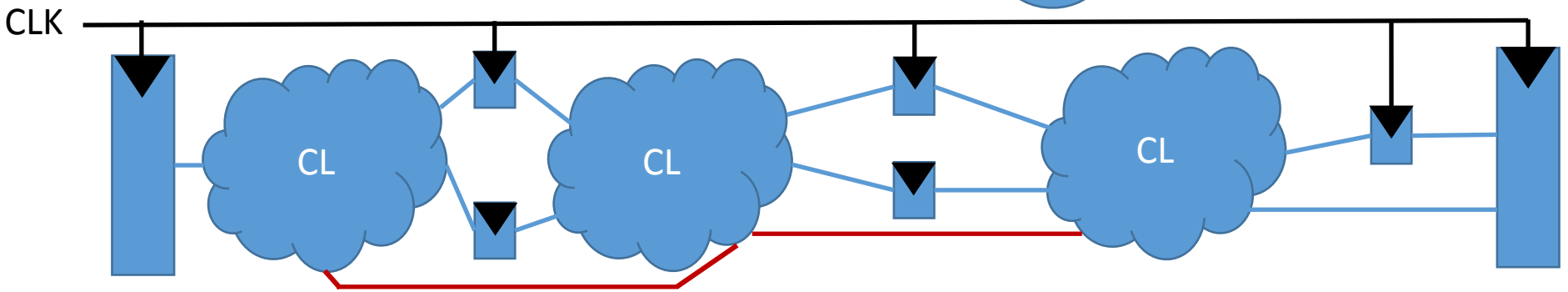
Comparison of datapaths





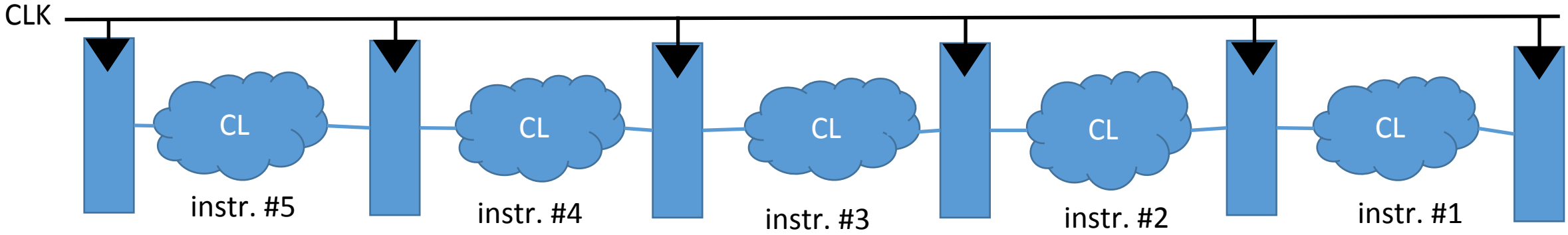
CL: Combinational Logic

Only one instr. In the datapath at an instant of time



Only one instr.

What if one instruction is here.



Microprocessor Design Trade-offs: Interconnects Vs Functional Units Vs IPC

- (clock) Cycle Per Instruction (CPI)
- Instructions Per (clock) Cycle/Seconds (IPC) = 1/CPI

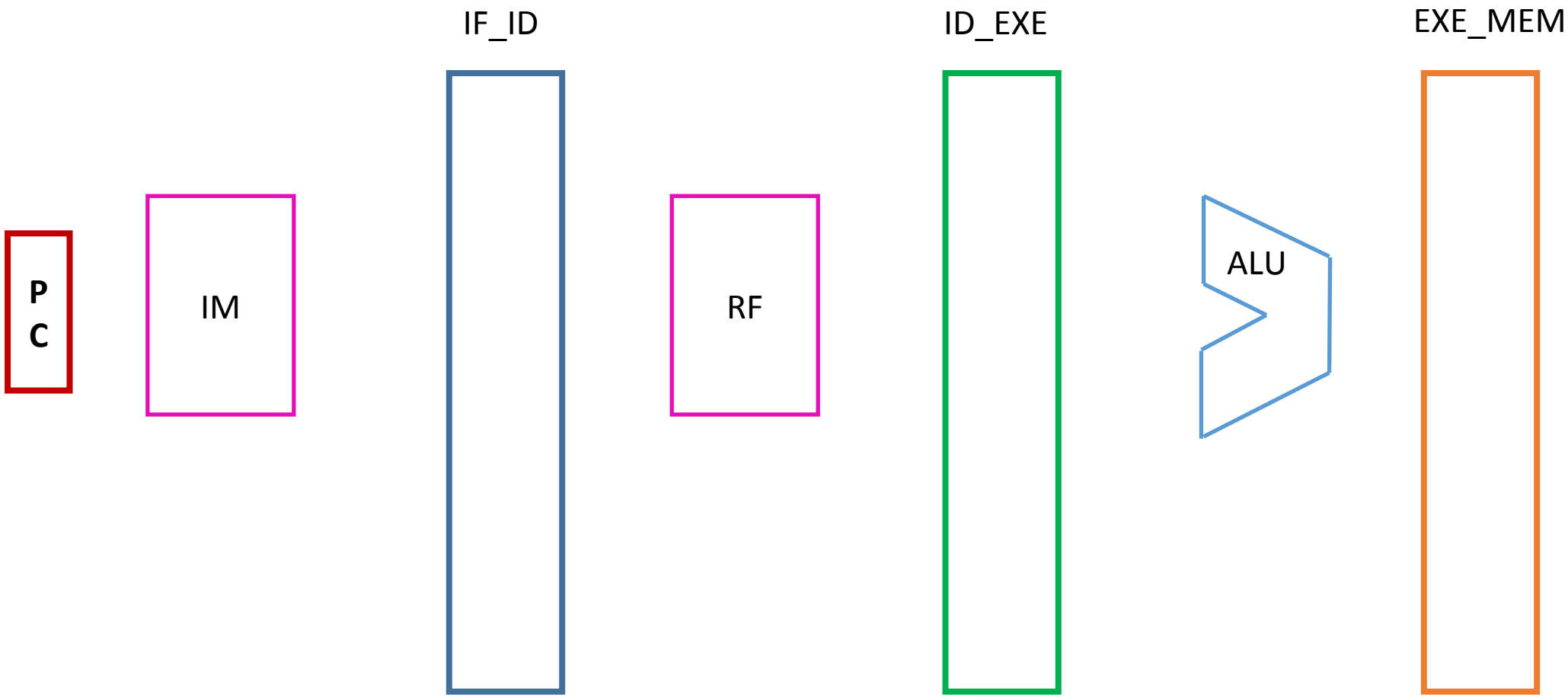
Interconnects (Bus)	Functional Units (FUs)	
	Less	More
Less	Single-bus & Single-FU (Multi-Cycle, IPC < 1)	Single-bus & Many-FUs (Multi-Cycle, IPC < 1)
More	Many-bus & Single-FU (Multi-Cycle, IPC < 1)	Many-bus & Many-FUs (Single-Cycle or Pipeline, IPC = 1)

Methods/Algorithms:
1) Multi-Cycle
2) Single-Cycle
3) Pipelined

- Pipeline: IPC = 1 (borrowed from Single-Cycle) and less clock period (T) (borrowed from Multi-Cycle), shared the Buses & FUs by more than one instruction.
- **Program Execution time: #instr. x (1/IPC) x Clk (T)**
- Can we have the IPC > 1?

Datapath for R-type: ADD R1, R2, R3

op	rs	rt	rd	shamt	funct
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Reg. File's write operation @posedge

&

Stage Reg.'s write operation @negedge

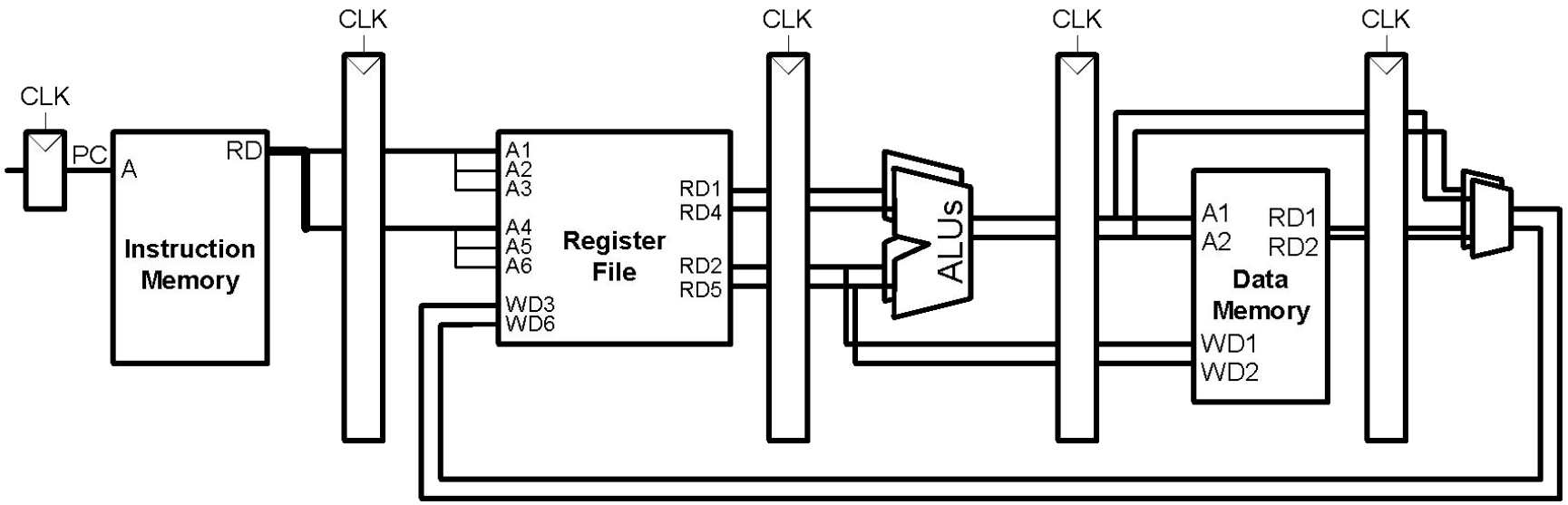
Can we have IPC > 1?

Multiple issue processor

Superscalar and VLIW

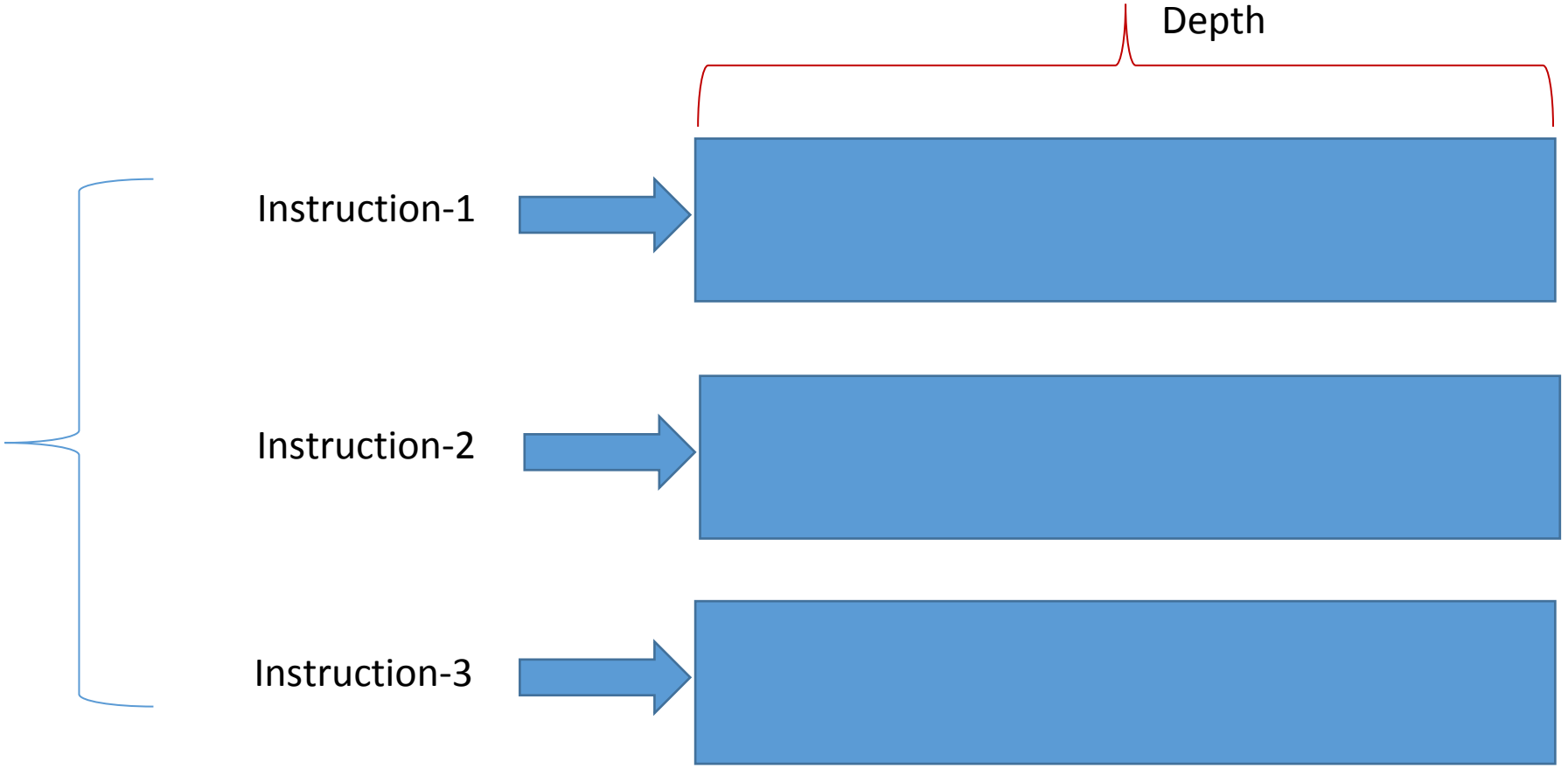
IPC = 2, Superscalar or multiple issue processor

IPC = 2, Superscalar or multiple issue processor



IPC = 3, Superscalar or 3 issue processor

- 3-instruction can be fetched or issued
- 3-instructions can be executed in parallel (in-order superscalar execution)



Spatial parallelism

Static Multiple Issue MIPS Processor

- Two issue Instruction
- Integer ALU operations – using one datapath
  - ADD, BNE, etc
- Data transfer operations – using 2nd datapath
  - LW & SW
- Instruction format



Static Multiple Issue Processor

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

Book-COD-P&H, CH-4

Static Multiple Issue Processor

Very Long Instruction Word (VLIW)

If one instruction of the pair cannot be used, we require that it be replaced with a noop. Thus, the instructions always issue in pairs, possibly with a noop in one slot.

In some designs, the compiler takes full responsibility for removing *all* hazards, scheduling the code and inserting no-ops so that the code executes without any need for hazard detection or hardware-generated stalls.

Static Multiple Issue Processor

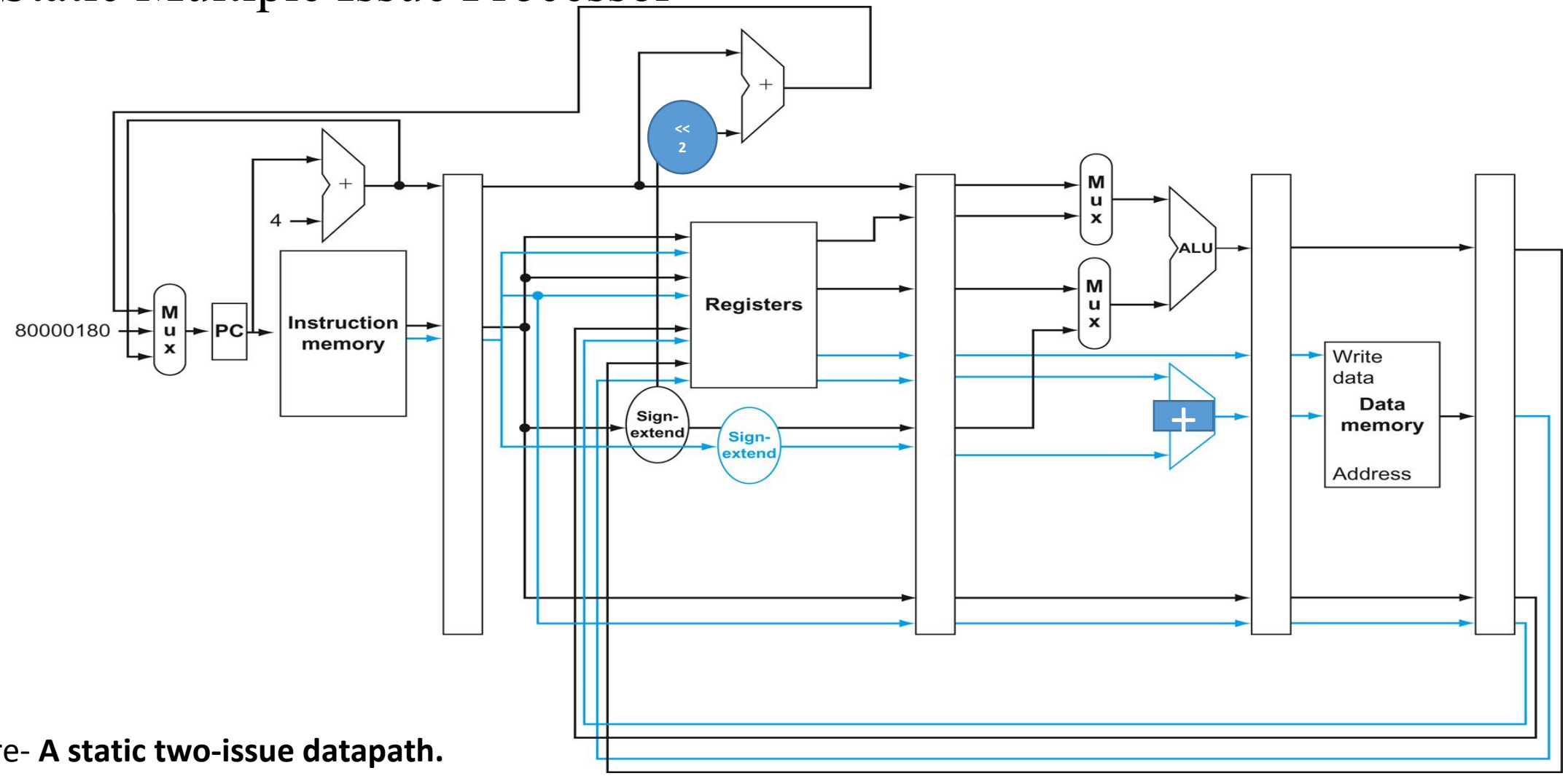


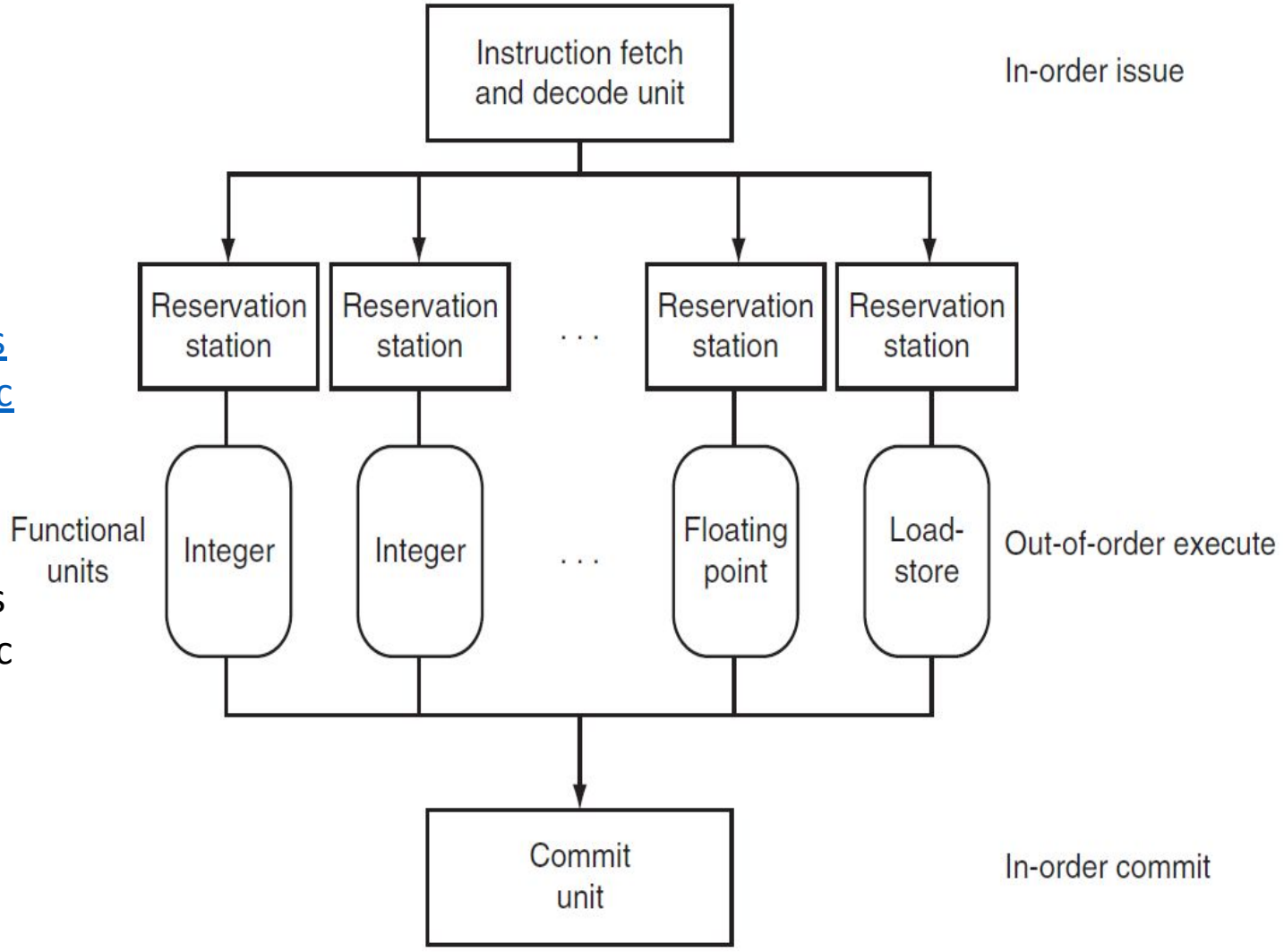
Figure- **A static two-issue datapath.**
The additions needed for double issue are highlighted: another 32 bits from instruction memory, two more read ports and one more write port on the register file, and another ALU. Assume the bottom ADDER handles address calculations for data transfers and the top ALU handles everything else.

Book-COD-P&H, CH-4

Dynamic Multiple Issue or Superscalar

1. Soreboard Technique
https://www.cs.umd.edu/~meesh/411/website/projects/dynamic/scoreboard.html

2. Tomasulo Technique:
https://www.cs.umd.edu/~meesh/411/website/projects/dynamic/tomasulo.html



Book-COD-P&H, CH-4

Difference between Superscalar and VLIW

- General Vs special instruction
- Dynamic Vs. Static pipeline scheduling
- Hazard detection
- Instruction format
- Different VLIW processor needs compilation of the application

ISA design steps

- Step-1:
  - Find out the instructions for the **Algorithm(s)**
- Step-2: [Microarchitecture design]
  - Find out the strategy (Sharedbus/Singlecycle/Multicycle/Pipeline[in order]/etc) for datapath and next
  - Design the datapath and its components for each instructions
- Step-3:
  - Design the combined datapaths for all instructions
- Step-4:
  - Decide the clock period based on the critical path [timing analysis]
  - Add setup time, clock-to-Q and etc. to the decided clock period [satisfy hold time constraints]
- Step-5:
  - Identify the control signals on the combined datapath
- Step-6:
  - Design the Control Unit (H/W or S/W) for generating the such control signals based on the strategy (Sharedbus/Singlecycle/Multicycle/Pipeline[in order]/etc) decided for datapath
- Step-7:
  - Test & verification of the designed processor

How about Single-purpose microprocessor like MinMax microprocessor?

Applying Pipeline Technique in Other Processors

- MinMax Processor
  - Recording is available in the Google-classroom

- Simple CPU
  - Recording is available in the Google-classroom

Homework

- Design the Pipelined MIPS ISA using Verilog HDL and C++
- Convert
  - MinMax microprocessor in Pipelined MinMax
- Design the Pipelined MinMax microprocessor using Verilog HDL and C++
- How does Intel manages to run CISC-type code onto RISC-based pipeline?

Summary

- Limitation of Multi-cycle approach
- CPI Vs IPC
- Comparison between single-cycle and pipelined approaches
- Views of pipeline in operation
- Comparison of datapaths
- Design tradeoffs of microprocessors
- Datapath and CU for pipelined processor